

Control IF Conditions

Control IF Conditions

PL/SQL Control IF-THEN-ELSE Conditions

- IF THEN Condition
- Structure of IF THEN Condition
- Example of IF THEN Condition
- Structure of IF THEN ELSE Condition
- Example of IF THEN ELSE Condition
- Structure of IF THEN ELSIF Condition
- Example of IF THEN ELSIF Condition

Control IF Conditions

IF THEN Condition

There are three ways to write IF condition:

- IF – END IF
- IF – ELSE – END IF
- IF – ELSIF – ELSE – END IF

Control IF Conditions

Structure of IF THEN Condition

There are three ways to write IF condition:

```
IF condition THEN
   statements;
END IF;
```

For example:

```
IF v_sal > 2000 Then
   total_sal := v_sal + 5000;
END IF;
```

Control IF Conditions

Example of IF THEN Condition

```
DECLARE
   v_sal EMP.SAL%TYPE;
BEGIN
   Select sal INTO v_sal
   From emp
   Where ename = 'SMITH';

   IF v_sal < 850 THEN
    DBMS_OUTPUT.PUT_LINE('Low paid salary person');
   END IF;
END;
/
```

Control IF Conditions

Structure of IF THEN ELSE Condition

There are three ways to write IF condition:

```
IF condition THEN
   statements;
ELSE
   statements;
END IF;
```

Control IF Conditions

Example of IF THEN ELSE Condition

```
DECLARE
   v_sal EMP.SAL%TYPE;
BEGIN
   Select sal INTO v_sal
   From emp
   Where ename = 'SMITH';

   IF v_sal < 850 THEN
    DBMS_OUTPUT.PUT_LINE('Low paid salary person');
   ELSE
    DBMS_OUTPUT.PUT_LINE('Higly paid salary person');
   END IF;
END;
/
```

Control IF Conditions

Structure of IF THEN ELSIF Condition

There are three ways to write IF condition:

```
IF condition THEN
   statements;
ELSIF
   statements;
ELSE
   statements;
END IF;
```

Control IF Conditions

Example of IF THEN ELSIF Condition

```
DECLARE
   v_sal EMP.SAL%TYPE;
BEGIN
   Select sal INTO v_sal
   From emp
   Where ename = 'SMITH';

   IF v_sal < 850 THEN
     DBMS_OUTPUT.PUT_LINE('Low paid salary person');
   ELSIF (v_sal >= 850 AND v_sal <=1000) THEN
     DBMS_OUTPUT.PUT_LINE('Average paid salary person');
   ELSE
     DBMS_OUTPUT.PUT_LINE('Higly paid salary person');
   END IF;
END;
/
```

Introduction to Database Administration



FOR & WHILE Loops

FOR & WHILE Loops

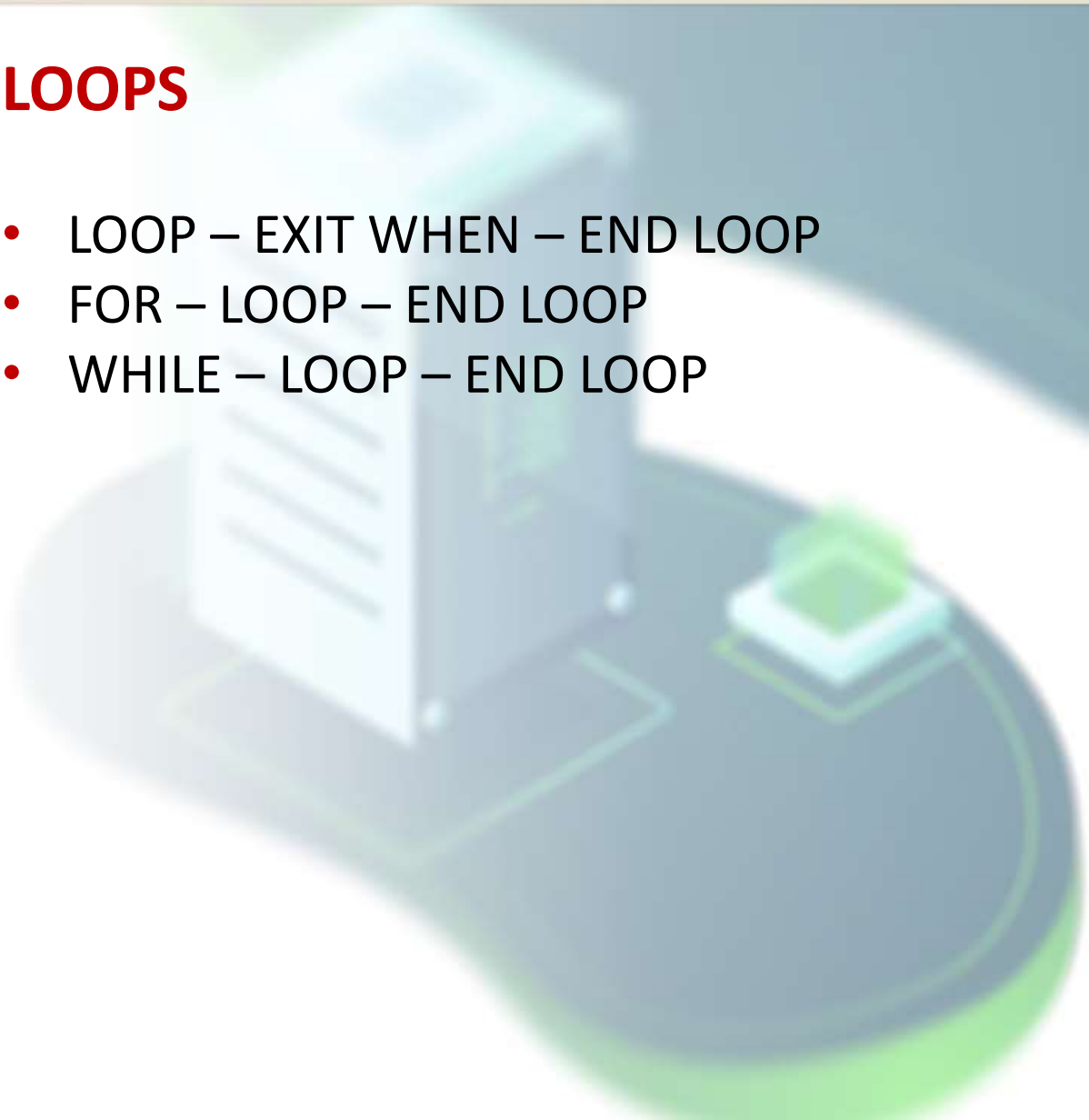
LOOPS, FOR & WHILE Loops

- LOOPS
- Loop with EXIT
- FOR Loop
- FOR Loop Syntax
- FOR Loop - Example
- FOR Loop - Bulk Data Insertion
- WHILE Loop
- WHILE Loop - Example
- WHILE Loop Syntax

FOR & WHILE LOOPS

LOOPS

- LOOP – EXIT WHEN – END LOOP
- FOR – LOOP – END LOOP
- WHILE – LOOP – END LOOP



FOR & WHILE LOOPS

Loop with EXIT

Following is its example (Repeat until LOOP):

```
DECLARE
      ...
      v_deptno   dept.deptno%TYPE   := 50;
        v_counter  integer := 1;
       ...
BEGIN
       ...
       LOOP
          INSERT INTO dept(deptno)
                      VALUES(v_deptno);
          v_counter := v_counter + 1;
          v_deptno := v_deptno + 10;
          EXIT WHEN v_counter > 5;
      END LOOP;
      ...
END;
/
```

FOR & WHILE LOOPS

FOR Loop

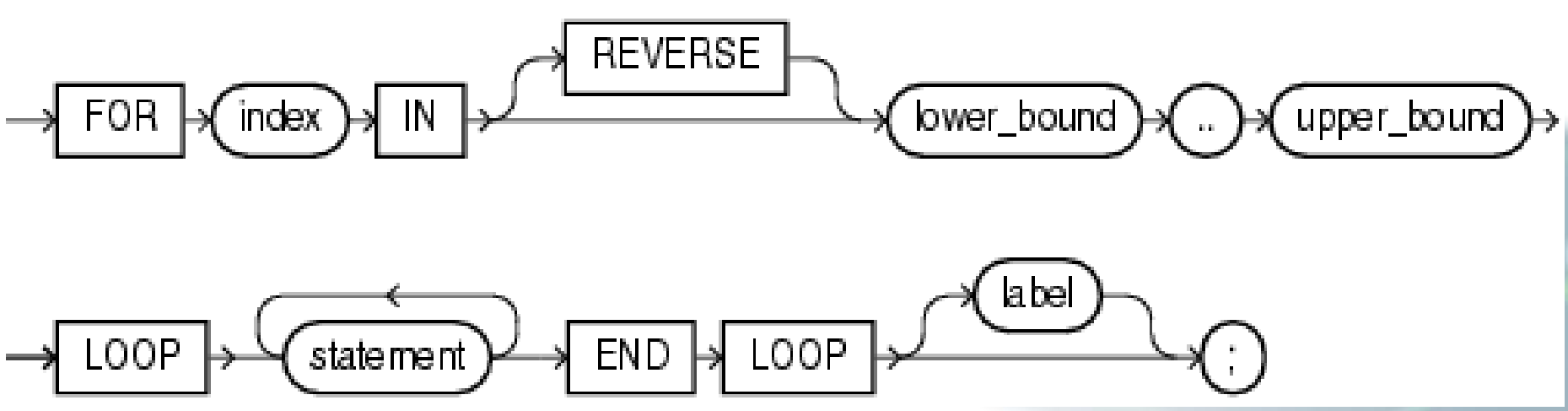
Following is its Syntax:

```
FOR index IN lower_bound .. upper_bound
LOOP
    statements;
END LOOP;
```

FOR & WHILE LOOPS

FOR Loop Syntax

Following is its generic syntax:



FOR & WHILE LOOPS

FOR Loop - Example

Following example displays serial numbers:

```
BEGIN
  FOR l_counter IN 1..5
  LOOP
    DBMS_OUTPUT.PUT_LINE( l_counter );
  END LOOP;
END;
/
```

FOR & WHILE LOOPS

FOR Loop - Example

Following example displays serial numbers in reverse:

```
BEGIN
  FOR l_counter IN REVERSE 1..3
  LOOP
    DBMS_OUTPUT.PUT_LINE( l_counter );
  END LOOP;
END;
/
```

FOR & WHILE LOOPS

FOR Loop - Bulk Data Insertion

Following example displays serial numbers in reverse:

```
BEGIN
  FOR i IN 55..70
  LOOP
    insert into dept(deptno, dname, loc)
    values (i, 'Dummy Dept-'||i, 'No Loc');
    DBMS_OUTPUT.PUT_LINE( i );
  END LOOP;
  Commit;
END;
/
```

```
 DEPTNO DNAME          LOC
------- -------------- -------------
     55 Dummy Dept-55  No Loc
     56 Dummy Dept-56  No Loc
     57 Dummy Dept-57  No Loc
     58 Dummy Dept-58  No Loc
     59 Dummy Dept-59  No Loc
     60 Dummy Dept-60  No Loc
     10 ACCOUNTING     NEW YORK
```

FOR & WHILE LOOPS

WHILE Loop

Following is its Syntax:

```
WHILE condition
LOOP
   statements;
END LOOP;
```

FOR & WHILE LOOPS

WHILE Loop - Example

Following is its example:

```
DECLARE
  n_counter NUMBER := 1;
BEGIN
  WHILE n_counter <= 5
  LOOP
    DBMS_OUTPUT.PUT_LINE( 'Counter : ' || n_counter );
    n_counter := n_counter + 1
  END LOOP;
END;
/
```

FOR & WHILE LOOPS

WHILE Loop Syntax

Following is its generic syntax:



Introduction to Database Administration



Anonymous Blocks with Examples

Anonymous Blocks with Examples

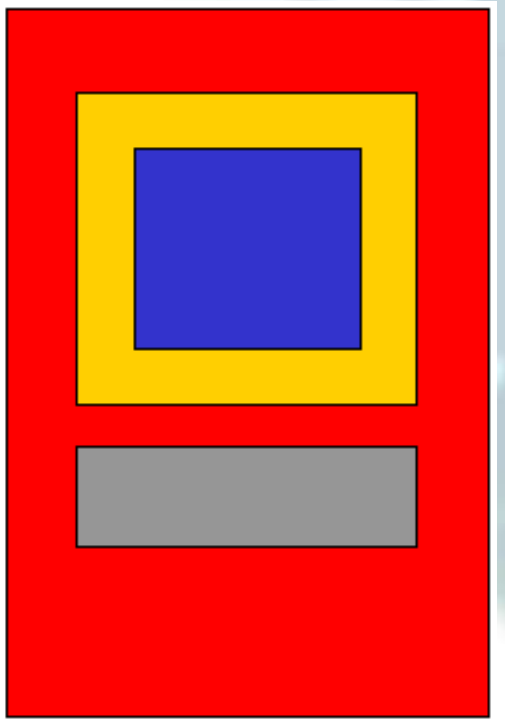
Anonymous Blocks with Examples

- Nesting Anonymous Blocks
- Nesting Anonymous Blocks – Example
- Repeat Until LOOP - Example

Anonymous Blocks with Examples

Nesting Anonymous Blocks

Blocks within blocks give the concept of Nested blocks



Anonymous Blocks with Examples

Repeat Until LOOP - Example

This loop executes once first time. It is same as DO-WHILE Loop.

```
LOOP
   monthly_value := daily_value * 31;
   EXIT WHEN monthly_value > 4000;
END LOOP;
```

Anonymous Blocks with Examples

Nesting Anonymous Blocks - Example

```
DECLARE
  n_counter NUMBER := 1;
BEGIN
  DBMS_OUTPUT.PUT_LINE('**************----Outer Block----**************');
  FOR i IN 55..70
  LOOP
    DBMS_OUTPUT.PUT_LINE( 'Outer Block Counter : ' || i );
  END LOOP;
  BEGIN
        WHILE n_counter <= 5
        LOOP
        DBMS_OUTPUT.PUT_LINE( 'Inner Block Counter : ' || n_counter );
        n_counter := n_counter + 1;
  END LOOP;
  END; --inner block end
END; --outer block end
/
```

Introduction to Database Administration



Exception Handlings-1

Exception Handlings-1

PL/SQL Exception Handlings

- PL/SQL Exception
- PL/SQL Exception Usage
- PL/SQL Exception - Example-1
- PL/SQL Exception - Example-2

Exception Handlings-1

PL/SQL Exception

- PL/SQL treats all errors that occur in an anonymous block, procedure, or function as exceptions.
- The exceptions can have different causes such as coding mistakes, bugs, even hardware failures.
- It is not possible to anticipate all potential exceptions, however, you can write code to handle exceptions to enable the program to continue running as normal.

Exception Handlings-1

PL/SQL Exception

- The code that you write to handle exceptions is called an exception handler.
- A PL/SQL block can have an exception-handling section, which can have one or more exception handlers.

Exception Handlings-1

PL/SQL Exception Usage

```
BEGIN
   -- executable section ...
   -- exception-handling section
   EXCEPTION
      WHEN e1 THEN
         -- exception_handler1
      WHEN e2 THEN
         -- exception_handler1
      WHEN OTHERS THEN
         -- other_exception_handler
END;
/
```

Exception Handlings-1

PL/SQL Exception – Example-1

Following program will terminate without successfully execution when employee no does not exist in table EMP.

```
DECLARE
    v_ename EMP.ename%TYPE;
    v_empno EMP.empno%TYPE := &empno;
BEGIN
    -- get the employee name by id
    SELECT ename INTO v_ename
    FROM EMP
    WHERE empno = v_empno;

    -- show the customer ename
    dbms_output.put_line('Employee name is ' || v_ename);
END;
/
```

Exception Handlings-1

PL/SQL Exception – Example-1

Following program will execute successfully when employee no does not exist in table EMP.

```
DECLARE
    v_ename EMP.ename%TYPE;
    v_empno EMP.empno%TYPE := &empno;
BEGIN
    -- get the employee name by id
    SELECT ename INTO v_ename
    FROM EMP
    WHERE empno = v_empno;

    -- show the customer ename
    dbms_output.put_line('Employee name is ' || v_ename);
EXCEPTION
        WHEN NO_DATA_FOUND THEN
          dbms_output.put_line('Employee ID' || v_empno ||   ' does not exist');
END;
/
```

Exception Handlings-1

PL/SQL Exception – Example-2

Following program will execute successfully using multiple Exceptions.

```
DECLARE
    v_ename EMP.ename%TYPE;
    v_job EMP.job%TYPE := &job;
BEGIN
    -- get the employee name by id
    SELECT ename INTO v_ename
    FROM EMP
    WHERE job = v_job;

    -- show the customer ename
    dbms_output.put_line('Employee name is ' || v_ename);
EXCEPTION
        WHEN NO_DATA_FOUND THEN
          dbms_output.put_line('Employee ID' || v_ename ||   ' does not exist');
        WHEN TOO_MANY_ROWS THEN
          dbms_output.put_line('Database returns many employees');
END;
/
```

```
Enter value for job: 'CLERK'
old   3:     v_job EMP.job%TYPE := &job;
new   3:     v_job EMP.job%TYPE := 'CLERK';
Database returns many employees

PL/SQL procedure successfully completed.
```

Introduction to Database Administration



Exception Handlings-2

Exception Handlings-2

PL/SQL Exception Handlings

- Raise Exception
- User-Defined Exception
- Affecting PL/SQL Exception
- Other Exceptions

Exception Handlings-2

Raise Exception

Exceptions are raised by the database server automatically whenever there is any internal database error, but exceptions can be raised explicitly by the programmer by using the command **RAISE**. Following is the simple syntax for raising an exception:

```
DECLARE
   exception_name EXCEPTION;
BEGIN
   IF condition THEN
      RAISE exception_name;
   END IF;
EXCEPTION
   WHEN exception_name THEN
   statement;
END;
/
```

Exception Handlings-2

User-Defined Exception

```
DECLARE
   v_empno EMP.empno%type := &cv_empno;
   v_name  EMP.ename%type;
   v_job EMP.job%type;
   -- user defined exception
   ex_invalid_empno  EXCEPTION;
BEGIN
   IF v_empno <= 0 THEN
      RAISE ex_invalid_empno;
   ELSE
      SELECT  ename, job INTO  v_name, v_job
      FROM EMP
      WHERE empno = v_empno;
      DBMS_OUTPUT.PUT_LINE ('Name: '||  v_name);
      DBMS_OUTPUT.PUT_LINE ('job: ' || v_job);
   END IF;

EXCEPTION |
   WHEN ex_invalid_empno THEN
      dbms_output.put_line('empno must be greater than zero!');
   WHEN no_data_found THEN
      dbms_output.put_line('No such employee!');
   WHEN others THEN
      dbms_output.put_line('Error!');
END;
/
```

Exception Handlings-2

Affecting PL/SQL Exception

One exception does not affect the rest of program

```
DECLARE
v_tname varchar2(100);
BEGIN
 BEGIN
  select tname into v_tname
  from tab
  where tname='CMG_EFFORTT';
  DBMS_OUTPUT.PUT_LINE('table name:'||v_tname);
  exception
    when no_data_found then
    DBMS_OUTPUT.PUT_LINE('No data');
 END;
 select tname into v_tname
  from tab
  where tname='CMG_EFFORT';
  DBMS_OUTPUT.PUT_LINE('table name:'||v_tname);
 exception
 when no_data_found then
    DBMS_OUTPUT.PUT_LINE('No data second exception');
 --go to 1;
END;
/
```

Exception Handlings-2

Other Exceptions

Examples are
NO_DATA_FOUND, ZERO_DIVIDE, OTHERS

Raise some exception

```
WHEN NO_DATA_FOUND THEN
      raise_application_error(-20011,'Invalid FK value');
```

Raise My_Excep;

To display details of oracle standard error message

```
EXCEPTION
      WHEN OTHERS THEN
                  DBMS_OUTPUT.PUT_LINE('Error detail is: '||SQLERRM)
```

Introduction to Database Administration



PL/SQL Procedure

PL/SQL Procedure

PL/SQL Procedure

- What is a Procedure in PL/SQL?
- Procedure - Example
- Compiling & Executing Procedure
- Invoking a Procedure in a Block
- Executing a Procedure & Show Errors
- Data Dictionary for Procedures
- Managing Procedure using SQL Developer

PL/SQL Procedure

What is a Procedure in PL/SQL?

- Is a block with a name
- The DECLARE key word is not used
- Parameters can be
  - IN
  - OUT
  - IN OUT
- Is stored (USER_SOURCE)

PL/SQL Procedure

What is Procedure in PL/SQL?

Writing PL/SQL procedure

```
CREATE OR REPLACE PROCEDURE pname(                ) IS
        
BEGIN
        
EXCEPTION
        
END;
/
```

PL/SQL Procedure

Procedure – Example

```
SET SERVEROUTPUT ON;

CREATE OR REPLACE PROCEDURE
    proc_test(p_empno IN VARCHAR2)  IS
       v_job      EMP.job%TYPE;
       v_sal      EMP.sal%TYPE;
BEGIN
         SELECT job, sal
         INTO  v_job,v_sal
         FROM  emp
         WHERE  empno = p_empno;
         DBMS_OUTPUT.PUT_LINE('job is '||v_job);
EXCEPTION
     WHEN OTHERS THEN
         DBMS_OUTPUT.PUT_LINE('ERROR...');
END;
/
```

PL/SQL Procedure

Compiling & Executing Procedure

```
SQL> Show errors


SQL> execute proc_test(5893);
```

PL/SQL Procedure

Invoking a Procedure in a Block

```
DECLARE
    …
BEGIN
    …
    proc_test('23');
    …
END;
/

Or
SQL> exec proc_test('1123')
```

PL/SQL Procedure

Executing a Procedure & Show Errors

```
  1  create or replace procedure test_proc is
  2       v_id  INTEGER;
  3      v_empno emp.empno%TYPE;
  4  BEGIN
  5      v_id := 1234567;
  6      select empno into v_empno
  7      FROM  EMP
  8      WHERE empno = v_id;
  9  EXCEPTION
 10      WHEN NO_DATA_FOUND THEN
 11       DBMS_OUTPUT.PUT_LINE('No record exists');
 12*   END;
SQL> /

Procedure created.

SQL> exec test_proc
No record exists

PL/SQL procedure successfully completed.

SQL> show errors                (to see errors for procedures, functions)
```

PL/SQL Procedure

Data Dictionary for Procedures

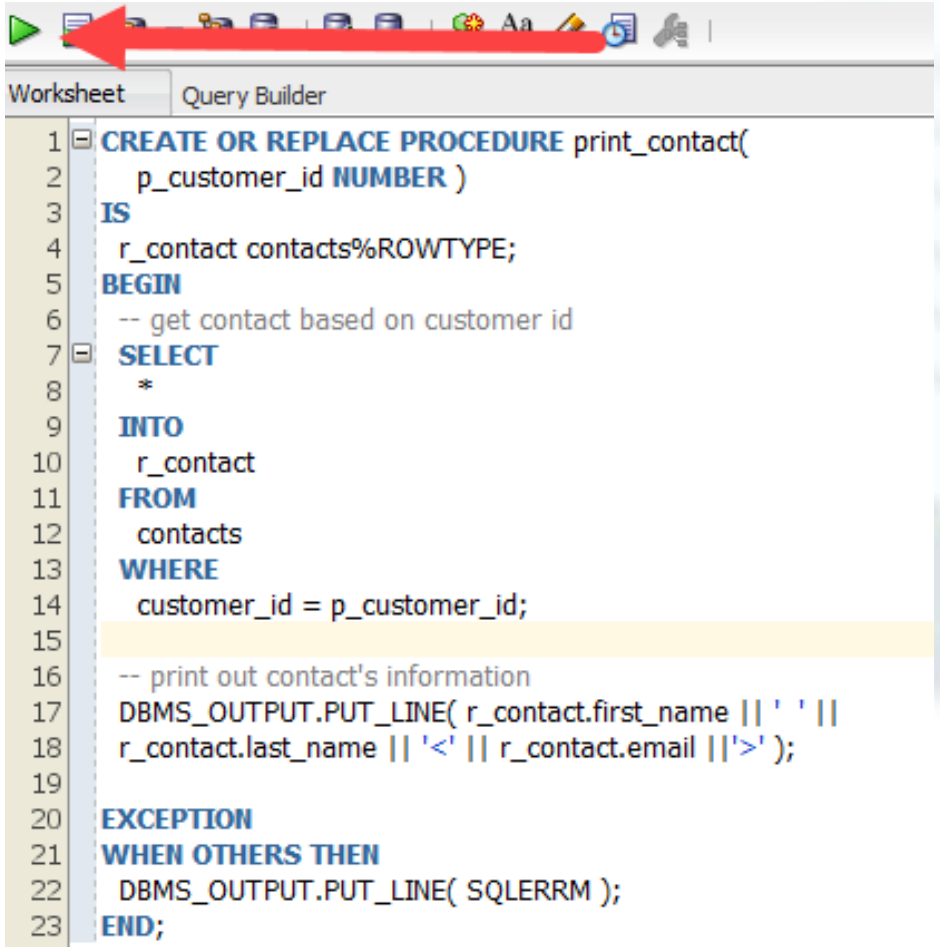
```
SQL> select Text from user_source
  2  where name='PROC_TEST';

TEXT
--------------------------------------------------
PROCEDURE
    proc_test(p_empno IN VARCHAR2)  IS
       v_job  EMP.job%TYPE;
       v_sal  EMP.sal%TYPE;
BEGIN
 SELECT job, sal
 INTO  v_job,v_sal
 FROM  emp
         WHERE  empno = p_empno;
 DBMS_OUTPUT.PUT_LINE('job is '||v_job);
EXCEPTION

TEXT
--------------------------------------------------
     WHEN OTHERS THEN
         DBMS_OUTPUT.PUT_LINE('ERROR...');
END;
```

PL/SQL Procedure

Managing Procedure using SQL Developer



Introduction to Database Administration



PL/SQL Function

PL/SQL Function

PL/SQL Function

- What is Function in PL/SQL?
- Function - Example
- Compiling & Executing Function
- Executing a Function & Show Errors
- Data Dictionary for Function

PL/SQL Function

What is Function in PL/SQL?

- Is a block with a name
- Returns one value only.
- The DECLARE key word is not used
- Parameters can only be IN
- Is stored (USER_SOURCE)

PL/SQL Function

What is Function in PL/SQL?

Writing PL/SQL Function

```
CREATE OR REPLACE FUNCTION fname( [          ] )
RETURN datatype IS
        [          ]
BEGIN
        [          ]
        [          ]
EXECPTION
        [          ]
END;
/
```

PL/SQL Function

Function - Example

```
CREATE OR REPLACE FUNCTION
    func_test(p_empno IN VARCHAR2)  RETURN NUMBER IS
       v_job      EMP.job%TYPE;
       v_sal      EMP.sal%TYPE;
BEGIN
         SELECT job, sal
         INTO  v_job,v_sal
         FROM  emp
         WHERE  empno = p_empno;
         DBMS_OUTPUT.PUT_LINE('job is '||v_job);
         return v_sal;
EXCEPTION
     WHEN OTHERS THEN
         DBMS_OUTPUT.PUT_LINE('ERROR...');
END;
/

SQL> Show errors
```

PL/SQL Function

Compiling & Executing Function

```
SET SERVEROUTPUT ON;
DECLARE
   …
   v_sal     emp.sal%TYPE;
   …
BEGIN
   …
   v_sal := func_test(3477);

DBMS_OUTPUT.PUT_LINE(TO_CHAR(v_sal)
);
   …
END;
/
```

PL/SQL Function

Executing a Function & Show Errors

```
SQL> CREATE OR REPLACE FUNCTION
  2      func_test(p_empno IN VARCHAR2)  RETURN NUMBER IS
  3         v_job  EMP.job%TYPE;
  4         v_sal  EMP.sal%TYPE;
  5  BEGIN
  6   SELECT job, sal
  7   INTO  v_job,v_sal
  8   FROM  emp
  9           WHERE  empno = p_empno;
 10   DBMS_OUTPUT.PUT_LINE('job is '||v_job);
 11           return v_sal;
 12  EXCEPTION
 13       WHEN OTHERS THENv_sal;
 14           DBMS_OUTPUT.PUT_LINE('ERROR...');
 15  END;
 16  /

Warning: Function created with compilation errors.

SQL> show error
Errors for FUNCTION FUNC_TEST:

LINE/COL ERROR
-------- -----------------------------------------------------------------
13/18    PLS-00103: Encountered the symbol "THENV_SAL" when expecting one
         of the following:
         then or
         The symbol "then" was substituted for "THENV_SAL" to continue.
```

PL/SQL Function

Data Dictionary for Function

```
SQL> select text from user_source
  2  where name='FUNC_TEST';

TEXT
--------------------------------------------------------------------------------
FUNCTION
    func_test(p_empno IN VARCHAR2)  RETURN NUMBER IS
       v_job  EMP.job%TYPE;
       v_sal  EMP.sal%TYPE;
BEGIN
 SELECT job, sal
 INTO  v_job,v_sal
 FROM  emp
         WHERE  empno = p_empno;
 DBMS_OUTPUT.PUT_LINE('job is '||v_job);
         return v_sal;

TEXT
--------------------------------------------------------------------------------
EXCEPTION
     WHEN OTHERS THEN
         DBMS_OUTPUT.PUT_LINE('ERROR...');
END;
```

End

Introduction to Database Administration



Calling Procedures & Functions

Calling Procedures & Functions

- Calling Procedure
- Calling Function
- Calling Procedure with Parameters

Calling Procedures & Functions

Calling Procedure

Procedure can be called after compilation in some other function, procedure, block or package. For example,

```
DECLARE
   …
BEGIN
   …
   proc_test('23');
   …
END;
/

Or
SQL> exec proc_test('1123')
```

Calling Procedures & Functions

Calling Function

Function can be called after compilation in some other function, procedure, block or package by assigning it to variable.

```
DECLARE
   ...
   v_sal        emp.sal%TYPE;
   ...
BEGIN
   ...
   v_sal := func_test(3477);
   DBMS_OUTPUT.PUT_LINE(TO_CHAR(v_sal));
   ...
END;
/
```

Calling Procedures & Functions

Calling Procedure with Parameters

```
create or replace procedure proc_inout
                            (p_empno in number,
                             p_sal out number) is
BEGIN
   select sal
   into p_sal
   from emp
     where empno=p_empno;
END;
/

create or replace procedure proc_out is
  v_sal EMP.SAL%TYPE;
BEGIN
    proc_inout(7369,v_sal);
    DBMS_OUTPUT.PUT_LINE('salary is :'||v_sal);
END;
/

SQL> exec proc_out
salary is :2000

PL/SQL procedure successfully completed.
```

Introduction to Database Administration



Exercises of Functions

Exercises of Functions

- Compiling & Executing Function
- Function - Exercise-1
- Function - Exercise-2

Exercises of Functions

Compiling & Executing Function

```
SET SERVEROUTPUT ON;
DECLARE
   …
   v_sal     emp.sal%TYPE;
   …
BEGIN
   …
   v_sal := func_test(3477);

DBMS_OUTPUT.PUT_LINE(TO_CHAR(v_sal)
);
   …
END;
/
```

Exercises of Functions

Function – Exercise-1

```
CREATE OR REPLACE FUNCTION
    func_MaxSal(p_deptno IN NUMBER)  RETURN NUMBER IS
       v_MaxSal          EMP.sal%TYPE;
BEGIN
          SELECT max(sal)
          INTO  v_MaxSal
          FROM  emp
          where deptno = p_deptno;
          DBMS_OUTPUT.PUT_LINE('Highest Salary is '||v_MaxSal);
          return v_MaxSal;
EXCEPTION
     WHEN OTHERS THEN
          DBMS_OUTPUT.PUT_LINE('ERROR...'||SQLERRM);
END;
/
```

```
SQL> DECLARE
  2    vv_MaxSal EMP.SAL%TYPE;
  3  BEGIN
  4    vv_MaxSal := func_MaxSal(10);
  5    DBMS_OUTPUT.PUT_LINE('Calling from block:'||vv_MaxSal);
  6  END;
  7  /
Highest Salary is 5000
Calling from block:5000
```

Exercises of Functions

Function – Exercise-2

```
CREATE OR REPLACE FUNCTION fact(x number)
RETURN number IS
   f number;
BEGIN
   IF x=0 THEN
      f := 1;
   ELSE
      f := x * fact(x-1);
   END IF;
RETURN f;
END;
/

DECLARE
   num number;
   factorial number;
BEGIN
   num:= 6;
   factorial := fact(num);
   dbms_output.put_line(' Factorial '|| num || ' is ' || factorial);
END;
/
```

```
Factorial 6 is 720

PL/SQL procedure successfully completed.
```

Introduction to Database Administration



Exercises of Procedures

Exercises of Procedures

Exercises of Procedures

- Compiling & Executing Function
- Procedure Exercise-I

Exercises of Procedures

Compiling & Executing Procedure

```
DECLARE
    …
BEGIN
    …
    proc_test('23');
    …
END;
/
```

Or

```
SQL> exec proc_test(‘1123’)
```

Exercises of Procedures

Procedure – Exercise-1

```
CREATE OR REPLACE PROCEDURE
    proc_MaxSal(p_deptno IN NUMBER, p_MaxSal OUT NUMBER) IS
       v_MaxSal         EMP.sal%TYPE;
BEGIN
         SELECT max(sal)
         INTO  v_MaxSal
         FROM  emp
         where deptno = p_deptno;
         DBMS_OUTPUT.PUT_LINE('Highest Salary is '||v_MaxSal);
         p_MaxSal := v_MaxSal;
EXCEPTION
     WHEN OTHERS THEN
         DBMS_OUTPUT.PUT_LINE('ERROR...'||SQLERRM);
END;
/

CREATE OR REPLACE PROCEDURE proc_CALLING(p_dno IN NUMBER) IS
  vv_MaxSal EMP.SAL%TYPE;
BEGIN
  proc_MaxSal(p_dno, vv_MaxSal);
  DBMS_OUTPUT.PUT_LINE('Calling from block:'||vv_MaxSal);
END;
/


SQL> execute proc_CALLING(20);
```

```
SQL> execute proc_CALLING(20);
Highest Salary is 3400
Calling from block:3400

PL/SQL procedure successfully completed.
```

Introduction to Database Administration



PL/SQL Package-1

PL/SQL Package-1

- Purpose of PL/SQL Package
- PL/SQL Package contains …
- Schematic Diagram of Package
- PL/SQL Package Specification
- PL/SQL Package Body
- Package Specification & its Body

PL/SQL Package-1

Purpose of PL/SQL Package

- In PL/SQL, a package is a schema object that contains definitions for a group of related functionalities.
- A package includes variables, constants, cursors, exceptions, procedures, functions, and subprograms.
- It is compiled and stored in the Oracle Database.

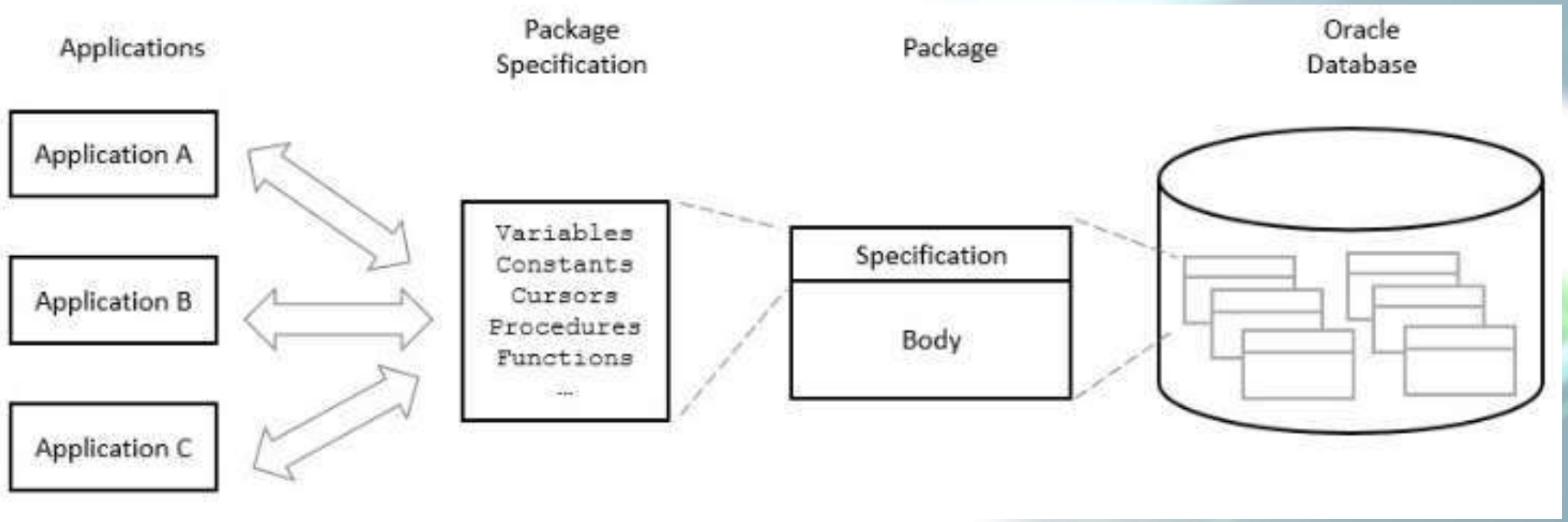
PL/SQL Package-1

PL/SQL Package contains …

- Typically, a package has a specification and a body. A package specification is mandatory while the package body can be required or optional, depending on the package specification.

PL/SQL Package-1

Schematic Diagram of Package



https://www.oracletutorial.com/plsql-tutorial/plsql-package/

PL/SQL Package-1

PL/SQL Package Specification

- The package specification declares the public objects that are accessible from outside the package.
- If a package specification whose public objects include cursors and subprograms, then it must have a body which defines queries for the cursors and code for the subprograms.

PL/SQL Package-1

PL/SQL Package Body

- A package body contains the implementation of the cursors or subprograms declared in the package specification. In the package body, you can declare or define private variables, cursors, etc., used only by package body itself.

PL/SQL Package-1

PL/SQL Package Body

- A package body can have an initialization part whose statements initialize variables or perform other one-time setups for the whole package.
- A package body can also have an exception-handling part used to handle exceptions.

PL/SQL Package-1

Package Specification & its Body

```
CREATE OR REPLACE PACKAGE emp_info IS



END emp_info;
/

CREATE OR REPLACE PACKAGE BODY emp_info is



END emp_info; --end of package body
/
```

Introduction to Database Administration



PL/SQL Package-2

PL/SQL Package-2

PL/SQL Package-2

- What is a Package in PL/SQL?
- Package Specification
- Package Body
- Add in a Package Body by hiding from User Specification
- Invoking Package Subprogram

PL/SQL Package-2

What is a Package in PL/SQL?

Selected and purposeful collection of procedures, functions and variables is managed in package.

- Package
  - Package Specification
  - Package Body
- Invoking Package subprogram
- Abstract Programming
- To avoid spreading of procedures & functions
- Is stored (USER_SOURCE)

PL/SQL Package-2

Package Specification

```
CREATE OR REPLACE PACKAGE emp_info IS

   v_count   INTEGER;  --global variable for all sub programs inside package

   PROCEDURE insert_record( p_empno IN NUMBER
                          , p_ename IN VARCHAR2
                          , p_job IN VARCHAR2
                          , p_sal IN NUMBER
                          , p_comm IN NUMBER
                          , p_deptno IN VARCHAR2);

   PROCEDURE delete_record(p_empno IN NUMBER);

   FUNCTION sum_dept_sal( p_deptno IN dept.deptno%TYPE) RETURN is dept.sal%TYPE;

END emp_info;
/
```

PL/SQL Package-2

Package Body

```
CREATE OR REPLACE PACKAGE BODY emp_info is

PROCEDURE insert_record(p_empno IN NUMBER,p_ename IN VARCHAR2,
p_job IN VARCHAR2,p_sal IN NUMBER,
p_comm IN NUMBER,p_deptno IN VARCHAR2) IS
BEGIN
      INSERT INTO EMP(empno,ename,job,sal,comm,deptno)
      VALUES(p_empno,p_ename,p_job,p_sal,p_comm,p_deptno);
END insert_record;

PROCEDURE delete_record(p_empno IN NUMBER) IS
BEGIN
    DELETE FROM EMP
    WHERE empno=p_empno;
END delete_record;

 FUNCTION  sum_dept_sal(p_deptno IN NUMBER) RETURN emp.sal%TYPE IS
 v_sum_sal    emp.sal%TYPE;
 BEGIN
    SELECT SUM(sal)
    INTO v_sum_sal
     FROM emp
     WHERE deptno = p_deptno;
    RETURN v_sum_sal;
END;

END emp_info; --end of package body
/
```

PL/SQL Package-2

Purpose of PL/SQL Package

Following are its main purposes:

- Make code more modular
- Hide implementation details
- Improve application performance
- Minimize unnecessary recompiling code
- Manage authorization easily

PL/SQL Package-2

Add in a Package Body by hiding from User Specification

```
Procedure call_from_out is
   v_empno EMP.empno%TYPE;
   v_ename EMP.ename%TYPE;
   v_job   EMP.job%TYPE;
   v_sal   EMP.sal%TYPE;
   v_comm  EMP.comm%TYPE;
   v_deptno EMP.dept%TYPE;
begin

   v_empno := 8899;
   v_ename := 'ALI';
   v_job   := 'MANAGER';
   v_sal   := 8000;
   v_comm  := 300;
   v_deptno := 10;

   insert_record(  v_empno, v_ename, v_job, v_sal, v_comm,v_deptno);
    commit;
EXCEPTION
   WHEN OTHERS THEN
     DBMS_OUTPUT.PUT_LINE('Except call_from_out:'||SQLERRM);
end;
/
```

PL/SQL Package-2

Invoking Package Subprogram

```
DECLARE
    v_sum_sal    emp.sal%TYPE;
BEGIN
    ...
    v_sum_sal := emp_info.sum_dept_sal(10);
    ...
    emp_info.delete_record(1234567);
    ...
END;
/
```

Or

SQL> exec emp_info.insert_record(...,...)

Introduction to Database Administration



Execution of Packages

Execution of Packages

Execution of Packages

- Compiling a Package using SQL*Plus
- Compiling a Package using Oracle Developer
- Compiling a Package using Files on SQL*Plus
- Calling Function and Procedure from Package

Execution of Packages

Compiling a Package using SQL*Plus

If you use SQL*Plus for compiling and creating a package body, you type forward slash (/) as follows:

```
44   return in_net_value;
45   EXCEPTION
46   WHEN no_data_found THEN
47   DBMS_OUTPUT.PUT_LINE( SQLERRM );
48     END get_net_value_by_customer;
49
50   END order_mgmt;
51   /

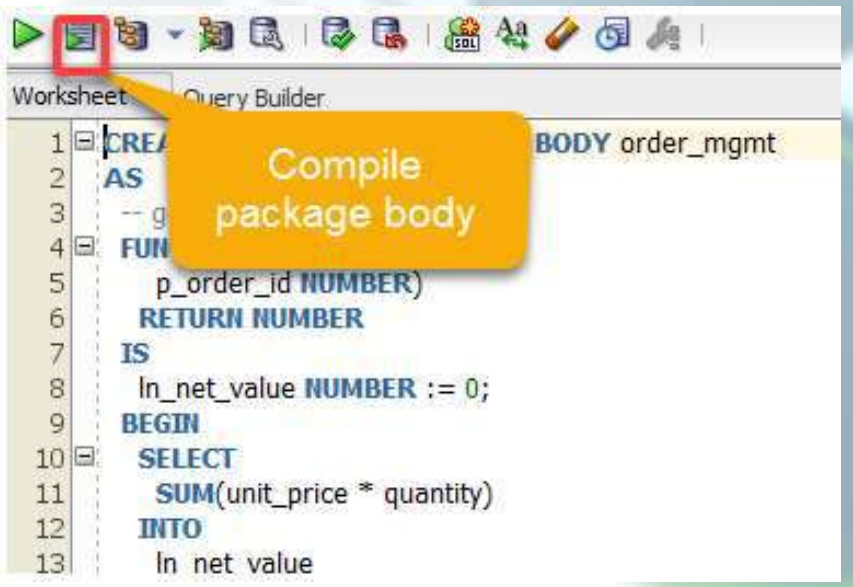
Package body created.

SQL>
```

Execution of Packages

Compiling a Package using Oracle Developer

From PL/SQL Developer, you click the **Run Script** button to compile the package body.



Execution of Packages

Compiling a Package using Files on SQL*Plus

If you manage package specification & body using separate files, you can compile separately using the following commands:

```
SQL> @path_to_packge_body_file
```

```
SQL > @path_to_package_body_file
2 /
```

Execution of Packages

Calling Function and Procedure from Package

```
DECLARE
   v_sum_sal    emp.sal%TYPE;
BEGIN
   ...
   v_sum_sal := emp_info.sum_dept_sal(10);
   ...
   emp_info.delete_record(1234567);
   ...
END;
/
```

Or

SQL> exec emp_info.insert_record(...,...)

End

Introduction to Database Administration



PL/SQL Triggers

PL/SQL Triggers

PL/SQL Triggers

- What is an Oracle Triggers?
- Oracle Trigger Usage
- Oracle Trigger Syntax
- Oracle Trigger Header
- Oracle Trigger Body
- Create or Replace a Trigger

PL/SQL Triggers

What is an Oracle Triggers?

- A trigger is a named PL/SQL block stored in the Oracle Database and executed automatically when a triggering event takes place. The event can be any of the following:

- A data manipulation language (DML) statement executed against a table e.g., INSERT, UPDATE, or DELETE.

PL/SQL Triggers

What is an Oracle Triggers?

- For example, if you define a trigger that fires before an INSERT statement on the customers table, the trigger will fire once before a new row is inserted into the customers table.
- A data definition language (DDL) statement executes e.g., CREATE or ALTER statement. These triggers are often used for auditing purposes to record changes of the schema.

PL/SQL Triggers

What is an Oracle Trigger?

- A system event such as startup or shutdown of the Oracle Database.
- A user event such as login or logout.
- The act of executing a trigger is also known as firing a trigger. We say that the trigger is fired.

PL/SQL Triggers

Oracle Trigger Usage

Oracle triggers are useful in many cases such as the following:

- Enforcing complex business rules that cannot be established using integrity constraint such as UNIQUE, NOT NULL, and CHECK.
- Preventing invalid transactions.

PL/SQL Triggers

Oracle Trigger Usage

- Gathering statistical information on table accesses.
- Generating value automatically for derived columns.
- Auditing sensitive data.

PL/SQL Triggers

Oracle Trigger Syntax

- Following is its syntax

```
CREATE [OR REPLACE] TRIGGER trigger_name
{BEFORE | AFTER } triggering_event ON table_name
[FOR EACH ROW]
[FOLLOWS | PRECEDES another_trigger]
[ENABLE / DISABLE ]
[WHEN condition]
DECLARE
    declaration statements
BEGIN
    executable statements
EXCEPTION
    exception_handling statements
END;
/
```

PL/SQL Triggers

Oracle Trigger Header

```
CREATE [OR REPLACE] TRIGGER trigger_name
   {BEFORE | AFTER } triggering_event ON table_name
   [FOR EACH ROW]
   [FOLLOWS | PRECEDES another_trigger]
   [ENABLE / DISABLE ]
   [WHEN condition]
```

PL/SQL Triggers

Oracle Trigger Body

```
DECLARE
   declaration statements
BEGIN
   executable statements
EXCEPTION
   exception_handling statements
END;
```

PL/SQL Triggers

Create or Replace a Trigger

- The CREATE keyword specifies that you are creating a new trigger. The OR REPLACE keywords are optional. They are used to modify an existing trigger.
- Even though the OR REPLACE keywords are optional, they appear with the CREATE keyword in most cases.

End



Trigger Examples

Trigger Examples

Trigger Examples

- Trigger Example-1
- Execution of Trigger Example-1
- Trigger Example-2
- Trigger & Function Example-3

Trigger Examples

Trigger Example-1

Display difference of salary when salary of employee is changed

```
CREATE OR REPLACE TRIGGER display_salary_changes
BEFORE DELETE OR INSERT OR UPDATE ON EMP
FOR EACH ROW
WHEN (NEW.EMPNO > 0)
DECLARE
   sal_diff number;
BEGIN
   sal_diff := :NEW.sal  - :OLD.sal;
   dbms_output.put_line('Old salary: ' || :OLD.sal);
   dbms_output.put_line('New salary: ' || :NEW.sal);
   dbms_output.put_line('Salary difference: ' || sal_diff);
END;
/
```

Trigger Examples

Execution of Trigger Example-1

Display difference of salary when salary of employee is changed

```
SQL> CREATE OR REPLACE TRIGGER display_salary_changes
  2  BEFORE DELETE OR INSERT OR UPDATE ON EMP
  3  FOR EACH ROW
  4  WHEN (NEW.EMPNO > 0)
  5  DECLARE
  6     sal_diff number;
  7  BEGIN
  8     sal_diff := :NEW.sal  - :OLD.sal;
  9     dbms_output.put_line('Old salary: ' || :OLD.sal);
 10     dbms_output.put_line('New salary: ' || :NEW.sal);
 11     dbms_output.put_line('Salary difference: ' || sal_diff);
 12  END;
 13  /

Trigger created.

SQL> set serveroutput on
SQL> update emp
  2  set sal=777
  3  where ename='SMITH';
Old salary: 800
New salary: 777
Salary difference: -23

1 row updated.
```

Trigger Examples

Trigger Example-2

Keeping log of old data into a log table

```
CREATE OR REPLACE TRIGGER del_emp
BEFORE DELETE ON emp
FOR EACH ROW
DECLARE
   x number(2);
BEGIN
     INSERT INTO emp_audit
            VALUES(:NEW.empno, USER, sysdate);
END;
/

--Other forms can be BEFORE INSERT OR UPDATE
--OF salary ON EMP
--Need to create table emp_audit
--select * from emp_audit;
```

Trigger Examples

Trigger & Function Example-3

```
create table emp1
as select * from emp;
```

```
alter table emp1
add bonus number(6,2);
CREATE OR REPLACE  FUNCTION "BONUS_CALC"
                    (p_sal IN NUMBER) RETURN NUMBER IS
BEGIN
    return(p_sal/600);
   dbms_output.put_line(' ***** THE RECORD HAS SUCCESSFULLY BEEN INSERTED *****');
END;
/


CREATE OR REPLACE TRIGGER "UPDATE_BONUS" before INSERT OR UPDATE  OF SAL ON "EMP1" FOR EACH ROW
BEGIN
    --AFTER option gives error of mutating
    --select Bonus_Calc(:New.sal) into :New.bonus from dual;
    --Or use the following asignment statement
    :New.bonus := Bonus_Calc(:New.sal);

   dbms_output.put_line(' ***** THE RECORD HAS SUCCESSFULLY BEEN INSERTED *****');
end;
/

Update emp1
set sal=1600
where ename in ('ALLEN');
```

Introduction to Database Administration



Trigger Exercises

Trigger Exercises

Trigger Exercises

- Exercise-1 (Off day DML Restriction)
- Exercise-2 (for Data Entry)
- Trigger’s Disable/ Enable
- Trigger’s Disable/ Enable using SQL Developer
- Exercise-3 (Mutating Error)
- Fired Triggers
- Updating & Deleting on Table with specific Columns

Trigger Exercises

Exercise-1 (Off day DML Restriction)

This is a statement trigger on the EMP table, and it fires before an INSERT, UPDATE, or DELETE statement is issued. First, the trigger determines the day of the week. If the day is Saturday or Sunday, an error message is generated.

Trigger Exercises

Exercise-1 (Off day DML Restriction)

```
SQL> CREATE OR REPLACE TRIGGER EMP_ModifyIssue
  2  BEFORE INSERT OR UPDATE OR DELETE ON EMP
  3  DECLARE
  4     v_day VARCHAR2(10);
  5  BEGIN
  6     v_day := RTRIM(TO_CHAR(SYSDATE, 'DAY'));
  7     IF v_day LIKE ('S%') THEN
  8   RAISE_APPLICATION_ERROR (-20000, 'A table cannot be modified during off hours');
  9  END IF;
 10  END;
 11  /

Trigger created.

SQL>
SQL> update emp
  2  set sal=700
  3  where ename='SMITH';
update emp
       *
ERROR at line 1:
ORA-20000: A table cannot be modified during off hours
ORA-06512: at "SCOTT.EMP_MODIFYISSUE", line 6
ORA-04088: error during execution of trigger 'SCOTT.EMP_MODIFYISSUE'
```

Trigger Exercises

Exercise-2 (for Data Entry)

```
CREATE OR REPLACE TRIGGER Derived
BEFORE INSERT OR UPDATE OF Ename ON Emp

/* Before updating the ENAME field, derive the values for
   the UPPERNAME and SOUNDEXNAME fields. Restrict users
   from updating these fields directly: */
FOR EACH ROW
BEGIN
  :NEW.Uppername := UPPER(:NEW.Ename);
  :NEW.Soundexname := SOUNDEX(:NEW.Ename);
END;
/
```

Trigger Exercises

Trigger's Disable/ Enable

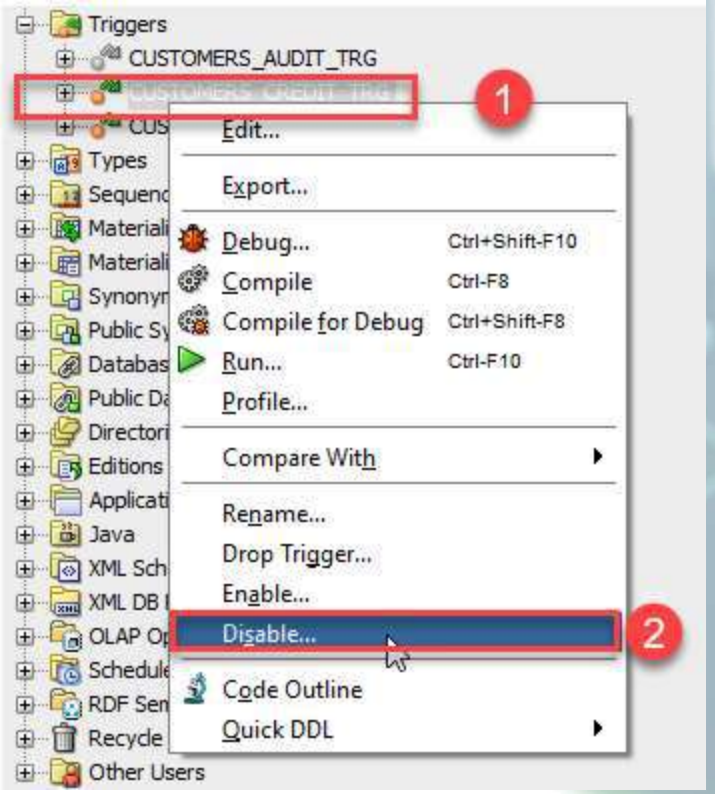
Sometimes, you may want to disable a trigger for testing and troubleshooting purposes. To disable a trigger, you use the ALTER TRIGGER DISABLE statement:

```
ALTER TRIGGER trigger_name DISABLE;
```

```
ALTER TABLE table_name DISABLE ALL TRIGGERS;
```

Trigger Exercises

Trigger's Disable/ Enable using SQL Developer



Trigger Exercises

Exercise-3 (Mutating Error)

```
SQL> CREATE OR REPLACE TRIGGER Update_EmpDeptno
  2  BEFORE INSERT OR UPDATE ON EMP
  3  FOR EACH ROW
  4  DECLARE
  5  v_cnt Number(3);
  6  BEGIN
  7  IF :NEW.deptno IS NOT NULL THEN
  8  BEGIN
  9  SELECT count(*)
 10  INTO v_cnt
 11  FROM EMP
 12  WHERE deptno = :NEW.deptno;
 13  EXCEPTION
 14  WHEN NO_DATA_FOUND THEN
 15  RAISE_APPLICATION_ERROR
 16  (-20001, 'This is not a valid deptno');
 17  END;
 18  END IF;
 19  END;
 20  /

Trigger created.

SQL> Update emp
  2  set deptno = 11
  3  where deptno=10;
Update emp
       *
ERROR at line 1:
ORA-04091: table SCOTT.EMP is mutating, trigger/function may not see it
ORA-06512: at "SCOTT.UPDATE_EMPDEPTNO", line 6
ORA-04088: error during execution of trigger 'SCOTT.UPDATE_EMPDEPTNO'
```

Trigger Exercises

Exercise-3 (Resolving Mutating Error)

```
CREATE OR REPLACE TRIGGER Update_EmpDeptno
BEFORE INSERT OR UPDATE ON EMP
FOR EACH ROW
DECLARE
v_cnt Number(3);
BEGIN
   IF :NEW.deptno IS NOT NULL THEN
   BEGIN
        SELECT count(*)
        INTO v_cnt
        FROM dept
        WHERE deptno = :NEW.deptno;
        DBMS_OUTPUT.PUT_LINE(:NEW.deptno);
   EXCEPTION
   WHEN NO_DATA_FOUND THEN
   RAISE_APPLICATION_ERROR (-20001, 'This is not a valid deptno');
   END;
   END IF;
END;
/
```

Trigger Exercises

Fired Triggers

Need to see which trigger names are fired on commands such as

```
SQL> update emp
  2  set deptno=11
  3  where deptno=10;
11
Old salary:
New salary:
Salary difference:
11
Old salary: 4000
New salary: 4000
Salary difference: 0
11
Old salary: 2450
New salary: 2450
Salary difference: 0
```

```
select distinct name
from user_source
where type='TRIGGER';

drop trigger DEL_EMP;
drop trigger UPDATE_EMPDEPTNO;
```

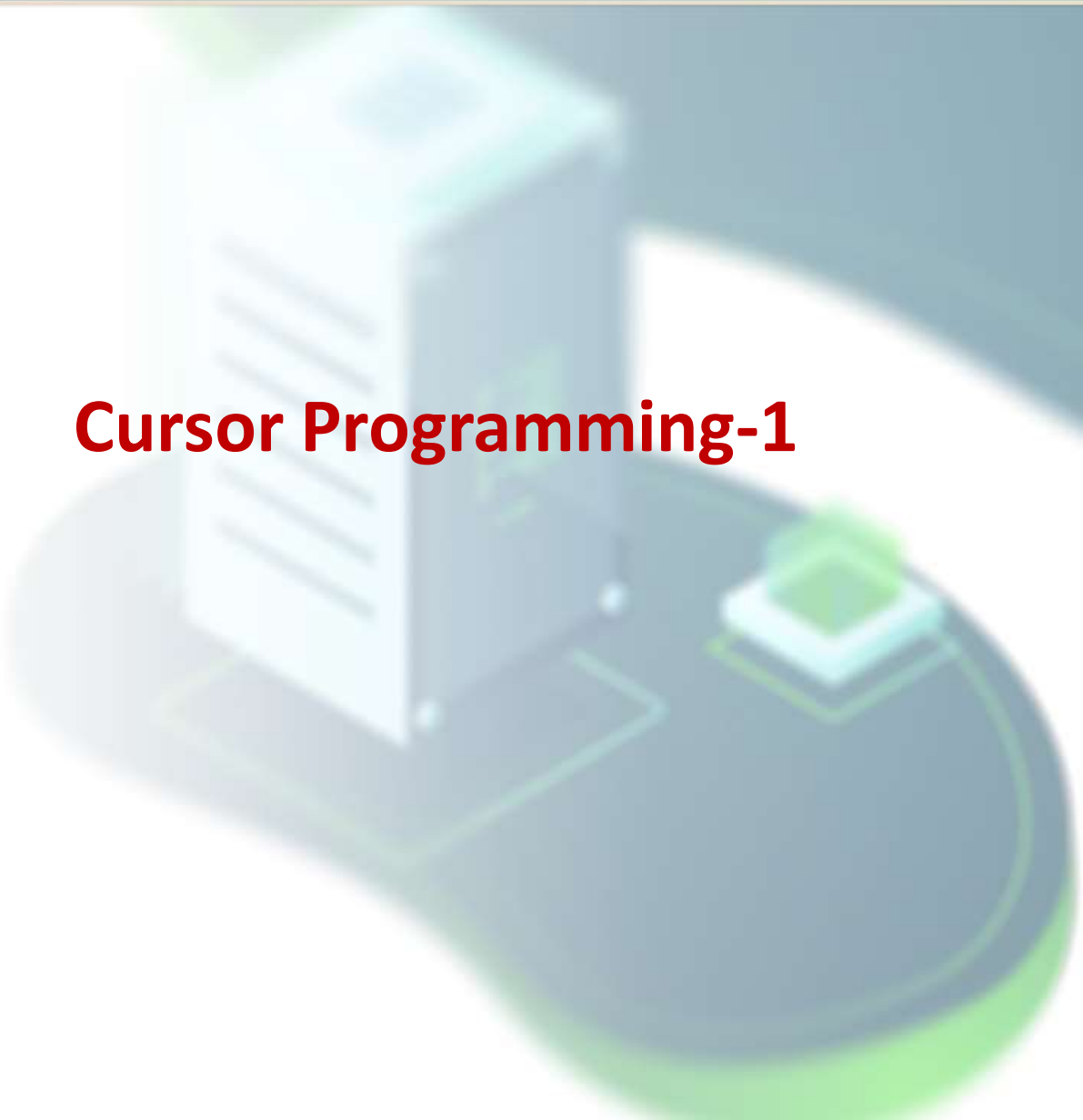
Trigger Exercises

Updating & Deleting on Table with specific Columns

```
CREATE OR REPLACE TRIGGER Updating_EMP
  BEFORE
    INSERT OR
    UPDATE OF sal, deptno OR
    DELETE
  ON EMP
BEGIN
  CASE
    WHEN INSERTING THEN
      DBMS_OUTPUT.PUT_LINE('Inserting');
    WHEN UPDATING('sal') THEN
      DBMS_OUTPUT.PUT_LINE('Updating sal');
    WHEN UPDATING('deptno') THEN
      DBMS_OUTPUT.PUT_LINE('Updating department ID');
    WHEN DELETING THEN
      DBMS_OUTPUT.PUT_LINE('Deleting');
  END CASE;
END;
/

update emp
set sal=2000
where ename='SMITH';
```

Introduction to Database Administration



Cursor Programming-1

Cursor Programming-1

- What is a Cursor?
- Implicit Cursor
- Execution Cycle of Explicit Cursor
- Cursor’s usage
- Explicit Cursor Attributes

Cursor Programming-1

What is a Cursor?

- A cursor is a pointer that points to a result of a query.
- It is mainly used with tables which return more than one row.
- It is handled implicitly and explicitly.

Cursor Programming-1

Implicit Cursor

- Whenever Oracle executes an SQL statement such as SELECT INTO, INSERT, UPDATE, and DELETE, it automatically creates an implicit cursor.
- Oracle internally manages the whole execution cycle of implicit cursors and reveals only the cursor's information and statuses such as SQL%ROWCOUNT, SQL%ISOPEN, SQL%FOUND, and SQL%NOTFOUND.

Cursor Programming-1

Implicit Cursor

- The implicit cursor is not elegant when the query returns zero or multiple rows which cause NO_DATA_FOUND or TOO_MANY_ROWS exception respectively.

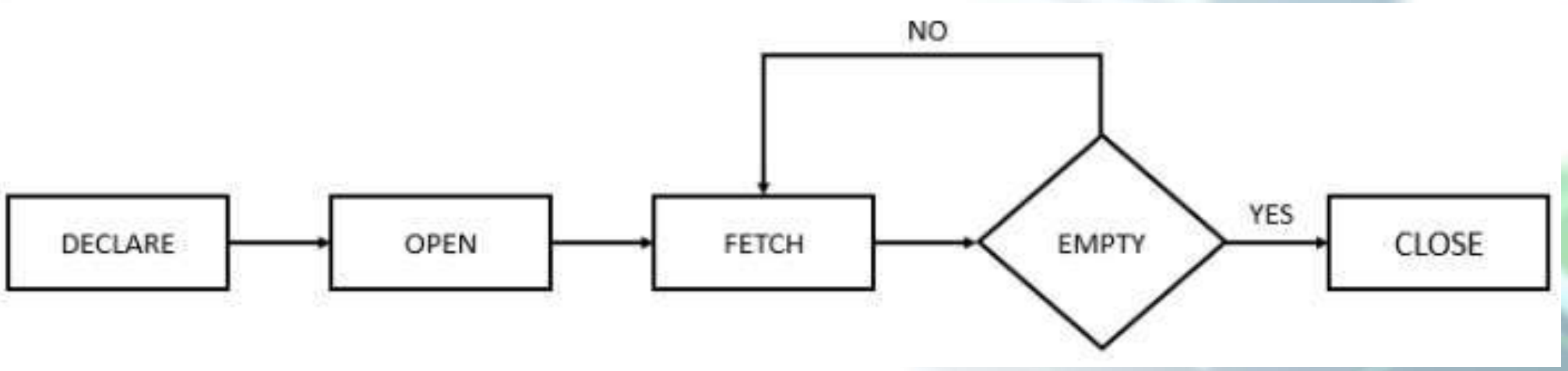
Cursor Programming-1

Explicit Cursor

- An explicit cursor is an SELECT statement declared explicitly in the declaration section of the current block or a package specification.
- For an explicit cursor, you have control over its execution cycle from OPEN, FETCH, and CLOSE.
- Oracle defines an execution cycle that executes an SQL statement and associates a cursor with it.

Cursor Programming-1

Execution Cycle of Explicit Cursor



Cursor Programming-1

Cursor's usage

- **Declare a cursor**

  CURSOR cursor_name IS query;

- **Open a cursor**

  OPEN cursor_name;

Cursor Programming-1

Cursor's usage

- **Fetch from a cursor**
  The FETCH statement places the contents of the current row into var.

  FETCH cursor_name INTO variable_list;

  To retrieve all rows in a result set, you need to fetch each row till the last one.

- **Closing a cursor**

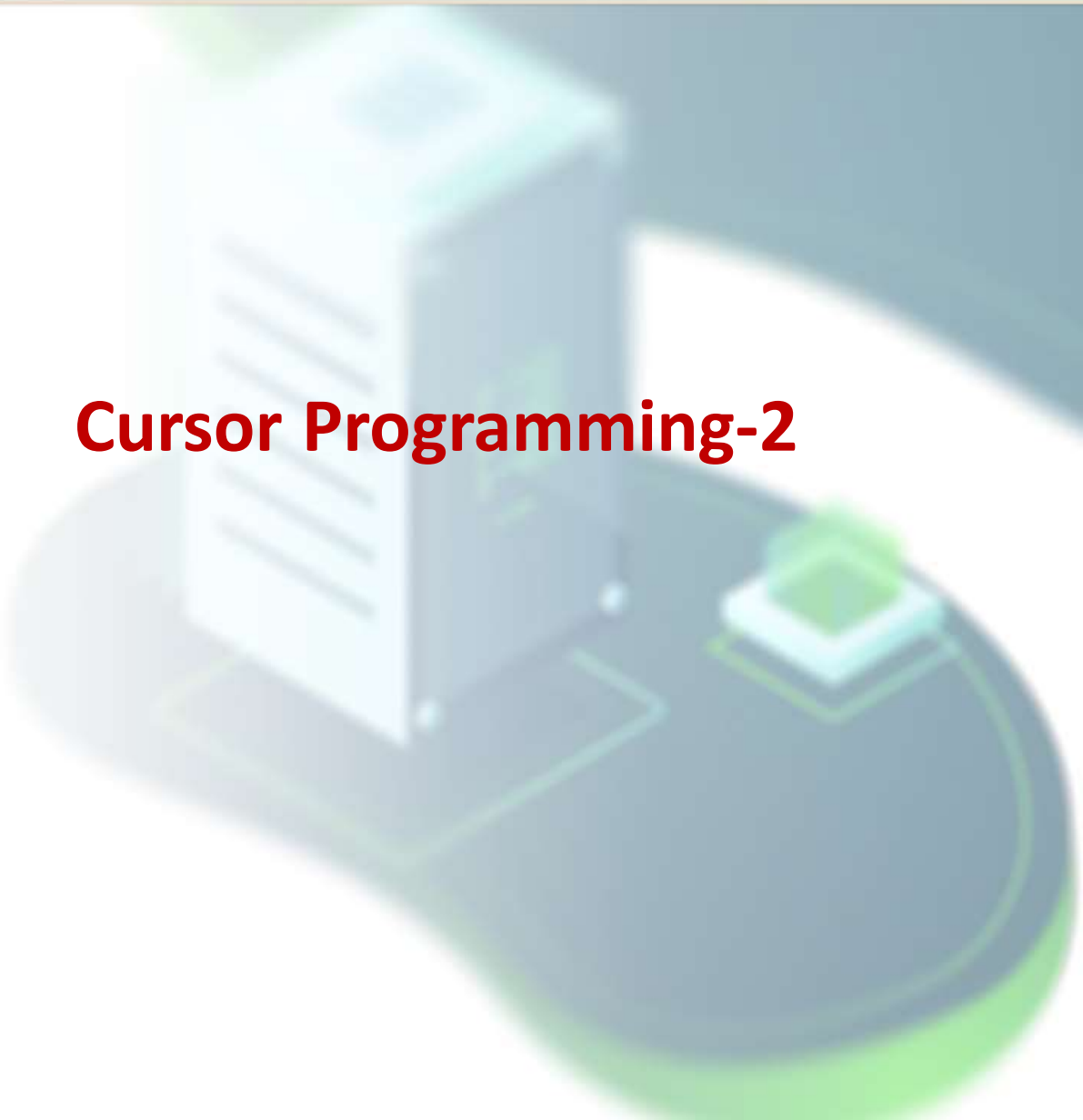
  CLOSE cursor_name;

Cursor Programming-1

Explicit Cursor Attributes

- %ISOPEN
- %FOUND
- %NOTFOUND
- %ROWCOUNT

Introduction to Database Administration



Cursor Programming-2

Cursor Programming-2

- Explicit Cursor Attributes
- Implicit Cursor - Record Type
- ROWTYPE Example - 1
- ROWCOUNT Example
- How to Declare Cursor?
- %ISOPEN, %FOUND, FETCH Example

Cursor Programming-2

Explicit Cursor Attributes

- %ISOPEN
- %FOUND
- %NOTFOUND
- %ROWCOUNT

Cursor Programming-2

Implicit Cursor – Record Type

- A record is a composite data structure, which means that it is composed of one or more elements.
- Records are very much like a row of a database table, but each element of the record does not stand on its own. PL/SQL supports three kinds of records: table-based, cursor-based, and programmer-defined.

```
v_EmpRec   EMP%TYPE;
```

Cursor Programming-2

Implicit Cursor – Record Type

```
CURSOR c_emp  IS
SELECT  empno, ename, job
FROM    emp
WHERE  deptno = 20;

vr_emp    c_emp%ROWTYPE;
```

Cursor Programming-2

ROWTYPE Example - 1

```
DECLARE
   v_emp  EMP%ROWTYPE;
BEGIN
   SELECT *
   INTO v_emp
   FROM EMP
   WHERE ename='SMITH';
   DBMS_OUTPUT.PUT_LINE('Records columns are '
   ||v_emp.empno||','||v_emp.ename);
EXCEPTION
WHEN NO_DATA_FOUND THEN
     DBMS_OUTPUT.PUT_LINE('The Employee '
     ||'is not in the database');
END;
/
```

Cursor Programming-2

ROWTYPE Example - 2

```
DECLARE
CURSOR c_emp IS
    SELECT *
    FROM EMP;
    vr_emp c_emp%ROWTYPE;
BEGIN
OPEN c_emp;
LOOP
FETCH c_emp INTO vr_emp;
EXIT WHEN c_emp%NOTFOUND;
DBMS_OUTPUT.PUT_LINE(vr_emp.empno||
' '||vr_emp.ename||' '||vr_emp.sal);
END LOOP;
END;
/
```

Cursor Programming-2

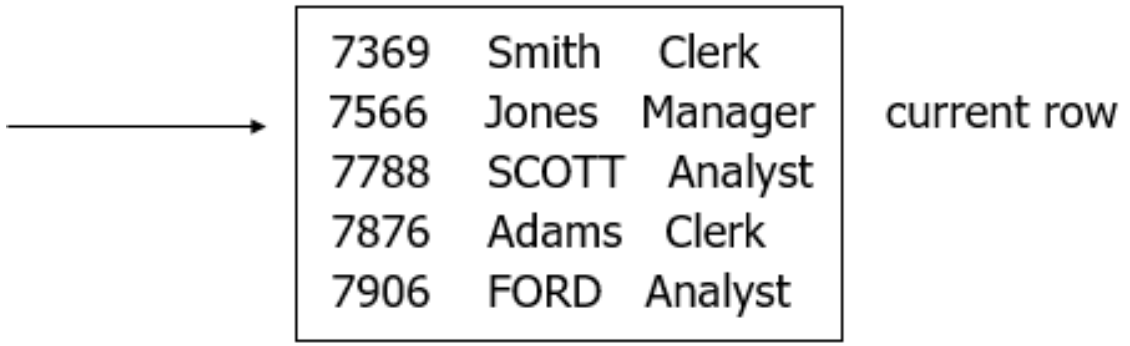
ROWCOUNT Example

```
DECLARE
    v_job EMP.job%type;
BEGIN
    SELECT job
    INTO v_job
    FROM EMP
    WHERE empno = 7698;
  IF SQL%ROWCOUNT = 1 THEN
    DBMS_OUTPUT.PUT_LINE(v_job ||' has a '||
    'Empno of 7698');
  ELSIF SQL%ROWCOUNT = 0 THEN
    DBMS_OUTPUT.PUT_LINE('The Empno 7698 is '||
    ' not in the database');
  ELSE
    DBMS_OUTPUT.PUT_LINE('Stop... ');
  END IF;
END;
/
```

Cursor Programming-2

How to Declare Cursor?

```
CURSOR c_emp  IS
SELECT  empno, ename, job
FROM    emp
WHERE  deptno = 20;
```

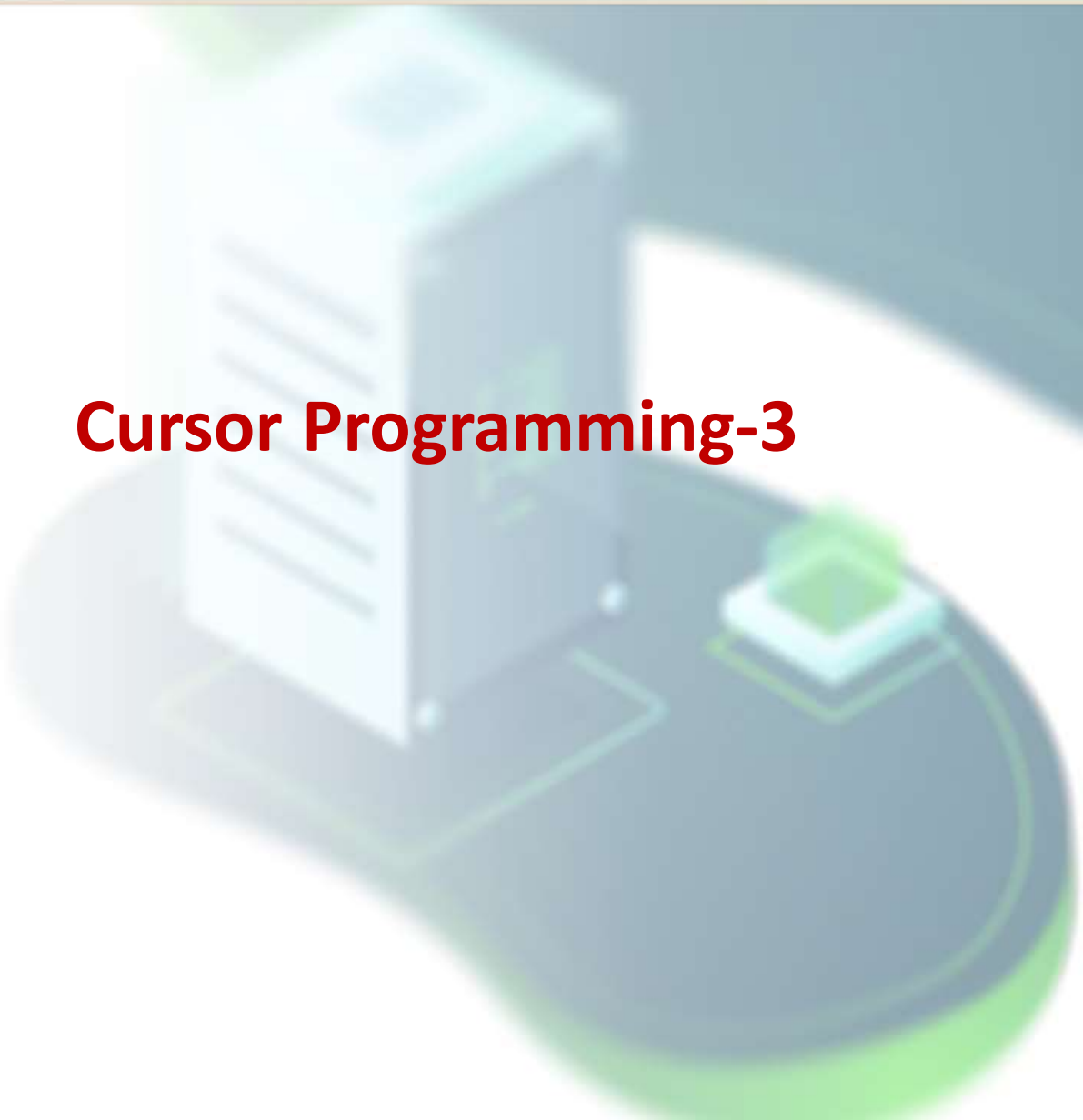


Cursor Programming-2

%ISOPEN, %FOUND, FETCH Example

```
DECLARE
        v_empno EMP.Empno%TYPE;
        CURSOR c_EMP IS
        SELECT Empno
          FROM EMP
          WHERE Empno > 1000;
BEGIN
        OPEN c_EMP;
        LOOP
        FETCH c_EMP INTO v_empno;
        IF c_EMP%FOUND THEN
           DBMS_OUTPUT.PUT_LINE('Just FETCHED row '
           ||TO_CHAR(c_EMP%ROWCOUNT)||' EMP ID: '||v_empno);
        ELSE
           EXIT;
        END IF;
        END LOOP;
        CLOSE c_EMP;
        EXCEPTION
        WHEN OTHERS THEN
           IF c_EMP%ISOPEN
        THEN
           CLOSE c_EMP;
        END IF;
END;
/
```

Introduction to Database Administration



Cursor Programming-3

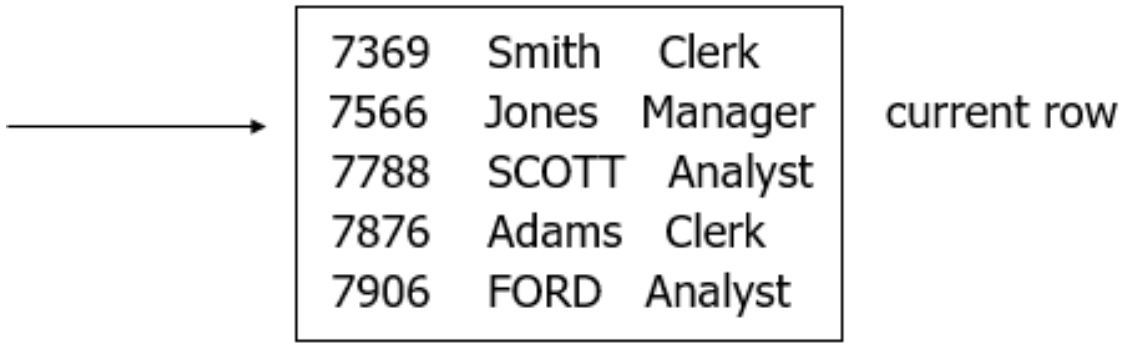
Cursor Programming-3

- How to Declare Cursor?
- Sample Program using Cursor
- Cursor for DML Insert
- Cursor with Parameters
- Cursor Program-1
- Cursor Program-1 (Solution)

Cursor Programming-3

How to Declare Cursor?

```
CURSOR c_emp  IS
SELECT  empno, ename, job
FROM    emp
WHERE  deptno = 20;
```



Cursor Programming-3

Sample Program using Cursor

```
DECLARE

     CURSOR c_emp IS
     SELECT  empno, ename, job
     FROM     emp
     WHERE  deptno = 20;

BEGIN

      FOR v_c IN c_emp  LOOP
       ...
        DBMS_OUTPUT.PUT_LINE(v_c.ename);
       ...
      END LOOP;
    ...
end;
/
```

Cursor Programming-3

Cursor for DML Insert

```
DECLARE
   CURSOR c_emp IS
   SELECT empno, ename, job
   FROM EMP
   WHERE empno > 1100;
BEGIN
   FOR r_emp IN c_emp
   LOOP
      INSERT INTO table_log
      VALUES(r_emp.ename);
   END LOOP;
END;
/
```

Cursor Programming-3

Cursor with Parameters

```
DECLARE
  CURSOR c_emp (p_job IN EMP.job%TYPE) IS
  SELECT *
  FROM EMP
  WHERE job = p_job;
BEGIN
  FOR r_emp IN c_emp('CLERK')
  LOOP
        DBMS_OUTPUT.PUT_LINE(r_emp.ename||'--'||r_emp.job);
  END LOOP;
END;
/
```

Cursor Programming-3

Cursor Program-1

Given a table with first three columns are composite keys. Write a PL/SQL program using cursor to update cat_template1 with LONG column cat_template. Assuming this tables contains more than 40,000 records.

```
SQL> desc patient_visit
 Name                                      Null?    Type
 ----------------------------------------- -------- --------------
 PATIENT_ID                                NOT NULL CHAR(7)
 DEPEND_SNO                                NOT NULL VARCHAR2(2)
 PATIENT_VISITNO                           NOT NULL NUMBER(4)
 EMP_ID                                             CHAR(7)
 DOCTOR_SNO                                         NUMBER(3)
 VISIT_DATE                                         DATE
 VISIT_NOTE                                         VARCHAR2(50)
 CAT_TEMPLATE                                       LONG
 MEDICATION                                         VARCHAR2(500)
 CHECKUP_STATUS                                     CHAR(1)
 SUB_CAT_ID                                         NUMBER(6)
 CAT_TEMPLATE1                                       VARCHAR2(3000)
```

Cursor Programming-3

Cursor Program-1 (Solution)

```
declare
  v_count number(7);
cursor c1 is
   select  PATIENT_ID,DEPEND_SNO,PATIENT_VISITNO,CAT_TEMPLATE
    from patient_visit;
begin
  v_count := 0;
  for v_c in c1 LOOP
    update  patient_visit
    set  CAT_TEMPLATE1   = v_c.CAT_TEMPLATE
    where PATIENT_ID = v_c.PATIENT_ID
        and DEPEND_SNO = v_c.DEPEND_SNO
        and PATIENT_VISITNO = v_c.PATIENT_VISITNO;
    If v_count = 300 Then
      v_count := 0;
      --DBMS_OUTPUT.PUT_LINE('commit test');
    else
      v_count := v_count + 1;
    end if;
  end LOOP;
  commit;
end;
/
```



Programming using an Array

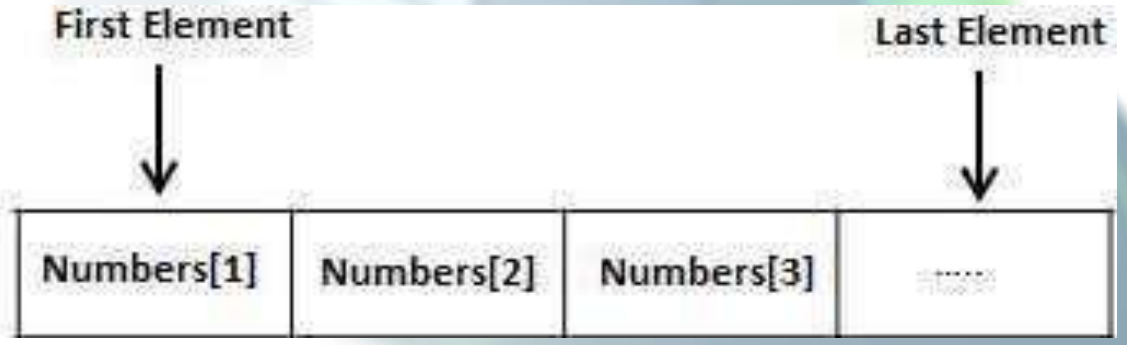
Programming using an Array

- What is an Array?
- Declaration of an Array
- Basic Example of an Array
- Associative Array
- Associative Array - An Example
- Use of Array with an Exception
- Array Null and Empty Collection

Programming using an Array

What is an Array?

All VARRAYS consist of contiguous memory locations. The lowest address corresponds to the first element and the highest address to the last element.



https://www.tutorialspoint.com/plsql/plsql_collections.htm

Programming using an Array

Declaration of an Array

A varray type is created with the **CREATE TYPE** statement. You must specify the maximum size and the type of elements stored in the varray.

```
CREATE Or REPLACE TYPE namearray AS
VARRAY(3) OF VARCHAR2(10);
/
```

Programming using an Array

Basic Example of an Array

```
DECLARE
   CURSOR c_emp is
   SELECT  ename FROM EMP;
   TYPE c_list is varray (20) of EMP.ename%type;
   ename_list c_list := c_list();
   counter integer :=0;
BEGIN
   FOR n IN c_emp LOOP
      counter := counter + 1;
      ename_list.extend;
      ename_list(counter)  := n.ename;
      dbms_output.put_line('Employee('||counter ||'):'
      ||ename_list(counter));
   END LOOP;
END;
/
```

Programming using an Array

Associative Array

An index-by table is a set of key-value pairs. Each key is unique and is used to locate the corresponding value. The key can be either an integer or a string.

TYPE type_name IS TABLE OF element_type [NOT NULL] INDEX BY subscript_type;

table_name type_name;

Programming using an Array

Associative Array – An Example

```
DECLARE
   TYPE salary IS TABLE OF NUMBER INDEX BY VARCHAR2(20);
   salary_list salary;
   name   VARCHAR2(20);
BEGIN
   -- adding elements to the table
   salary_list('Sarah') := 62000;
   salary_list('Jamil') := 75000;
   salary_list('Wisal') := 100000;
   salary_list('Umar') := 78000;

   -- printing the table
   name := salary_list.FIRST;
   WHILE name IS NOT null LOOP
      dbms_output.put_line
      ('Salary of ' || name || ' is ' || TO_CHAR(salary_list(name)));
      name := salary_list.NEXT(name);
   END LOOP;
END;
/
```

Programming using an Array

Use of Array with an Exception

```
SQL> DECLARE
  2    CURSOR name_cur IS
  3    SELECT ename
  4    FROM EMP
  5    WHERE rownum <= 10;
  6    TYPE ename_type IS TABLE OF EMP.ename%TYPE
  7    INDEX BY BINARY_INTEGER;
  8    ename_tab ename_type;
  9    v_counter INTEGER := 0;
 10  BEGIN
 11    FOR name_rec IN name_cur LOOP
 12  v_counter := v_counter + 1;
 13  ename_tab(v_counter) := name_rec.ename;
 14  DBMS_OUTPUT.PUT_LINE ('ename('||v_counter||'): '||
 15  ename_tab(v_counter));
 16    END LOOP;
 17    DBMS_OUTPUT.PUT_LINE ('ename(11): '|| ename_tab(11));
 18  EXCEPTION
 19    WHEN NO_DATA_FOUND THEN
 20    DBMS_OUTPUT.PUT_LINE(SQLERRM);
 21  END;
 22  /
ename(1):
ename(2): ABC
ename(3): XYZ
ename(4): SMITH
ename(5): ALLEN
ename(6): WARD
ename(7): JONES
ename(8): MARTIN
ename(9): BLAKE
ename(10): CLARK
ORA-01403: no data found
```

Programming using an Array

Array Null and Empty Collection

```
DECLARE
   TYPE integer_type IS TABLE OF INTEGER;
   integer_tab integer_type;

   v_counter integer := 1;
BEGIN
   DBMS_OUTPUT.PUT_LINE (integer_tab(v_counter));
END;
```

```
DECLARE
*
ERROR at line 1:
ORA-06531: Reference to uninitialized collection
ORA-06512: at line 7
```

```
DECLARE
   TYPE integer_type IS TABLE OF INTEGER;
   integer_tab integer_type := integer_type();

   v_counter integer := 1;
BEGIN
   DBMS_OUTPUT.PUT_LINE (integer_tab(v_counter));
END;
```

```
DECLARE
*
ERROR at line 1:
ORA-06533: Subscript beyond count
ORA-06512: at line 7
```

Introduction to Database Administration



Examples with Advance Topics

Examples with Advance Topics

- Use of Records Arrays
- Use of PRAGMA Exception
- PL/SQL Dynamic SQL
- PL/SQL Dynamic SQL - Example-1
- PL/SQL Dynamic SQL - Example-2

Examples with Advance Topics

Use of Records Arrays

```
Declare
Cursor c1 is
  select sal
  from emp;
TYPE sal_type IS VARRAY(20) OF Integer;
v_col sal_TYPE := sal_type();
k number := 0;
Begin
  for i in C1 LOOP
    k := k+1;
    v_col.extend; --collection method
    v_col(k) := i.sal;
  end loop;
  for j in 1..4 LOOP
    DBMS_OUTPUT.PUT_LINE('sal is col('||j||') '
    ||v_col(j));
  end loop;
End;
```

Examples with Advance Topics

Use of PRAGMA Exception

```
CREATE OR REPLACE TRIGGER Employee_permit_changes
  BEFORE INSERT OR DELETE OR UPDATE ON EMP
DECLARE
  Dummy              INTEGER;
  Not_on_weekends    EXCEPTION;
  Nonworking_hours   EXCEPTION;
  PRAGMA EXCEPTION_INIT (Not_on_weekends, -4097);
  PRAGMA EXCEPTION_INIT (Nonworking_hours, -4099);
BEGIN
   -- Check for weekends:
   IF (TO_CHAR(Sysdate, 'DAY') = 'SAT' OR
     TO_CHAR(Sysdate, 'DAY') = 'SUN') THEN
       RAISE Not_on_weekends;
   END IF;
  -- Check for work hours (8am to 6pm):
  IF (TO_CHAR(Sysdate, 'HH24') < 8 OR
    TO_CHAR(Sysdate, 'HH24') > 18) THEN
      RAISE Nonworking_hours;
  END IF;
EXCEPTION
  WHEN Not_on_weekends THEN
    Raise_application_error(-20324,'Might not change '
      ||'employee table during the weekend');
  WHEN Nonworking_hours THEN
    Raise_application_error(-20326,'Might not change '
     ||'emp table during Nonworking hours');
END;
/
```

Examples with Advance Topics

PL/SQL Dynamic SQL

- Dynamic SQL is a programming methodology for generating and running SQL statements at run time.
- It is useful when writing general-purpose and flexible programs like ad hoc query systems, when writing programs that must run database definition language (DDL) statements, or when you do not know at compilation time the full text of a SQL statement or the number or data types of its input and output variables.

Examples with Advance Topics

PL/SQL Dynamic SQL – Example-1

Creating a table during run-time.

```
BEGIN
   execute immediate 'create table MyTable('
                                          ||'ID CHAR(3),'
                                          ||'Name VARCHAR2(50))';
END;
/
```

Note: grant create table to <user> is required

Examples with Advance Topics

PL/SQL Dynamic SQL - Example-2

Set to Null a column at run time

```
CREATE TABLE EMP_temp AS SELECT * FROM EMP;
/

DECLARE
  -- Set to NULL automatically at run time
  a_null  CHAR(1);
BEGIN
  EXECUTE IMMEDIATE 'UPDATE EMP_TEMP SET sal = :x'
    USING a_null;
END;
/
```

End

Introduction to Database Administration



Concepts of Oracle Instance-1

Concepts of Oracle Instance-1

- Introduction
- Configuration Option
- Instance Structure
- Database Instance Framework

Concepts of Oracle Instance-1

Introduction

- A database instance is a set of memory structures that manage database files.
- A Database is a set of physical files on disk created by the CREATE DATABASE statement.

Concepts of Oracle Instance-1

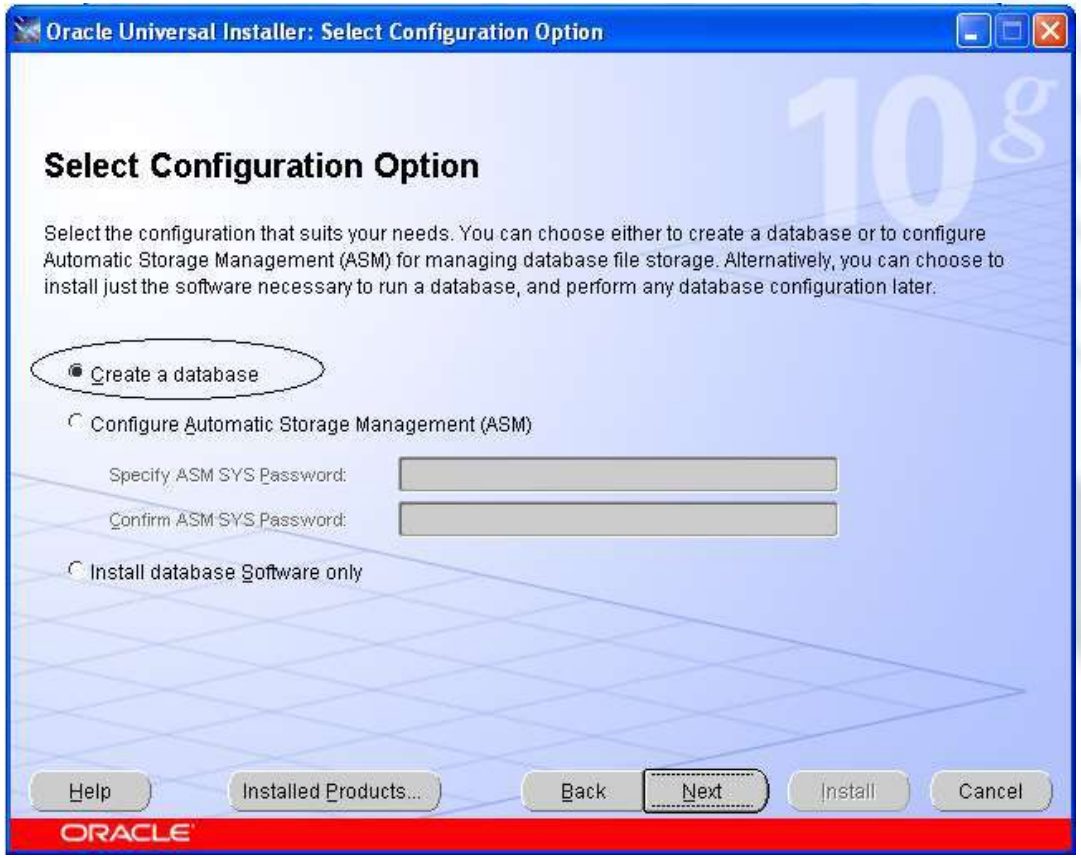
Introduction

- The instance manages its associated data and serves the users of the database.
- Every running Oracle database is associated with at least one Oracle database instance. Because an instance exists in memory and a database exists on disk, an instance can exist without a database and a database can exist without an instance.

Concepts of Oracle Instance-1

Configuration Option

During installation, oracle has an option to install instance, database or both



Concepts of Oracle Instance-1

Instance Structure

When an instance is started, Oracle Database allocates a memory area called the system global area (SGA)and starts one or more background processes. The SGA serves various purposes, including the following:

- Maintaining internal data structures that are accessed by many processes and threads concurrently
- Caching data blocks read from disk
- Buffering redo data before writing it to the online redo log files

Concepts of Oracle Instance-1

Instance Structure

- Storing SQL execution plans

The SGA is shared by the Oracle processes, which include server processes and background processes, running on a single computer. The way in which Oracle processes are associated with the SGA varies according to operating system

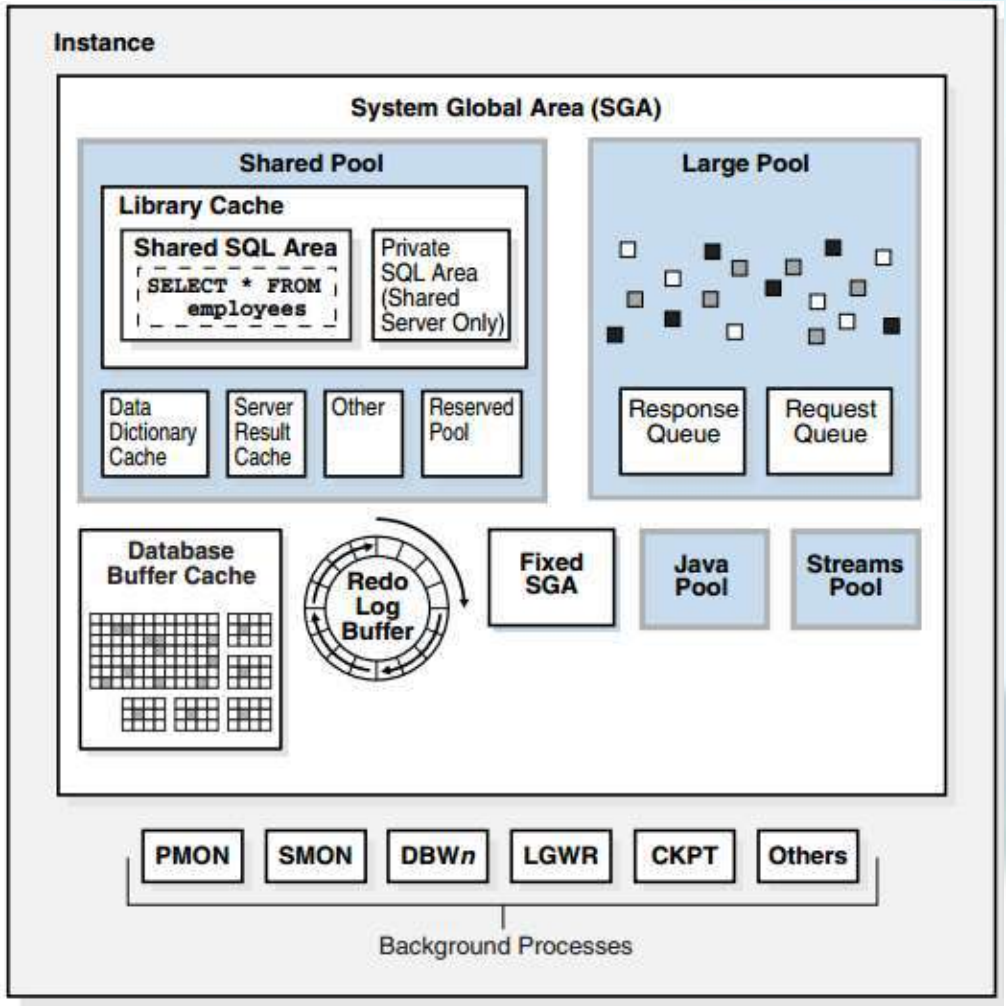
Concepts of Oracle Instance-1

Instance Structure

- A database instance includes background processes. Server processes, and the process memory allocated in these processes, also exist in the instance. The instance continues to function when server processes terminate.

Concepts of Oracle Instance-1

Database Instance Framework



Introduction to Database Administration



Concepts of Oracle Instance-2

Concepts of Oracle Instance-2

- Database Instance Configuration
- Database Instance Configuration
- Database Single & RAC Instance Configuration

Concepts of Oracle Instance-2

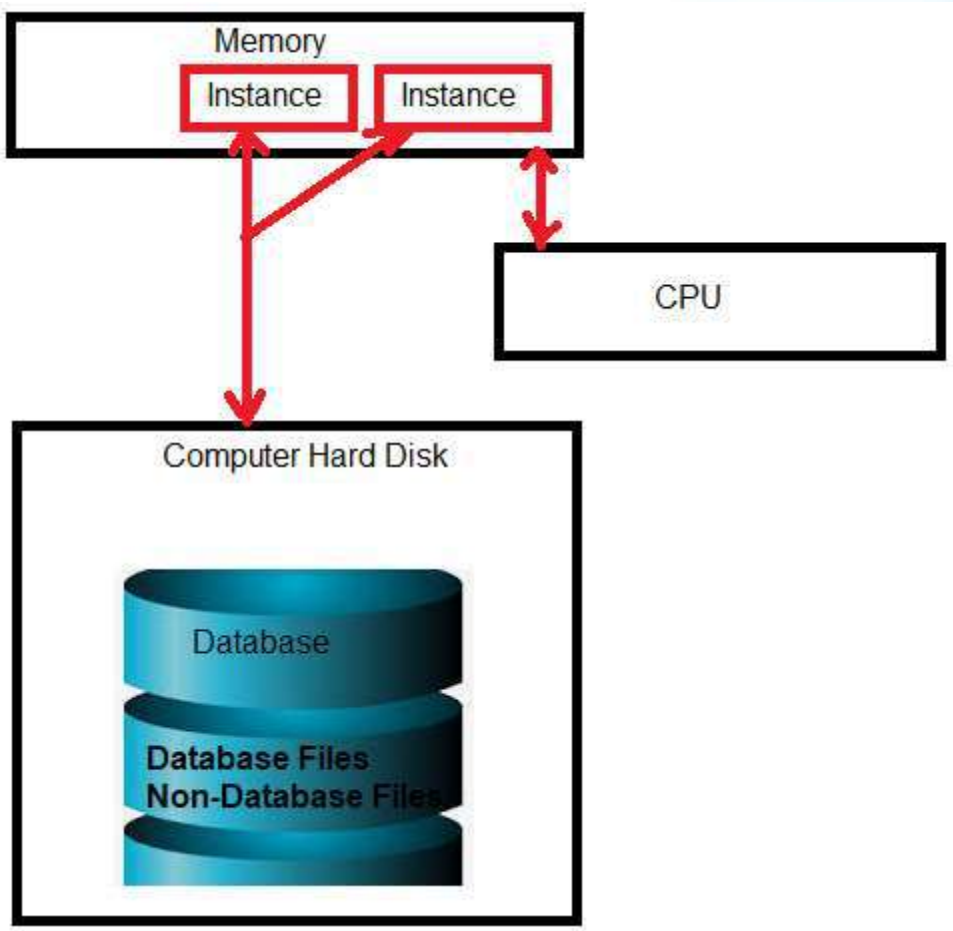
Database Instance Configuration

You can run Oracle Database in either of the following mutually exclusive configurations:

- Single-instance configuration
  A one-to-one relationship exists between the database and an instance.
- Oracle Real Application Clusters (Oracle RAC) configuration
  A one-to-many relationship exists between the database and instances.

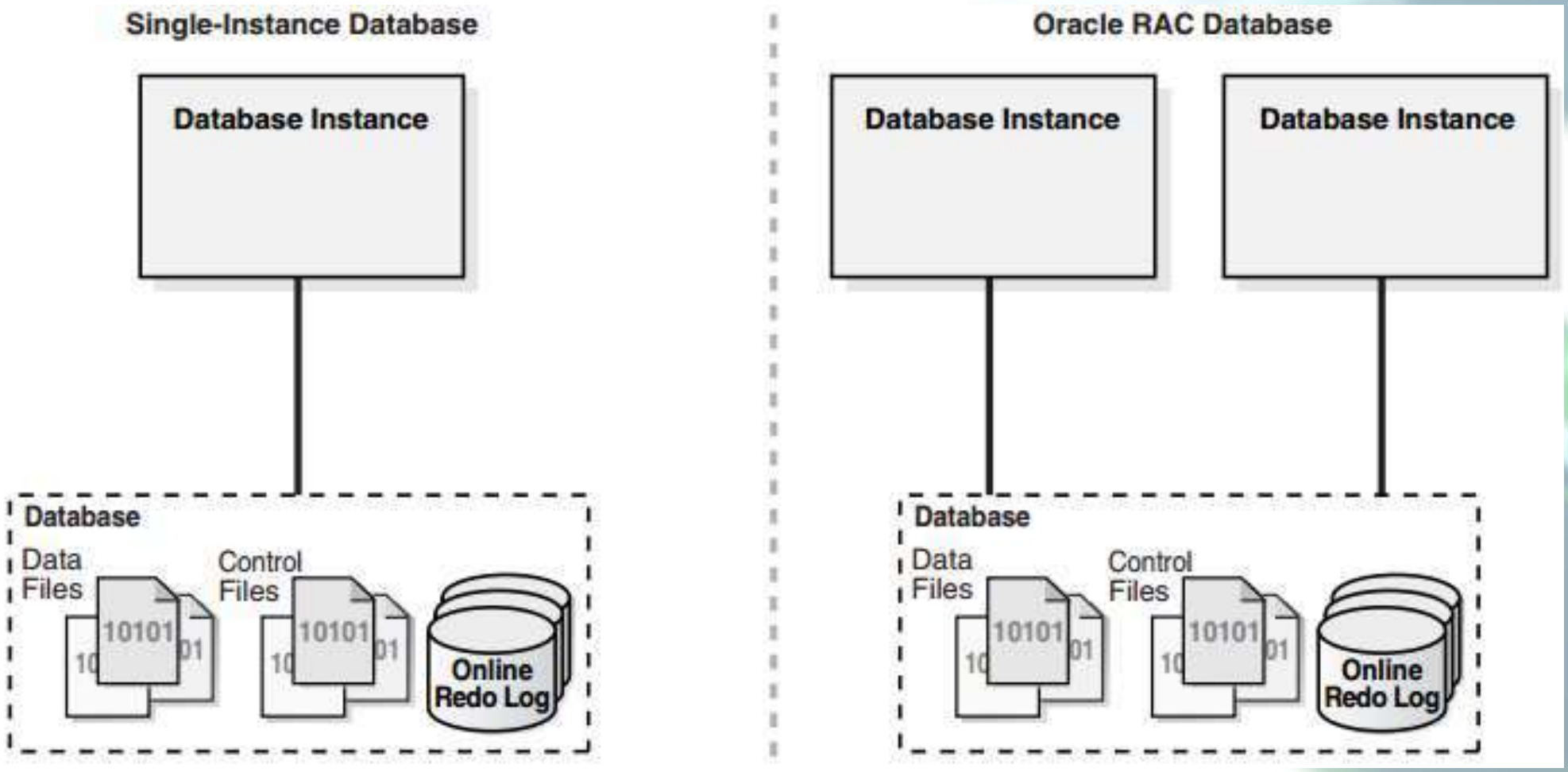
Concepts of Oracle Instance-2

Database Instance Configuration



Concepts of Oracle Instance-2

Database Single & RAC Instance Configuration



Introduction to Database Administration



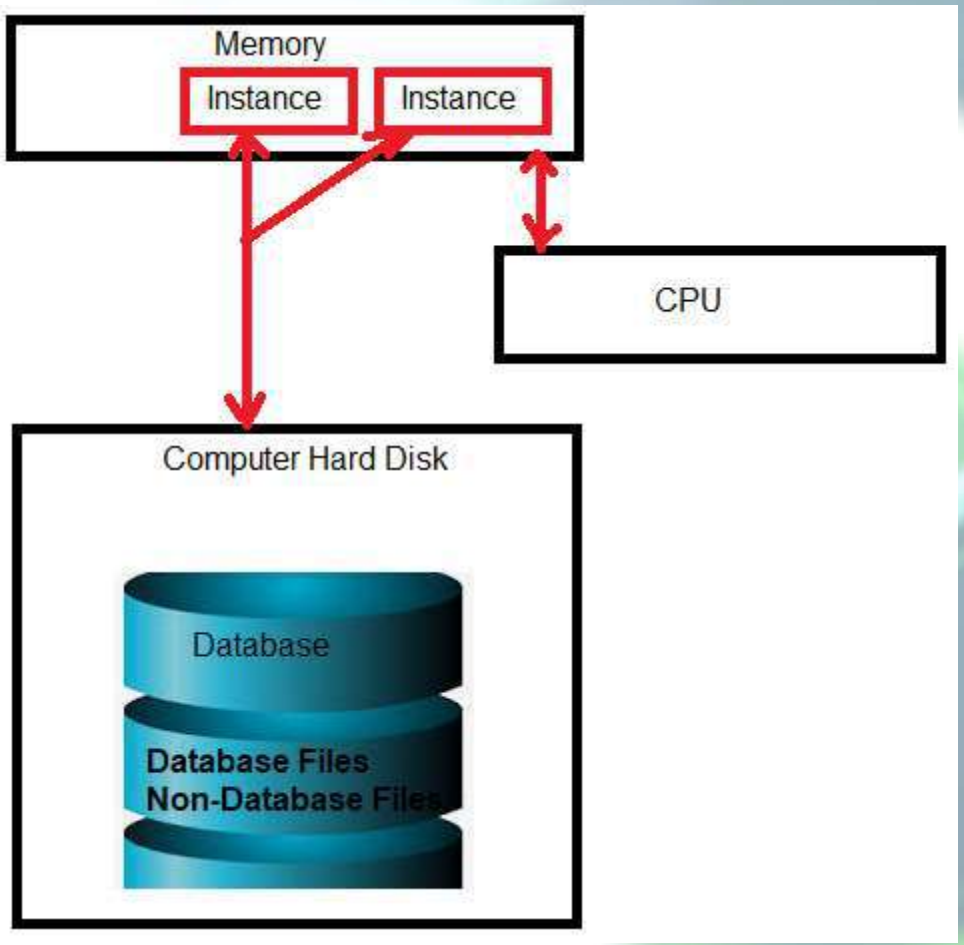
Concepts of Oracle Instance-3

Concepts of Oracle Instance-3

- Database Instance Configuration
- Duration of an Instance
- Instance Startup
- Instance Shutdown & Startup
- Instance & SID
- Oracle Home & SID at Windows OS
- Display Instance Name using Data Dictionary View

Concepts of Oracle Instance-3

Database Instance Configuration



Concepts of Oracle Instance-3

Duration of an Instance

- An instance begins when it is created with the STARTUP command and ends when it is terminated. During this period, an instance can associate itself with one and only one database.
- Furthermore, the instance can mount a database only once, close it only once, and open it only once. After a database has been closed or shut down, you must start a different instance to mount and open this database.

Concepts of Oracle Instance-3

Instance Startup

- Login with user sys / as sysdba
- After shutdown database, run the following:

```
SQL> STARTUP
ORACLE instance started.

Total System Global Area  468729856 bytes
Fixed Size                  1333556 bytes
Variable Size             440403660 bytes
Database Buffers           16777216 bytes
Redo Buffers               10215424 bytes
Database mounted.
Database opened.
```

Concepts of Oracle Instance-3

Instance Startup

This query shows the time that the current instance was started:

```
SQL> SELECT
  2  TO_CHAR(STARTUP_TIME,'MON-DD-RR HH24:MI:SS')
  3  AS "Inst Start Time" FROM V$INSTANCE;

Inst Start Time
------------------
SEP-30-20 20:56:54
```

Concepts of Oracle Instance-3

Instance Shutdown & Startup

This query shows the time that the current instance was started:

```
SQL>
SQL> shutdown immediate
Database closed.
Database dismounted.
ORACLE instance shut down.
SQL> startup
ORACLE instance started.

Total System Global Area 1071333376 bytes
Fixed Size                  1375792 bytes
Variable Size             696254928 bytes
Database Buffers          369098752 bytes
Redo Buffers                4603904 bytes
Database mounted.
Database opened.
SQL> SELECT
  2  TO_CHAR(STARTUP_TIME,'MON-DD-RR HH24:MI:SS')
  3  AS "Inst Start Time" FROM V$INSTANCE;

Inst Start Time
------------------
OCT-01-20 08:48:52
```

Concepts of Oracle Instance-3

Instance & SID

- The system identifier (SID)is a unique name for an Oracle database instance on a specific host. On UNIX and Linux, Oracle Database uses the SID and Oracle home values to create a key to shared memory.
- Also, the SID is used by default to locate the parameter file, which is used to locate relevant files such as the database control files.

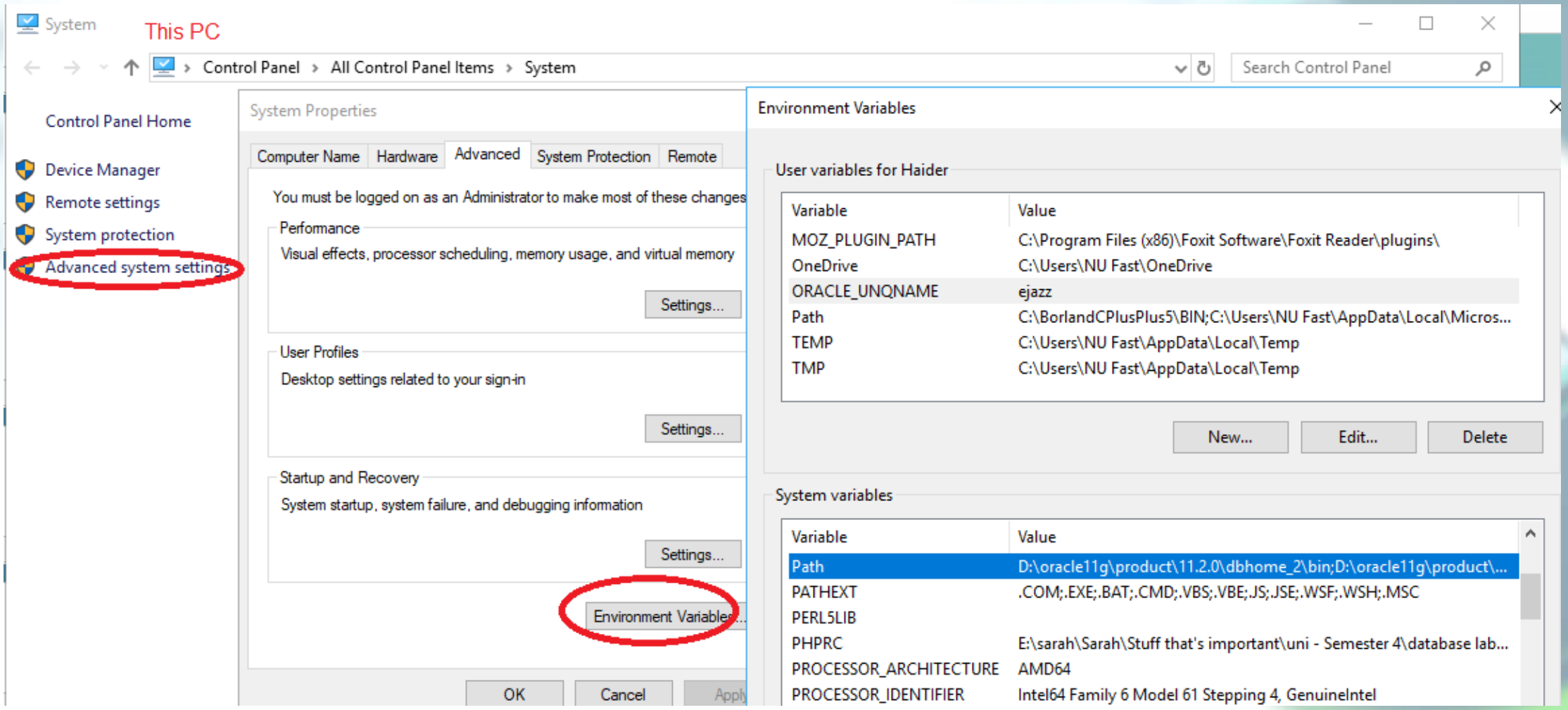
Concepts of Oracle Instance-3

Instance & SID

- On most platforms, the ORACLE_SID environment variable sets the SID, whereas the ORACLE_HOME variable sets the Oracle home.
- When connecting to an instance, clients can specify the SID in an Oracle Net connection or use a net service name.
- Oracle Database converts a service name into an ORACLE_HOME and ORACLE_SID.

Concepts of Oracle Instance-3

Oracle Home & SID at Windows OS



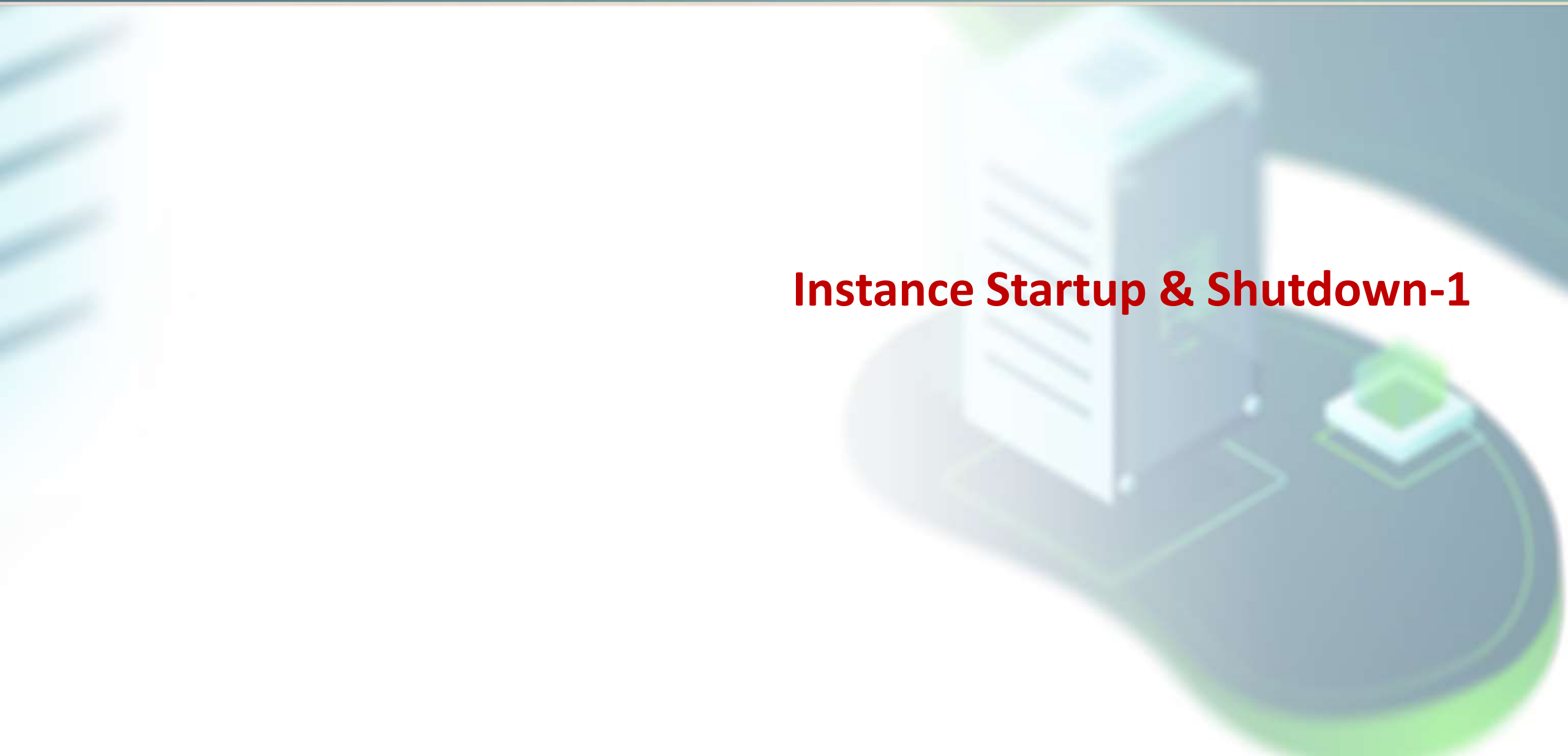
Concepts of Oracle Instance-3

Display Instance Name using Data Dictionary View

```
SQL> SELECT TO_CHAR(STARTUP_TIME,'MON-DD-RR HH24:MI:SS') AS "Inst Start Time",
  2  Instance_name , substr(Host_name,1,20)
  3  FROM V$INSTANCE;

Inst Start Time     INSTANCE_NAME    SUBSTR(HOST_NAME,1,2
------------------- ---------------- --------------------
OCT-01-20 08:48:52 orcl             HAIDER
```

Introduction to Database Administration



Instance Startup & Shutdown-1

Instance Startup & Shutdown-1

- Overview of Instance and Database Startup
- SVRCTL Utility Help
- Example - SVRCTL command
- Progress from Shutdown & Startup OPEN
- Instance Startup
- Instance Startup MOUNT
- Instance Startup NOMOUNT

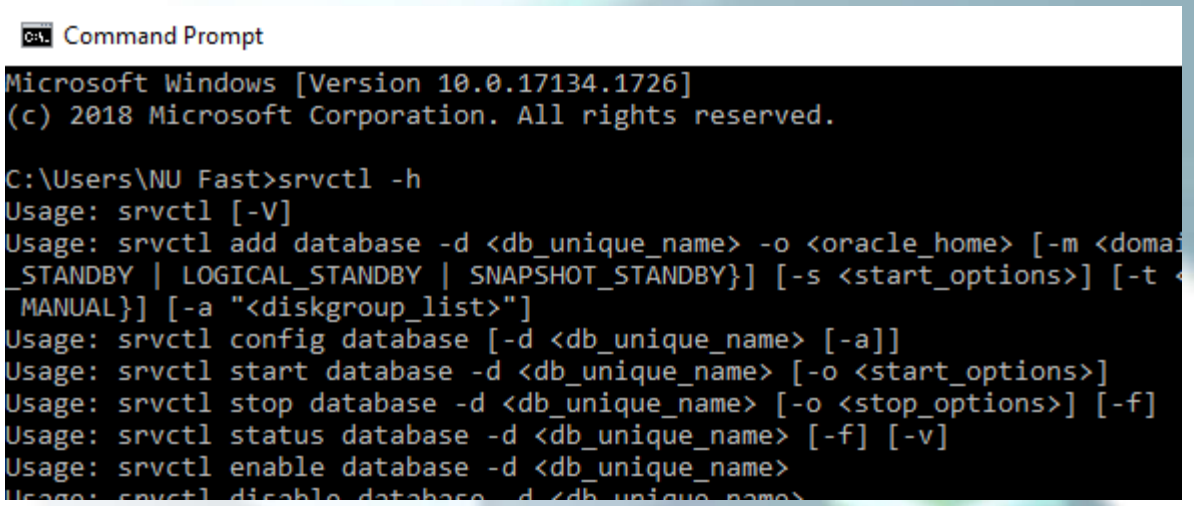
Instance Startup & Shutdown-1

Overview of Instance and Database Startup

- A database instance provides user access to a database.
- In a typical use case, you manually start an instance and then mount and open the database, making it available for users. You can use the SQL*Plus STARTUP command, Oracle Enterprise Manager (Enterprise Manager), or the SRVCTL utility to perform these steps.

Instance Startup & Shutdown-1

SVRCTL Utility Help



Instance Startup & Shutdown-1

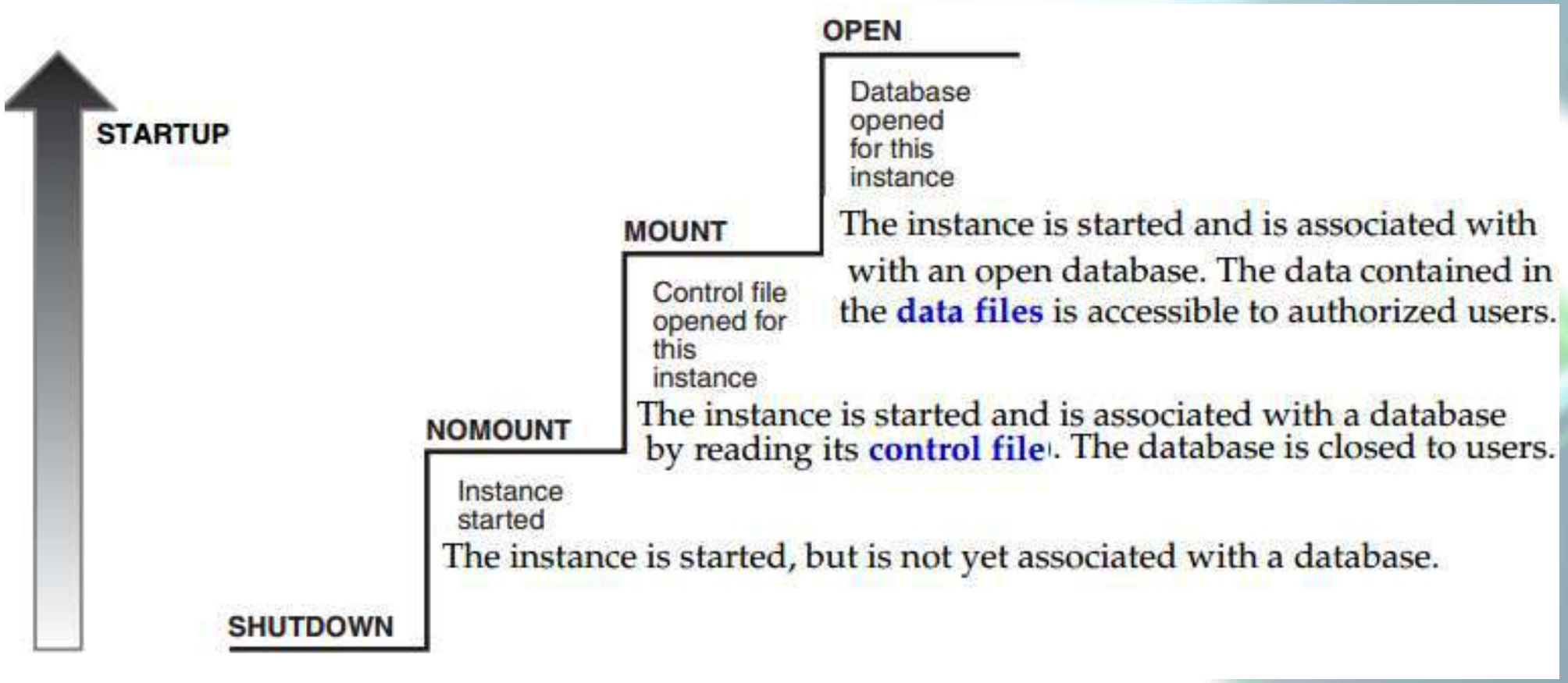
Example - SVRCTL command

Syntax to add a named service to a database with preferred instances in list one and available instances in list two, using basic failover for the available instances:

```
srvctl add  service -d crm -s sales -r crm01,crm02 -a crm03
```

Instance Startup & Shutdown-1

Progress from Shutdown & Startup OPEN



Instance Startup & Shutdown-1

Instance Startup

- Login with user sys / as sysdba
- After shutdown database, run the following:

```
SQL> STARTUP
ORACLE instance started.

Total System Global Area  468729856 bytes
Fixed Size                  1333556 bytes
Variable Size             440403660 bytes
Database Buffers           16777216 bytes
Redo Buffers               10215424 bytes
Database mounted.
Database opened.
```

Instance Startup & Shutdown-1

Instance Startup

This query shows the time that the current instance was started:

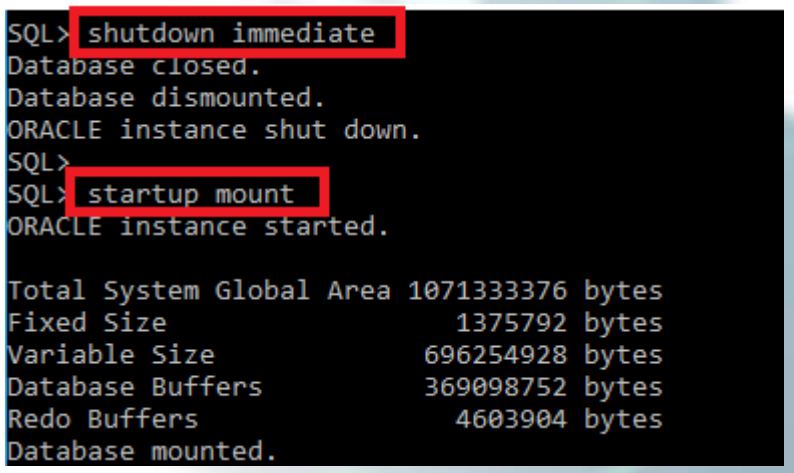
```
SQL> SELECT
  2  TO_CHAR(STARTUP_TIME,'MON-DD-RR HH24:MI:SS')
  3  AS "Inst Start Time" FROM V$INSTANCE;

Inst Start Time
------------------
SEP-30-20 20:56:54
```

Instance Startup & Shutdown-1

Instance Startup MOUNT

First perform shutdown immediate then startup mount:



Instance Startup & Shutdown-1

Instance Startup NOMOUNT

First perform shutdown immediate then startup nomount:

```
SQL> shutdown immediate
ORA-01109: database not open


Database dismounted.
ORACLE instance shut down.
SQL>
SQL> startup nomount
ORACLE instance started.

Total System Global Area 1071333376 bytes
Fixed Size                  1375792 bytes
Variable Size             696254928 bytes
Database Buffers          369098752 bytes
Redo Buffers                4603904 bytes
SQL>
```

End

Database creation and certain backup & recovery operations are performed.



Instance Startup & Shutdown-2

Instance Startup & Shutdown-2

- Connection with Administrator Privileges
- Instance Started to perform steps
- How Database is Mounted?
- How Database is opened?
- Read-only Opened Database
- Progress from Shutdown & Startup OPEN

Instance Startup & Shutdown-2

Connection with Administrator Privileges

- Database startup and shutdown are powerful administrative options that are restricted to users who connect to Oracle Database with administrator privileges.
- Normal users do not have control over the current status of an Oracle database.

Instance Startup & Shutdown-2

Connection with Administrator Privileges

- Depending on the operating system, one of the following conditions establishes administrator privileges for a user:
- The operating system privileges of the user enable him or her to connect using administrator privileges.

Instance Startup & Shutdown-2

Connection with Administrator Privileges

- The user is granted the SYSDBA or SYSOPER system privileges and the database uses password files to authenticate database administrators over the network.
- When you connect with the SYSDBA system privilege, you are in the schema owned by SYS. When you connect as SYSOPER, you are in the public schema.

Instance Startup & Shutdown-2

Instance Started to perform steps

- Searches for a server parameter file in a platform-specific default location and, if not found, for a text initialization parameter file(specifying STARTUP with the SPFILE or PFILE parameters overrides the default behavior)
- Reads the parameter file to determine the values of initialization parameters

Instance Startup & Shutdown-2

Instance Started to perform steps

- Allocates the SGA based on the initialization parameter settings
- Starts the Oracle background processes
- Opens the alert log and trace files and writes all explicit parameter settings to the alert login valid parameter syntax

Instance Startup & Shutdown-2

How Database is Mounted?

- The instance mounts a database to associate the database with this instance. To mount the database, the instance obtains the names of the database control files specified in the CONTROL_FILES initialization parameter and opens the files.

Instance Startup & Shutdown-2

How Database is Mounted?

- Oracle Database reads the control files to find the names of the data files and the online redo log files that it will attempt to access when opening the database.
- In a mounted database, the database is closed and accessible only to database administrators.

Instance Startup & Shutdown-2

How Database is Mounted?

- Administrators can keep the database closed while completing specific maintenance operations. However, the database is not available for normal operations

Instance Startup & Shutdown-2

How Database is opened?

- Opening a mounted database makes it available for normal database operations
- Opens the online data files in tablespaces other than undo tablespaces
- Acquires an undo tablespace
- Opens the online redo log files
- By default, the database opens in read/write mode.

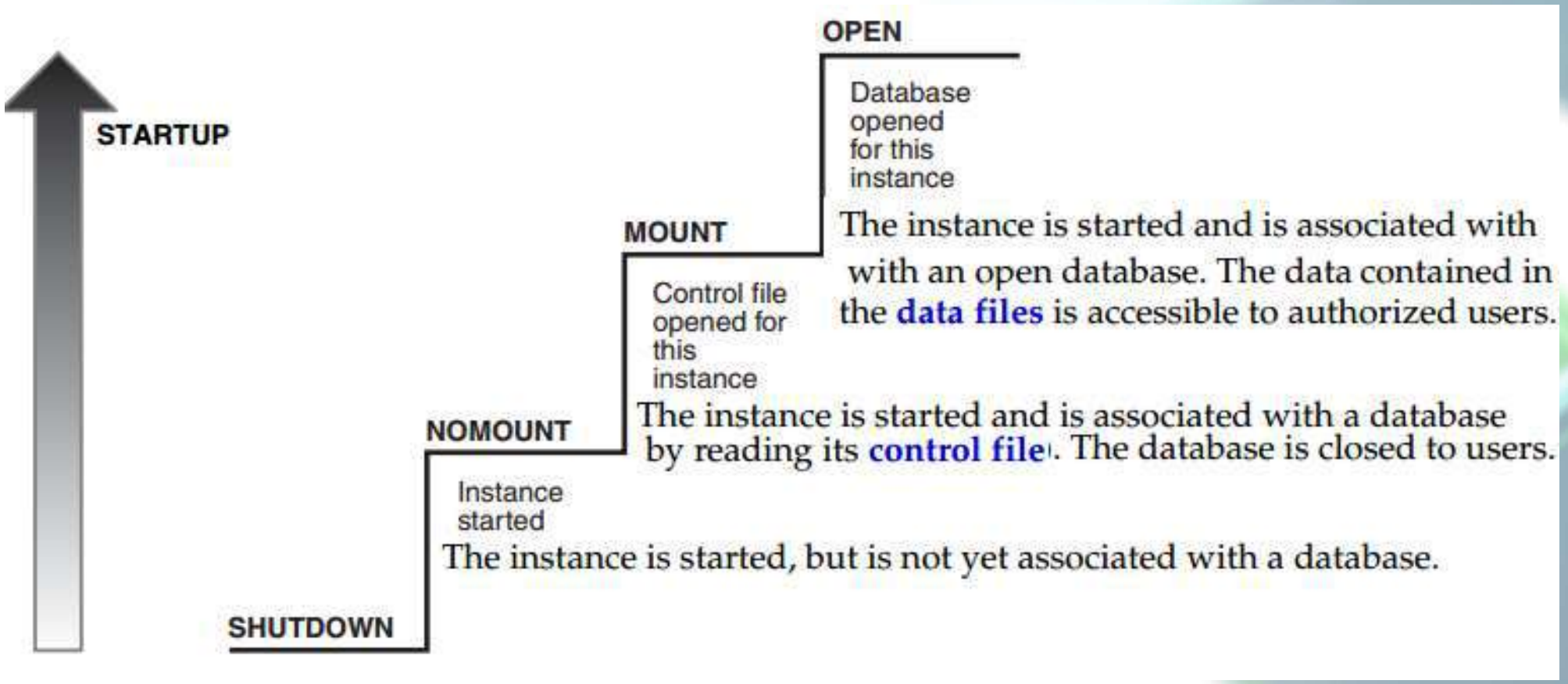
Instance Startup & Shutdown-2

Read-only Opened Database

- Data files can be taken offline and online. However, you cannot take permanent tablespaces offline.
- Offline data files and tablespaces can be recovered.
- The control file remains available
- Temporary tablespaces created with the CREATE TEMPORARY TABLESPACE statement are read/write.
- Writes to operating system audit trails, trace files, and alert logs can continue.

Instance Startup & Shutdown-2

Progress from Shutdown & Startup OPEN





Instance Startup & Shutdown-3

Instance Startup & Shutdown-3

- Database with Shutdown Instance
- Progress from OPEN to Shutdown
- Steps to perform Database Shutdown
- Shutdown ABORT
- Shutdown IMMEDIATE
- Shutdown TRANSACTIONAL
- Shutdown NORMAL
- Shutdown Modes Summary

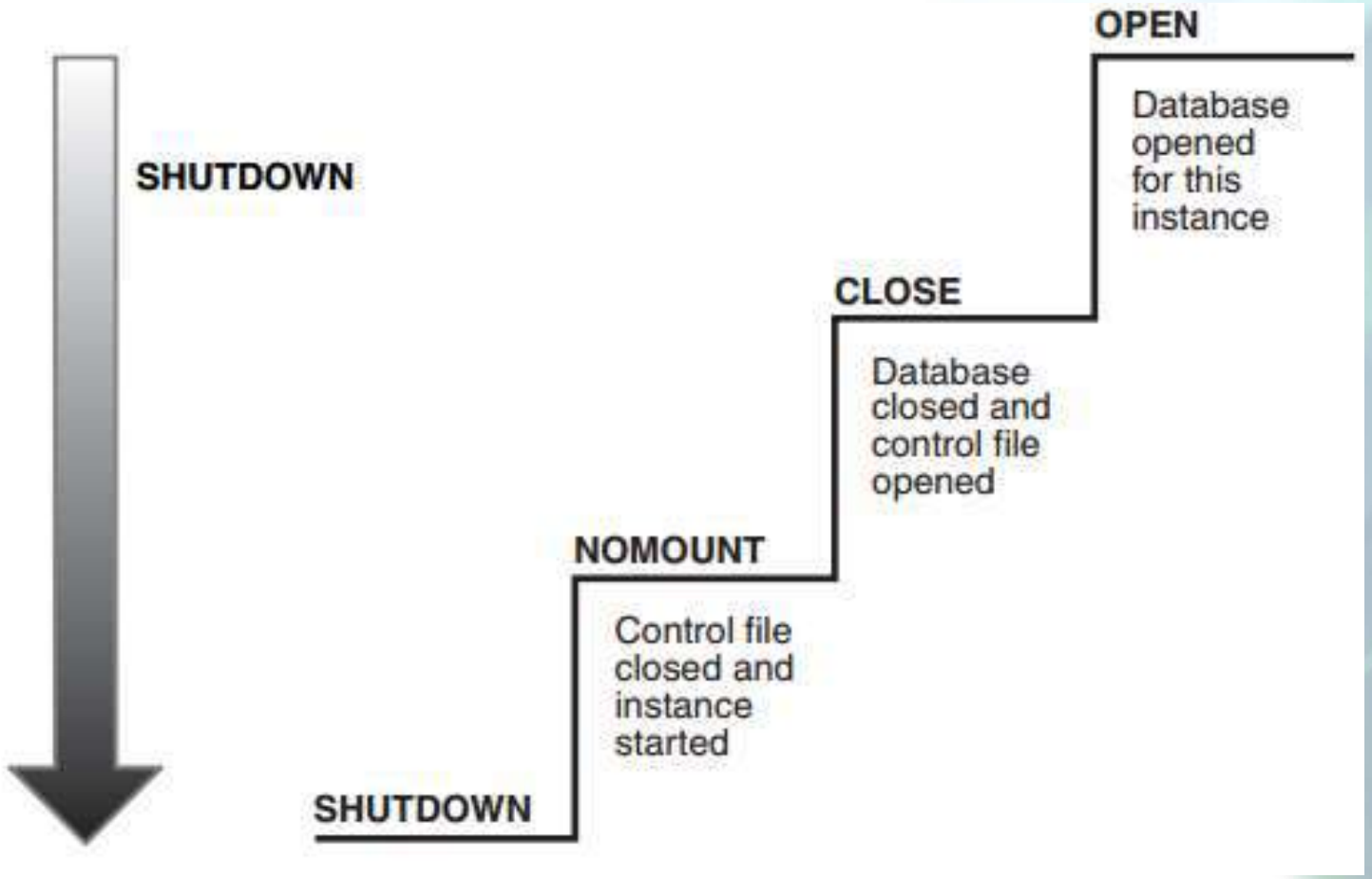
Instance Startup & Shutdown-3

Database with Shutdown Instance

- In a typical use case, you manually shut down the database, making it unavailable for users while you perform maintenance or other administrative tasks.
- You can use the SQL*Plus SHUTDOWN command or Enterprise Manager to perform these steps.

Instance Startup & Shutdown-3

Progress from OPEN to Shutdown



Instance Startup & Shutdown-3

Steps to perform Database Shutdown

- Database closed
  The database is mounted, but online data files and redo log files are closed.

- Database unmounted
  The instance is started, but is no longer associated with the control file of the database.
- Database instance shut down
  The database instance is no longer started.

Instance Startup & Shutdown-3

Steps to perform Database Shutdown

Oracle Database does not go through all of the preceding steps in an instance failure or SHUTDOWN ABORT, which immediately terminates the instance.

Instance Startup & Shutdown-3

Shutdown ABORT

Used in emergency situation. Shuting down instance may take substantially longer. SHUTDOWN ABORT does not checkpoint the open data files

Instance Startup & Shutdown-3

Shutdown IMMEDIATE

Fastest way to manage. Active transactions are terminated and uncommitted changes are rolled back.



Instance Startup & Shutdown-3

Shutdown TRANSACTIONAL

This mode prevents users from starting new transactions, but waits for all current transactions to complete before shutting down.

Instance Startup & Shutdown-3

Shutdown NORMAL

This is the default mode of shutdown. The database waits for all connected users to disconnect before shutting down.



Instance Startup & Shutdown-3

Shutdown Modes Summary

A database administrator with SYSDBA or SYSOPER privileges can shut down the database using the SQL*Plus SHUTDOWN command or Enterprise Manager.

Database Behavior	ABORT	IMMEDIATE	TRANSACTIONAL	NORMAL
Permits new user connections	No	No	No	No
Waits until current sessions end	No	No	No	Yes
Waits until current transactions end	No	No	Yes	Yes
Performs a checkpoint and closes open files	No	Yes	Yes	Yes

Introduction to Database Administration



Overview of Checkpoints

Overview of Checkpoints

- What is a Checkpoint?
- Purpose of Checkpoint?
- Recovery without Checkpoints
- Recovery using Checkpoints

Overview of Checkpoints

What is a Checkpoint?

- A checkpoint is a crucial mechanism in consistent database shutdowns, instance recovery, and Oracle Database operation generally.
- A data structure that indicates the checkpoint position, which is the SCN in the redo stream where instance recovery must begin The checkpoint position is determined by the oldest dirty buffer in the database buffer cache.

Overview of Checkpoints

What is Checkpoint?

- The checkpoint position acts as a pointer to the redo stream and is stored in the control file and in each data file header.
- The writing of modified database buffers in the database buffer cache to disk.

Whenever checkpoint happens all the dirty buffers of DB buffer cache (volatile storage) moves into data files (Non volatile storage)

Overview of Checkpoints

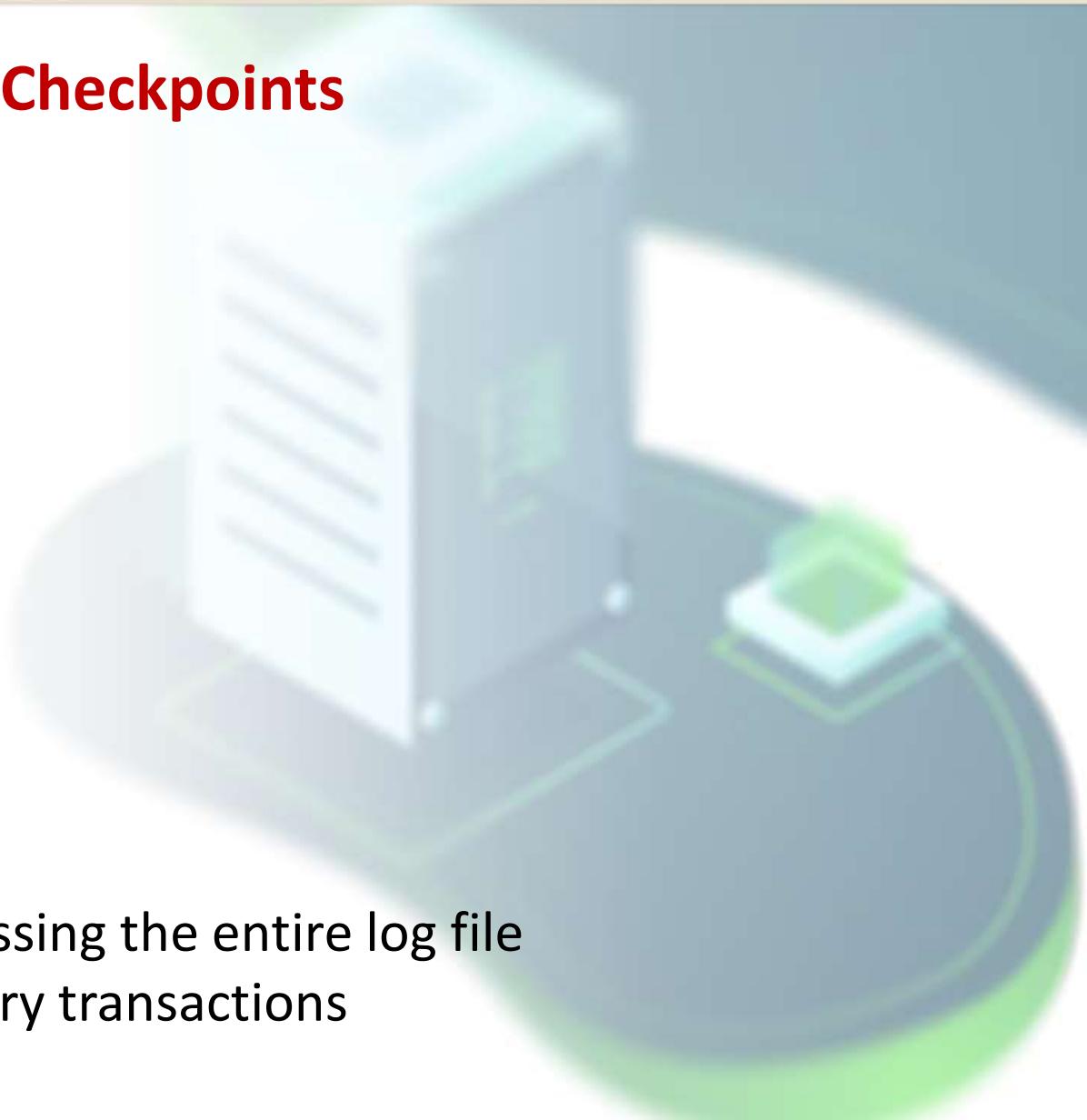
Purpose of Checkpoint?

Oracle Database uses checkpoints to achieve the following goals:

- Reduce the time required for recovery incase of an instance or media failure
- Ensure that dirty buffers in the buffer cache are written to disk regularly
- Ensure that all committed data is written to disk during a consistent shutdown
- Establish data consistency

Overview of Checkpoints

Recovery without Checkpoints

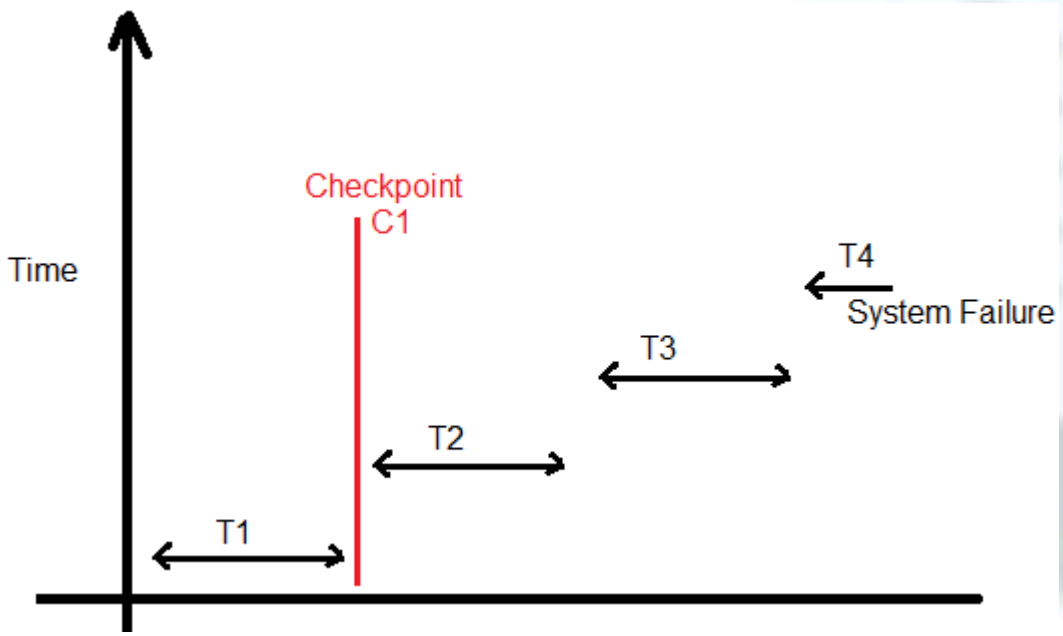


Extra time is wasted in processing the entire log file
Redoing of unnecessary transactions

Overview of Checkpoints

Recovery using Checkpoints

Checkpoint is a mechanism in which all the previous logs are removed from the system and are stored permanently in the storage disk.



Introduction to Database Administration



Initiate Checkpoints

Initiate Checkpoints

Initiate Checkpoints

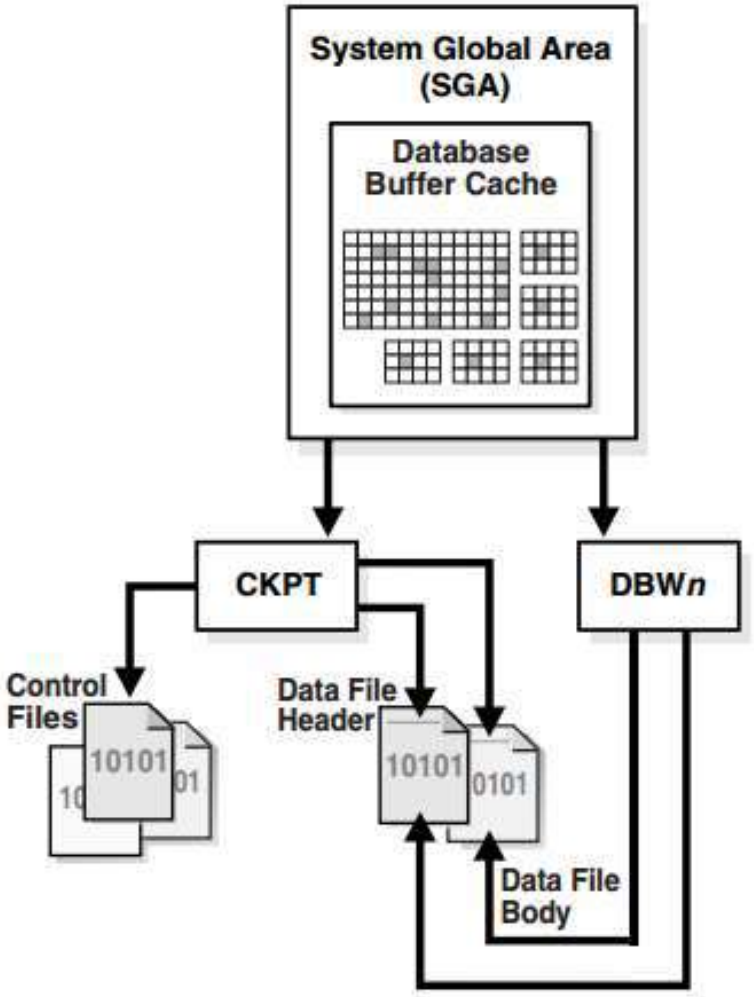
- When to initiate Checkpoint
- Checkpoint Process
- Thread Checkpoints Type
- Tablespace and Data File Checkpoints Type
- Incremental Checkpoints Type
- Checkpoints Processor and Database files

Initiate Checkpoints

When to initiate Checkpoint

The checkpoint process (CKPT) is responsible for writing checkpoints to the data file headers and control file.
Checkpoints occur in a variety of situations

Initiate Checkpoints



Checkpoint Process

The checkpoint process (CKPT)updates the control file and data file headers with checkpoint information and signals DBWn to write blocks to disk. Checkpoint information includes the checkpoint position, SCN, location in online redo log to begin recovery, and so on.

Initiate Checkpoints

Thread Checkpoints Type

The database writes to disk all buffers modified by redo in a specific thread before a certain target. The set of thread checkpoints on all instances in a database is a database checkpoint. Thread checkpoints occur in the following situations:

- Consistent database shutdown
- ALTER SYSTEM CHECKPOINT statement
- Online redo log switch
- ALTER DATABASE BEGIN BACKUP statement

Initiate Checkpoints

Tablespace and Data File Checkpoints Type

The database writes to disk all buffers modified by redo before a specific target. A tablespace checkpoint is a set of data file checkpoints, one for each data file in the tablespace. These checkpoints occur in a variety of situations, including making a tablespace read-only or taking it offline normal, shrinking a data file, or executing

ALTER TABLESPACE BEGIN BACKUP;

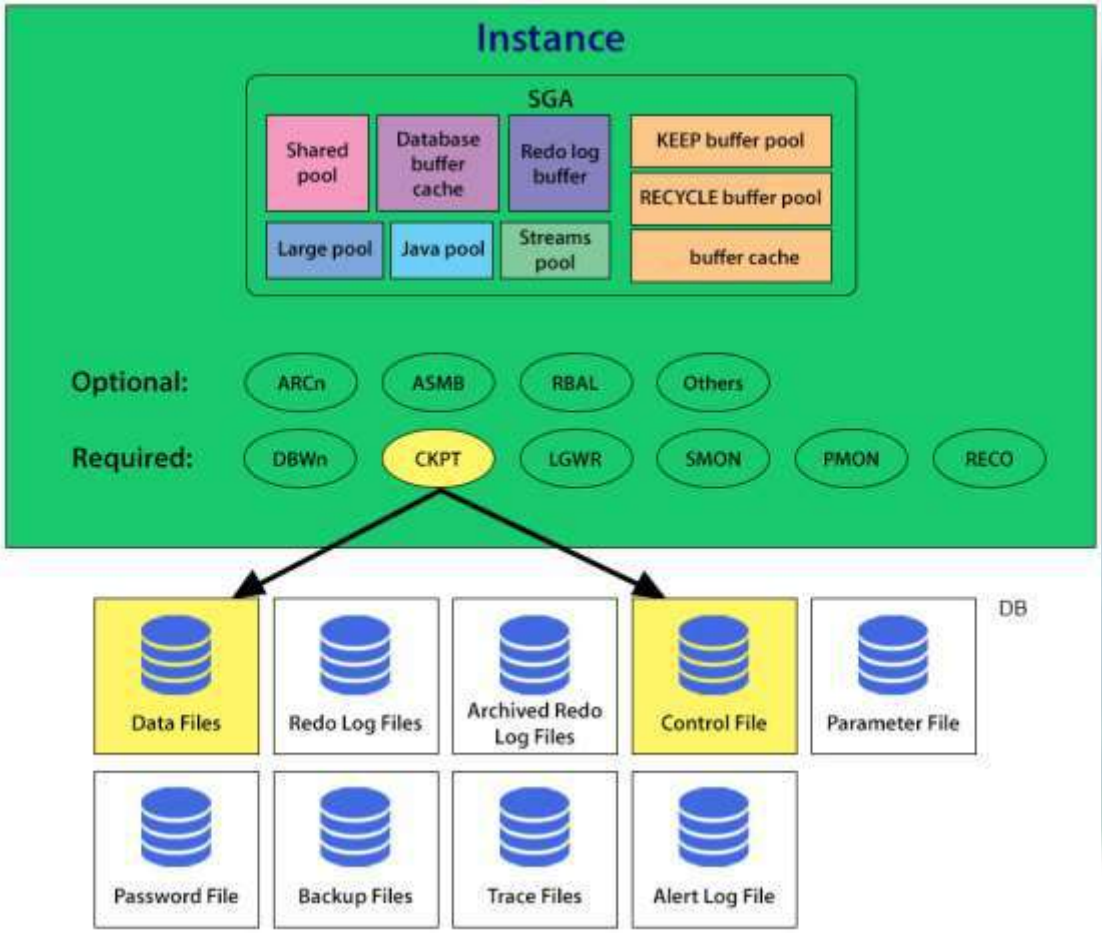
Initiate Checkpoints

Incremental Checkpoints Type

- An incremental checkpoint is a type of thread checkpoint partly intended to avoid writing large numbers of blocks at online redo log switches. DBWn checks at least every three seconds to determine whether it has work to do.
- When DBWn writes dirty buffers, it advances the checkpoint position, causing CKPT to write the checkpoint position to the control file, but not to the data file headers.

Initiate Checkpoints

Checkpoints Processor and Database files



Introduction to Database Administration



Checkpoints Commands Samples

Checkpoints Commands Samples

Checkpoints Commands Samples

- Checkpoints Commands Samples
- When will a Checkpoint occur
- Checkpoint Occurring Command
- Data Dictionary View for Checkpoint
- SCN is Changed After another Checkpoint
- SCN is not Changed After Switch Logfile

Checkpoints Commands Samples

When will a Checkpoint occur

- At each switch of the redo log files
- Once the number of seconds defined in the LOG_CHECKPOINT_TIMEOUT is reached Once the current redo log file reaches the size
  LOG Checkpoint Interval x size of IO-OS blocks, Example 10000*512=5MB

- Directly by the Alter System switch Logfile command
- Directly by the Alter System Checkpoint command

Checkpoints Commands Samples

Checkpoint Occurring Command

- Run the following command with sys user

Alter system checkpoint;

Checkpoints Commands Samples

Data Dictionary View for Checkpoint

```
SQL> select Checkpoint_change#, TO_CHAR(checkpoint_time, 'DD/MM/YYYY HH:MI:SS AM') CHKP_Time,
  2  Bytes/1024/1024
  3  from v$datafile;

CHECKPOINT_CHANGE# CHKP_TIME              BYTES/1024/1024
------------------ ---------------------- ---------------
          19352187 02/10/2020 10:06:26 AM             710
          19352187 02/10/2020 10:06:26 AM             710
          19352187 02/10/2020 10:06:26 AM             305
          19352187 02/10/2020 10:06:26 AM            8.75
          19352187 02/10/2020 10:06:26 AM            1000
          19352187 02/10/2020 10:06:26 AM            1000
```

```
SQL> select Checkpoint_change#, checkpoint_time, sum(Bytes)/1024/1024 DataSize
  2  from v$datafile
  3  group by Checkpoint_change#, checkpoint_time;

CHECKPOINT_CHANGE# CHECKPOIN   DATASIZE
------------------ --------- ----------
          19352187 02-OCT-20   18749.75
```

Checkpoints Commands Samples

SCN is Changed After another Checkpoint

```
        19352187 02-OCT-20   18749.75

SQL> shutdown immediate
Database closed.
Database dismounted.
ORACLE instance shut down.
SQL> startup
ORACLE instance started.

Total System Global Area 1071333376 bytes
Fixed Size                  1375792 bytes
Variable Size             696254928 bytes
Database Buffers          369098752 bytes
Redo Buffers                4603904 bytes
Database mounted.
Database opened.
SQL> select Checkpoint_change#, TO_CHAR(checkpoint_time, 'DD/MM/YYYY HH:MI:SS AM') CHKP_Time,
  2  Bytes/1024/1024
  3  from v$datafile;

CHECKPOINT_CHANGE# CHKP_TIME              BYTES/1024/1024
------------------ ---------------------- ---------------
        19352722 02/10/2020 10:19:42 AM             710
        19352722 02/10/2020 10:19:42 AM             710
        19352722 02/10/2020 10:19:42 AM             305
        19352722 02/10/2020 10:19:42 AM            8.75
        19352722 02/10/2020 10:19:42 AM            1000
        19352722 02/10/2020 10:19:42 AM            1000
        19352722 02/10/2020 10:19:42 AM             500
```

Checkpoints Commands Samples

SCN is not Changed After Switch Logfile

```
          19353247 02/10/2020 10:25:15 AM             1024
          19353247 02/10/2020 10:25:15 AM              800

22 rows selected.

SQL>
SQL> Alter System switch Logfile;

System altered.

SQL> select Checkpoint_change#, checkpoint_time, sum(Bytes)/1024/1024 DataSize
  2  from v$datafile
  3  group by Checkpoint_change#, checkpoint_time;

CHECKPOINT_CHANGE# CHECKPOIN   DATASIZE
------------------ --------- ----------
          19353247 02-OCT-20   18749.75
```

Introduction to Database Administration



Instance Recovery

Instance Recovery

Instance Recovery

- Overview of Instance Recovery
- Purpose of Instance Recovery
- How Instance Recovery is Performed?

Instance Recovery

Overview of Instance Recovery

- Instance recovery is the process of applying records in the online redo log to data files to reconstruct changes made after the most recent checkpoint.
- Instance recovery occurs automatically when an administrator attempts to open a database that was previously shut down inconsistently.

Instance Recovery

Overview of Instance Recovery

- Instance recovery occurs automatically when an administrator attempts to open a database that was previously shut down inconsistently.

```
SQL> shutdown immediate
Database closed.
Database dismounted.
ORACLE instance shut down.
SQL> startup
ORACLE instance started.

Total System Global Area 1071333376 bytes
Fixed Size                  1375792 bytes
Variable Size             696254928 bytes
Database Buffers          369098752 bytes
Redo Buffers                4603904 bytes
Database mounted.
Database opened.
```

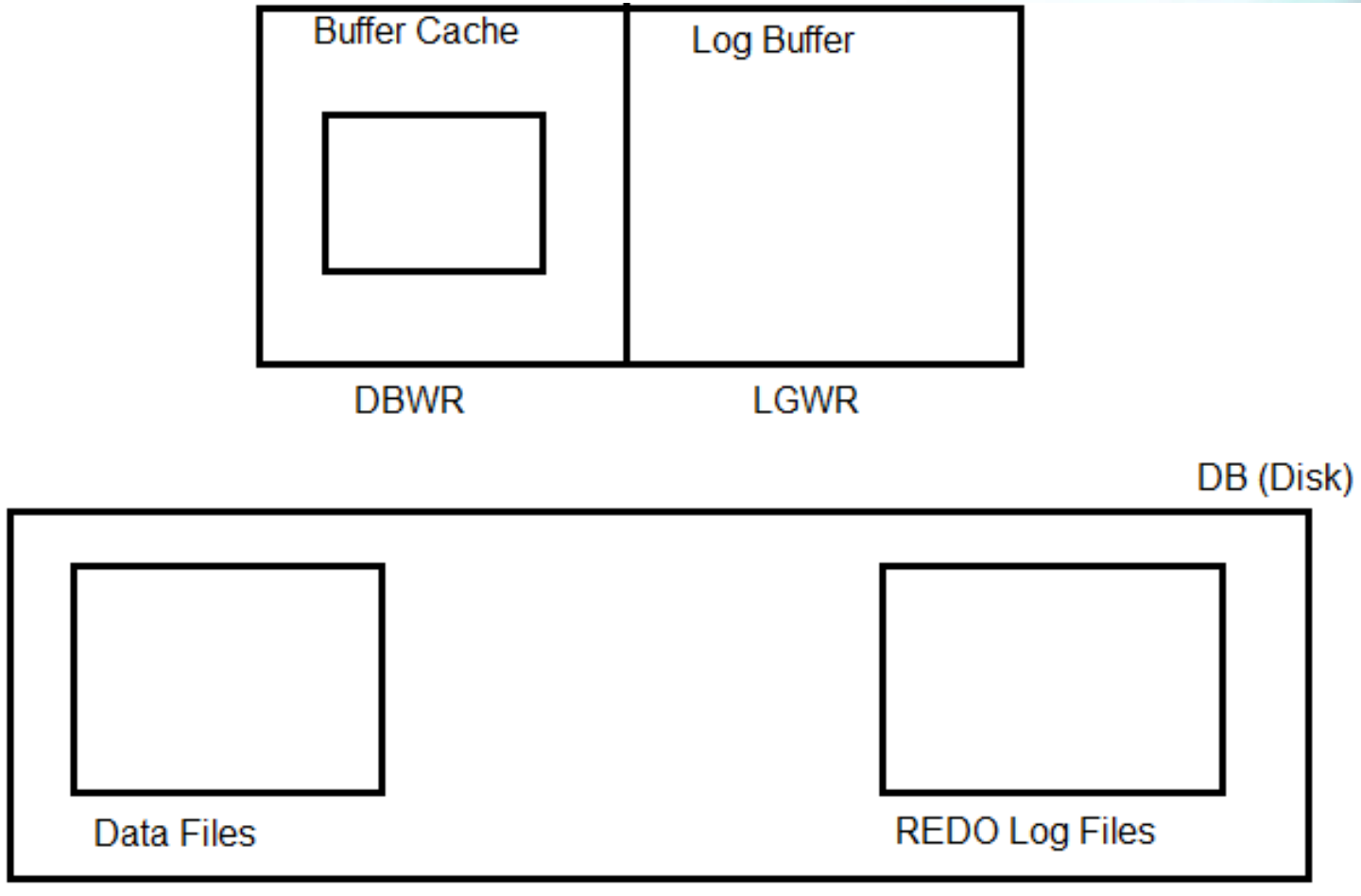
Instance Recovery

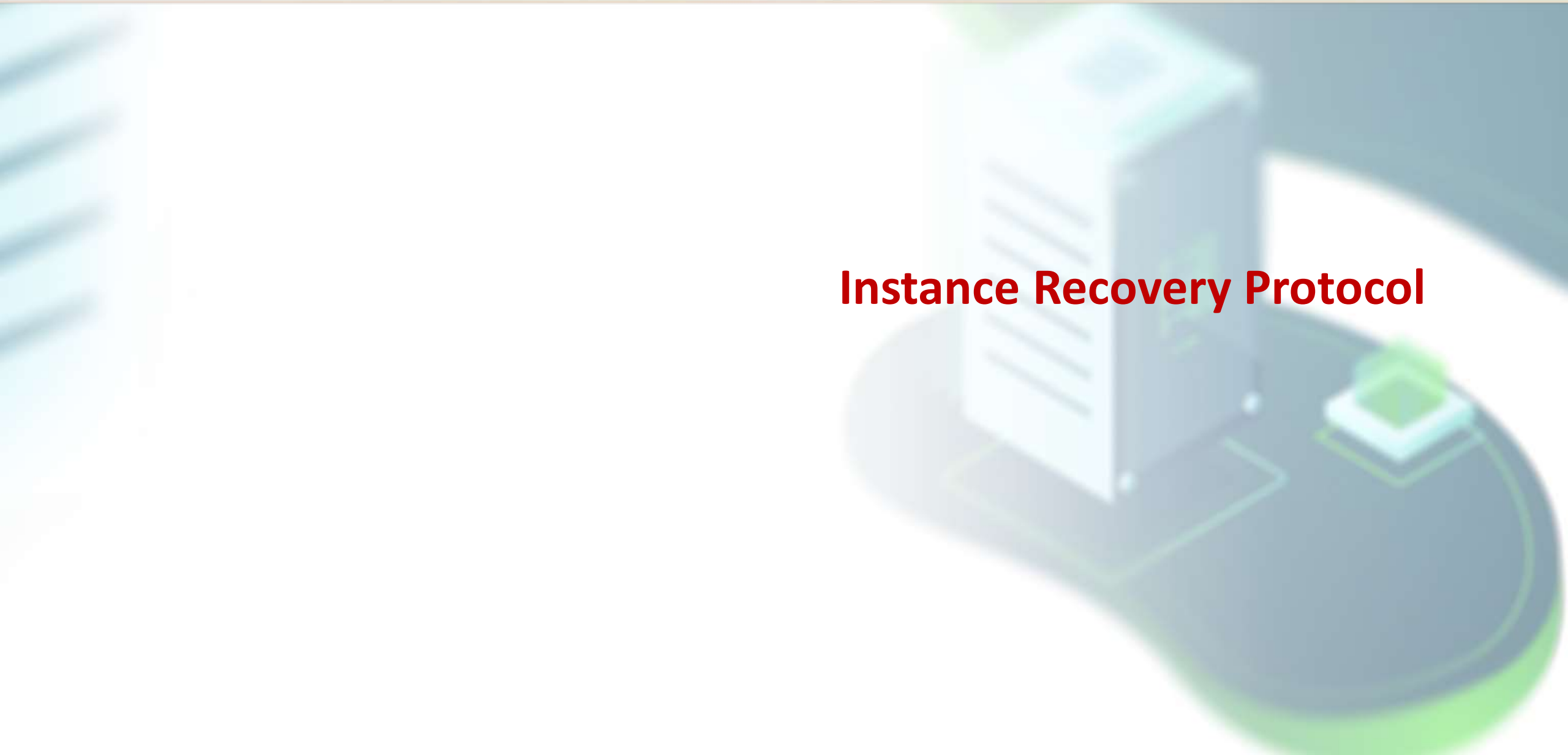
Purpose of Instance Recovery

- Instance recovery ensures that the database is in a consistent state after an instance failure.
- The files of a database can be left in an inconsistent state because of how Oracle Database manages database changes.

Instance Recovery

How Instance Recovery is Performed?





Instance Recovery Protocol

Instance Recovery Protocol

Instance Recovery

- Situations for Instance Recovery
- Performing Instance Recovery
- Importance of CKPT for Instance Recovery

Instance Recovery Protocol

Situations for Instance Recovery

If an instance of an open database fails, either because of a SHUTDOWN ABORT statement or abnormal termination, then the following situations can result:

- Data blocks committed by a transaction are not written to the data files and appear only in the online redo log. These changes must be reapplied to the database.

Instance Recovery Protocol

Situations for Instance Recovery

- The data files contains changes that had not been committed when the instance failed. These changes must be rolled back to ensure transactional consistency.

Instance recovery uses only online redo log files and current online data files to synchronize the data files and ensure that they are consistent

Instance Recovery Protocol

Performing Instance Recovery

- Whether instance recovery is required depends on the state of the redo threads. A redo thread is marked open in the control file when a database instance opens in read/write mode, and is marked closed when the instance is shut down consistently.
- If redo threads are marked open in the control file, but no live instances hold the thread enqueues corresponding to these threads, then the database requires instance recovery.

Instance Recovery Protocol

Performing Instance Recovery

Oracle Database performs instance recovery automatically in the following situations:

- The database opens for the first time after the failure of a single-instance database or all instances of an Oracle RAC database. This form of instance recovery is also called crash recovery. Oracle Database recovers the online redo threads of the terminated instances together.

Instance Recovery Protocol

Performing Instance Recovery

- Some but not all instances of an Oracle RAC database fail. Instance recovery is performed automatically by a surviving instance in the configuration.

The SMON background process performs instance recovery, applying online redo automatically. No user intervention is required

Instance Recovery Protocol

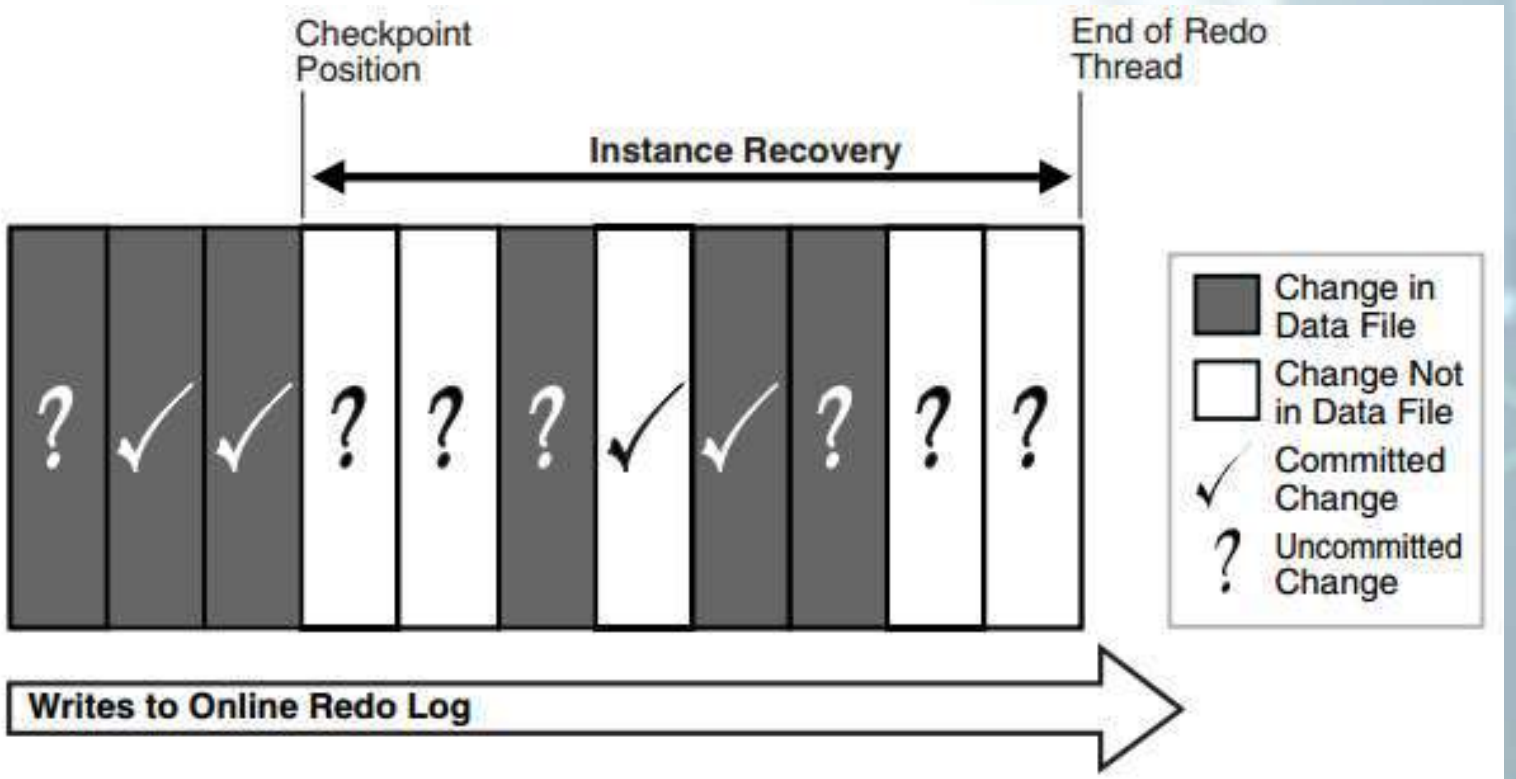
Importance of CKPT for Instance Recovery

- Instance recovery uses checkpoints to determine which changes must be applied to the data files. The checkpoint position guarantees that every committed change with an SCN lower than the checkpoint SCN is saved to the data files.

Instance Recovery Protocol

Importance of CKPT for Instance Recovery

During instance recovery, the database must apply the changes that occur between the checkpoint position and the end of the redo thread.



Introduction to Database Administration



Managing Recovery using MS SQL Server

Managing Recovery using MS SQL Server

MS SQL Server's Recovery

- Restore & Recovery Overview
- Overview of Restore Scenarios
- Steps to Restore a Database
- Accelerated Database Recovery
- Steps to Restore a Database
- Decision to adopt ADR
- To View & Change the Recovery Model
  - Transact SQL
- Without ADR, DB Recovery Follows ARIES
- DB Recovery Follows ARIES using ADR

Managing Recovery using MS SQL Server

Restore & Recovery Overview

To recover a SQL Server database from a failure, a database administrator has to restore a set of SQL Server backups in a logically correct and meaningful restore sequence.:

- The database (a complete database restore)
- The data file (a file restore)
- The data page (Bulk load, restore individual DB)

Managing Recovery using MS SQL Server

Overview of Restore Scenarios

A *restore scenario* in SQL Server is the process of restoring data from one or more backups and then recovering the database. The supported restore scenarios depend on the recovery model of the database and the edition of SQL Server.

- Complete DB Restore
- File Restore (individual files)

Managing Recovery using MS SQL Server

Steps to Restore a Database

To perform a database restore, the Database Engine executes three steps:

- Creates the database and transaction log files if they do not already exist.
- Copies all the data, log, and index pages from the backup media of a database to the database files.
- Applies the transaction log in what is known as the recovery process.

Managing Recovery using MS SQL Server

Accelerated Database Recovery

- Accelerated Database Recovery (ADR) improves database availability, especially in the presence of long running transactions, by redesigning the SQL database engine recovery process. ADR is new in SQL Server 2019.
- ADR is available for Azure SQL Database, Azure SQL Managed Instance and Azure Synapse SQL.

Managing Recovery using MS SQL Server

Steps to Restore a Database

To perform a file restore, the Database Engine executes two steps:

- Creates any missing database file(s).
- Copies the data from the backup devices to the database file(s).

If you restore a file, you cannot recover it and bring it online until it has been rolled far enough forward to be consistent with the database.

Managing Recovery using MS SQL Server

Decision to adopt ADR

The following types of customers should consider enabling ADR:

- Customers that have workloads with long running transactions.
- Customers that have seen cases where active transactions are causing the transaction log to grow significantly.
- Customers that have experienced long periods of database unavailability due to SQL Server long running recovery (such as unexpected SQL Server restart or manual transaction rollback).

Managing Recovery using MS SQL Server

To View & Change the Recovery Model - Transact SQL

Connect to the Database Engine. From the Standard bar, click **New Query**.
Copy and paste the following example into the query window and click **Execute**. This example shows how to query the sys.databases catalog view to learn the recovery model of the **model** database.

SQL

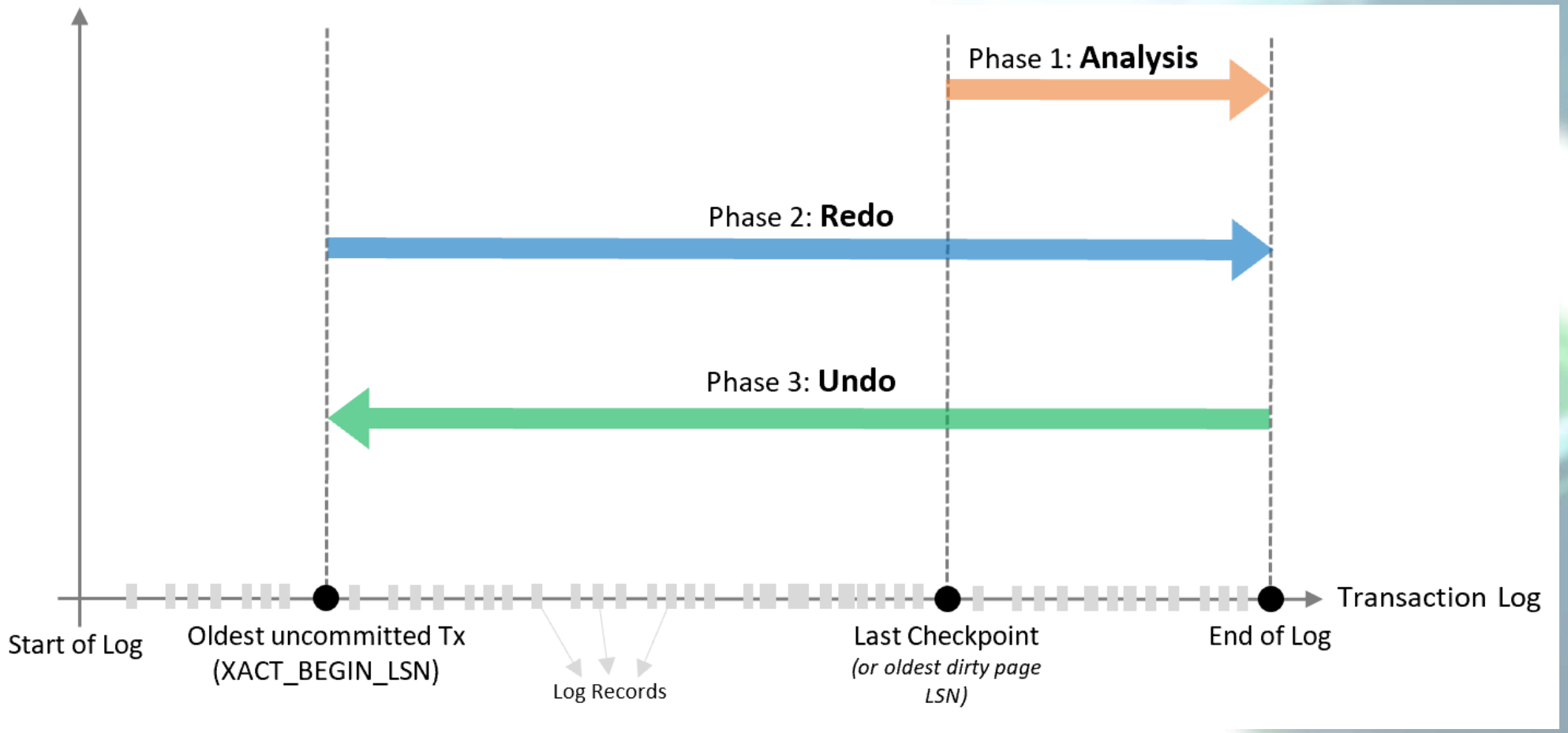
```
SELECT name, recovery_model_desc
   FROM sys.databases
      WHERE name = 'model' ;
GO
```

SQL

```
USE [master] ;
ALTER DATABASE [model] SET RECOVERY FULL ;
```

Managing Recovery using MS SQL Server

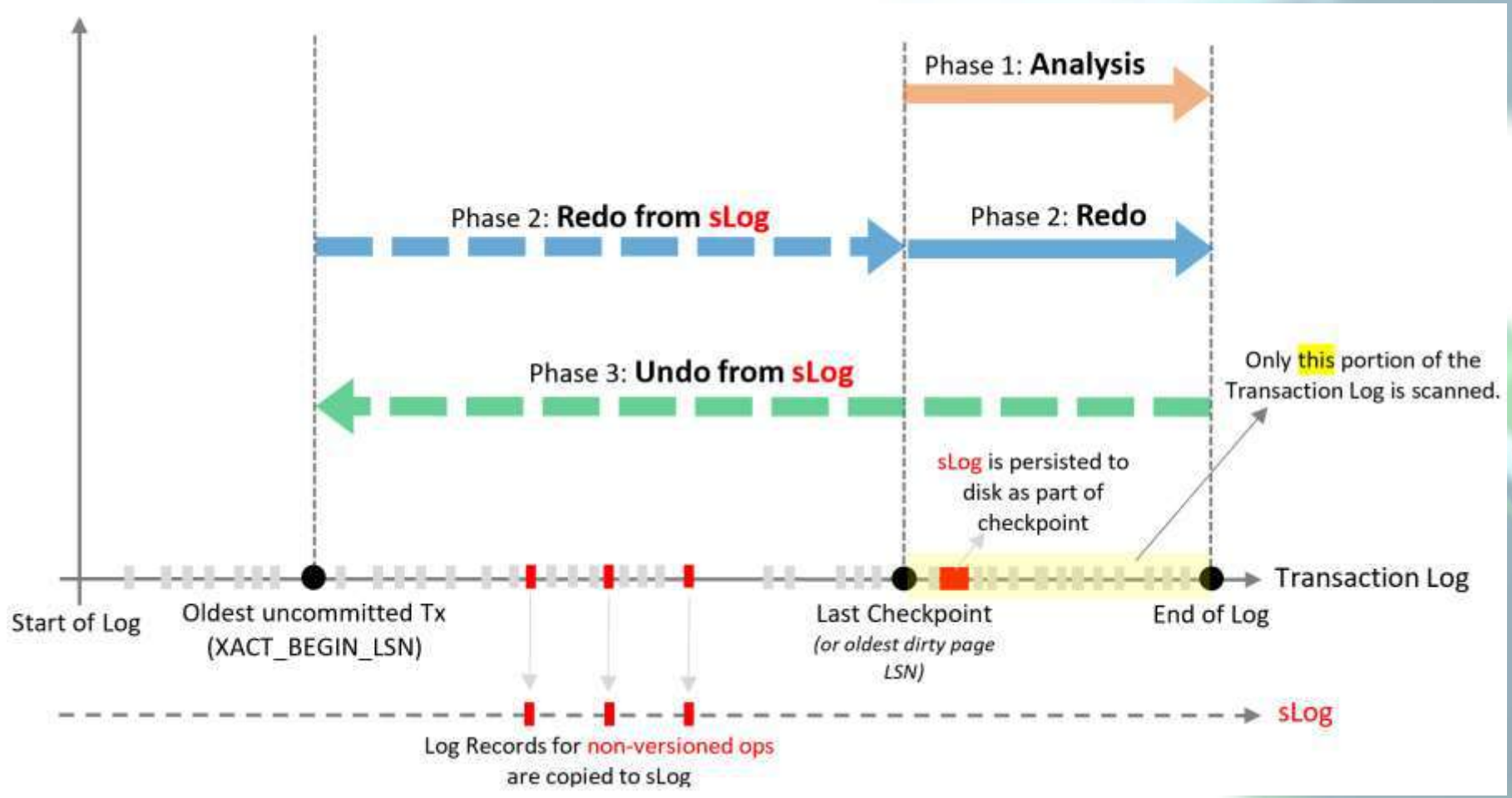
Without ADR, DB Recovery Follows ARIES



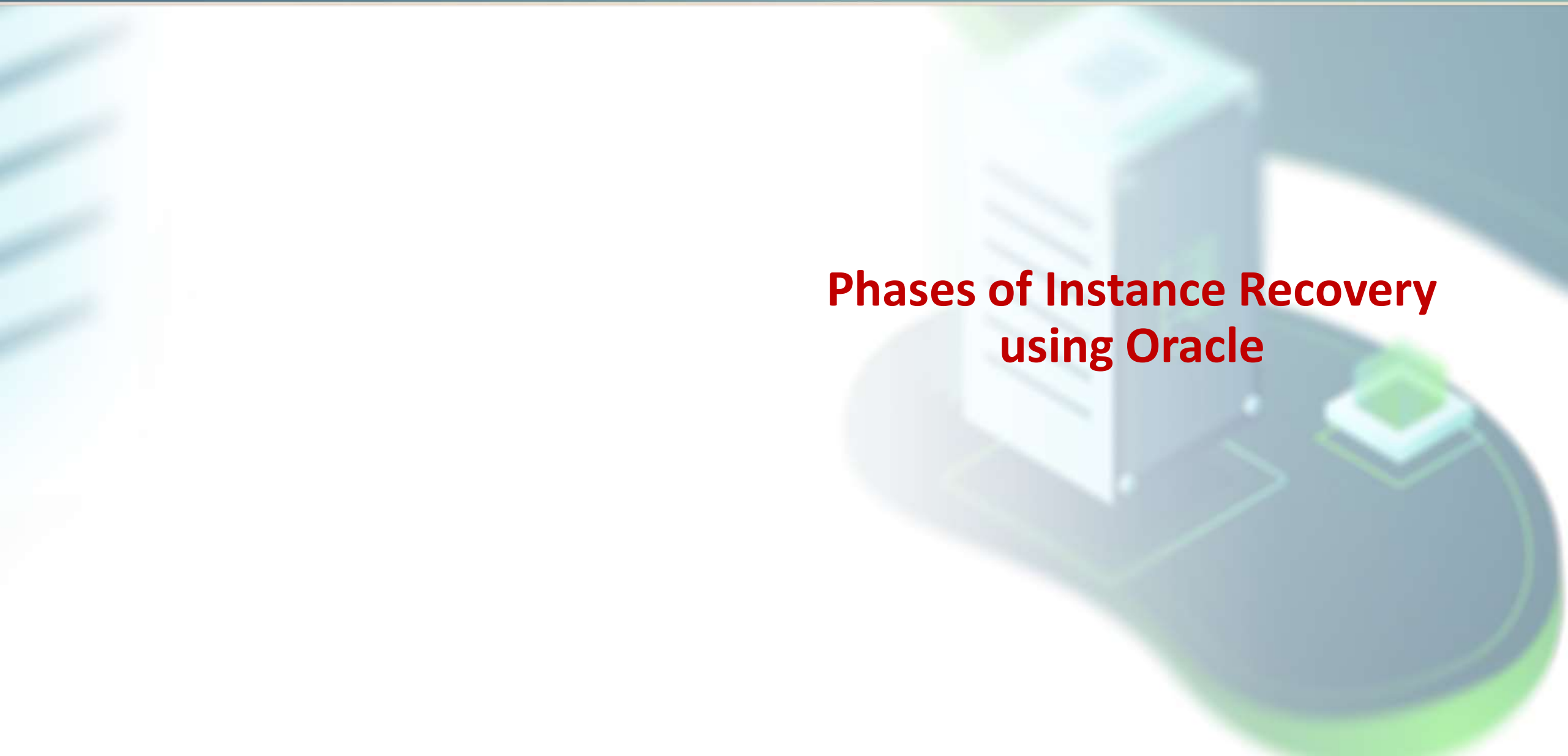
docs.microsoft.com

Managing Recovery using MS SQL Server

DB Recovery Follows ARIES using ADR



docs.microsoft.com



Phases of Instance Recovery using Oracle

Phases of Instance Recovery

Phases of Instance Recovery

- Introduction
- Rolling Forward
- After Roll Forward
- Rolling Forward & Rolling Backward

Phases of Instance Recovery

Introduction

The first phase of instance recovery is called cache recovery or rolling forward, and involves reapplying all of the changes recorded in the online redo log to the data files.

Because rollback data is recorded in the online redo log, rolling forward also regenerates the corresponding undo segments.

Phases of Instance Recovery

Rolling Forward

Rolling forward proceeds through as many online redo log files as necessary to bring the database forward in time.
After rolling forward, the data blocks contain all committed changes recorded in the online redo log files. These files could also contain uncommitted changes that were either saved to the data files before the failure, or were recorded in the online redo log and introduced during cache recovery.

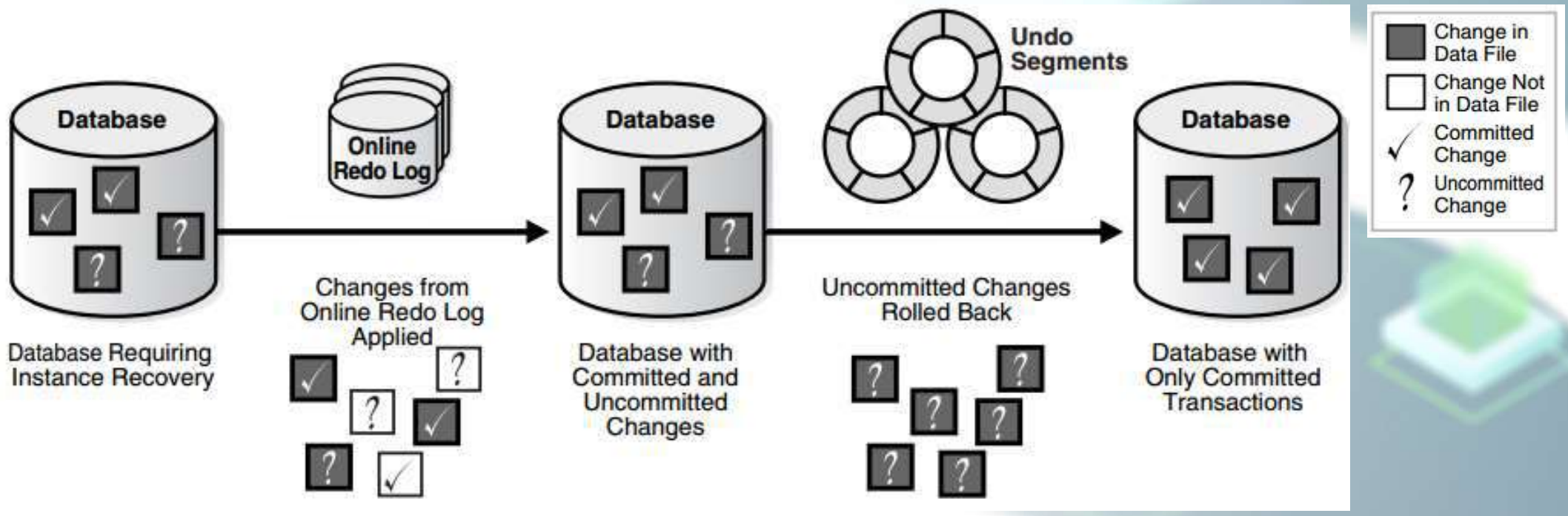
Phases of Instance Recovery

After Roll Forward

After the roll forward, any changes that were not committed must be undone. Oracle Database uses the checkpoint position, which guarantees that every committed change with an SCN lower than the checkpoint SCN is saved on disk. Oracle Database applies undo blocks to roll back uncommitted changes in data blocks that were written before the failure or introduced during cache recovery. This phase is called rolling back or transaction recovery.

Phases of Instance Recovery

Rolling Forward & Rolling Backward





Overview of Parameter Files with commands

Overview of Parameter Files with commands

Overview of Parameter Files with commands

- Introduction
- Initialization Parameters
- Functional Group of Initialization Parameters
- Purpose of Variable Parameters
- Basic Initialization Parameters
- Advance Initialization Parameters
- Parameters during Installation
- Parameters Files spfile and pfile

Overview of Parameter Files with commands

Introduction

To start a database instance, Oracle Database must read either a server parameter file, which is recommended, or a text initialization parameter file, which is a legacy implementation. These files contain a list of configuration parameters.

Overview of Parameter Files with commands

Initialization Parameters

- Initialization parameters are configuration parameters that affect the basic operation of an instance. The instance reads initialization parameters from a file at startup.
- Oracle Database provides many initialization parameters to optimize its operation in diverse environments. Only a few of these parameters must be explicitly set because the default values are adequate in most cases.

Overview of Parameter Files with commands

Functional Group of Initialization Parameters

Most initialization parameters belong to one of the following functional groups:

- Parameters that name entities such as files or directories
- Parameters that set limits for a process, database resource, or the database itself
- Parameters that affect capacity, such as the size of the SGA (these parameters are called variable parameters)

Overview of Parameter Files with commands

Purpose of Variable Parameters

Variable parameters are of particular interest to database administrators because they can use these parameters to improve database performance.

Overview of Parameter Files with commands

Basic Initialization Parameters

In most cases, you must set and tune only the approximately 30 basic parameters to obtain reasonable performance. The basic parameters set characteristics such as the database name,
locations of the control files,
database block size, and
undo tablespace

Overview of Parameter Files with commands

Advance Initialization Parameters

- In rare situations, modification to the advanced parameters may be required for optimal performance.
- The advanced parameters enable expert DBAs to adapt the behavior of the Oracle Database to meet unique requirements

Overview of Parameter Files with commands

Parameters during Installation

Oracle Database provides values in the starter initialization parameter file provided with your database software, or as created for you by the Database Configuration Assistant.

```
db_name=sample
control_files=/disk1/oradata/sample_cf.dbf
db_block_size=8192
open_cursors=52
undo_management=auto
shared_pool_size=280M
pga_aggregate_target=29M
.
.
```

Overview of Parameter Files with commands

Parameters during Installation

create spfile from pfile;

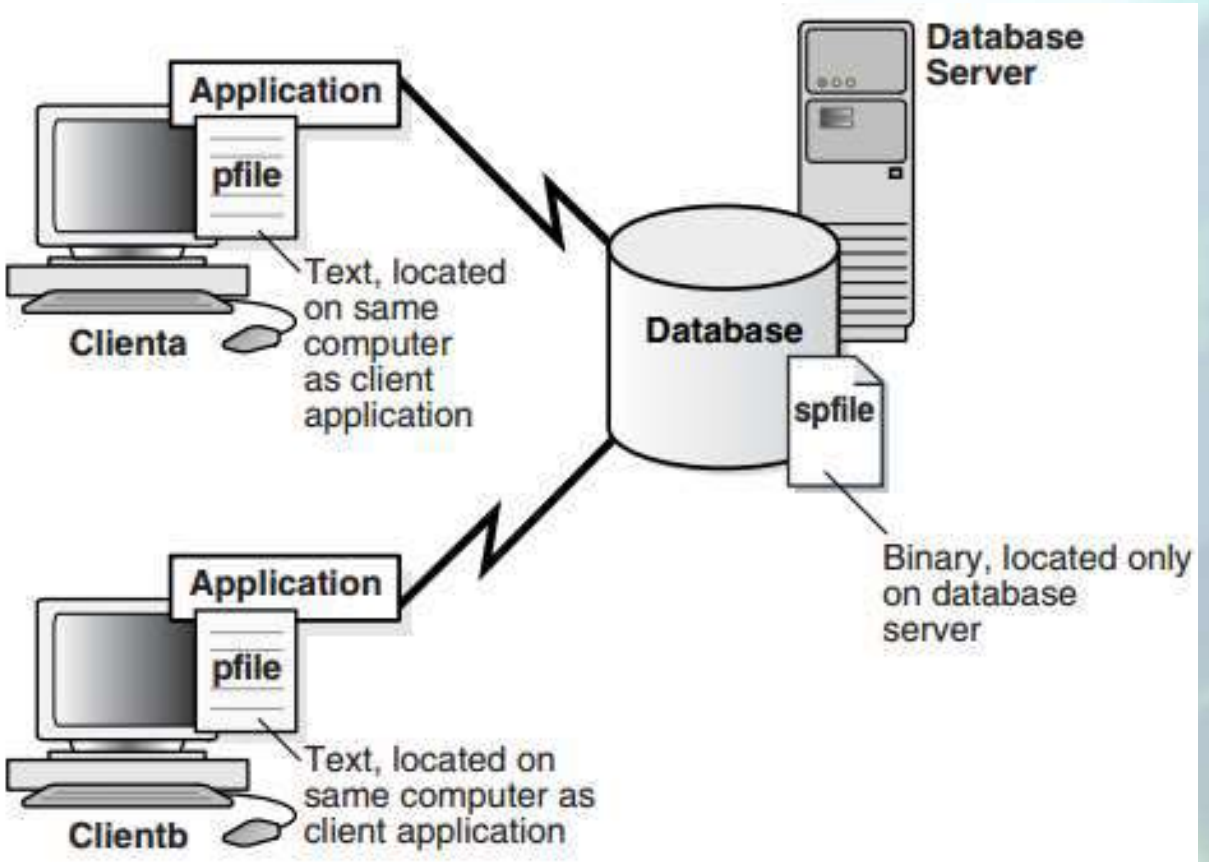
create pfile from spfile;

show parameter processes;

show parameter spfile;

Overview of Parameter Files with commands

Parameters Files spfile and pfile





Initializing Parameter Files

Initializing Parameter Files

Initializing Parameter Files

- Server's Parameters Characteristics
- Text Parameters File
- Text Parameters Characteristics
- Use of alter system command
- Static and Dynamic Parameters

Initializing Parameter Files

Server's Parameters Characteristics

- Only one server parameter file (spfile) exists for a database. This file must reside on the database host.
- The server parameter file is written to and read by only by Oracle Database, not by client applications.
- The server parameter file is binary and cannot be modified by a text editor.

Initializing Parameter Files

Server's Parameters Characteristics

- Initialization parameters stored in the server parameter file are persistent. Any changes made to the parameters while a database instance is running can persist across instance shutdown and startup.

Initializing Parameter Files

Text Parameters File

- A text initialization parameter file (pfile) is a text file that contains a list of initialization parameters. It is readable and writeable to DBA and its changes are reflected into spfile on Server.
- The text initialization parameter file contains a series of key=value pairs, one per line

Initializing Parameter Files

Text Parameters Characteristics

- When starting up or shutting down a database, the text initialization parameter file must reside on the same host as the client application that connects to the database.
- A text initialization parameter file is text-based, not binary.
- Oracle Database can read but not write to the text initialization parameter file. To change the parameter values you must manually alter the file with a text editor.

Initializing Parameter Files

Text Parameters Characteristics

- Changes to initialization parameter values by ALTER SYSTEM are only in effect for the current instance. You must manually update the text initialization parameter file and restart the instance for the changes to be known.

Initializing Parameter Files

```
global_txn_processes                integer     1
job_queue_processes                 integer     1000
log_archive_max_processes           integer     4
processes                           integer     150
SQL> shutdown immediate
Database closed.
Database dismounted.
ORACLE instance shut down.
SQL> startup  open
ORACLE instance started.

Total System Global Area 1071333376 bytes
Fixed Size                  1375792 bytes
Variable Size             696254928 bytes
Database Buffers          369098752 bytes
Redo Buffers                4603904 bytes
Database mounted.
Database opened.
SQL> show parameter processes;

NAME                                 TYPE        VALUE
------------------------------------ ----------- ----------
aq_tm_processes                      integer     0
db_writer_processes                  integer     1
gcs_server_processes                 integer     0
global_txn_processes                 integer     1
job_queue_processes                  integer     1000
log_archive_max_processes            integer     4
processes                            integer     200
SQL>
```

Use of Alter System command

Alter system set processes=200 scope=spfile;

create pfile='E:\app\Haider\product\11.2.0\dbhome_1\database\pfileORCL.ora' from SPFILE;

This change can be seen by opening spfile

Initializing Parameter Files

Static and Dynamic Parameters

Characteristic	Static	Dynamic
Requires modification of the parameter file (text or server)	Yes	No
Requires database instance restart before setting takes affect	Yes	No
Described as "Modifiable" in *Oracle Database Reference* initialization parameter entry	No	Yes
Modifiable only for the database or instance	Yes	No

Static parameters include DB_BLOCK_SIZE, DB_NAME, and COMPATIBLE. Dynamic parameters are grouped into session-level parameters, which affect only the current user session, and system-level parameters, which affect the database and all sessions.

MEMORY_TARGET=system-level & NLS_DATE_FORMAT=session level parameters



Diagnostic Instance Files

Diagnostic Instance Files

Diagnostic Instance Files

- Overview of Diagnostic Files
- Goals of fault Diagnosability Infrastructure
- Automatic Diagnostic Repository
- Problems & Incidence
- DDL & DEBUG Log files - Example
- ADR Directory Structure

Diagnostic Instance Files

Overview of Diagnostic Files

Oracle Database includes a fault diagnosability infrastructure for preventing, detecting, diagnosing, and resolving database problems. Problems include critical errors such as code bugs, metadata corruption, and customer data corruption.

Diagnostic Instance Files

Goals of fault Diagnosability Infrastructure

- Detecting problems proactively
- Limiting damage and interruptions after a problem is detected
- Reducing problem diagnostic and resolution time
- Simplifying customer interaction with Oracle Support

Diagnostic Instance Files

Automatic Diagnostic Repository

Automatic Diagnostic Repository (ADR) is a file-based repository that stores database diagnostic data such as trace files, the alert log, and Health Monitor reports. Key characteristics of ADR include:

- Unified directory structure
- Consistent diagnostic data formats
- Unified tool set

Diagnostic Instance Files

Problems & Incidence

- ADR proactively tracks problems, which are critical errors in the database. Critical errors manifest as internal errors, such as ORA-600, or other severe errors. Each problem has a problem key, which is a text string that describes the problem.

Diagnostic Instance Files

Problems & Incidence

- When a problem occurs multiple times, ADR creates a time-stamped incident for each occurrence. An incident is uniquely identified by a numeric incident ID. When an incident occurs, ADR sends an incident alert to Enterprise Manager. Diagnosis and resolution of a critical error usually starts with an incident alert.
- ADR informs you that a critical error is ongoing without overloading the system with diagnostic data

Diagnostic Instance Files

DDL & DEBUG Log Files - Example

DDL Log: In the alert log file, DDL statements can be recorded when using parameter ENABLE_DDL_LOGGING to TRUE with NOMOUNT startup of database (in Oracle 11g version).
Following commands can be recorded with TRUE option:

Create Table
Create Index
Create User
Drop Table

Diagnostic Instance Files

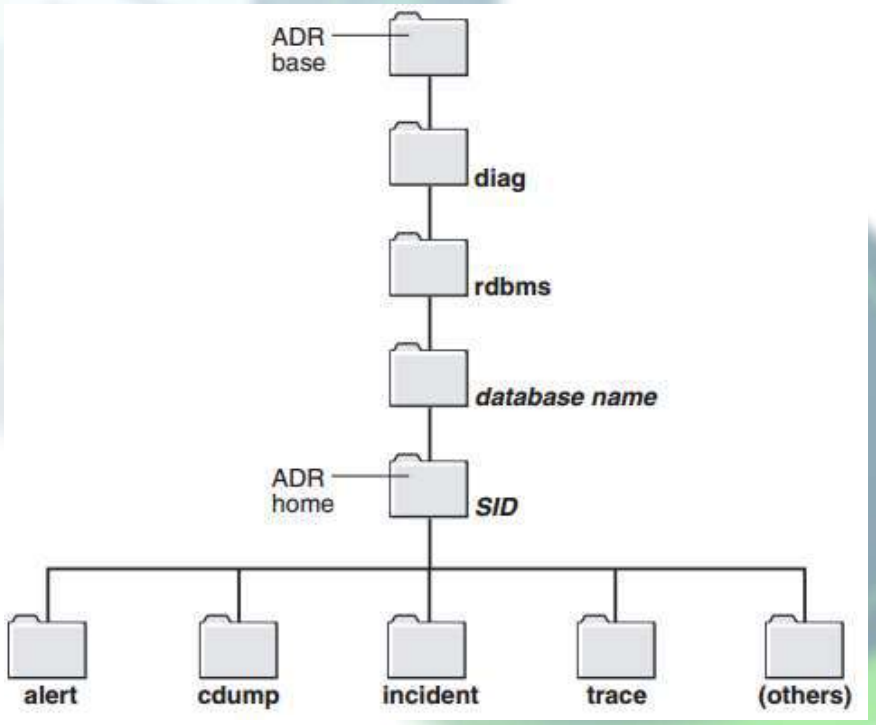
ADR Directory Structure

The ADR base is the ADR root directory. The ADR base can contain multiple ADR homes, where each ADR home is the root directory for all diagnostic data—traces, dumps, the alert log, and so on—for an instance of an Oracle product or component.

```
SQL> SELECT substr(NAME,1,30) Diag_name, substr(VALUE, 1, 50) diag_val
  2  FROM V$DIAG_INFO;

DIAG_NAME                      DIAG_VAL
------------------------------ --------------------------------------------------
Diag Enabled                   TRUE
ADR Base                       e:\app\haider
ADR Home                       e:\app\haider\diag\rdbms\orcl\orcl
Diag Trace                     e:\app\haider\diag\rdbms\orcl\orcl\trace
Diag Alert                     e:\app\haider\diag\rdbms\orcl\orcl\alert
Diag Incident                  e:\app\haider\diag\rdbms\orcl\orcl\incident
Diag Cdump                     e:\app\haider\diag\rdbms\orcl\orcl\cdump
Health Monitor                 e:\app\haider\diag\rdbms\orcl\orcl\hm
Default Trace File             e:\app\haider\diag\rdbms\orcl\orcl\trace\orcl_ora_
Active Problem Count           0
Active Incident Count          0

11 rows selected.
```



Introduction to Database Administration



Commands & Log Alerts

Commands & Log Alerts

- Overview of Alert File
- Alert Log File
- Alert Log File – Contents
- Alert Log File - Maintenance
- Alert Log File - Location

Commands & Log Alerts

Overview of Alert File

Each database has an alert log, which is an XML and text file containing a chronological log of database messages and errors. The alert log contents include the following:

- Deadlock occurs
- Any error written by Oracle Server process
- Any unexpected messages and corruptions happened
- Any DBA DDL operations are managed

Commands & Log Alerts

Alert Log File

The Alert log file (ALERT.LOG) is a chronological log of messages and errors written out by an Oracle Database

This file should constantly be monitored to detect unexpected messages and corruptions

Commands & Log Alerts

Alert Log File - Contents

- Non default initialization parameter used at startup or
- internal errors (Ora-600)
- Block corruption error (Ora-1578)
- DDL and STARTUP, SHUTDOWN, ARCHIVE LOG, and RECOVER

Commands & Log Alerts

Alert Log File - Contents

The alert log file contains the following information that can be used to aid in tuning the database

- Time to perform archiving
- Instance recovery start and complete times
- Deadlock and timeout errors
- Incomplete checkpoints
- Checkpoint start and end times

Commands & Log Alerts

Alert Log File - Maintenance

As Alert file grows bigger,

- DBA can periodically backup the alert log file and delete the current file

When database attempts to write to the alert file again, it will create a new file

Commands & Log Alerts

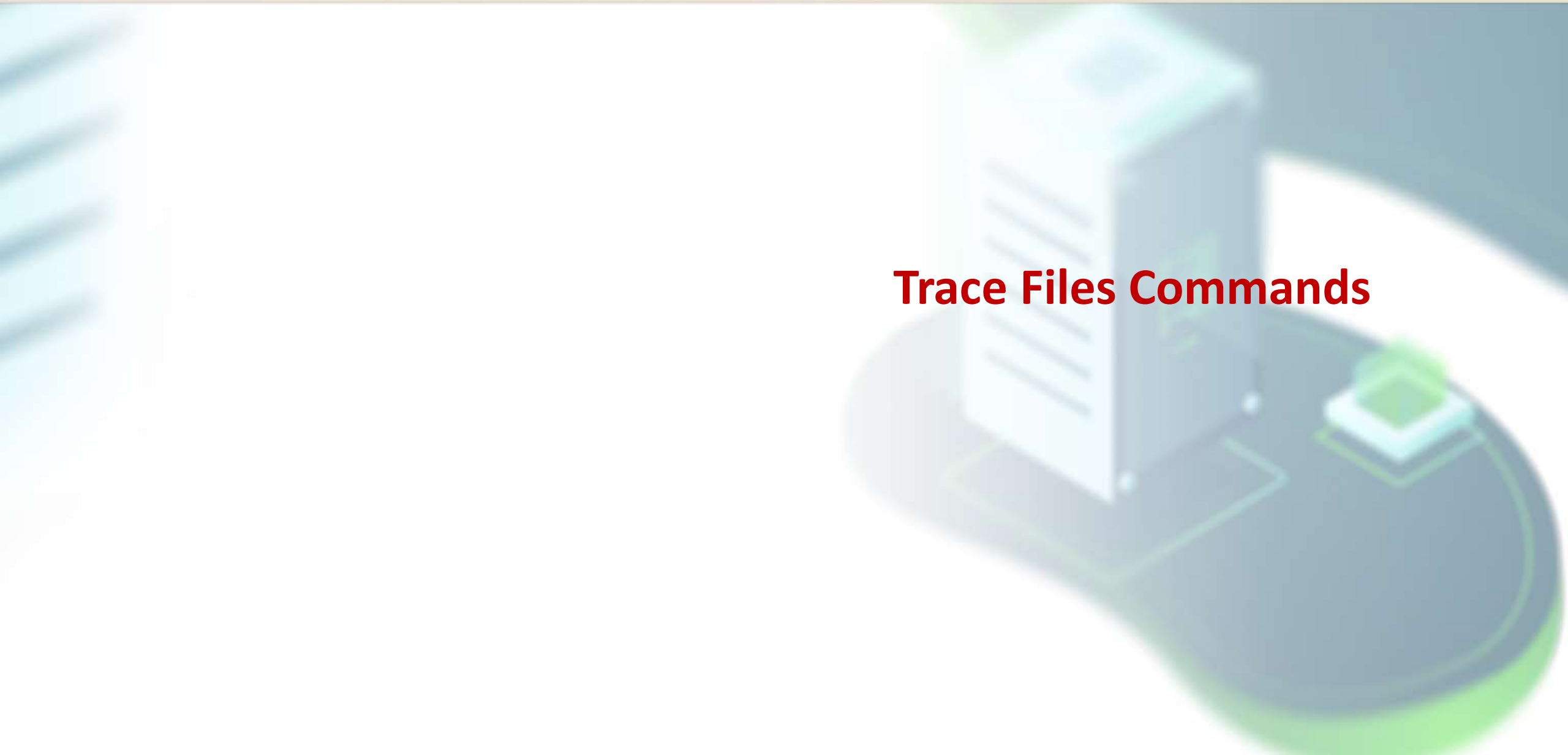
Alert Log File - Location

Each database has an Alert_SID.log file

Resides on Server with the database

Default location
Oracle-
Base/diag/rdbms/DB_Name/SID/trace

End



Trace Files Commands

Trace Files Commands

Trace Files Commands

- Overview of Trace Files
- Types of Trace Files
- Contents of Trace Files
- Locations of Trace Files

Trace Files Commands

Overview of Trace Files

A trace file is an administrative file that contain diagnostic data used to investigate problems. Also, trace files can provide guidance for tuning applications or an instance.

Trace Files Commands

Types of Trace Files

- Each server and background process can periodically write to an associated trace file. The files information on the process environment, status, activities, and errors.
- The SQL trace facility also creates trace files, which provide performance information on individual SQL statements.

Trace Files Commands

Types of Trace Files

- To enable tracing for a client identifier, service, module, action, session, instance, or database, you must execute the appropriate procedures in the DBMS_MONITOR package or use Oracle Enterprise Manager.
- A dump is a special type of trace file. Whereas a trace tends to be continuous output of diagnostic data, a dump is typically a one-time output of diagnostic data in response to an event (such as an incident).

Trace Files Commands

Types of Trace Files

- When an incident occurs, the database writes one or more dumps to the incident directory created for the incident. Incident dumps also contain the incident number in the file name

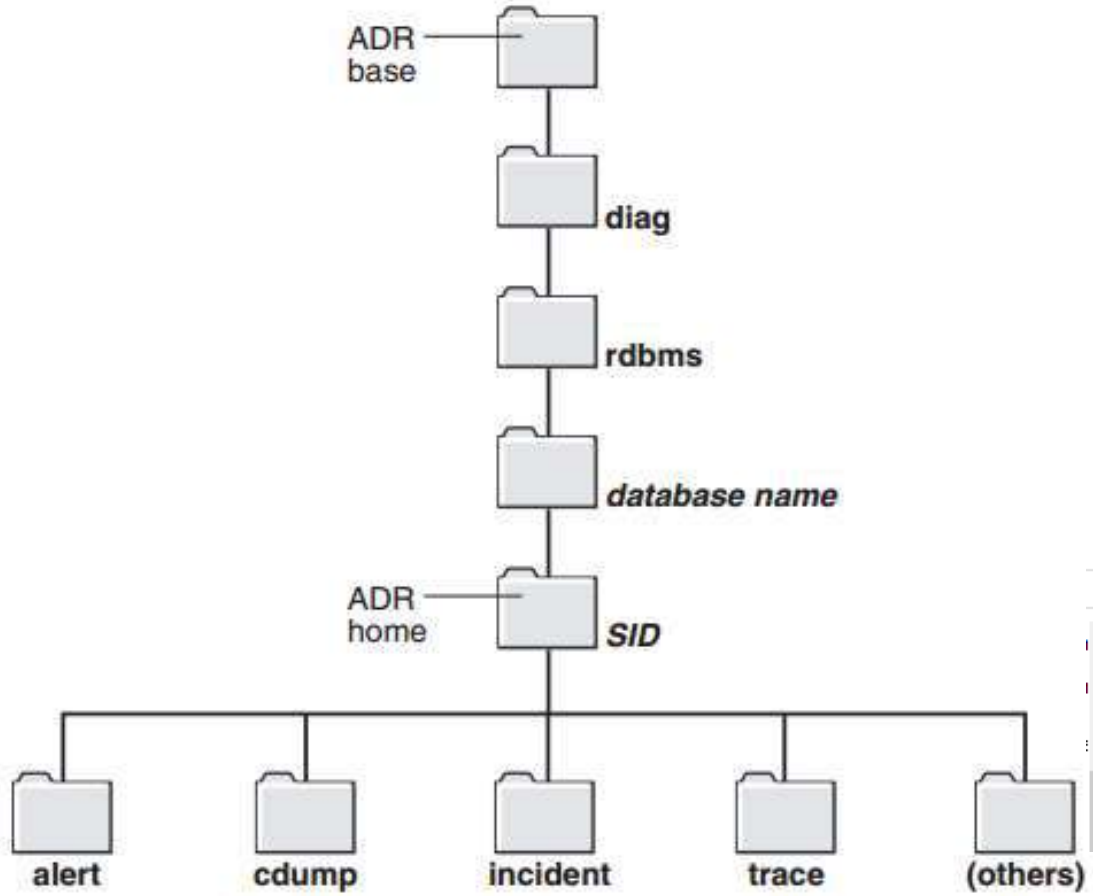
Trace Files Commands

Contents of Trace Files

- Typically, database background process trace file names contain the
- Oracle SID,
- background process name, and
- operating system process number

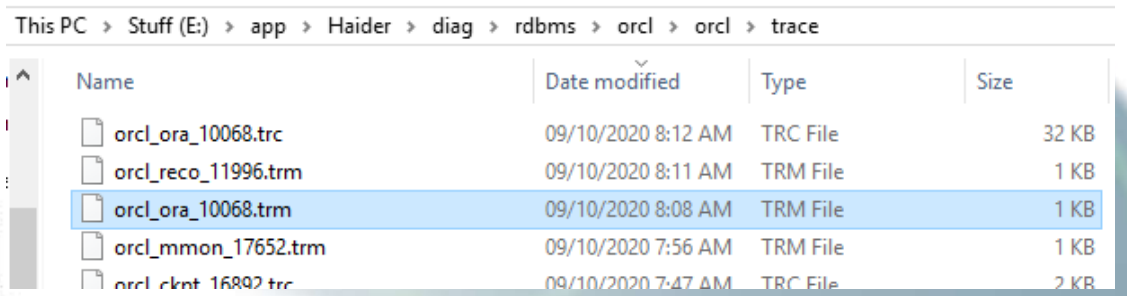
An example of a trace file for the RECO process is mytest_reco_10355.trc

Trace Files Commands

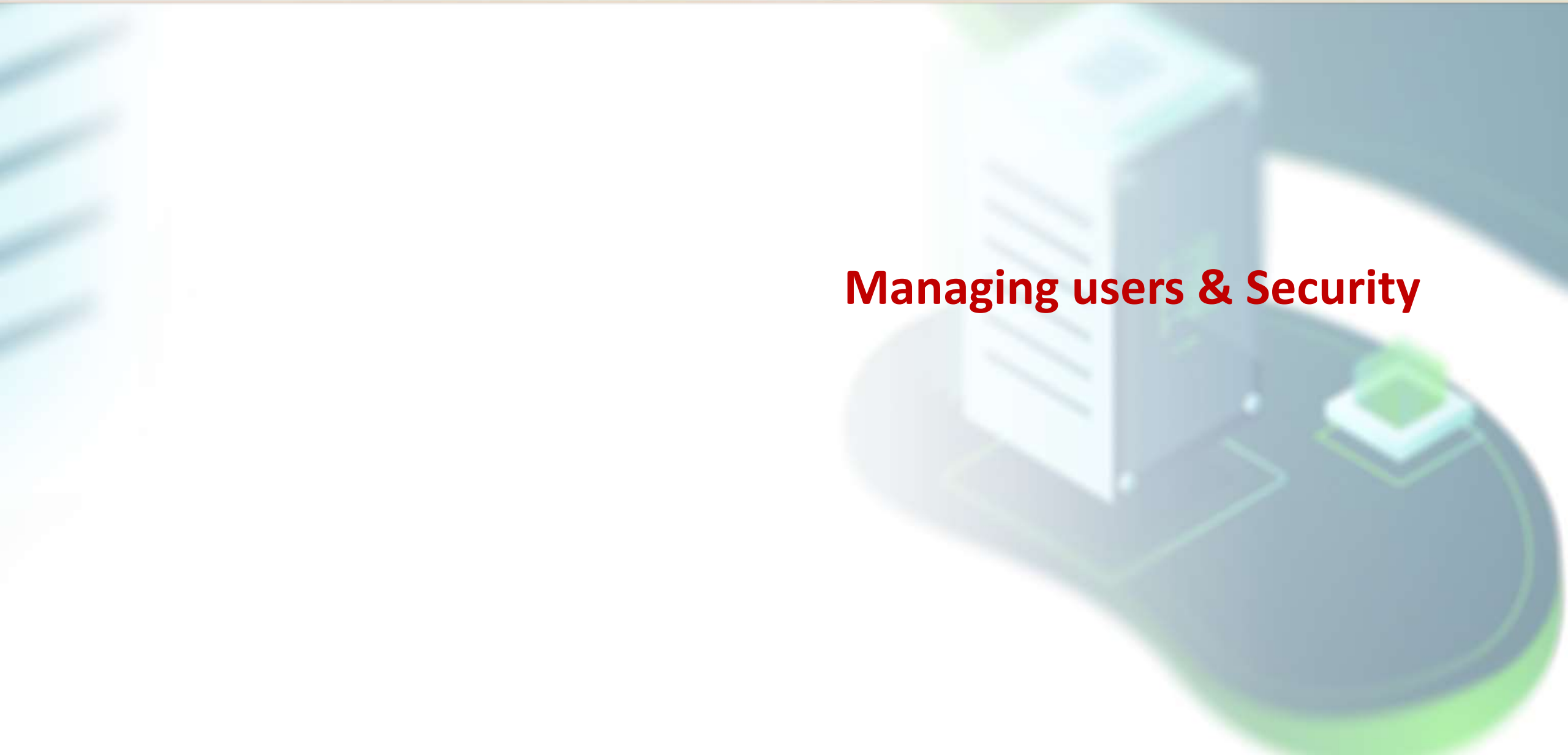


Locations of Trace Files

ADR stores trace files in the trace sub directory, as shown in Figure Trace file names are platform dependent and use the extension .trc.



Introduction to Database Administration



Managing users & Security

Managing users & Security

- Types of Security
- Threats to Databases
- Security Domain
- Schema Objects & User
- User & Database
- Creating Oracle User
- Pre-defined Accounts

Managing users & Security

Types of Security

Following are the database security types and issues

- Legal & Ethical Issues
- Policy Issues in Enterprises
- Systems' related Issues
- Needs of Enterprises

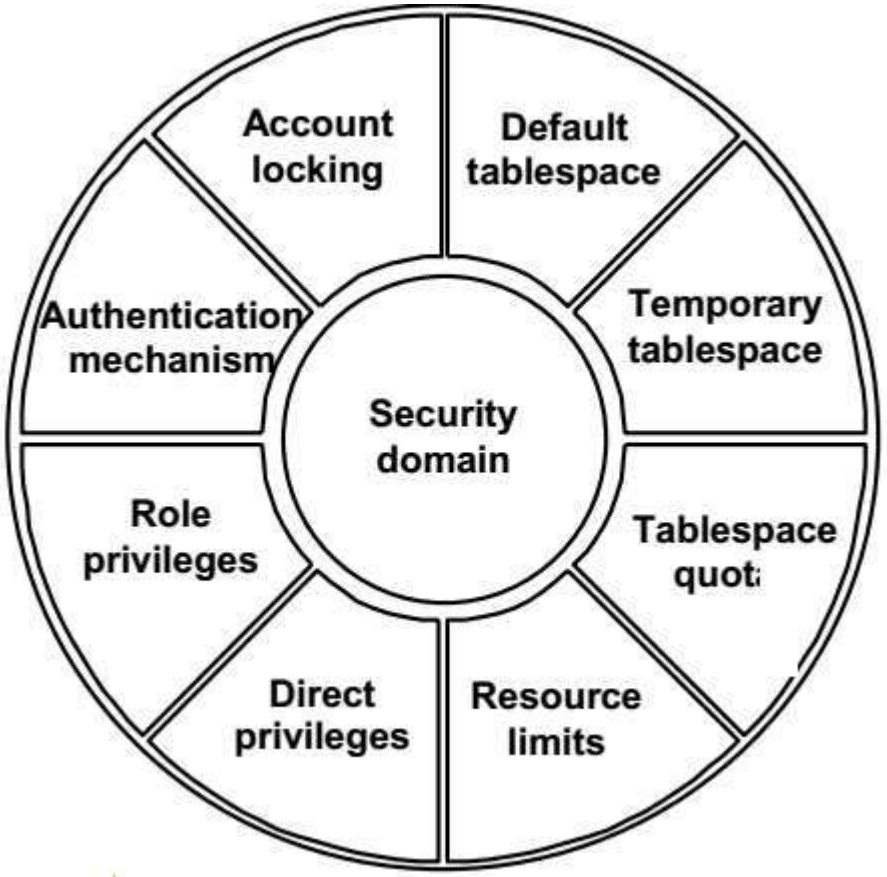
Managing users & Security

Threats to Databases

Following are the database related threats and issues

- Loss of integrity
- Loss of availability
- Loss of Confidentiality

Managing users & Security



Security Domain

The database administrator defines the names of the users who are allowed to access a database. A security domain defines the settings that apply to the user.

Authentication mechanisms are

- Data Dictionary
- Operating System
- Network

Managing users & Security

Schema Objects & User

- A Schema is a collection of objects
- A user is associated (owner) of schema
- Creating schema means creating a user as an owner of schema

Managing users & Security

User & Database

- Accessing Database (connection with the database)
- Specifying database unique user account of ID
- Specifying authentication (roles & privileges) to user account

Allocating space, quota, buffer etc. are additional features in security.

Managing users & Security

Creating Oracle User

- A unique user name or account
- Authentications (roles/ rights)
- A default tablespace
- A temporary tablespace
- A user profile
- An account status (lock/unlock)

Managing users & Security

Pre-defined Accounts

- SYS account (sys as sysdba)
  - is granted the DBA role
  - has all privileges with ADMIN OPTION
  - is used for startup, shutdown etc.
  - owns the data dictionary
- SYSTEM account is granted the DBA role
  - Normally not used for routine operations

End



Creating Users with Commands

Creating Users with Commands

Creating Users with Commands

- Creating Oracle User
- Syntax for Creating Oracle User
- Data Dictionary View DBA_USERS

Creating Users with Commands

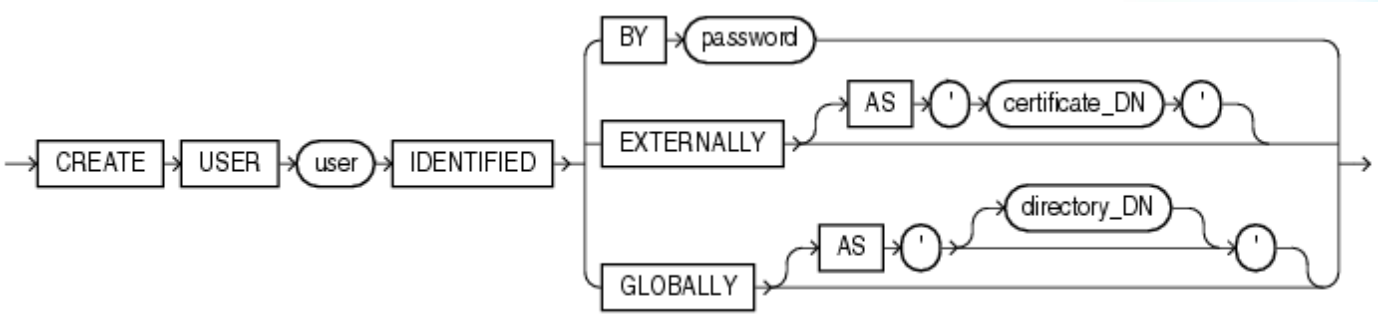
Creating Oracle User

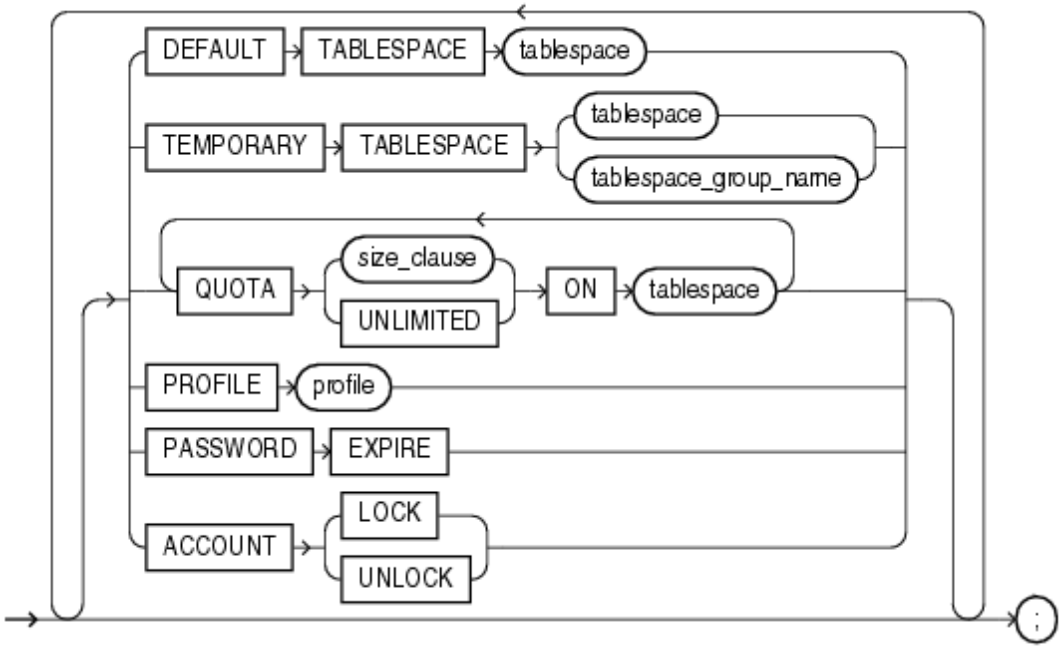
Checklist's clauses

- Tablespaces
- Quota
- Roles/ privileges

Creating Users with Commands

Syntax for Creating Oracle User





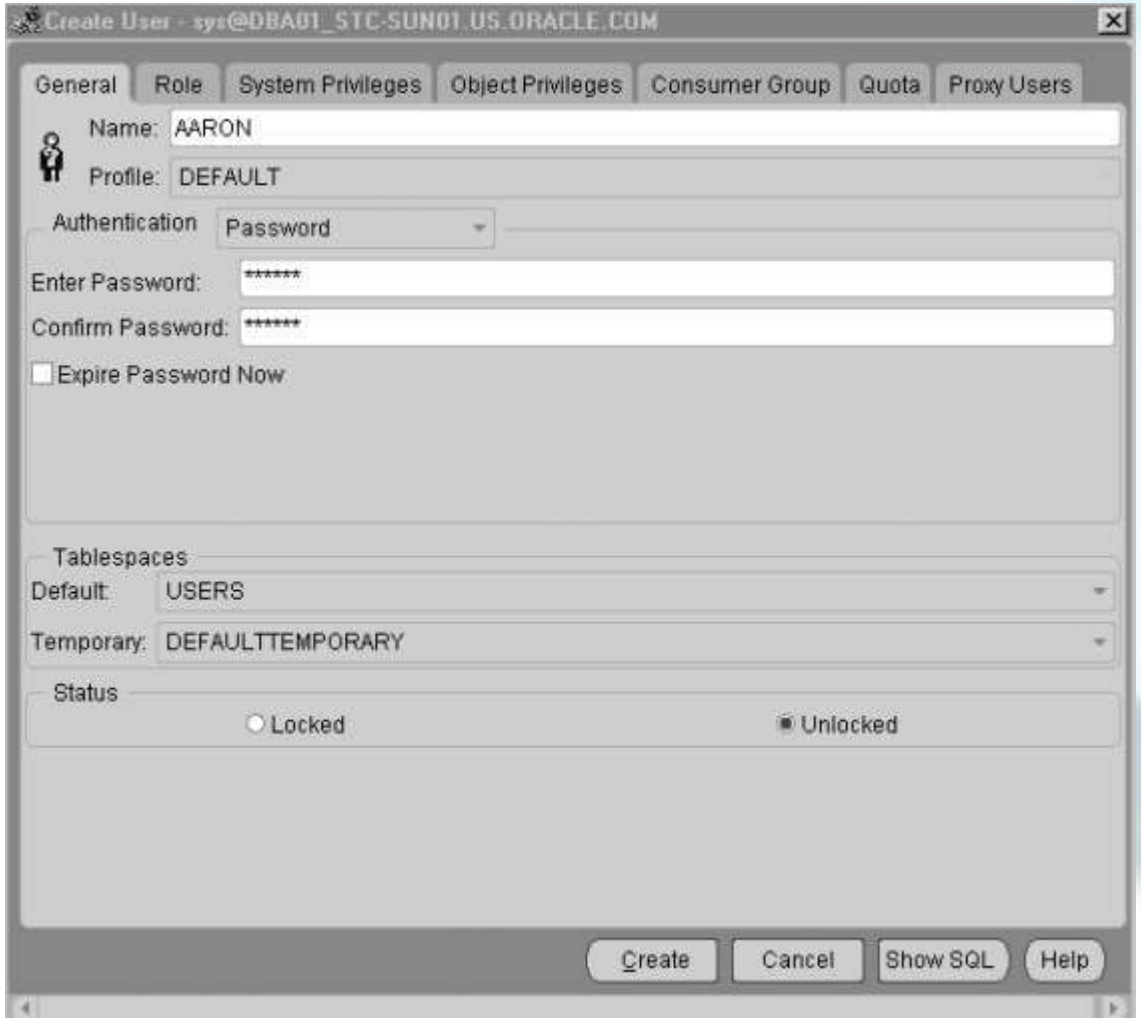

```
CREATE USER sidney
    IDENTIFIED BY out standing1
    DEFAULT TABLESPACE example
    QUOTA 10M ON example
    TEMPORARY TABLESPACE temp
    QUOTA 5M ON system
    PROFILE app_user
    PASSWORD EXPIRE;
```

```
CREATE USER global_user
   IDENTIFIED GLOBALLY AS 'CN=analyst
   DEFAULT TABLESPACE example
   QUOTA 5M ON example;
```

docs.oracle.com

Creating Users with Commands

Creating Oracle User using Oracle-EM



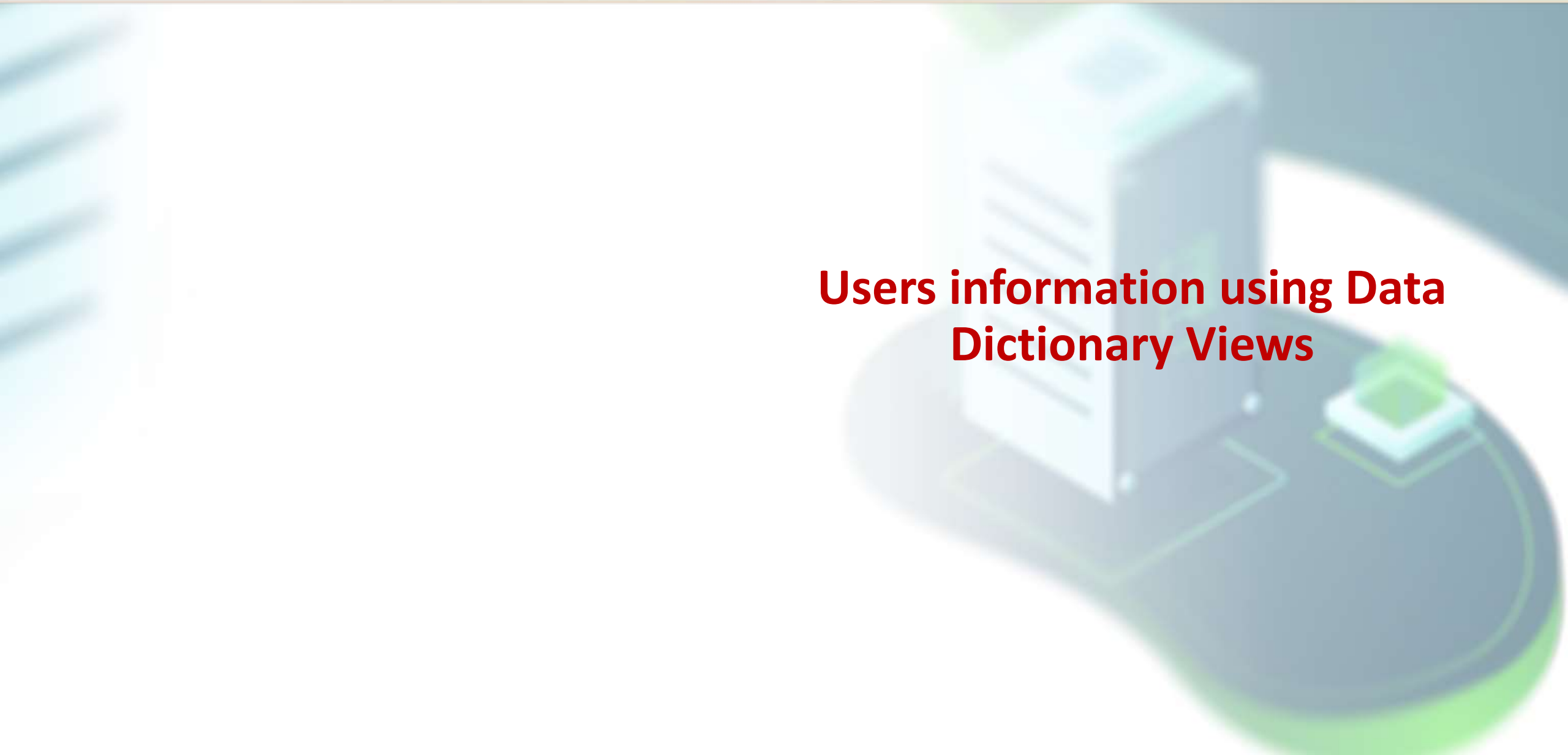
docs.oracle.com

Creating Users with Commands

Data Dictionary View DBA_USERS

```
SQL> select username, account_status, created
  2  from dba_users;

USERNAME                       ACCOUNT_STATUS                   CREATED
------------------------------ -------------------------------- ---------
NOVUS                          OPEN                             18-DEC-17
NOVUSDB                        OPEN                             16-OCT-17
PROD                           OPEN                             19-JUL-17
NOVUSDB2                       OPEN                             13-DEC-17
NEONDEV                        EXPIRED                          23-JAN-17
SCOTT                          OPEN                             02-APR-10
NEONDB                         OPEN                             31-MAY-17
TEST                           OPEN                             14-FEB-17
MDDATA                         EXPIRED                          02-APR-10
ORACLE_OCM                     EXPIRED & LOCKED                 02-APR-10
```



Users information using Data Dictionary Views

Users information using Data Dictionary Views

Users information using Data Dictionary Views

- Searching Security Dictionary Views
- Single SQL to Display All Views
- Listing All Tablespace Quotas
- Listing All Profiles & Assigned Limit

Users information using Data Dictionary Views

Searching Security Dictionary Views

Searching Data Dictionary Views closest to security users' information. Search is managed by following SQL LIKE operator: '%QUOTA%', '%PASSWORD%' '%PROFILE%', '%COST%' '%SESS%', '%PROXY%' and *_USERS

Users information using Data Dictionary Views

Single SQL to Display All Views

```
SQL> SELECT *
  2  FROM DICTIONARY
  3  where table_name like '%QUOTA%'
  4  or table_name like '%PASSWORD%'
  5  or table_name like '%PROFILE%'
  6  or table_name like '%COST%'
  7  or table_name like '%SESS%'
  8  or table_name like '%PROXY%';

TABLE_NAME                     COMMENTS
------------------------------ --------------------------------------------------


DBA_HIST_SESS_TIME_S CPU and I/O time for interesting (STREAMS)
TATS                 sessions

DBA_LOGMNR_SESSION
DBA_PROFILES         Display all profiles and their limits
DBA_SQL_PROFILES     set of sql profiles
DBA_TS_QUOTAS        Tablespace quotas for all users
USER_AUDIT_SESSION   All audit trail records concerning CONNECT and
                     DISCONNECT
```

Users information using Data Dictionary Views

Listing All Tablespace Quotas

```
SELECT * FROM DBA_TS_QUOTAS;

TABLESPACE     USERNAME     BYTES      MAX_BYTES     BLOCKS     MAX_BLOCKS
---------      ---------   --------    ----------    -------    ----------
USERS          JFEE                0       512000          0           250
USERS          DCRANNEY            0           -1          0            -1
```

Users information using Data Dictionary Views

Listing All Profiles & Assigned Limit

```
SELECT * FROM DBA_PROFILES
   ORDER BY PROFILE;

PROFILE              RESOURCE_NAME                  RESOURCE   LIMIT
-------------------- ------------------------------ ---------- ----------
CLERK                COMPOSITE_LIMIT                KERNEL     DEFAULT
CLERK                FAILED_LOGIN_ATTEMPTS          PASSWORD   DEFAULT
CLERK                PASSWORD_LIFE_TIME             PASSWORD   DEFAULT
CLERK                PASSWORD_REUSE_TIME            PASSWORD   DEFAULT
CLERK                PASSWORD_REUSE_MAX             PASSWORD   DEFAULT
CLERK                PASSWORD_VERIFY_FUNCTION       PASSWORD   DEFAULT
CLERK                PASSWORD_LOCK_TIME             PASSWORD   DEFAULT
CLERK                PASSWORD_GRACE_TIME            PASSWORD   DEFAULT
CLERK                PRIVATE_SGA                    KERNEL     DEFAULT
CLERK                CONNECT_TIME                   KERNEL     600
CLERK                IDLE_TIME                      KERNEL     30
```

Introduction to Database Administration



Managing Privileges

Managing Privileges

Managing Privileges

- Types of Oracle User Privileges
- System Privileges
- System Privileges with ADMIN OPTION
- Object Privileges Syntax
- Object Privileges - Examples
- Objects Privileges Options
- Object Privileges using Oracle EM

Managing Privileges

Types of Oracle User Privileges

- System: Enables users to perform particular actions in the database
- Object: Enables users to access and manipulate a specific object

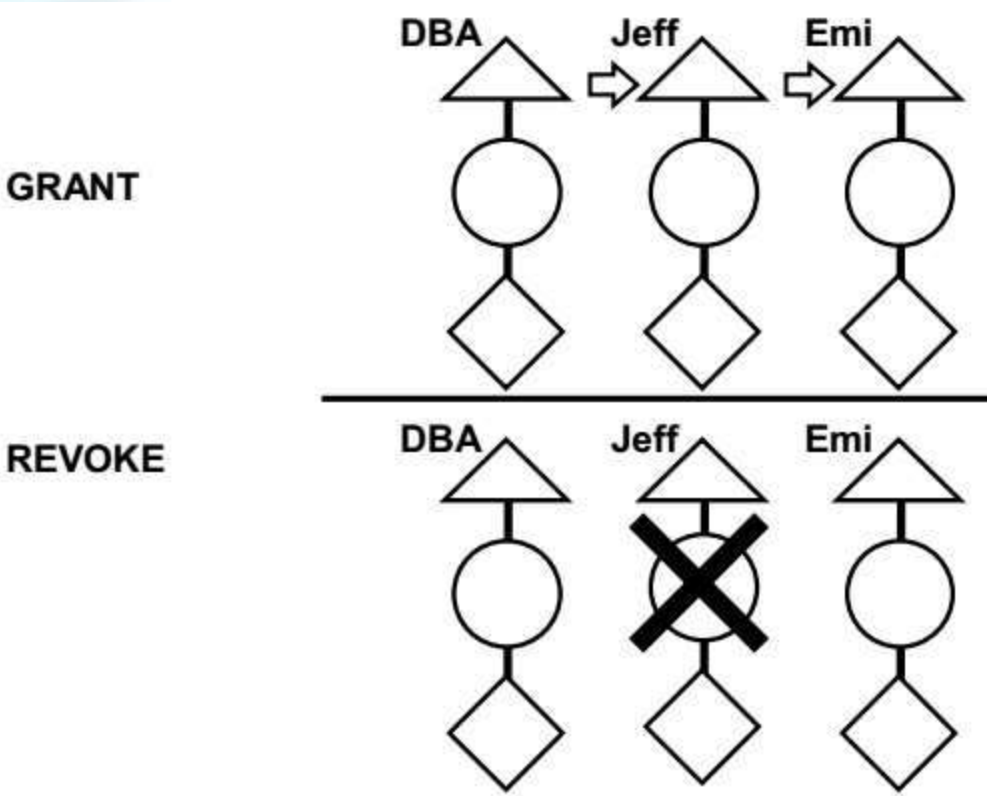
Managing Privileges

System Privileges

- More than 100 distinct system privileges
- ANY keyword in privileges signifies that users have the privilege in any schema
- GRANT command adds a privilege to a user or a group of users
- REVOKE command deletes the privileges

Managing Privileges

System Privileges with ADMIN OPTION



Scenario

1. The DBA grants the `CREATE TABLE` system privilege to Jeff with the `ADMIN OPTION`.
2. Jeff creates a table.
3. Jeff grants the `CREATE TABLE` system privilege to Emi.
4. Emi creates a table.
5. The DBA revokes the `CREATE TABLE` system privilege from Jeff.

The result

Jeff's table still exists, but no new tables can be created.

Emi's table still exists and she still has the `CREATE TABLE` system privilege.

Oracle Internal & OAI

Managing Privileges

Object Privileges Syntax

```
GRANT { object_privilege [(column_list)]
      [, object_privilege [(column_list)] ]...
      |ALL [PRIVILEGES]}
ON    [schema.]object
TO    {user|role|PUBLIC}[, {user|role|PUBLIC} ]
      [WITH GRANT OPTION]
```

Managing Privileges

Object Privileges - Examples

```
grant select on EMP to utest;


grant select, update on DEPT to utest;


grant select on v_emp to utest;

--login with utest
select *
from scott.dept;
```

Managing Privileges

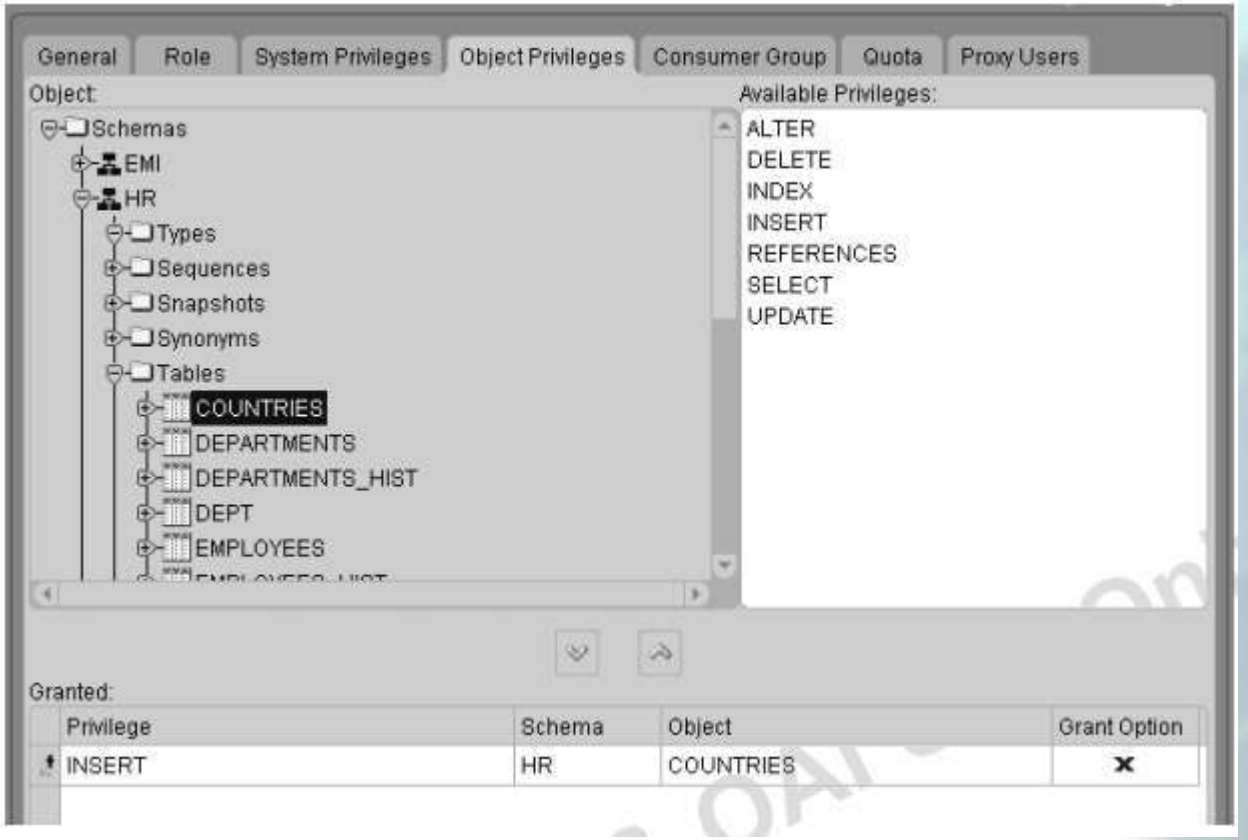
Objects Privileges Options

Object priv.	Table	View	Sequence	Procedure
ALTER	√	√	√	
DELETE	√	√		
EXECUTE				√
INDEX	√	√		
INSERT	√	√		
REFERENCES	√			
SELECT	√	√	√	
UPDATE	√	√		

--scott is the owner of EMP table and granting
GRANT select on EMP to jef;

Managing Privileges

Object Privileges using Oracle EM





System Privileges Commands

System Privileges Commands

System Privileges Commands

- System Privileges
- System Privileges with ADMIN OPTION
- System Privileges - Examples
- System Privileges SYSOPER & SYSDBA
- System Privileges using Oracle EM

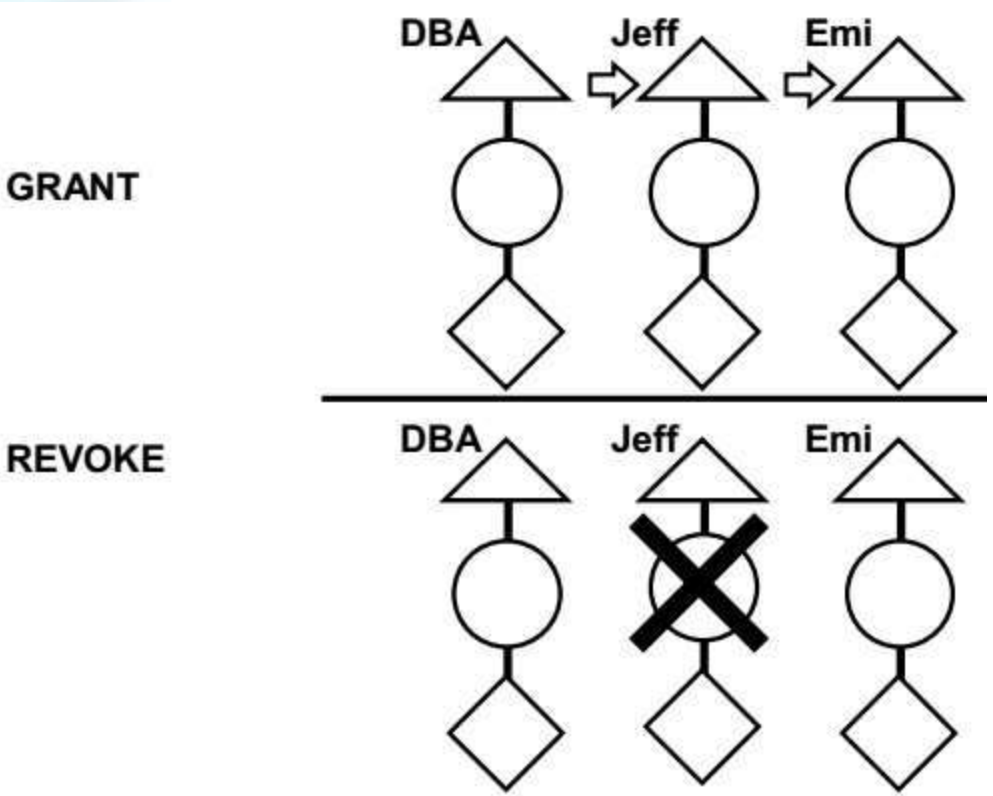
System Privileges Commands

System Privileges

- More than 100 distinct system privileges
- ANY keyword in privileges signifies that users have the privilege in any schema
- GRANT command adds a privilege to a user or a group of users
- REVOKE command deletes the privileges

System Privileges Commands

System Privileges with ADMIN OPTION



Scenario

1. The DBA grants the `CREATE TABLE` system privilege to Jeff with the `ADMIN OPTION`.
2. Jeff creates a table.
3. Jeff grants the `CREATE TABLE` system privilege to Emi.
4. Emi creates a table.
5. The DBA revokes the `CREATE TABLE` system privilege from Jeff.

The result

Jeff's table still exists, but no new tables can be created.

Emi's table still exists and she still has the `CREATE TABLE` system privilege.

Oracle Internal & OA

System Privileges Commands

System Privileges - Examples

Grant create table to utest;

Grant create view, create procedure to utest;

Grant create view to scott;

Revoke create table from utest;

System Privileges Commands

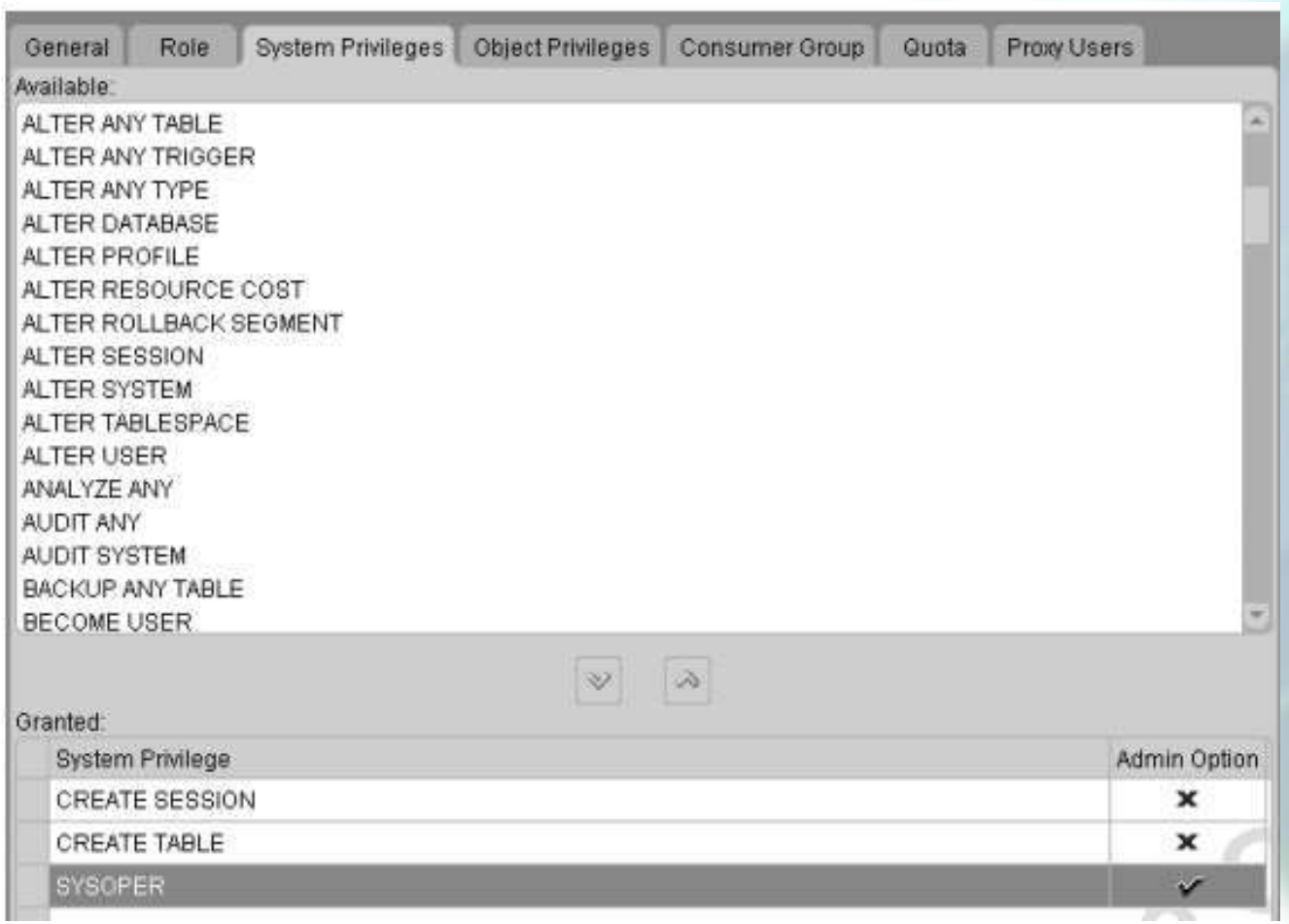
System Privileges SYSOPER & SYSDBA

Category	Examples
SYSOPER	STARTUP SHUTDOWN ALTER DATABASE OPEN | MOUNT ALTER DATABASE BACKUP CONTROLFILE TO RECOVER DATABASE ALTER DATABASE ARCHIVELOG RESTRICTED SESSION
SYSDBA	SYSOPER PRIVILEGES WITH ADMIN OPTION CREATE DATABASE ALTER TABLESPACE BEGIN/END BACKUP RESTRICTED SESSION RECOVER DATABASE UNTIL

--sys is giving grants to utest
GRANT create table, create table to utest;

System Privileges Commands

System Privileges using Oracle EM





System Multiple Privileges Commands

System Multiple Privileges Commands

System Multiple Privileges Commands

- What is User Privileges?
- Giving Privileges to Users
- Script Create Synonym
- Synonym Privileges to Users
- System and Object Privileges

System Multiple Privileges Commands

What is User Privileges?

A user privilege is the right to run a particular type of SQL statement, or the right to access an object that belongs to another user, run a PL/SQL package, and so on.

- System Privileges
- Object Privileges
- Roles

System Multiple Privileges Commands

Giving Privileges to Users

-login sys / as sysdba

create user u1
identified by u1
temporary tablespace temp;

conn u1/u1

--error ...

grant create session, create table to u1;

System Multiple Privileges Commands

Script Create Synonym

Run following SQL in owner's (scott) schema and run each command of script in granting user (say u1). Synonyms will be created in u1 schama.

```
SQL> select 'create synonym '||tname||' for scott.'||tname||';'
  2  from tab;

'CREATESYNONYM'||TNAME||'FORSCOTT.'||TNAME||';'
--------------------------------------------------------------
create synonym BLOBS for scott.BLOBS;
create synonym BONUS for scott.BONUS;
create synonym CAMPUS for scott.CAMPUS;
create synonym CHAINED_ROWS for scott.CHAINED_ROWS;
create synonym CLIENT for scott.CLIENT;
create synonym CUSTOMER for scott.CUSTOMER;
```

System Multiple Privileges Commands

Synonym Privileges to Users

User u1 is using object table as scott.emp, for example

Select * from scott.EMP; --when scott is authorizing select on EMP to u1

User u1 can run the following command
create synonym emp for scott.emp;

select * from tab; --can display EMP as synonym
select * from emp;
(Subject to all assigned privileges)

System Multiple Privileges Commands

System and Object Privileges

SYSTEM PRIVILEGES	OBJECT PRIVILEGES
In System privilege are more than 100 privileges	All DML are come into object privileges.
CREATE SESSION	SELECT
CREATE TABLE	INSERT
CREATE USER	UPDATE
CREATE VIEW	DELETE
CREATE SEQUENCE	REFERENCES
CREATE PROCEDURE	ALTER
CREATE ROLE etc.	INDEX
	EXECUTE



Revoke Privileges

Revoke Privileges

Revoke Privileges

- Data Dictionary for Privileges
- Revoke & Example
- To See Existing Privileges
- Revoke Privileges CASCADE

Revoke Privileges

Data Dictionary for Privileges

We can use string search PRIV to display possible data dictionary views.

For Example,

```
select table_name
from dict
where table_name like '%PRIV%';
```

Revoke Privileges

Data Dictionary for Privileges

We choose following suitable PRIV View:

desc USER_TAB_PRIVS is showing columns

GRANTEE
OWNER
TABLE_NAME
GRANTOR
PRIVILEGE
GRANTABLE
HIERARCHY

Revoke Privileges

Revoke & Example

The REVOKE statement is used to revoke object privileges.

- You previously granted the object privilege to the user or role.
- You possess the GRANT ANY OBJECT PRIVILEGE system privilege that enables you to grant and revoke privileges on behalf of the object owner.

REVOKE REFERENCES ON dept FROM jward CASCADE CONSTRAINTS;

Revoke Privileges

Revoke & Example

Assuming you are the original grantor, the following statement revokes the SELECT and INSERT privileges on the emp table from the users jfee and tsmith:

REVOKE SELECT, insert ON emp FROM jfee, tsmith;

Revoke Privileges

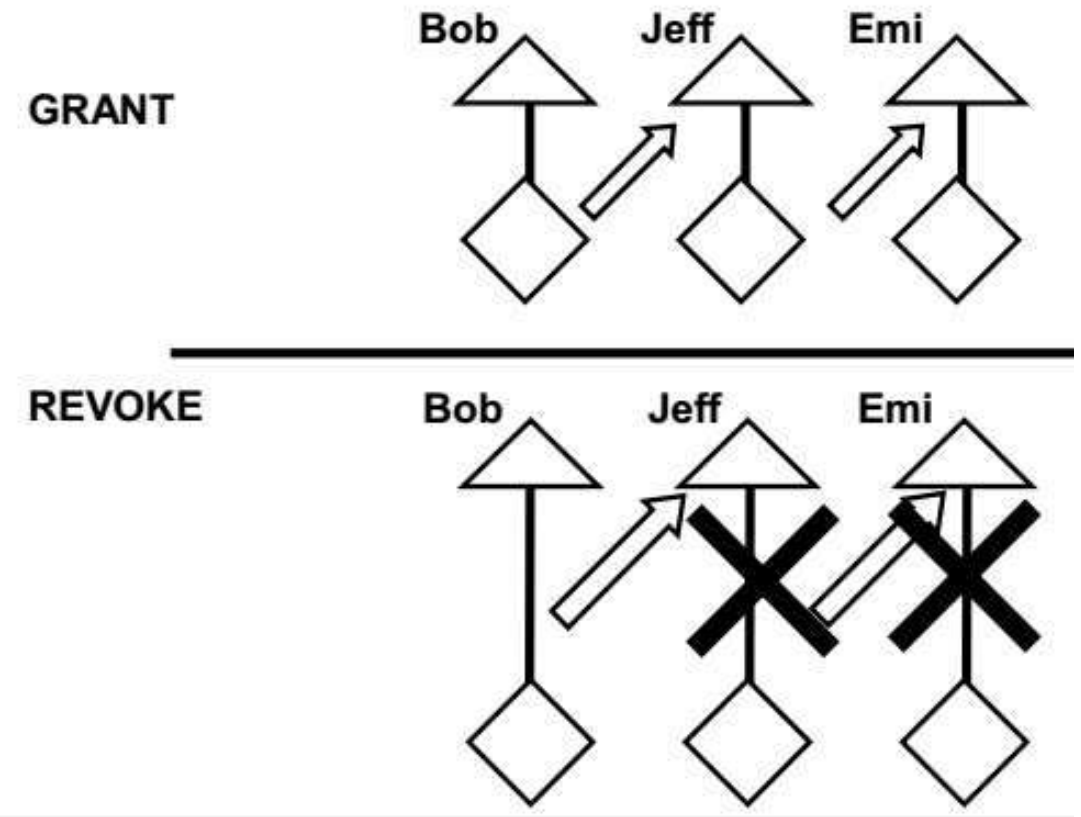
To See Existing Privileges

GRANT select on EMP to u1;

```
SQL> COLUMN OWNER FORMAT A20
SQL> COLUMN GRANTOR FORMAT A20
SQL> COLUMN GRANTEE FORMAT A20
SQL> COLUMN privilege FORMAT A20
SQL> select OWNER, GRANTOR, GRANTEE, privilege
  2  from USER_TAB_PRIVS
  3  where table_name='EMP';

OWNER                GRANTOR              GRANTEE              PRIVILEGE
-------------------- -------------------- -------------------- ----------
SCOTT                SCOTT                U1                   SELECT
```

Revoke Privileges



Revoke Privileges CASCADE

--Login with Bob
Grant select on EMP to Jeff with grant option;
--Login Jeff
Grant select on EMP to Emi;

--Login with Bob
Revoke select on EMP from Jeff;
--This revoke is cascaded to Emi

Introduction to Database Administration



Managing Roles

Managing Roles

- Introduction
- Data Dictionary for Privileges
- Privileges & Roles
- Roles & Users

Managing Roles

Introduction

- Roles are named groups of related privileges or rights.
- Each user can be granted roles and privileges for each role, giving them a tailored set of abilities.
- Roles define what actions the user can take, and privileges specify the targets to which their roles apply.
- You can view the permissions granted by each role, add and remove users, and assign roles and notification profiles to users.

Managing Roles

Data Dictionary for Privileges

We can use string search PRIV to display possible data dictionary views.

For Example,

```
select table_name
from dict
where table_name like '%PRIV%';
```

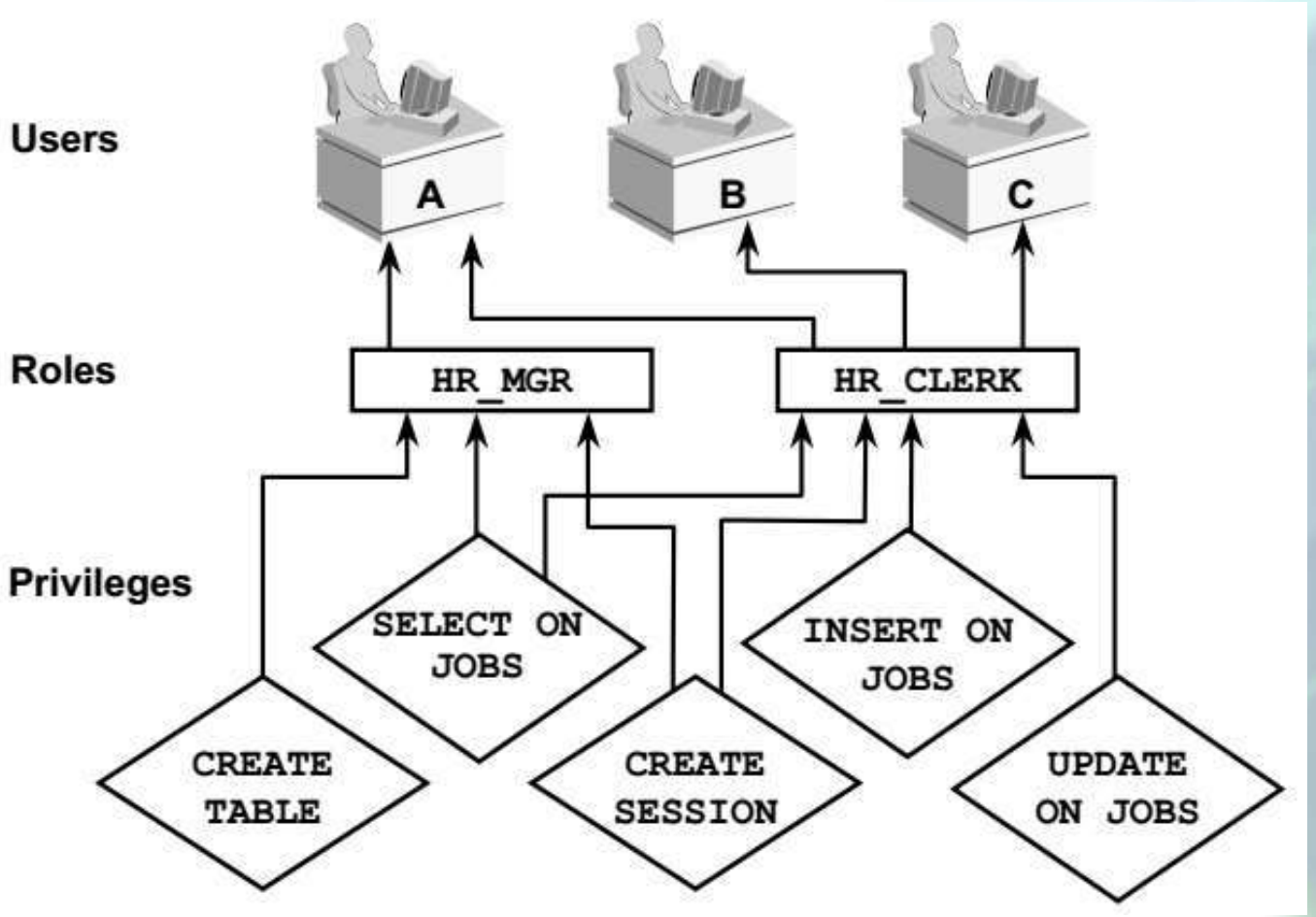
Managing Roles

Privileges & Roles

Type	Description
System privilege	A system-defined privilege usually granted only by administrators. These privileges allow users to perform specific database operations.
Object privilege	A system-defined privilege that controls access to a specific object.
Role	A collection of privileges and other roles. Some system-defined roles exist, but most are created by administrators. Roles group together privileges and other roles, which facilitates the granting of multiple privileges and roles to users.

Managing Roles

Roles & Users





Benefits of Roles-1

Benefits of Roles-1

Benefits of Roles-1

- Benefits of Roles
- Standard and Default Roles
- Roles of User SYS
- Analyzing Roles & Privileges to Users

Benefits of Roles-1

Benefits of Roles

- Easier privilege management
- Dynamic privilege management
- Selective availability of privileges
- Can be granted through the operating system

Benefits of Roles-1

Benefits of Roles

Easier privilege management

Use roles to simplify privilege management. Rather than granting the same set of privileges to several users, you can grant the privileges to a role, and then grant that role to each user.

Dynamic privilege management

If the privileges associated with a role are modified, all the users who are granted the role acquire the modified privileges automatically and immediately.

Benefits of Roles-1

Benefits of Roles

Selective availability of privileges

Roles can be enabled or disabled to turn privileges on or off temporarily. Enabling a role can also be used to verify that a user has been granted that role.

Granting through the operating system

Operating system commands or utilities can be used to assign roles to users in the database.

Benefits of Roles-1

Standard and Default Roles

- CONNECT is a role that Enterprise Manager automatically grants to a user when you create a user as shown in "Creating Users". This role has the CREATE SESSION privilege.

- RESOURCE extends the privileges of a user beyond those granted by the CONNECT role. It includes CREATE PROCEDURE, CREATE TRIGGER, and other system privileges.

Benefits of Roles-1

Standard and Default Roles

- DBA is the standard role that can be granted by an administrator to another administrator. It includes all system privileges and should only be granted to the most trusted and qualified of users. Assigning this role to a user enables the user to administer the database.

Benefits of Roles-1

Roles of User SYS

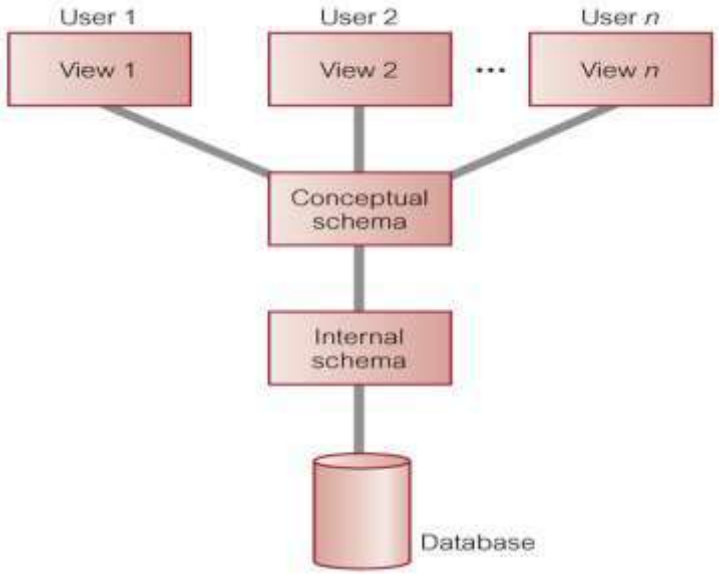
- When you create an Oracle database, the user SYS is automatically created and granted the DBA role.
- All base tables and views for the database data dictionary are stored in the schema SYS. These base tables and views are critical for the operation of Oracle Database.

Benefits of Roles-1

Roles of User SYS

- To maintain the integrity of the data dictionary, tables in the SYS schema are manipulated only by the database. They should never be modified by any user or database administrator.
- Also, you should not create any tables in the schema of user SYS, although you can change the storage parameters of the data dictionary settings if necessary.

Benefits of Roles-1



Analyzing Roles & Privileges to Users

- Assuming there are three departments in some Enterprise
  - Sales
  - Accounting
  - Human Development & Resource (HR)
- Employees are working as an external users in these departments
- Analyzing how Roles and Privileges are need to be decided
- Users Ali(Sales), Jamil(HR) and Nadeem (Accounting)



Benefits of Roles-2

Benefits of Roles-2

Benefits of Roles-2

- Characteristics of Roles
- Roles with Synonyms
- Comparing Roles with Privileges

Benefits of Roles-2

Characteristics of Roles

- Roles can be granted to and revoked from users with the same commands that are used to grant and revoke system privileges.
- Roles can be granted to any user or role. However, a role cannot be granted to itself and cannot be granted circularly.
- A role can consist of both system and object privileges.

Benefits of Roles-2

Characteristics of Roles

- A role can be enabled or disabled for each user who is granted the role.
- A role can be required a password to be enabled.
- Each role name must be unique among existing usernames and role names.
- Roles are not owned by anyone; and they are not in any schema.
- Roles have their description stored in the data dictionary.

Benefits of Roles-2

Roles with Synonyms

- Roles and privileges are assigned to base table or view. Any Synonym will be granted as per base table or view.
- For example,

--Table EMP owner gives grant:

grant select on emp to u1;

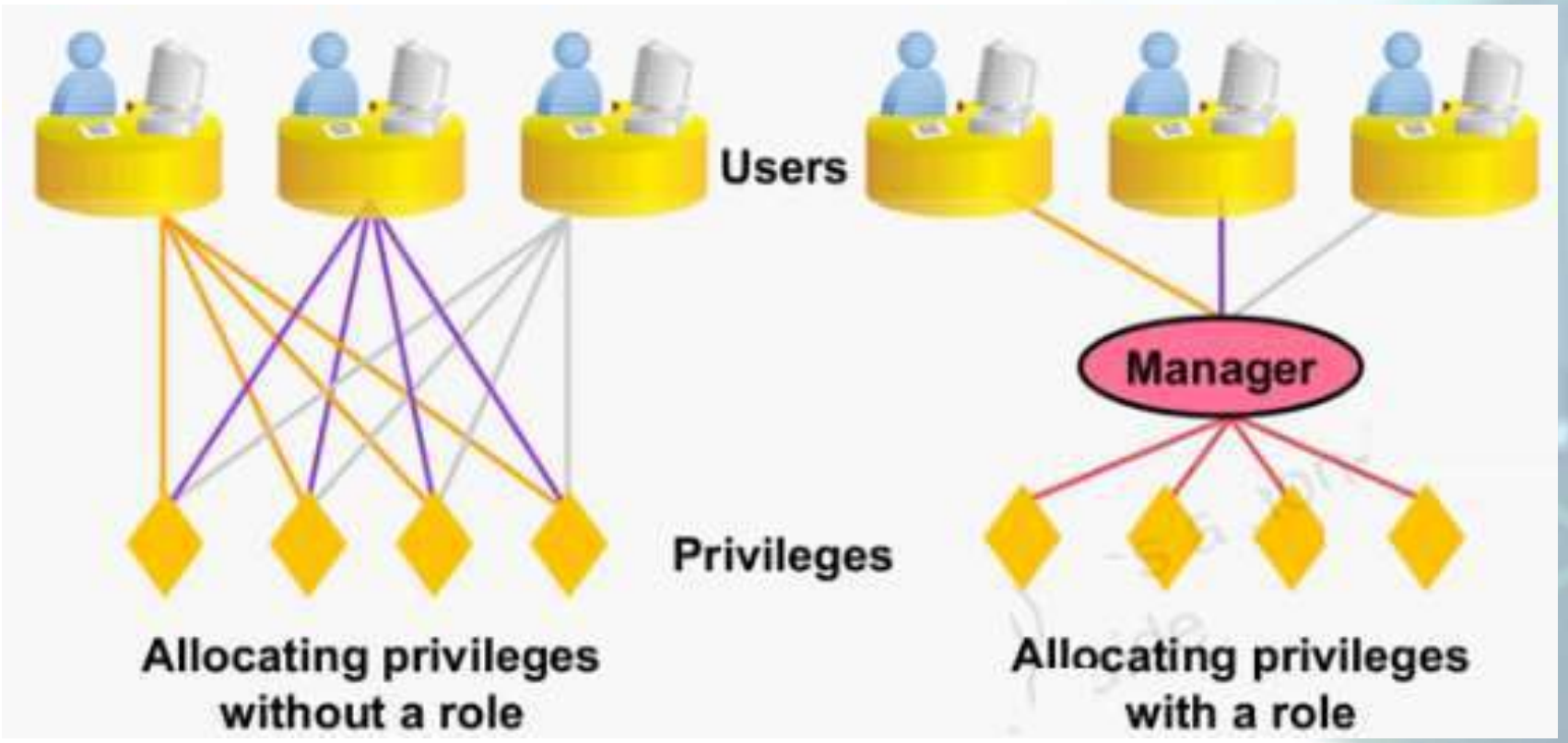
--run in u1's schema

create synonym emp for scott.emp;

select * from emp;

Benefits of Roles-2

Comparing Roles with Privileges



Introduction to Database Administration



Creating Roles

Creating Roles

Creating Roles

- SQL-Create Role
- Syntax of SQL-Create Role
- Roles with ADMIN option
- Syntax of SQL-Create Role using EM

Creating Roles

SQL-Create Role

- Use the CREATE ROLE statement to create roles. You must have the CREATE ROLE system privilege to create roles. When you create a role that is not identified externally or by password, the role is granted with the ADMIN option.
- You need to decide set of privileges to put in one role and a user who will use those privileges by assigning just role.

Creating Roles

Syntax of SQL-Create Role

Use the following command to create a role:

```
CREATE ROLE role [NOT IDENTIFIED | IDENTIFIED
  {BY password | EXTERNALLY | GLOBALLY | USING package}]
```

where:

- `role`: Is the name of the role
- `NOT IDENTIFIED`: Indicates that no verification is required when enabling the role
- `IDENTIFIED`: Indicates that verification is required when enabling the role
- `BY password`: Provides the password that the user must specify when enabling the role
- `USING package`: Creates an application role, which is a role that can be enabled only by applications using an authorized package
- `EXTERNALLY`: Indicates that a user must be authorized by an external service (such as the operating system or a third-party service) before enabling the role
- `GLOBALLY`: Indicates that a user must be authorized to use the role by the enterprise directory service before the role is enabled with the `SET ROLE` statement, or at login.

Creating Roles

Roles with ADMIN option

- Not identified:

  Create role oe_clerk;

- By password:

  Create role hr_clerk

  IDENTIFIED BY bonus;

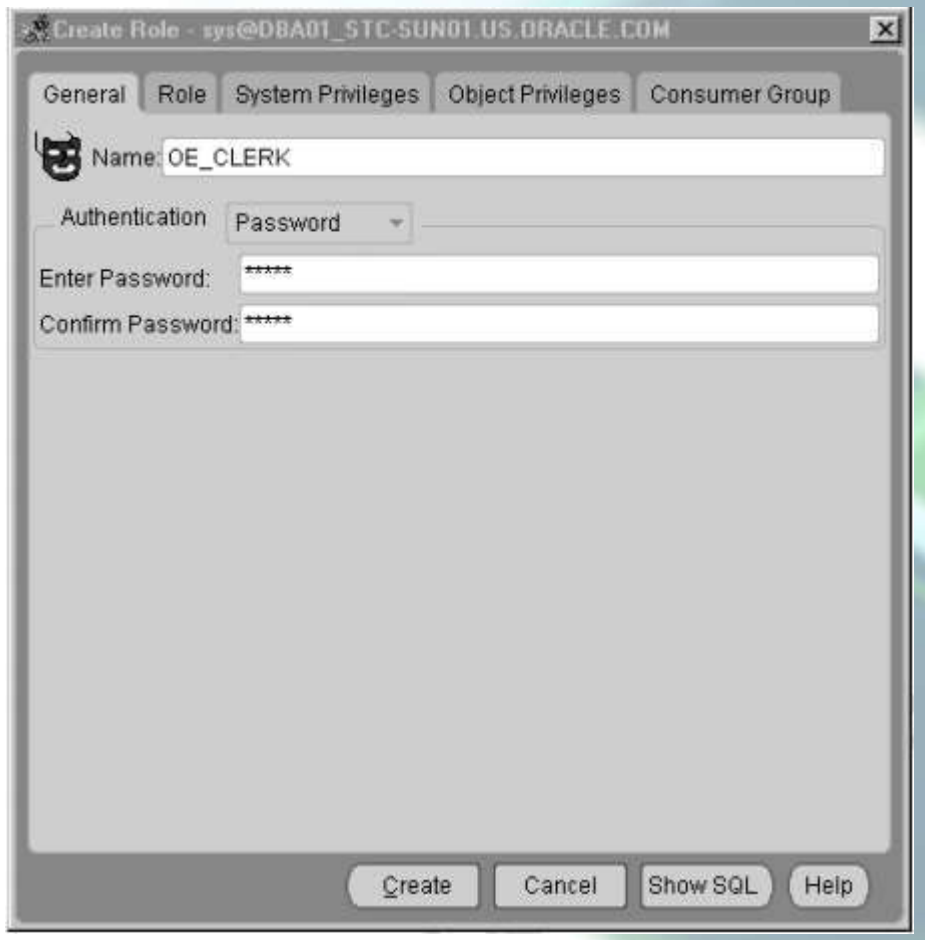
- Identified externally:

  Create role hr_manager

  identified Externally;

Creating Roles

Syntax of SQL-Create Role using EM



End



Pre-defined Roles & Assigning Roles

Pre-defined Roles & Assigning Roles

Pre-defined Roles & Assigning Roles

- Meaning of Pre-defined Roles & Assigning Roles
- Other Special Roles
- List of Pre-defined Roles & Assigning Roles
- Assigning Roles - An Enterprise Example

Pre-defined Roles & Assigning Roles

Meaning of Pre-defined Roles & Assigning Roles

The roles listed are defined automatically for Oracle databases when you run database creation scripts.

CONTENT, RESOURCE, and DBA roles are provided for backward compatibility to earlier versions of the Oracle server.

The EXP_FULL_DATABASE and IMP_FULL_DATABASE roles are provided for convenience in using the Import and Export utilities.

Pre-defined Roles & Assigning Roles

Meaning of Pre-defined Roles & Assigning Roles

The roles DELETE_CATALOG_ROLE, EXECUTE_CATALOG_ROLE, and SELECT_CATALOG_ROLE are provided for accessing data dictionary views and packages.

These roles can be granted to users who do not have the DBA role but who requires an access to the views and tables in the data dictionary.

Pre-defined Roles & Assigning Roles

Other Special Roles

The Oracle server also creates other roles that authorize you to administer the database. On many operating systems, these roles are called OSOPER and OSDBA. Their names may be different on your operating system.

Other roles are defined by SQL scripts provided with the database. For example, AQ_ADMINISTRATOR_ROLE provides privileges to administer advanced queuing. AQ_USER_ROLE is obsolete but is kept mainly for release 8.0 compatibility.

SELECT * FROM DBA_SYS_PRIVS;

Pre-defined Roles & Assigning Roles

List of Pre-defined Roles & Assigning Roles

Role Name	Description
CONNECT, RESOURCE, DBA	These roles are provided for backward compatibility
EXP_FULL_DATABASE	Privileges to export the database
IMP_FULL_DATABASE	Privileges to import the database
DELETE_CATALOG_ROLE	DELETE privileges on data dictionary tables
EXECUTE_CATALOG_ROLE	EXECUTE privilege on data dictionary packages
SELECT_CATALOG_ROLE	SELECT privilege on data dictionary tables

Pre-defined Roles & Assigning Roles

Assigning Roles – An Enterprise Example

Mumtaz (Manager)
Usma
Admin Dept



Adnan (Manager)
Abbasi
HR Dept



Fida (Manager)
Ali
Accounting Dept



Empno	Name	DOB	CNIC	Address	City	TelNo	Deptno	EmpCard#

Empno	Name	DOB	CNIC	Address	City	TelNo	Salary	Bonus	Deptno

Empno	Name	DOB	CNIC	Address	City	TelNo	BloodGroup	Salary	Bonus	Deptno	EmpCard#



Modifying & Align Roles

Modifying & Align Roles

- Alter Role
- Alter Role - Sample SQLs
- Alter Role - Syntax
- Alter Role - EM

Modifying & Align Roles

Alter Role

- ALTER ROLE modifies the authentication method.
- Modifying roles requires the ADMIN option or ALTER ANY ROLE privilege

Modifying & Align Roles

Alter Role – Sample SQLs

- ALTER ROLE oe_clerk
  IDENTIFIED BY order;

- ALTER ROLE hr_clerk
  IDENTIFIED EXTERNALLY;

- ALTER ROLE hr_manager
  NOT IDENTIFIED;

Modifying & Align Roles

Alter Role – Syntax

```
ALTER ROLE role {NOT IDENTIFIED | IDENTIFIED
 {BY password |USING package| EXTERNALLY | GLOBALLY }};
```

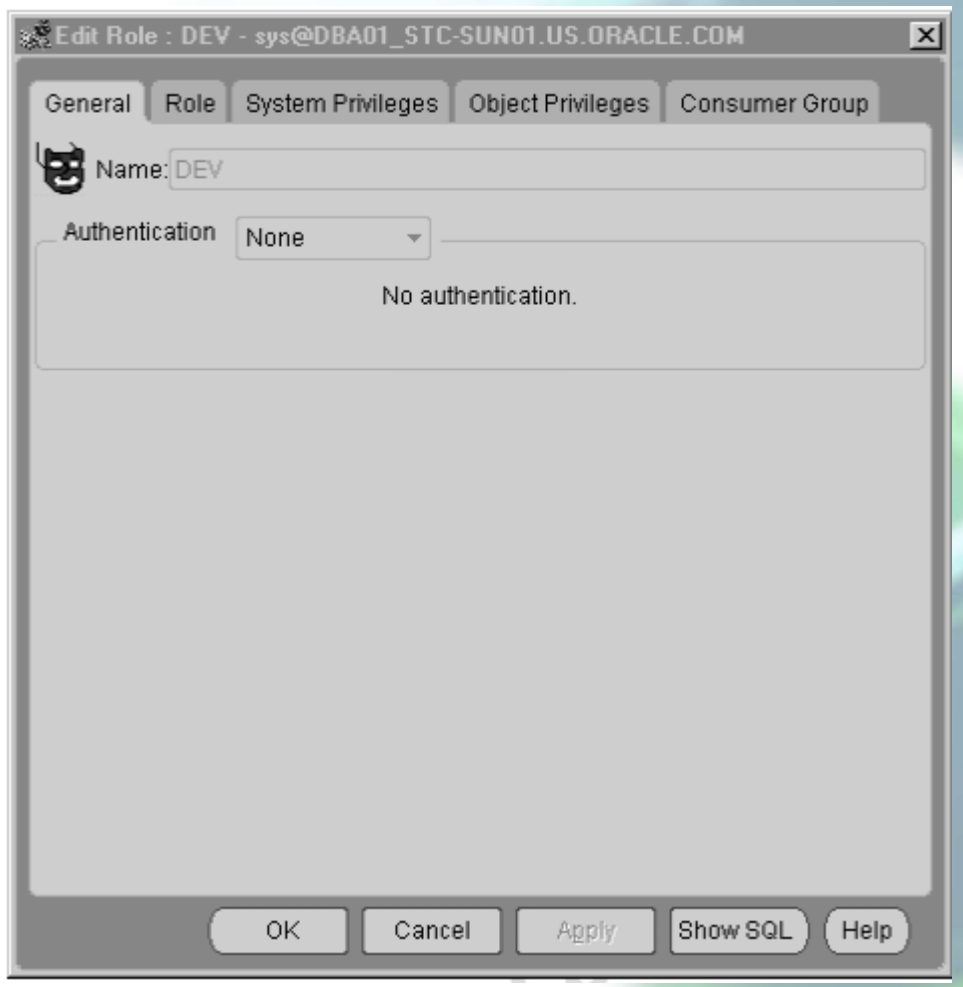
where:

- `role`: Is the name of the role
- `NOT IDENTIFIED`: Indicates that no verification is required when enabling the role
- `IDENTIFIED`: Indicates that verification is required when enabling the role
- `BY password`: Provides the password used when enabling the role
- `EXTERNALLY`: Indicates that a user must be authorized by an external service (such as the operating system or a third-party service) before enabling the role
- `GLOBALLY`: Indicates that a user must be authorized to use the role by the enterprise directory service before the role is enabled with the `SET ROLE` statement, or at login

Oracle Internal & Oracle

Modifying & Align Roles

Alter Role – EM



End

Introduction to Database Administration



Default Roles

Default Roles

Default Roles

- Default Roles
- Assigning & Disable Roles - Sample SQLs
- Establishing Default Roles

Default Roles

Assigning & Disable Roles – Sample SQLs

- GRANT oe_clerk TO scott;
- GRANT hr_clerk TO hr_manager;
- GRANT hr_manager TO scott WITH ADMIN OPTION;
- Disable role during session of user u1

  SQL> SET ROLE NONE;

Default Roles

Establishing Default Roles

- A user can be assigned many roles.
- A user can be assigned a default role.
- Limit the number of default roles for a user.

```
ALTER USER scott
      DEFAULT ROLE hr_clerk, oe_clerk;
```

```
ALTER USER scott DEFAULT ROLE ALL;
```

```
ALTER USER scott DEFAULT ROLE ALL EXCEPT
      hr_clerk;
```

```
ALTER USER scott DEFAULT ROLE NONE;
```

Default Roles

Establishing Default Roles

A user can be assigned many roles. A default role is a subset of these roles that is automatically enabled when the user logs on. By default, all the roles are assigned to a user are enabled at logon without the need of a password. Limit the default roles for a user with the *ALTER USER* command.

Default Roles

Establishing Default Roles

The DEFAULT ROLE clause applies only to roles that have been granted directly to the user with a GRANT statement. The DEFAULT ROLE clause cannot be used to enable the following:

Roles not granted to the user.

Roles granted through other roles.

Roles managed by an external service (such as the operating system).

Default Roles

Establishing Default Roles

Use the following syntax to assign default roles to a user:

ALTER USER user *DEAFULT ROLE*

{role [,role]…| ALL [EXCEPT role [,role]…] | NONE}

Where:

User: is the name of the user who is granted the roles

Role: is the role to be made the default role for the user

End

Introduction to Database Administration



Application Roles

Application Roles

Application Roles

- Application Roles
- Application Roles - SQL

Application Roles

Application Roles

- Application roles can be enabled only by authorized PL/SQL packages

- The USING package clause creates an application role

```
CREATE ROLE admin_role
IDENTIFIED USING hr.employee;
```

Application Roles

Application Roles

- The USING package clause in the CREATE ROLE statement creates an application role. An application role can be enabled only by applications by using an authorized PL/SQL package.
- Application developers do not need to secure a role by embedding passwords inside applications. Instead, they can create an application role and specify which PL/SQL package is authorized to enable the role.

Application Roles

Application Roles - SQL

```
CREATE ROLE admin_role
IDENTIFIED USING hr.employee;
```

In this example, admin_role is an application role and the role can be enabled only by modules that are defined inside the hr.employee PL/SQL package.

End



Enable & Disable Role

Enable & Disable Role

Enable & Disable Role

- Restrictions on Enable & Disable Role
- Specifying Roles to be Enabled
- Restrictions on Enable & Disable Role - Sample
- Restrictions on Enable & Disable Role - Syntax

Enable & Disable Role

Enable & Disable Role

- Disable a role to temporarily revoke the role from a user.
- Enable a role to grant it temporarily.
- The SET ROLE command enables and disables roles
- Default roles are enabled for a user at login.
- A password may be required to enable a role.

Enable & Disable Role

Enable & Disable Roles

- Enable and disable roles to temporarily activate and deactivate the privileges associated with the roles.
- To enable a role, a role must first be granted to the user.

Enable & Disable Role

Enable & Disable Roles

- When a role is enabled, the user can use the privileges granted to that role. If a role is disabled, the user cannot use the privileges associated with that role unless those privileges are granted directly to a user or to another role enabled for that user. Roles are enabled for a session. At the next session, the user's active roles revert to default roles.

Enable & Disable Role

Specifying Roles to be Enabled

- The SET command and the DBMS_SESSION.SET_ROLE procedure enable all of the roles included in the command and disable all other roles. Roles can be enabled from any tool or program that allows PL/SQL commands; however, a role cannot be enabled in a stored procedure.
- You can use the ALTER USER-DEFAULT ROLE command to indicate which roles will be enabled for a user at login. All other roles are disabled.

Enable & Disable Role

Specifying Roles to be Enabled

- A password may be required to enable a role. The password must be included in the SET ROLE command to enable the role.
- Default roles assigned to a user do not require a password; they are enabled at login, the same as a role without a password.

Enable & Disable Role

Restrictions on Enable & Disable Role

- A role cannot be enabled from a stored procedure, because this action may change the security domain (set of privileges) that allowed the procedure to be called in the first place.
- So, in PL/SQL, Role can be enabled and disabled in anonymous blocks and application procedures (for example, client Oracle Forms procedures), but not in stored procedures.

Enable & Disable Role

Restrictions on Enable & Disable Role

- IF a stored procedure contains the command SET ROLE, the following error is generated at run time:

ORA-06565: cannot execute SET ROLE from within stored procedure

Enable & Disable Role

Restrictions on Enable & Disable Role - Sample

```
SET ROLE hr_clerk;
```

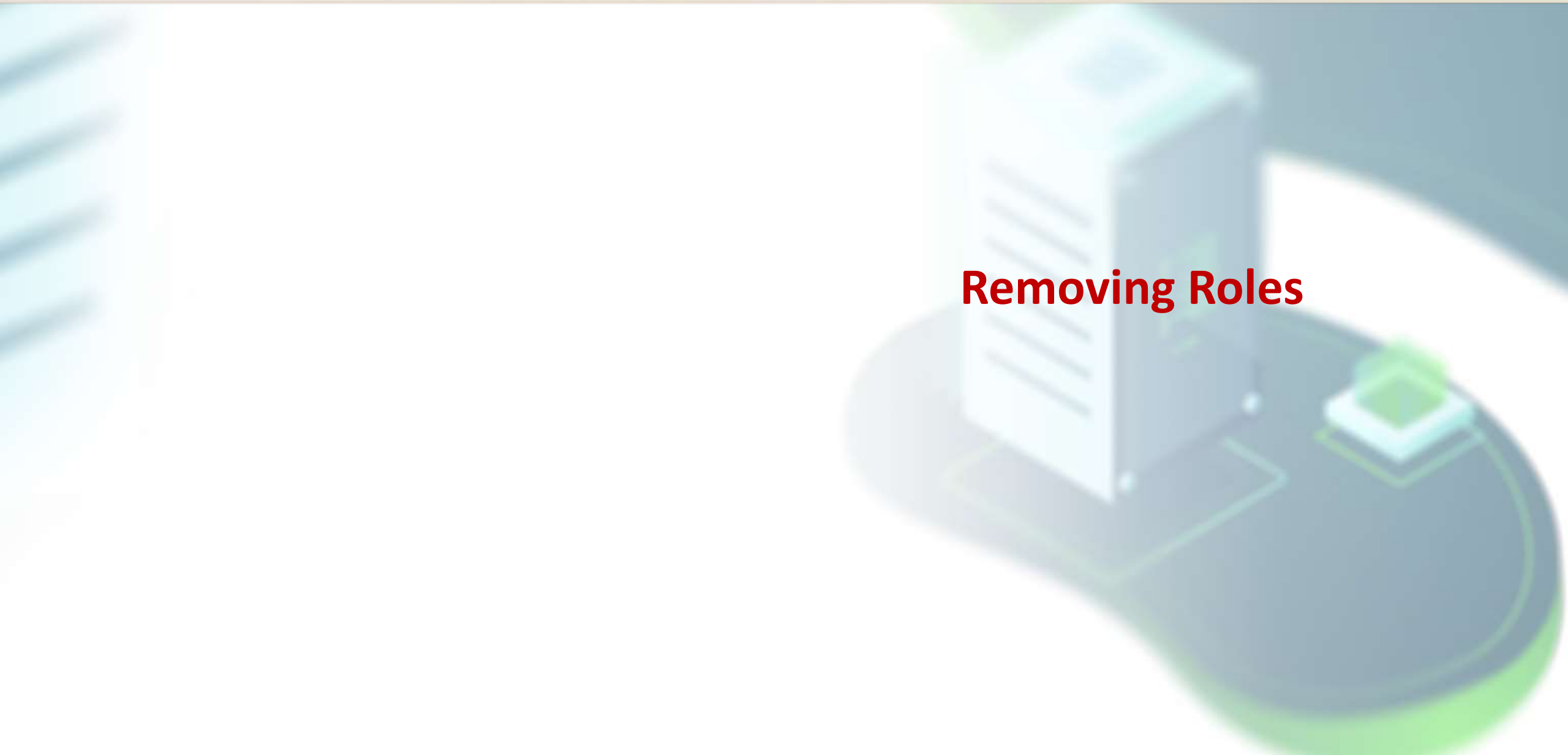
```
SET ROLE oe_clerk IDENTIFIED BY order;
```

```
SET ROLE ALL EXCEPT oe_clerk;
```

Enable & Disable Role

Restrictions on Enable & Disable Role - Syntax

```
SET ROLE {role [ IDENTIFIED BY password ]
   [, role [ IDENTIFIED BY password ]]
   | ALL [ EXCEPT role [, role ]     ]
   | NONE }
```



Removing Roles

Removing Roles

Removing Roles

- Revoke Roles
- Revoke Roles from Users
- Revoke Roles using OEM
- Removing Roles

Removing Roles

Revoke Roles

- Disable a role to temporarily revoke the role from a user.
- Enable a role to grant it temporarily.
- The SET ROLE command enables and disables roles
- Default roles are enabled for a user at login.
- A password may be required to enable a role.

Removing Roles

Revoke Roles from Users

- To revoke a role from a user, use the SQL statement REVOKE. Any user with the admin option for a role can revoke the role from any other database user or role. Also, users with the GRANT ANY ROLE privilege can revoke any role.

Removing Roles

Revoke Roles from Users

```
REVOKE role [, role ]
   FROM {user|role|PUBLIC}
   [, {user|role|PUBLIC} ]
```

where

- role: Is the role to be revoked or the role from which roles are revoked.
- user: is the user from which the system privileges or roles are revoked.
- PUBLIC: Revokes the privilege or role from all users

Removing Roles

Revoke Roles using OEM

Using Oracle Enterprise Manager to Revoke a Role from a User

From the OEM Console:

- Navigate to Security>Users.
- Highlight the user for whom a role is to be revoked.
- Navigate to Roles Granted.
- Select the role to be revoked.
- Select Revoke from the right-mouse menu.
- Select Yes to confirm revocation.

Removing Roles

Removing Roles

To remove a role from the database, use the following syntax:

```
SQL> DROP ROLE role;
```

When you drop a role, the Oracle server revokes it from all users and roles to whom it has been granted and removes it from the database.

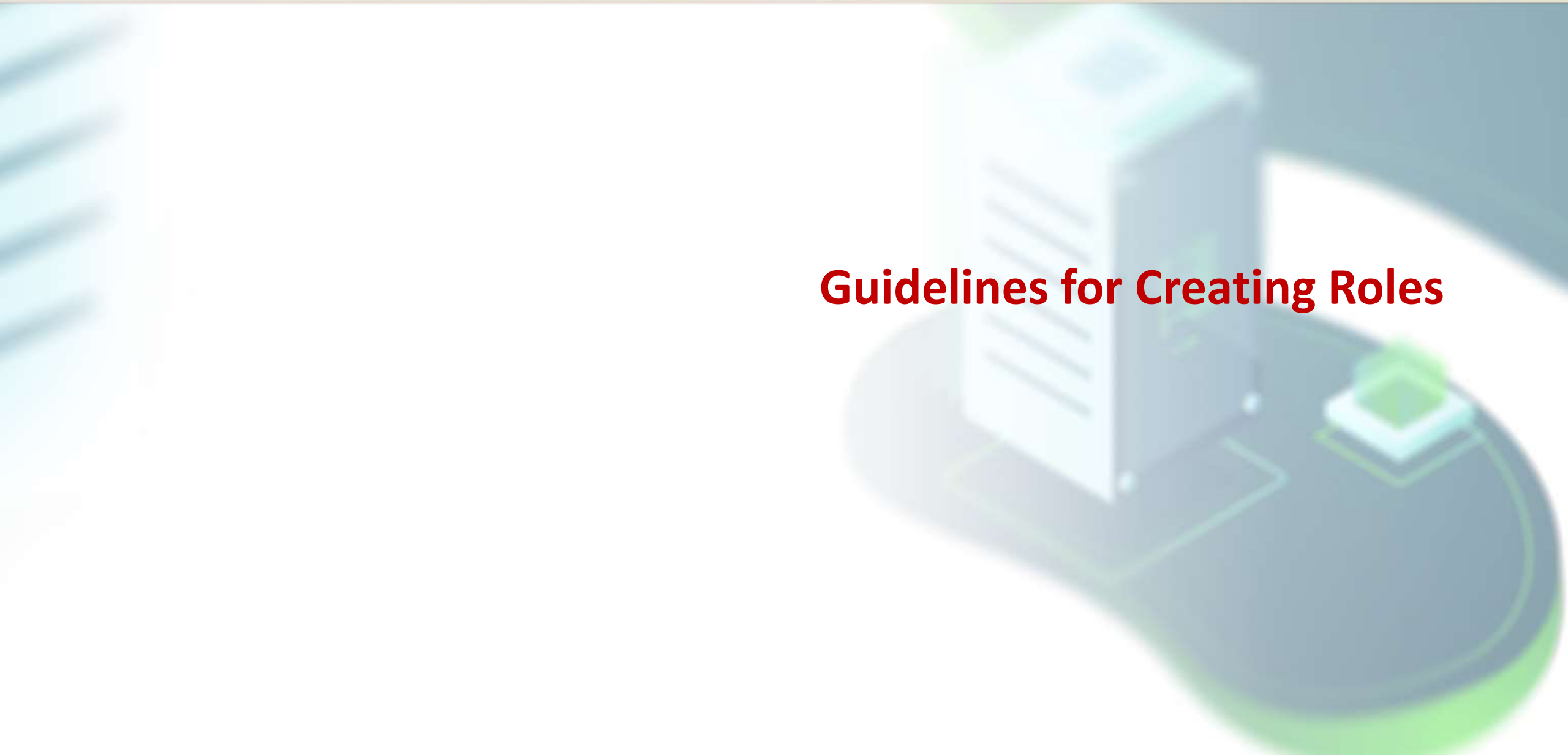
In order to drop the role, you must have been granted the role with ADMIN OPTION or have the DROP ANY ROLE system privilege.

Removing Roles

Removing Roles

- Dropping a role:
  - Removes it from all users and roles it was granted
  - Removes it from the database
- Requires the `ADMIN OPTION` or `DROP ANY ROLE` privilege
- To drop a role:

```
DROP ROLE hr_manager;
```



Guidelines for Creating Roles

Guidelines for Creating Roles

Guidelines for Creating Roles

- Guidelines
- Guidelines for an Example
- Guidelines for Assigning Roles
- Guidelines for using Password and Default Roles
- Roles with Password

Guidelines for Creating Roles

Guidelines

Because a role includes the privileges that are necessary to perform a task, the role name is usually an application task or a job title.

Our example uses both application tasks and job titles for role names.

Guidelines for Creating Roles

Guidelines for an Example

Use the following steps to create, assign, and grant users roles:

- Create a role for each application task. The name of the application role corresponds to a task in the application, such as PAYROLL.
- Assign the privileges necessary to perform the task to the application role.

Guidelines for Creating Roles

Guidelines for an Example

- Create a role for each type of user. The name of the user role corresponds to a job title, such as PAY_CLERK.
- Grant application roles to user's roles.
- Grant user's roles to users.

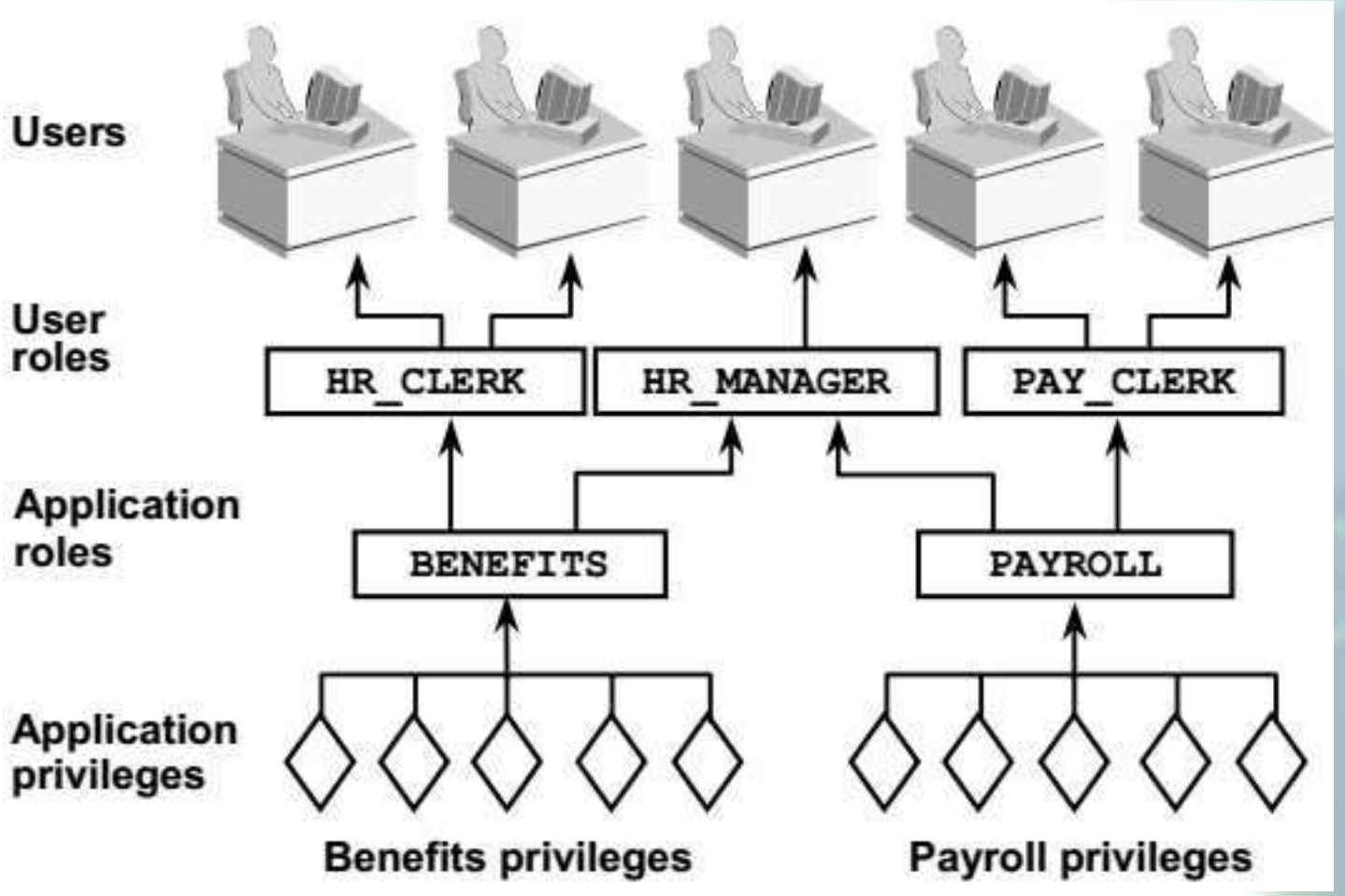
Guidelines for Creating Roles

Guidelines for an Example

If a modification to the application requires that the new privileges are needed to perform the payroll task, then the DBA only needs to assign the new privileges to the PAYROLL application role. All of the users that are currently performing this task will receive the new privileges.

Guidelines for Creating Roles

Guidelines for Assigning Roles



Guidelines for Creating Roles

Guidelines for using Password and Default Roles

Passwords provide an additional level of security when enabling a role.

For example, the application might require a user to enter a password when enabling the PAY_CLERK role, because this role can be used to issue checks.

Guidelines for Creating Roles

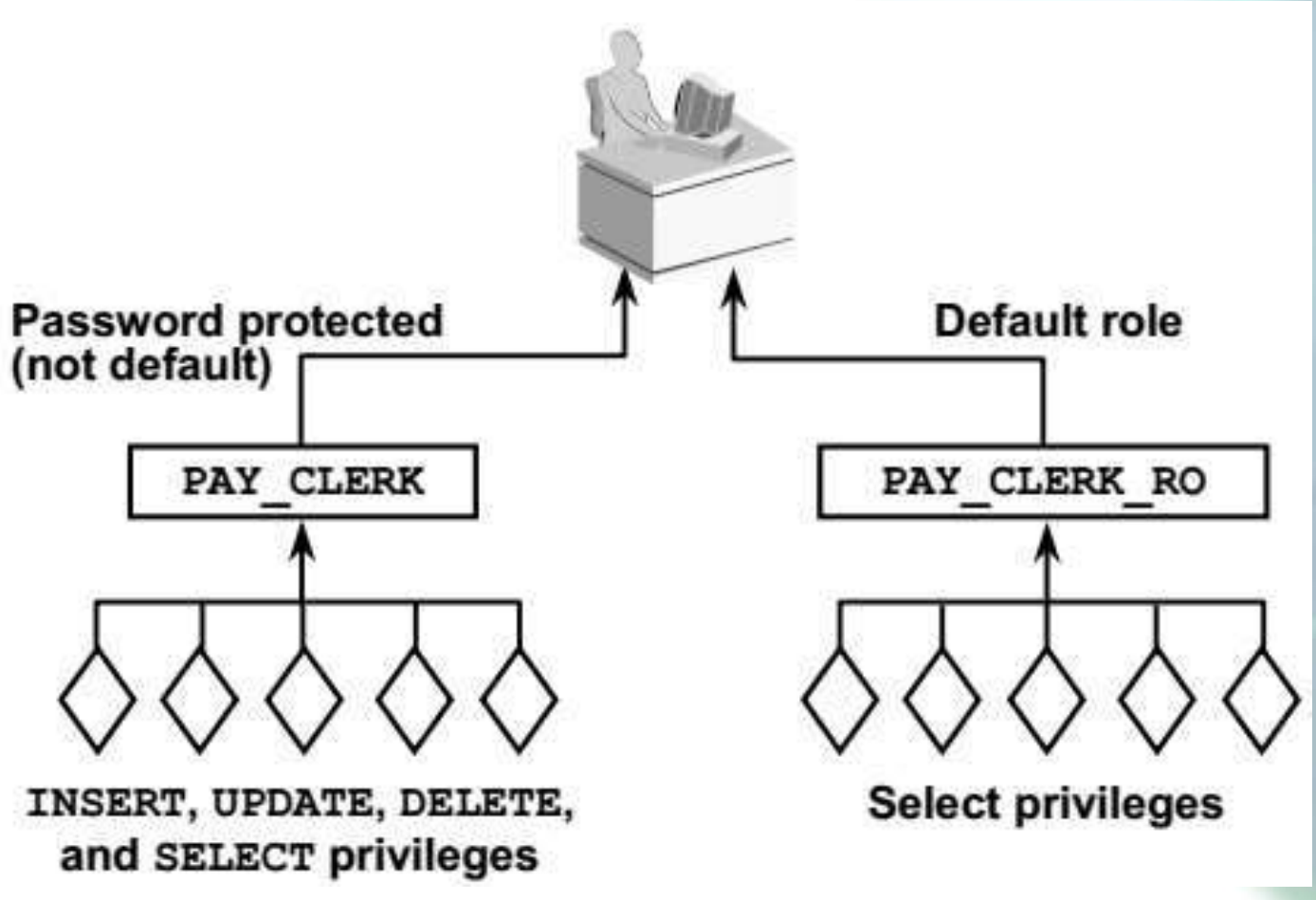
Guidelines for using Password and Default Roles

Passwords allow a role to be enabled only through an application. This technique is shown in the example in the slide.

- The DBA has granted the user two roles, PAY_CLERK and PAY_CLERK_RO.
- The PAY_CLERK has been granted all of the privileges that are necessary to perform the payroll clerk function.

Guidelines for Creating Roles

Roles with Password





Using Roles Data Dictionary Views

Using Roles Data Dictionary Views

Using Roles Data Dictionary Views

- Data Dictionary Views of Roles
- Roles' Views
- Privileges and Roles - An Example

Using Roles Data Dictionary Views

Data Dictionary Views of Roles

Information about roles can be obtained by querying the following SQL

```
select table_name, comments
from DICT
where table_name like '%ROLE%';
```

Using Roles Data Dictionary Views

Roles' Views

DBA_ROLES: All roles that exist in the database

DBA_ROLE_PRIVS: Roles granted to users and roles

ROLE_ROL_PRIVS: Roles that are granted to roles

DBA_SYS_PRIVS: System privileges granted to users and roles

ROLE_SYS_PRIVS: System privileges granted to roles

ROLE_TAB_PRIVS: Object privileges granted to roles

SESSION_ROLES: Roles that the user currently has enabled

Using Roles Data Dictionary Views

Privileges and Roles – An Example

User is u1

Privileges are
select, update for table EMP

Assigning Role EMP_SU

Assigning EMP_SU to user u1

Using Roles Data Dictionary Views

Privileges and Roles – An Example

To user has which roles

```
SQL> select Grantee, granted_role
  2  from DBA_ROLE_PRIVS
  3  where rownum<=10;

GRANTEE                        GRANTED_ROLE
------------------------------ --------------------
U1                             EMP_SU
DBA                            OLAP_DBA
DBA                            XDBADMIN
DBA                            JAVA_ADMIN
DBA                            JAVA_DEPLOY
DBA                            OLAP_XS_ADMIN
DBA                            WM_ADMIN_ROLE
DBA                            SCHEDULER_ADMIN
DBA                            XDB_SET_INVOKER
DBA                            EXP_FULL_DATABASE
```



Introduction to Data Migration & Data Loading

Introduction to Data Migration & Data Loading

Introduction to Data Migration & Data Loading

- Data Migration
- Data Migration - Source to Target Schema
- Data Migration Choices
- Data Migration & Staging Schema

Introduction to Data Migration & Data Loading

Data Migration

Data migration is the process of transferring data from one storage system or computing environment to another.

Non integrated/ distributed data can be integrated through data migration

Introduction to Data Migration & Data Loading

Data Migration

There are many reasons your enterprise might need to undertake a data migration project.

For example, you might be replacing servers or storage devices or consolidating or decommissioning data center.

Data migration is also an essential step in the overall process of migrating on-premises IT infrastructure to a cloud computing environment.

Introduction to Data Migration & Data Loading

Data Migration

Whether you are moving to a public cloud, private cloud, hybrid cloud, or multi-cloud environment, you will need to find a secure, cost-effective, and efficient method of migrating your data to its new storage location.

Introduction to Data Migration & Data Loading

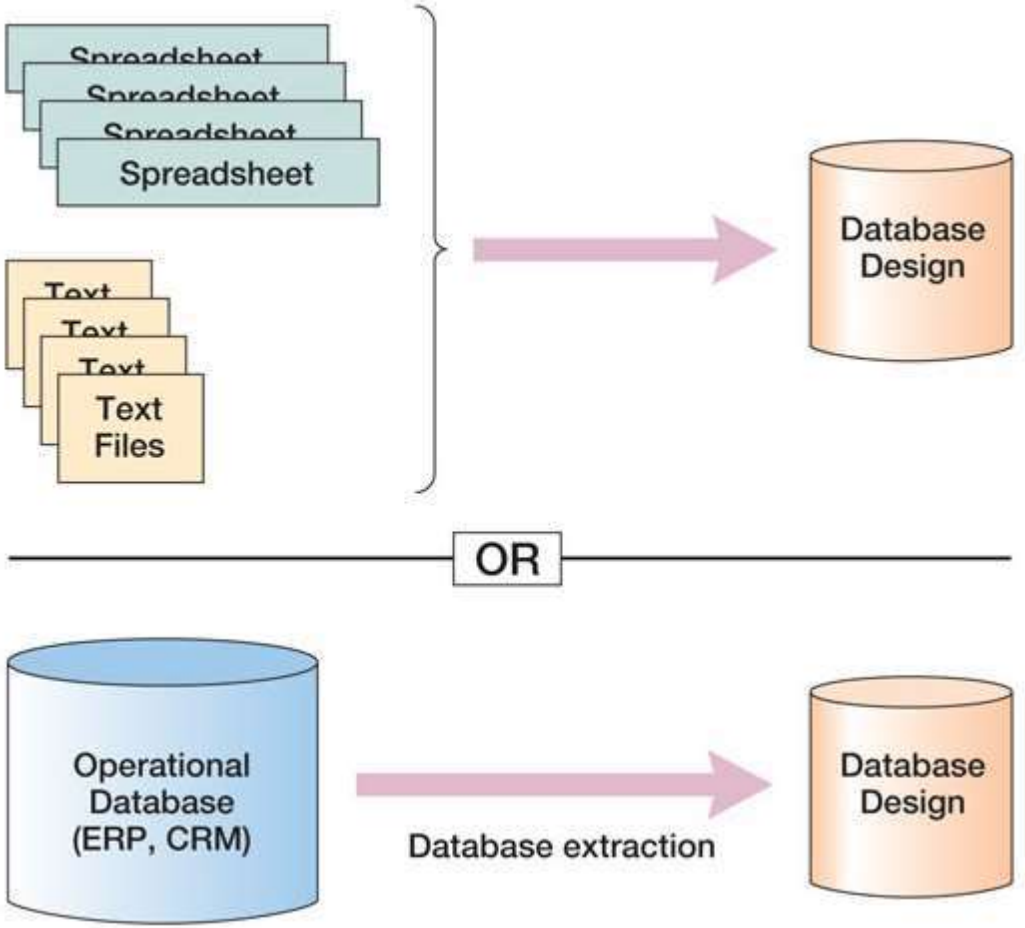
Data Migration – Source to Target Schema

Typically, the first step in the database migration process involves converting the source database's schema (if necessary) so that it's compatible with the target database. A database's schema is like a blueprint for how it is organized, controlling its logical architecture and structure. If the target database management system uses a data definition language (DDL) that is not compatible with the source's, the schema will need to be converted.

Introduction to Data Migration & Data Loading

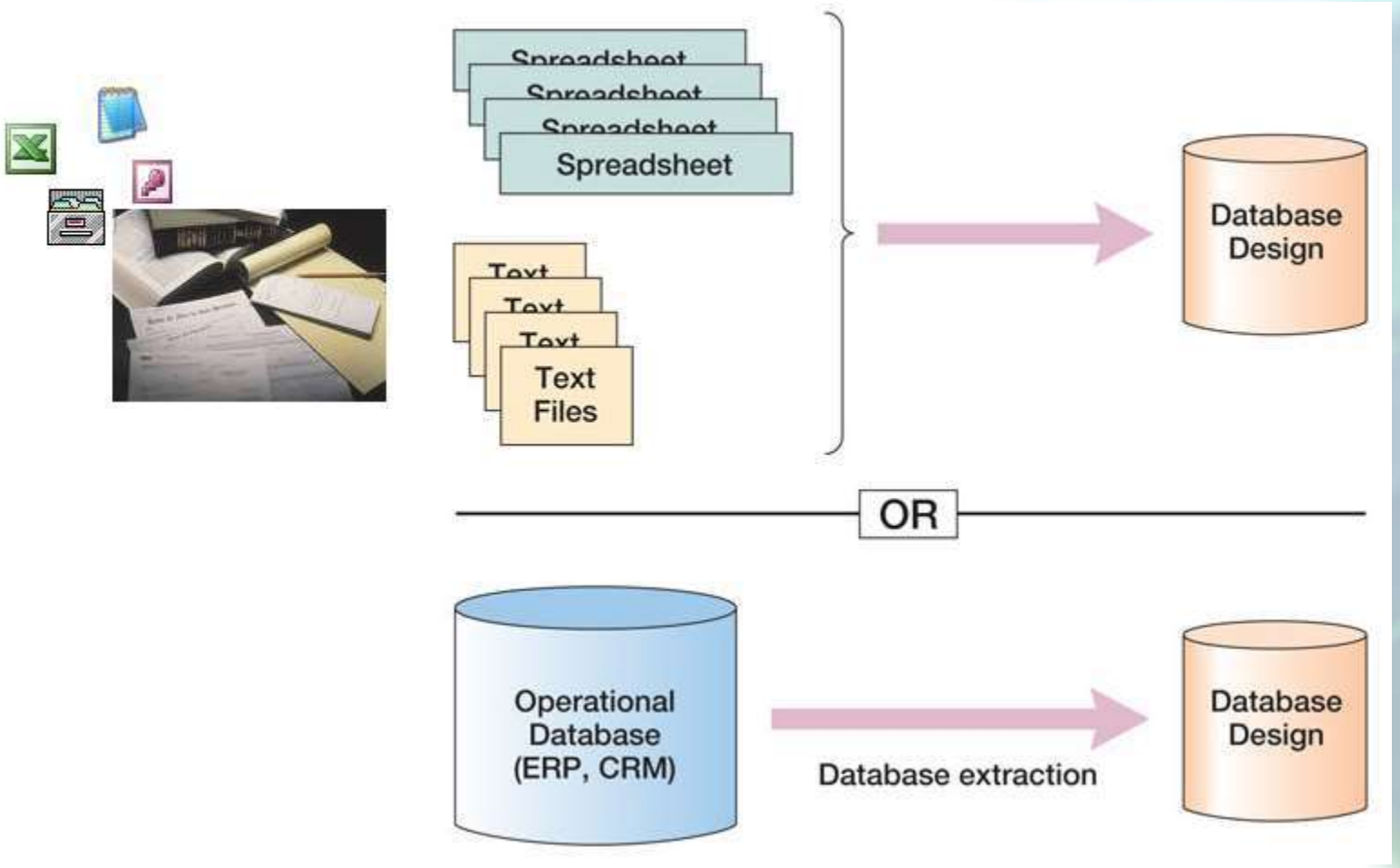
Data Migration Choices





Introduction to Data Migration & Data Loading

Data Migration Ways



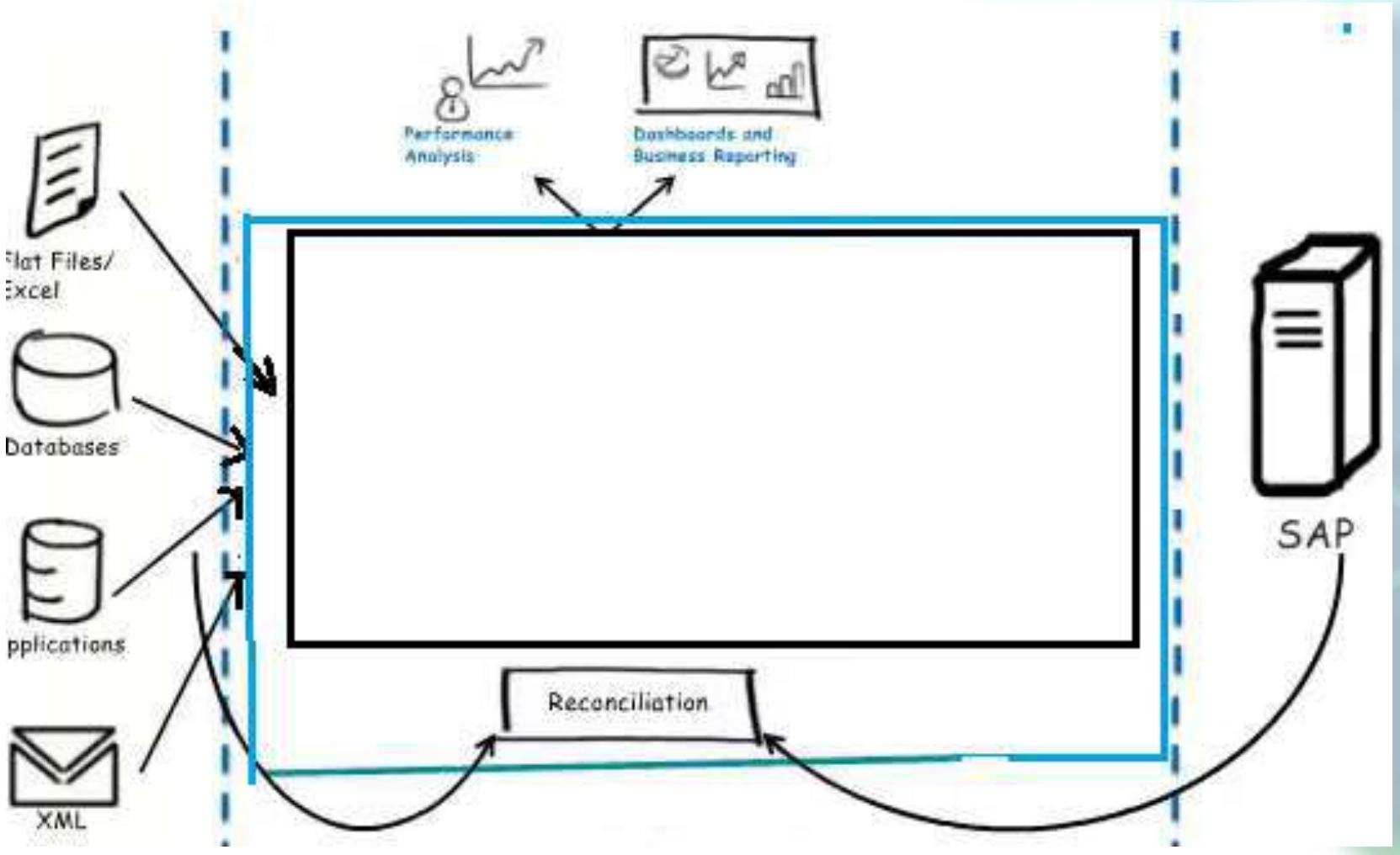
Introduction to Data Migration & Data Loading

Data Migration Ways



Introduction to Data Migration & Data Loading

Data Migration & Staging Schema



Introduction to Database Administration



Data Loading Methods

Data Loading Methods

Data Loading Methods

- Introduction
- SQL*Loader Method
- Direct Load Method
- Oracle Export and Oracle Import

Data Loading Methods

Introduction

Several methods are available for loading data into tables in an Oracle database. Of the methods available, Direct Load insert and SQL*Loader are discussed here.

Data Loading Methods

SQL*Loader Method

SQL*Loader loads data from external files into tables of an Oracle database. It has a powerful data parsing engine that places little limitation on the format of the data in the data file.

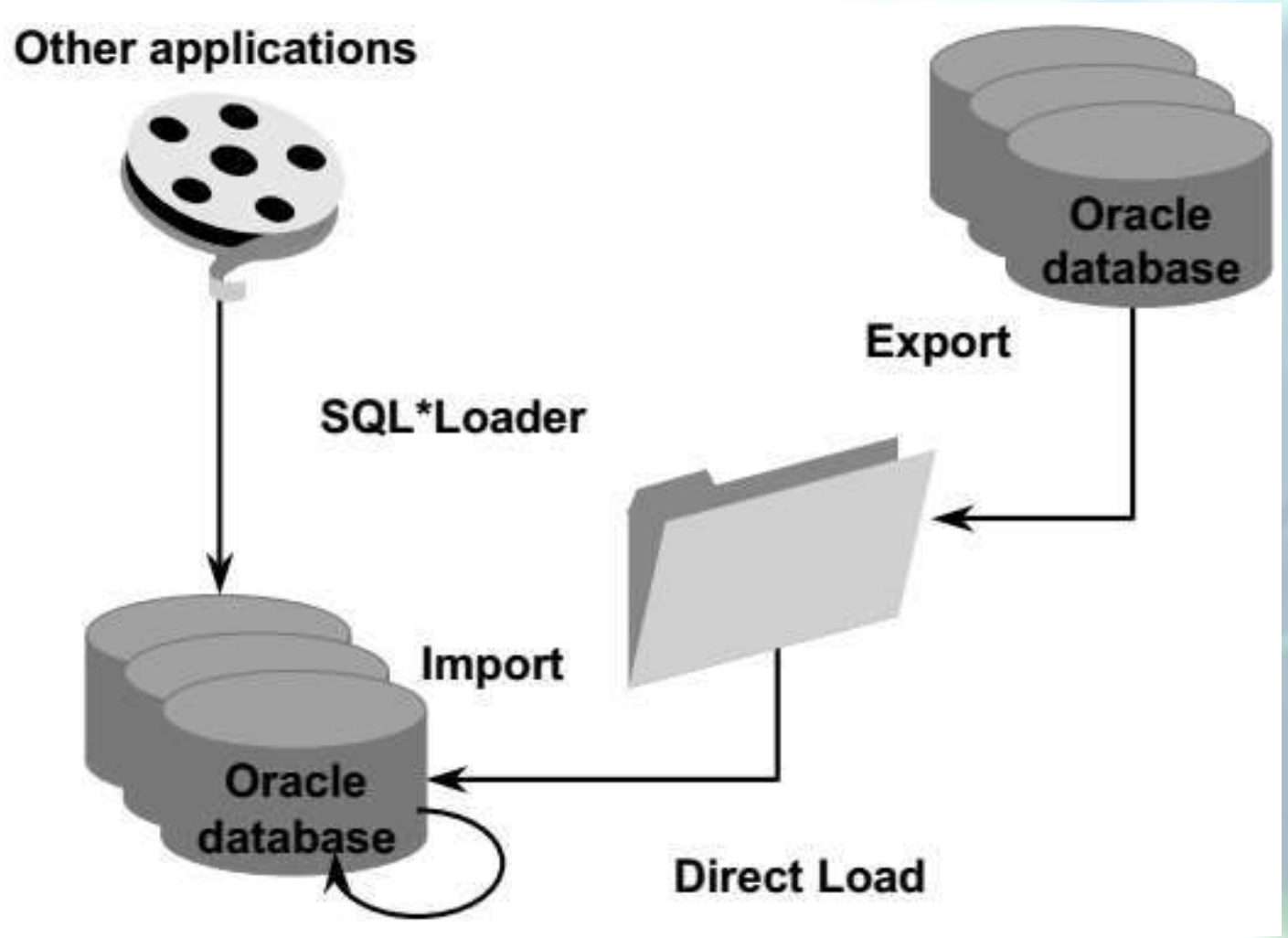
Data Loading Methods

Direct Load Method

Direct Load insert can be used to copy data from one table to another table within the same database. It speeds up the insert operation, bypassing the database buffer cache and writing data directly into the data files.

Data Loading Methods

Oracle Export and Oracle Import





Direct & Parallel Load

Direct & Parallel Load

Direct & Parallel Load

- Direct Load
- Serial Direct Load - Example
- Parallel Direct Load
- Parallel Direct Load - Example

Direct & Parallel Load

Direct Load

Direct Load insert can be performed in the following ways:

- Normal (Serially), or in parallel
- Into partition tables, non-partitioned tables, or single partitions of a table
- With or without logging of redo data

Direct & Parallel Load

Direct Load

Direct Load insert (serial or parallel) can only support the INSERT …

SELECT syntax of an INSERT statement, not the INSERT Values syntax. The parallelism for INSERT

SELECT is determined from either parallel hints or parallel table definition.

Oracle provides syntax extensions that extend the scope of the INSERT SELECT statement, so that you can insert rows into multiple tables as part of a single DML statement.

Direct & Parallel Load

Direct Load

A Direct Load insert can be invoked by using the APPEND hint, as shown in following command:

INSERT /*+APPEND * / INTO [schema.] table [[NO] LOGGING]
sub query;

Where

schema: Is the owner of the table

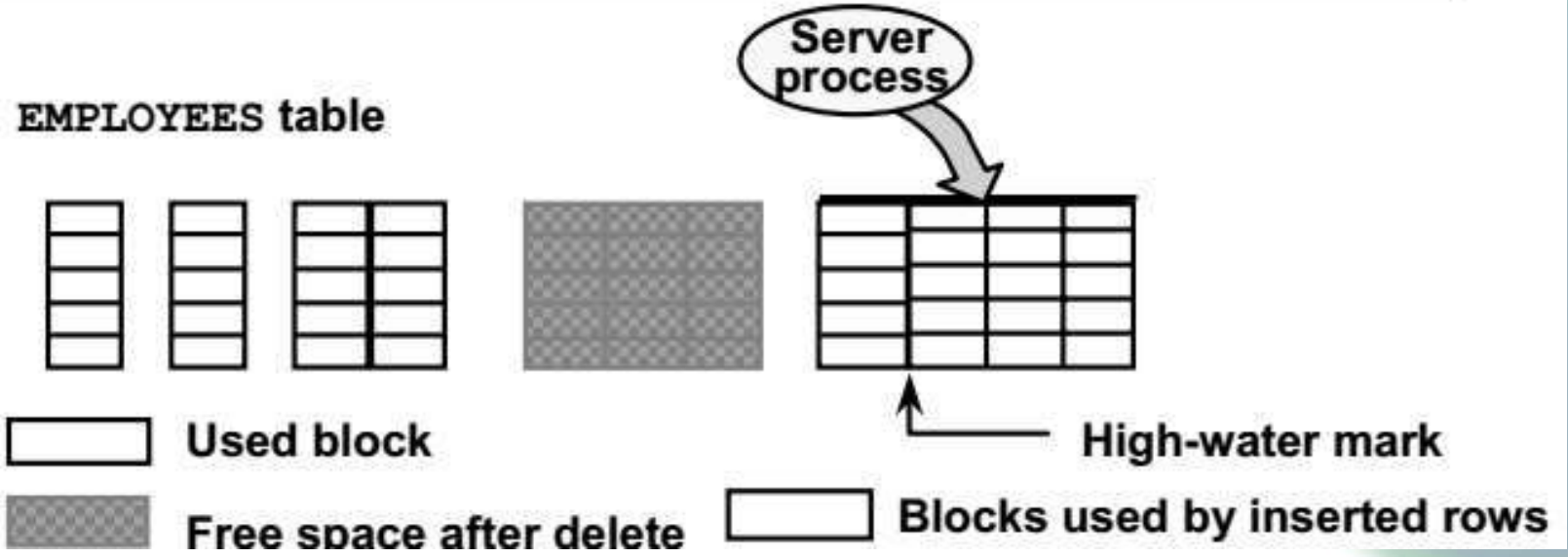
table: Is the name of the table

sub-query: Is the sub-query used to select the columns and rows for insert

Direct & Parallel Load

Serial Direct Load - Example

```
INSERT /*+ APPEND */ INTO emp
NOLOGGING
SELECT * FROM t_employees;
COMMIT;
```



Direct & Parallel Load

Parallel Direct Load

Direct Load inserts can be made in parallel by one of the following methods:

- Using a PARALLEL hint in the INSERT statement, as in the above example
- Creating the table or altering it to specify the PARALLEL clause

When parallel Direct Load inserts are made, the Oracle server uses several processes, known as parallel query slaves, to insert data into the table.

Direct & Parallel Load

Parallel Direct Load

Temporary segments are allocated to store the data inserted by each slave process.

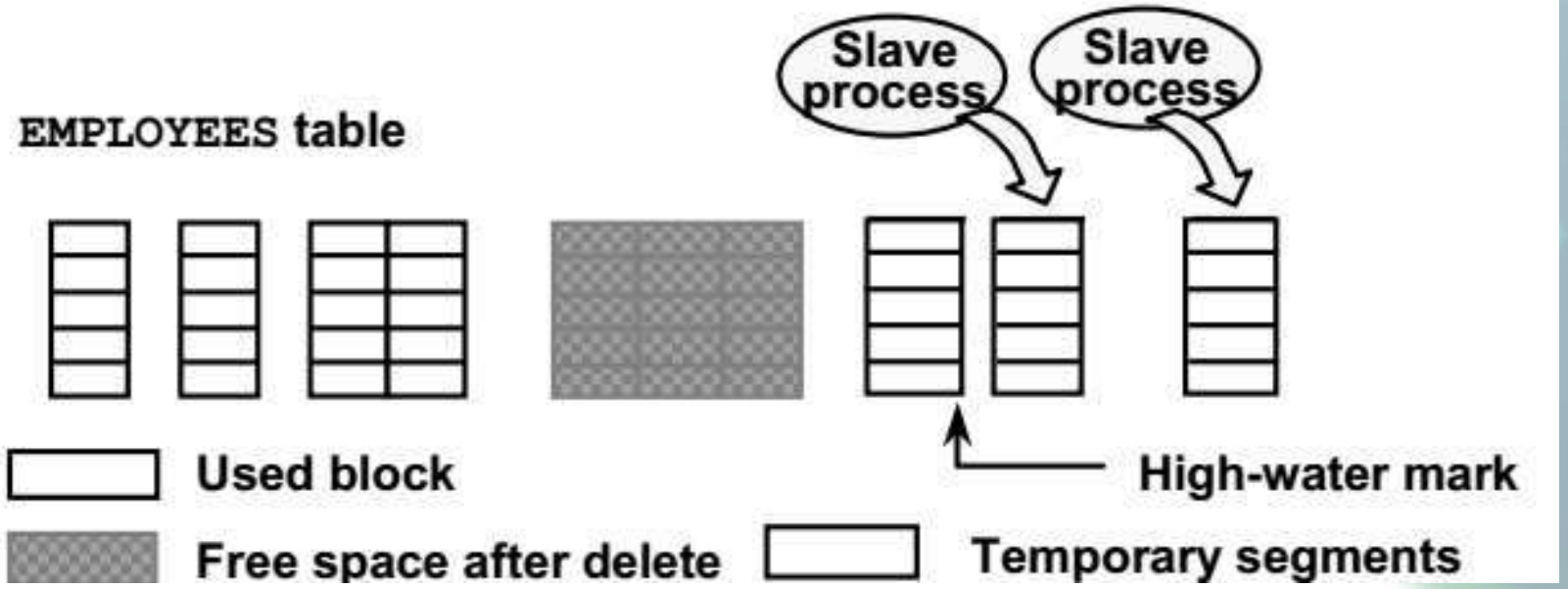
When the transaction commits, the extents in these individual segments become a part of the table in which records are inserted.

- The ALTER SESSION ENABLE PARALLEL DML command must be executed at the beginning of a transaction.
- An object that is modified using parallel Direct Load insert cannot be queried or modified again within the same transaction.

Direct & Parallel Load

Parallel Direct Load - Example

```
ALTER SESSION ENABLE PARALLEL DML;
INSERT /*+PARALLEL(hr.employees,2) */
INTO hr.employees NOLOGGING
SELECT * FROM hr.old_employees;
```



Introduction to Database Administration



SQL*Loader

SQL*Loader

SQL*Loader

- Introduction
- Files used by SQL*Loader
- Using SQL*Loader

SQL*Loader

Introduction

SQL*Loader loads data from external files into tables in an Oracle database.
SQL*Loader has the following features:

- SQL*Loader can use one or more input files.
- Several input records can be combined into one logical record for loading.
- Input fields can be of fixed or variable lengths.
- Input data can be in any format: character, binary, packed decimal, date, and zoned decimal.

SQL*Loader

Introduction

- Data can be loaded from different types of media such as disk, tape, or named pipes.
- Data can be loaded into several tables in one run.
- Options are available to replace or to append to existing data in the tables.
- SQL functions can be applied on the input data before the row is stored in the database.

SQL*Loader

Introduction

- Column values can be auto generated based on rules. For example, a sequential key value can be generated and stored in a column.
- Data can be loaded directly into the table, bypassing the database buffer cache.

SQL*Loader

Files used by SQL*Loader

SQL*Loader uses the following files:

- Loader control file: Specifies the input format, output tables, and optional conditions that can be used to load only part of the records found in the input data files
- Input data files: Contain the data in the format defined in the control file
- Parameter file: Is an optional file that can be used to define the command line parameters for the load

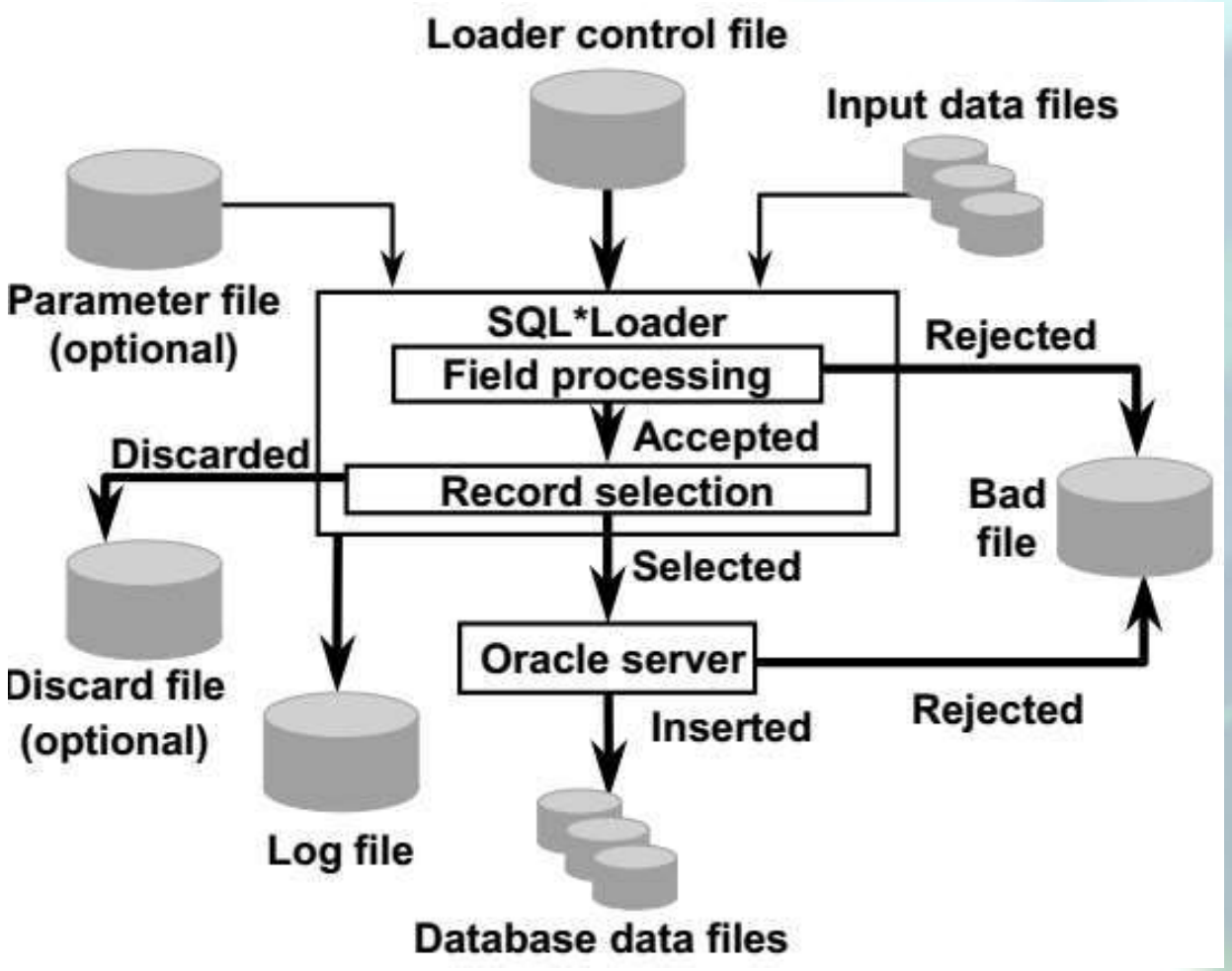
SQL*Loader

Files used by SQL*Loader

- Log file: Is created by SQL*Loader and contains a record of the load
- Bad file: Is used by the utility to write the records that are rejected during the load.
- Discard file: Is a file that can be created, if necessary, to store all records that did not satisfy the selection criteria

SQL*Loader

Files used by SQL*Loader



SQL*Loader

Using SQL*Loader

When you invoke SQL*Loader, you can specify parameters that establish session characteristics. Parameters can be entered in any order, optionally separated by commas.

You can specify values for parameters, or in some cases, you can accept the default without entering a value.

SQL*Loader

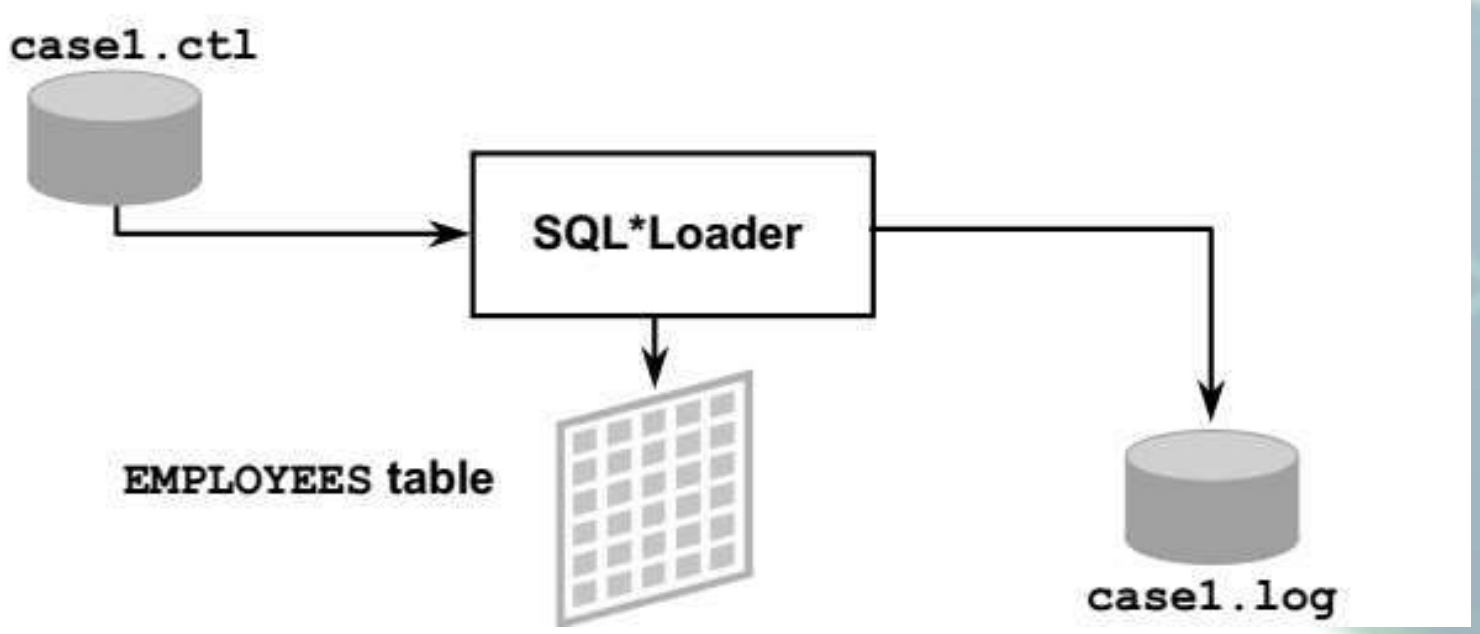
Using SQL*Loader

If you invoke SQL*Loader without specifying any parameters, SQL*Loader displays a Help screen that lists the available parameters and their default values.

SQL*Loader

Using SQL*Loader

```
$sqlldr hr/hr \
> control=case1.ctl \
> log=case1.log direct=Y
```





SQL*Loader Control Files

SQL*Loader Control Files

SQL*Loader Control Files

- Introduction
- Three Sections of Control Files
- Sample Control File
- Sample Control File Explanation

SQL*Loader Control Files

Introduction

The loader control file tells SQL*Loader:

- Where to find the load data
- The data format
- Configuration details:
  - Memory management
  - Record rejection
  - Interrupted load handling details
- How to manipulate the data

SQL*Loader Control Files

Introduction

The SQL*Loader control file is a text file that contains DDL instructions. DDL is used to control the following aspects of a SQL*Loader session:

- Where SQL*Loader finds the data to load
- How SQL*Loader expects that data to be formatted
- How SQL*Loader configures (memory management, rejecting records, interrupted load handling, and so on) as it loads the data

SQL*Loader Control Files

Introduction

- How SQL*Loader manipulates the data being loaded



SQL*Loader Control Files

Three Sections of Control Files

- The first section contains session-wide information, for example:
  - Global options such as bind size, rows, records to skip, and so on
  - INFILE clauses to specify where the input data is located
  - How data is to be loaded

SQL*Loader Control Files

Three Sections of Control Files

- The second section consists of one or more INTO TABLE blocks. Each of these blocks contain information about the table into which the data is to be loaded, such as the table name and the columns of the table.
- The third section is optional and, if present, contains input data.

SQL*Loader Control Files

Sample Control File

```
1   -- This is a sample control file
2  LOAD DATA
3  INFILE 'SAMPLE.DAT'
4  BADFILE 'sample.bad'
5  DISCARDFILE 'sample.dsc'
6  APPEND
7  INTO TABLE emp
8  WHEN (57) = '.'
9  TRAILING NULLCOLS
10 (hiredate SYSDATE,
       deptno POSITION(1:2) INTEGER EXTERNAL(3)
        NULLIF deptno=BLANKS,
       job POSITION(7:14) CHAR TERMINATED BY WHITESPACE
       NULLIF job=BLANKS "UPPER(:job)",
       mgr POSITION(28:31) INTEGER EXTERNAL
       TERMINATED BY WHITESPACE, NULLIF mgr=BLANKS,
       ename POSITION(34:41) CHAR
```

SQL*Loader Control Files

Sample Control File Explanation

- This is how comments are entered in a control file. Comments can appear anywhere in the command section of the file, but they should not appear within the data.
- The LOAD DATA statement tells SQL*Loader that this is the beginning of a new data load. If you were continuing a load that had been interrupted in progress, you would use the CONTINUE LOAD DATA statement.
- The INFILE keyword specifies the name of a data file containing data that you want to load.

SQL*Loader Control Files

Control File Syntax Consideration

The syntax is free-format

Syntax is not case sensitive

Comments extend from the two hyphen (--) that mark the beginning of the comment to the end of the line

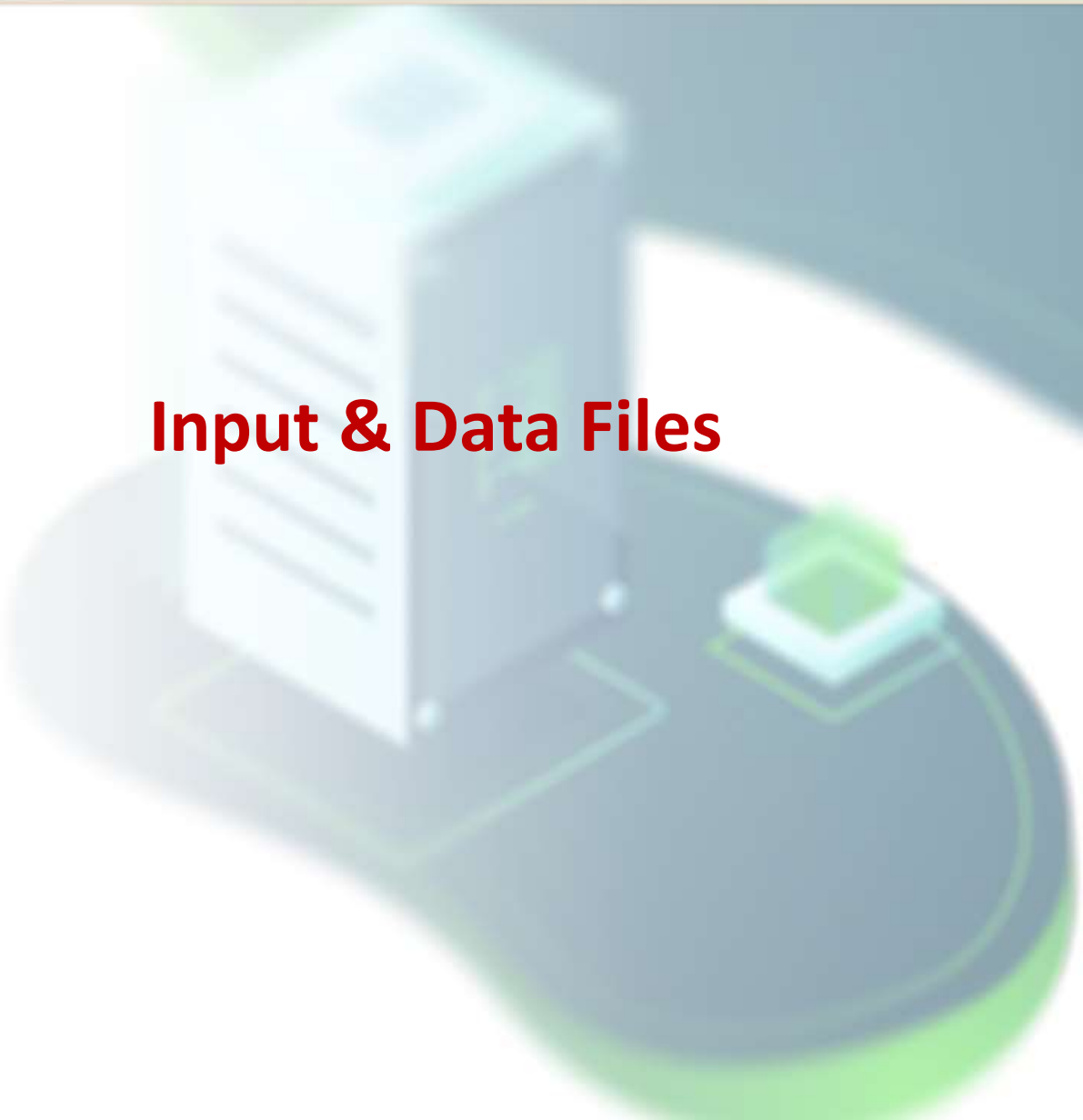
The CONSTANT keyword is reserved

SQL*Loader Control Files

Control File Syntax Consideration

- The syntax is free format. (Statements can extend over multiple lines.)
- It is case insensitive; however, strings enclosed in single or double quotation marks are taken literally, including case.
- In control file syntax, comments extend from the two hyphens (—) that mark the beginning of the comment to the end of the line. The optional third section of the control file is interpreted as data rather than as control file syntax; consequently, comments in this section are not supported.
- The CONSTANT keyword has special meaning to SQL*Loader and is therefore reserved. To avoid potential conflicts, do not use the word CONSTANT as a name for any tables or columns.

Introduction to Database Administration



Input & Data Files

Input & Data Files

- Introduction
- Fixed-Record Format
- Variable-Record Format
- Stream-Record Format

Input & Data Files

Introduction

SQL*Loader reads data from one or more files specified in the control file.

From SQL*Loader's perspective, the data in the data file is organized as records.

A data file can be in one of three formats:

- Fixed-record format
- Variable-record format
- Stream-record format

Input & Data Files

Fixed-Record Format

Records in a file are of same byte

Format is specified as

INFILE <data file> "fix n"

Following is a control file contents for two columns

load data

infile 'example.dat' "fix 10"

into table example

fields terminated by ' ' (col1, col2)

Input & Data Files

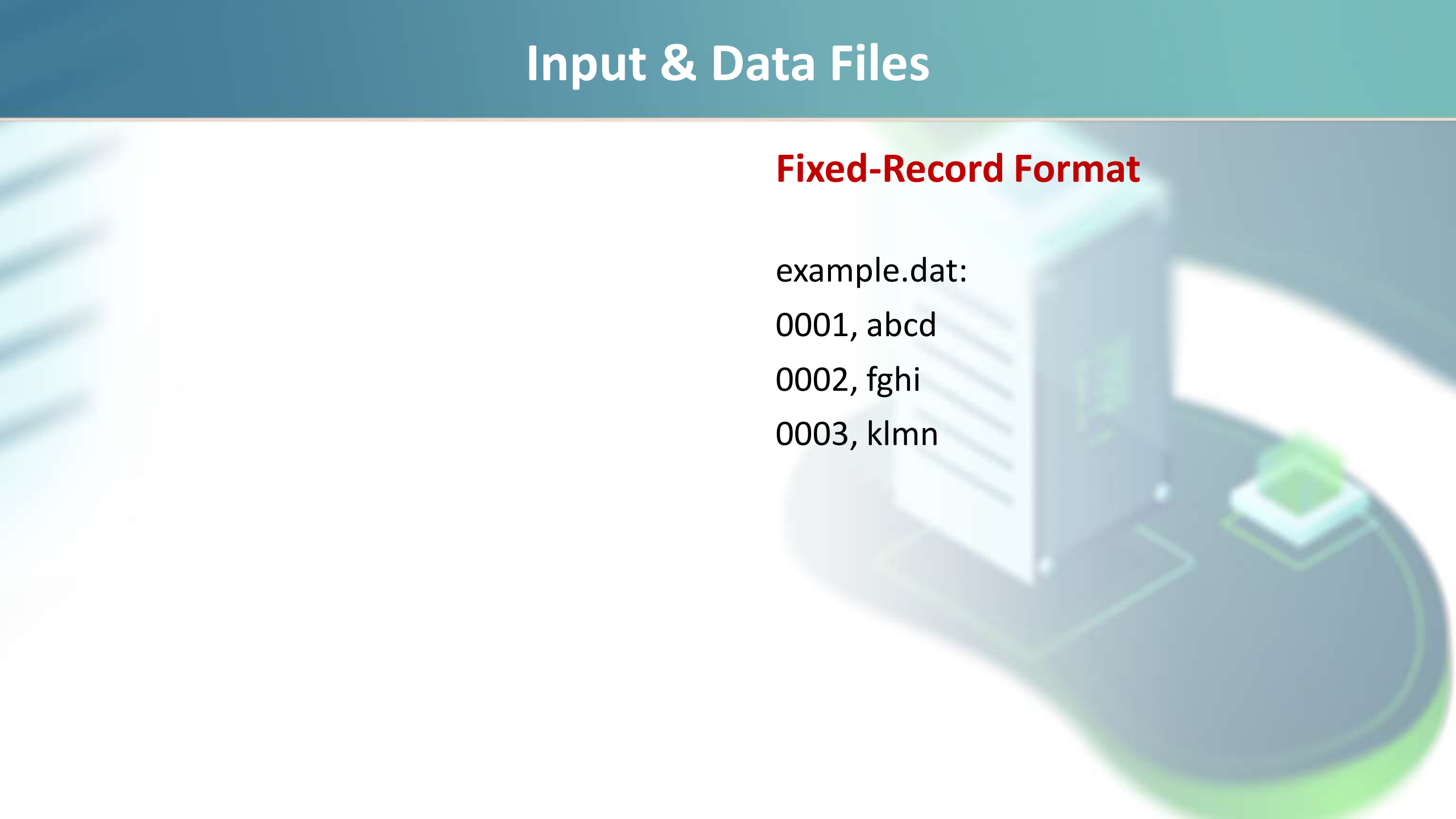
Fixed-Record Format

example.dat:

0001, abcd

0002, fghi

0003, klmn



Input & Data Files

Variable-Record Format

A file is in variable format when the length of each record in a character field is included at the beginning of each record in the data file.

For example, specifying data file as

INFILE "datafile_name" "var n"

n represents the number of bytes in the record length field

If n is not specified, SQL*Loader assumes a length of 6.

Input & Data Files

Variable-Record Format

In example.dat, record length fields/ columns are 3 bytes long

```
load data
infile 'example.dat' "var 3"
into table example
fields terminated by ',' optionally enclosed by '"'
(col1 char(5),col2 char(7))
example.dat:
009hello,cd,
010world,im,
012my,name is,
```

Input & Data Files

Stream-Record Format

A file is in stream-record format when records are not specified by size; instead SQL*Loader forms records by scanning for the record terminator. It is flexible but it has performance issues.

For example, specifying data file as

INFILE <datafile_name> ["str terminator_string"]

Input & Data Files

Stream-Record Format

Non printable characters will be

\n new line

\t horizontal tab

\f form feed

\v vertical tab

\r carriage return

Input & Data Files

Stream-Record Format

In example.dat, data loading with stream-record format

```
load data
infile 'example.dat' "str '|\n'"
into table example
fields terminated by ',' optionally enclosed by '"'
(col1 char(5),
col2 char(7))
example.dat:
hello,world,|
james,bond,|
```



Logical Records & Loading Methods

Logical Records & Loading Methods

Logical Records & Loading Methods

- Logical Records
- Using CONCATENATE to Assemble Logical Records
- Using CONTINUEIF to Assemble Logical Records
- Conventional Path Load
- Direct Path Load
- Loading Methods

Logical Records & Loading Methods

Logical Records

SQL*Loader can be instructed to follow one of the following two logical record-forming strategies:

Combine a fixed number of physical records to form each logical record

Combine physical records into logical records while a certain condition is true

Logical Records & Loading Methods

Logical Records

SQL*Loader organizes the input data into physical records, according to the specified record format. By default, a physical record is a logical record. However, for added flexibility, SQL*Loader can be instructed to combine a number of physical records into a logical record. SQL*Loader can do

- Combine fixed number of physical records to form each logical record
- Combine physical records into logical records while a certain condition is true

Logical Records & Loading Methods

Using CONCATENATE to Assemble Logical Records

CONCATENATE is used when SQL*Loader should always add the same number of physical records to form one logical record. For example,

CONCATENATE integer

Means number of physical records to combine

Logical Records & Loading Methods

Using CONTINUEIF to Assemble Logical Records

CONTINUEIF must be used if the number of physical records to be continued varies. The CONTINUEIF keyword is followed by a condition that is evaluated for each physical record as it is read.

For example, two records might be combined if there was a pound sign (#) in character position 80 of the first record.

Logical Records & Loading Methods

Conventional Path Load

Conventional path Load builds an array of rows to be inserted and uses the SQL INSERT statement to load the data. During conventional path loads, input records are parsed based on field specifications, and an array of records is built and inserted into the table specified in the control file.

Records that do not conform to the field specifications are rejected and those records that do not satisfy the selection criteria are discarded.

Logical Records & Loading Methods

Conventional Path Load

Conventional path loads can be used to load data into both the clustered and unclustered tables.

Redo generation is controlled by the logging attribute for the table that is being loaded.

Logical Records & Loading Methods

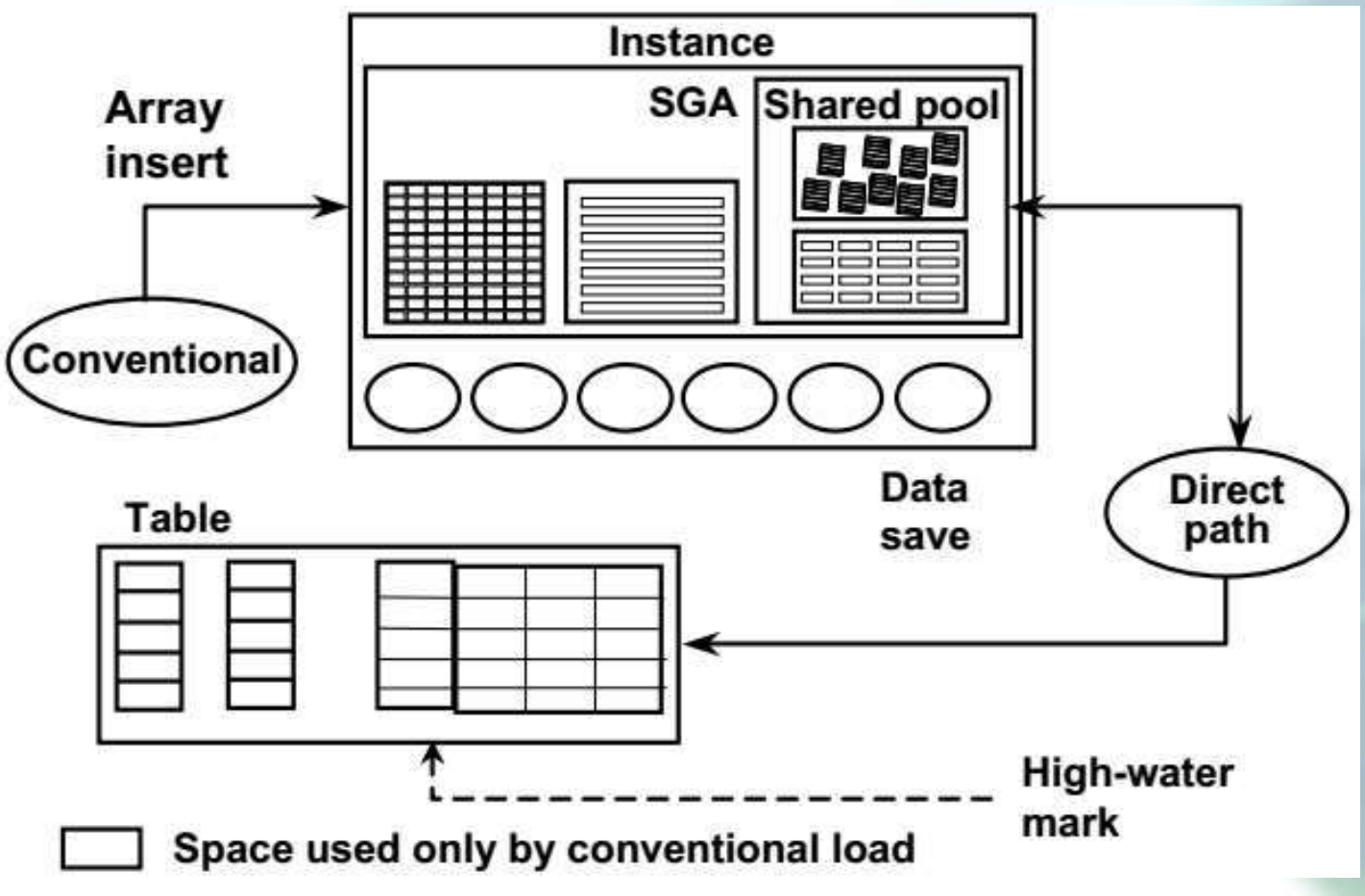
Direct Path Load

A Direct path load builds blocks of data in memory and saves these blocks directly into the extents allocated for the table being loaded. Online redo log file entries are not generated unless the database is in ARCHIVELOG mode.

Direct load path is generally faster than conventional path load.

Logical Records & Loading Methods

Loading Methods





Direct & Conventional Path Loads

Direct & Conventional Path Loads

Direct & Conventional Path Loads

- Method of Saving Data
- Comparison of Conventional and Direct Path Loads
- Logging Changes
- Enforcing Constraints
- Parallel Direct Paths
- Enforcing Constraints

Direct & Conventional Path Loads

Method of Saving Data

Conventional path loads use SQL processing and database COMMITS for saving data. The insertion of an array of records is followed by a commit operation. Each data load might involve several transactions.

Direct & Conventional Path Loads

Method of Saving Data

Direct path loads use data saves to write blocks of data to Oracle data files. The following features differentiate a data save from a COMMIT:

During a data save, only full database blocks are written to the database

The blocks are written after the high-water mark of the table

After a data save, the high-water mark is moved

Internal resources are not released after a data save

A data save does not end the transaction

Indexes are not updated at each data save

Direct & Conventional Path Loads

Comparison of Conventional and Direct Path Loads

Conventional Load	Direct Path Load
Uses COMMITs to make changes permanent	Uses data saves
Redo entries always generated	Generates redo only under specific conditions
Enforces all constraints	Enforces only primary key, unique, and NOT NULL
INSERT triggers fire	INSERT triggers do not fire
Can load into clustered tables	Cannot load into clustered tables
Other users can make changes to tables	Other users cannot make changes to tables

Direct & Conventional Path Loads

Logging Changes

Conventional path loading generates redo entries just as any DML statement. When using a Direct path load, redo entries are not generated if:

The database is in NOARCHIVELOG mode
The database is in ARCHIVELOG mode, but logging is disabled. Logging can be disabled by setting the NOLOGGING attribute for the table or by using the UNRECOVERABLE clause in the control file

Direct & Conventional Path Loads

Enforcing Constraints

During Direct path loads, the constraints are handled as follows:

- NOT NULL constraints are checked when arrays are built.
- FK and CHECK constraints are disabled
- PK and unique constraints are checked during and at the end of the run and may be disabled

Direct & Conventional Path Loads

Parallel Direct Paths

Multiple SQL*Loader sessions improve the performance of a Direct path load. Three models of concurrency can be used to minimize the time required for data loading:

- Parallel conventional path loads
- Intersegment concurrency with Direct path load method
- Intra segment concurrency with Direct path load method

Direct & Conventional Path Loads

Parallel Direct Paths

- Concurrent conventional path

When trigger or integrity constraints pose a problem

- Intersegment concurrency with Direct path load method

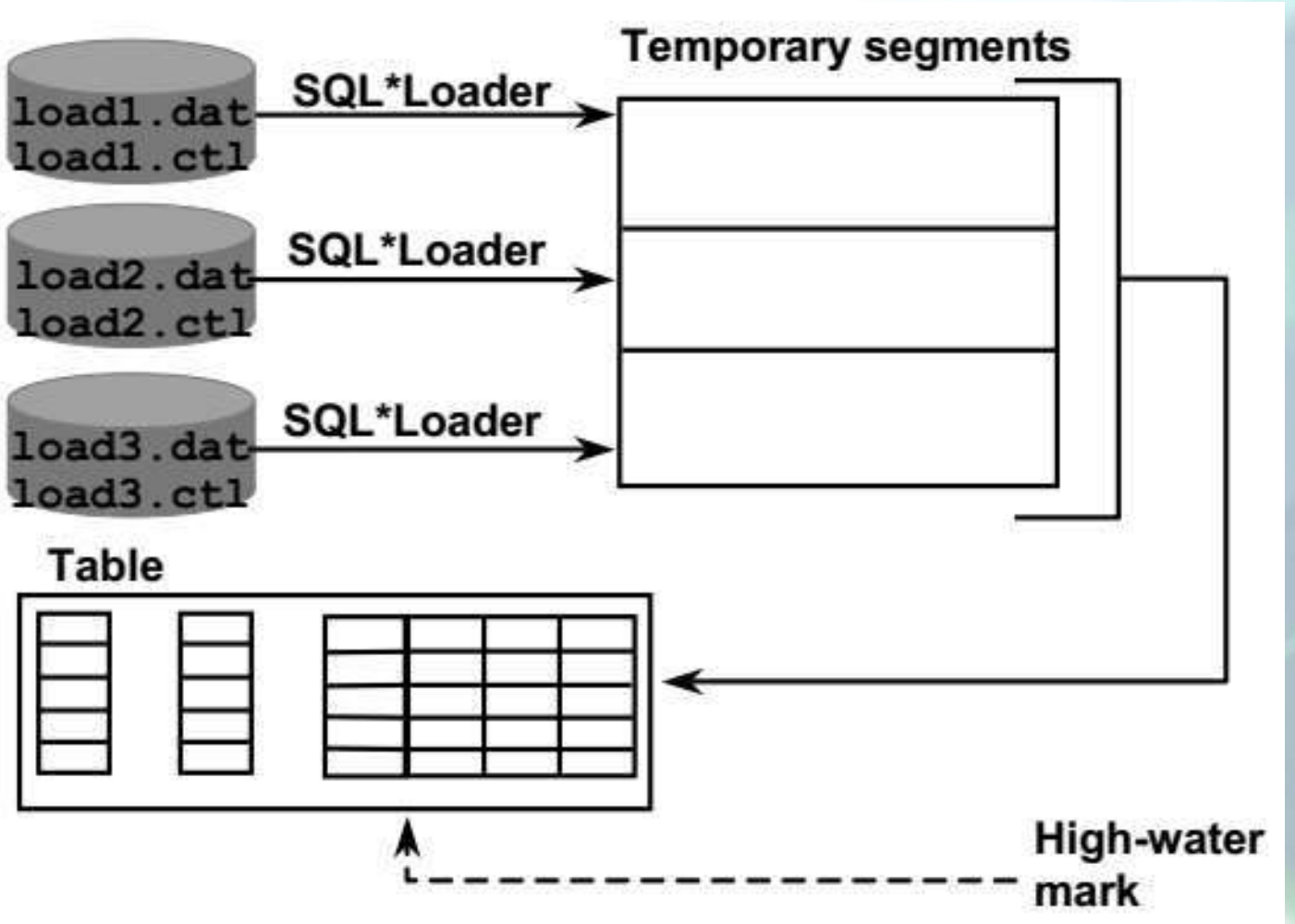
Loading of different objects such as tables and partitions of same table.

- Intra segment concurrency with Direct path load method

Multiple Direct path load session concurrently into same table or partition

Direct & Conventional Path Loads

Enforcing Constraints



Introduction to Database Administration



Data Conversion

Data Conversion

Data Conversion

- Introduction
- SQL*Loader Rejects
- Discarded or Rejected Records

Data Conversion

Introduction

During a conventional path load, data fields in the data file are converted into columns in the database in two steps:

- The field specifications in the control file are used to interpret the format of the data file and convert it to a SQL INSERT statement using that data.

- The Oracle database server accepts the data and executes the INSERT statement to sore the data in the database

Data Conversion

SQL*Loader Rejects

When input format is invalid

- When delaminated field exceeds its maximum length
- All bad records are saved in bad file
- Oracle determines row is valid or not.
- If row or record is rejected, SQL*Loader puts it in the bad file
- Why row can be rejected?

  (unique or not null constraints)

Data Conversion

Discarded or Rejected Records

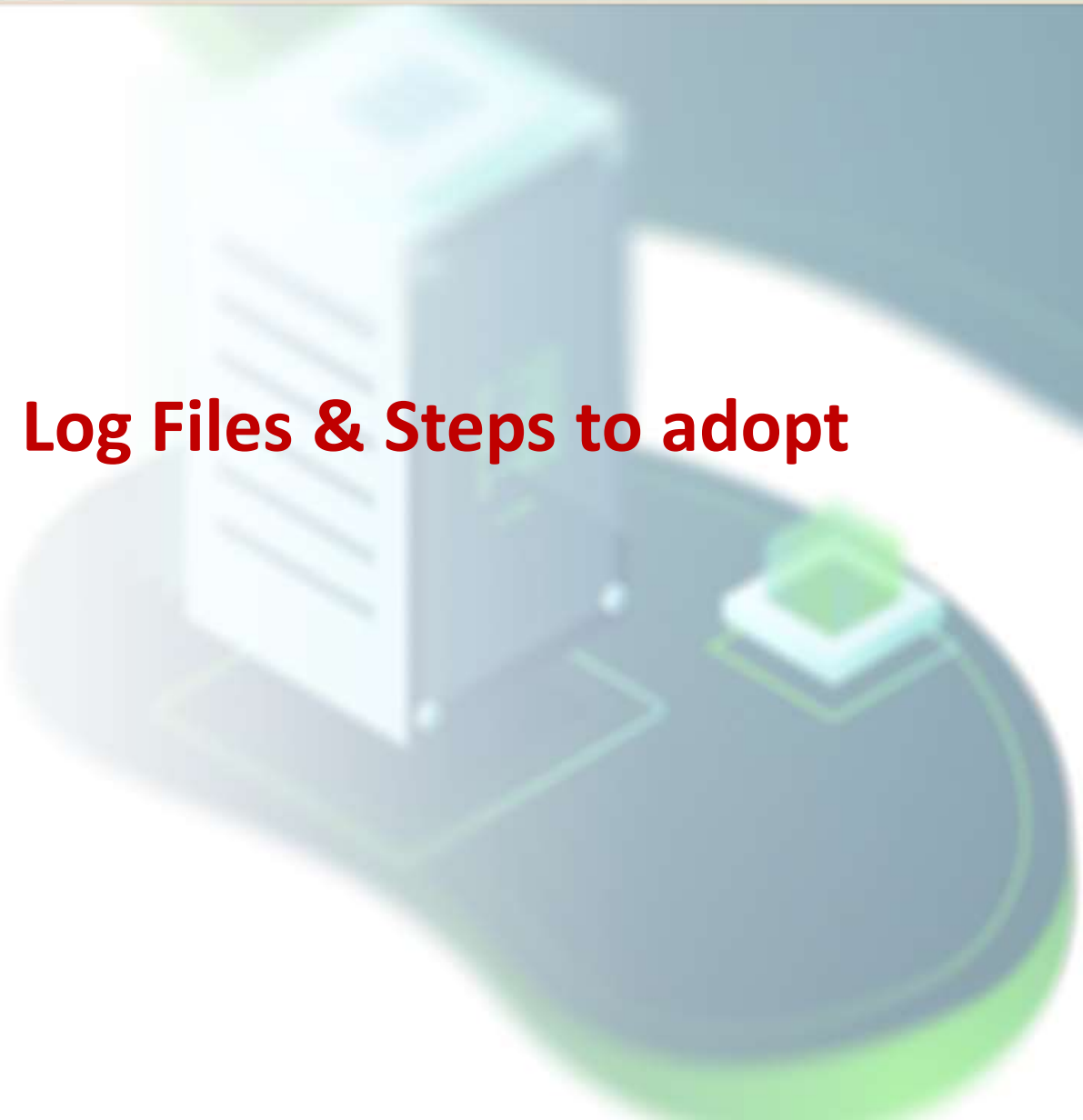
Bad file:

- SQL*Loader rejects records when the input format is invalid.
- If the Oracle database finds that the row is invalid, then the record is rejected and SQL*Loader puts it in the bad file.

Discard file:

- This can be used only if it has been enabled.
- This file contains records that were filtered out because they did not match any record-selection criteria specified in the control file.

Introduction to Database Administration



Log Files & Steps to adopt

Log Files & Steps to adopt

Log File Contents

- Header Information
- Global Information
- Table Information
- Data File Information
- Table Load Information
- Summary Statistics
- Additional statistics for Direct path loads and multithreading Information

Log Files & Steps to adopt

Header Information

Date of the run

Software version number

Log Files & Steps to adopt

Global Information

Name of all input/ output file

Echo of command-line arguments

Continuation character specification

Log Files & Steps to adopt

Table Information

Table Name

Load Conditions

INSERT, APPEND or REPLACE specifications

Columns, length, data type and deliminator

Log Files & Steps to adopt

Data File Information

SQL*Loader and Oracle data record errors



Log Files & Steps to adopt

Table Load Information

Number of rows loaded, rejected, discarded, Null fields discarded

Log Files & Steps to adopt

Summary Statistics

Amount of space used for bind array

Cumulative load statistics for data files, skipped, read and rejected records

Log Files & Steps to adopt

Additional statistics for Direct path loads and multithreading Information

Direct path load of a partitioned table reports per-partition statistics

Conventional-path load cannot report per-partition statistics



Sample Example to Load Data-1

Sample Example to Load Data-1

Example's Contents

- Data Loading
- Data Migration – From File to Database
- Example of Data Loading

Sample Example to Load Data-1

Data Loading

Loading data from files

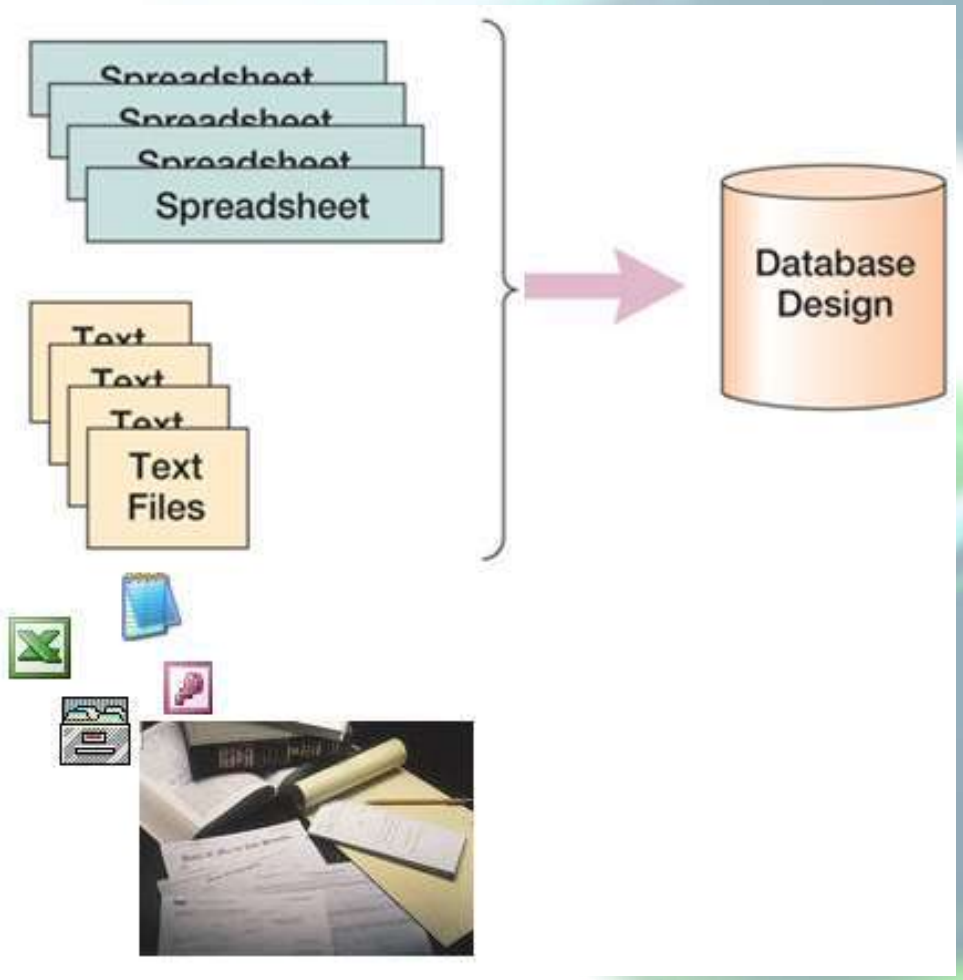
Files can be comma delaminated

Files can be in XML format

Binary files (Oracle to Oracle)

Sample Example to Load Data-1

Data Migration – Files to Database



Sample Example to Load Data-1

Example of Data Loading

```
Login scott
create table TEMP_EMP
(Empno Number(4),
Ename  VARCHAR2(50),
Salary Number(8)
);

c:\test\control.ctl
load data
infile 'c:\test\DBA-EXP-194.csv'
truncate
into table scott.temp_emp
fields terminated by ','
(empno, ename, salary)

cd E:\app\Haider\product\11.2.0\dbhome_1\BIN
c:\test>sqlldr control=c:\test\control.ctl
data=c:\test\DBA-EXP-194.csv log=c:\test\ulcase2.log
```

Sample Example to Load Data-1

Example of Data Loading

```
E:\app\Haider\product\11.2.0\dbhome 1\BIN>sqlldr control=c:\test\control.ctl
                        data=c:\test\DBA-EXP-194.csv log=c:\test\ulcase2.log
Username:scott
Password:
SQL*Loader: Release 11.2.0.1.0 - Production on Wed Nov 4 19:13:30 2020

Copyright (c) 1982, 2009, Oracle and/or its affiliates.  All rights reserved.

Commit point reached - logical record count 6

E:\app\Haider\product\11.2.0\dbhome 1\BIN>
```

Sample Example to Load Data-1

Example of Data Loading

```
SQL> create table TEMP_EMP
  2  (Empno Number(4),
  3  Ename  VARCHAR2(50),
  4  Salary Number(8)
  5  );

Table created.

SQL> select * from temp_emp;

     EMPNO ENAME                                              SALARY
---------- -------------------------------------------------- ----------
      1001 Ejaz                                                     5000
      1002 Ali                                                      3500
      1003 Qadeer                                                   3400
      1004 Yaseen                                                   4000
      1005 Asim                                                     3400
```

Sample Example to Load Data-1

Camtesia





Sample Example to Load Data-2

Sample Example to Load Data-2

Example's Contents

- Material's Data Loading
- Data Migration - Files to Database
- Reasons to Load Data into TEMP_Material
- Required Files Data Loading of Material
- Data Loading Files with Contents

Sample Example to Load Data-2

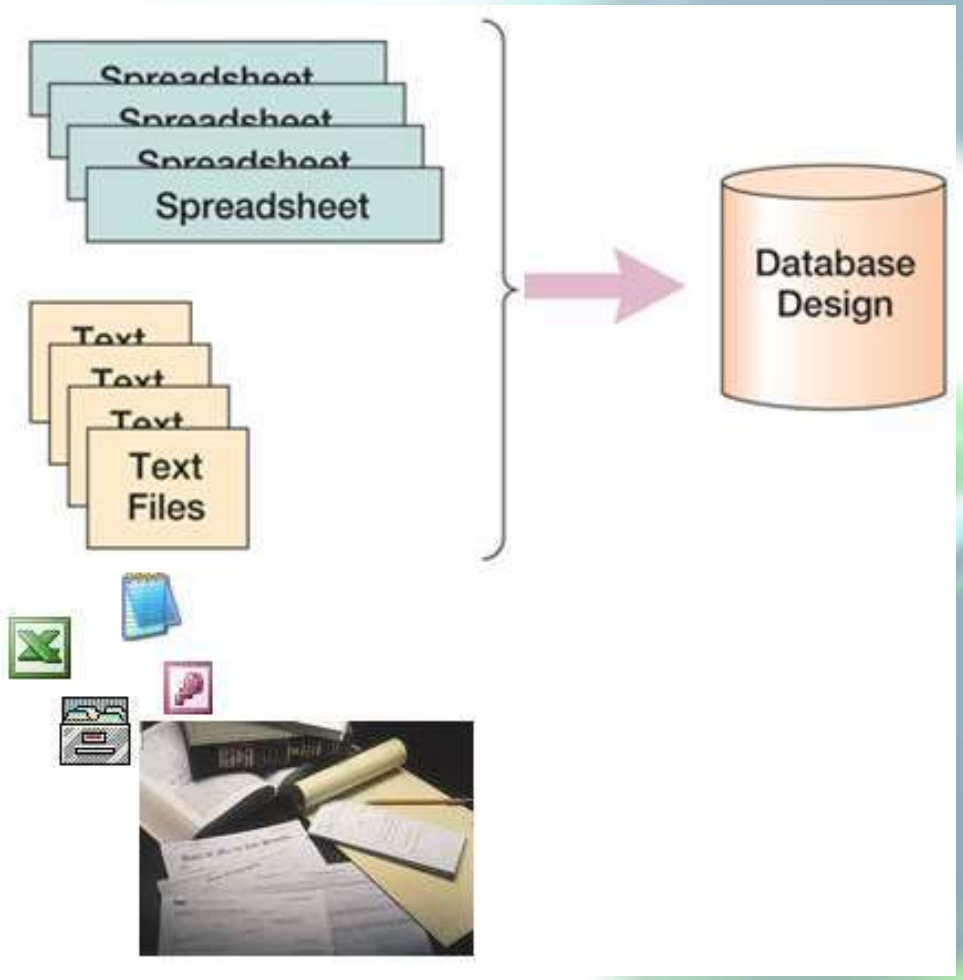
Material's Data Loading

Data is comma delaminated regarding material or products

Need to load into table Material via TMP_Material

Sample Example to Load Data-2

Data Migration – Files to Database



Sample Example to Load Data-2

Reasons to Load Data into TEMP_Material

Try to avoid to load data into bad files

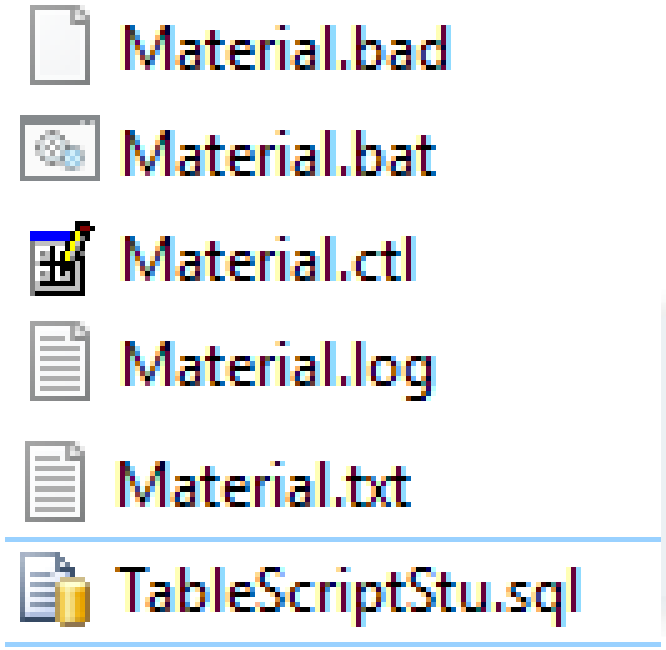
Format of data

For example $ symbol is given with numeric currency column

Size of column may less in the target table

Sample Example to Load Data-2

Required Files Data Loading of Material



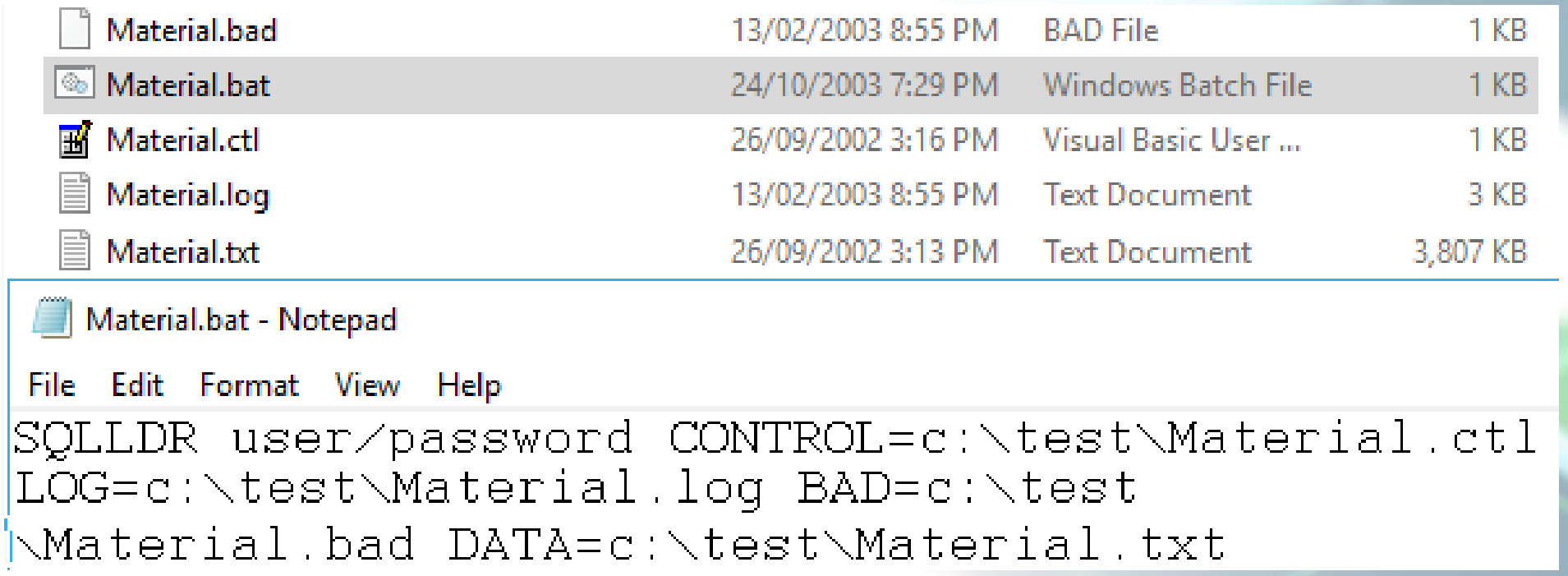
Material.bat - Notepad

File Edit Format View Help

```
SQLLDR user/password CONTROL=c:\test\Material.ctl LOG=c:\test\Material.log BAD=c:\test
\Material.bad DATA=c:\test\Material.txt
```

Sample Example to Load Data-2

Data Loading Files with Contents



Material.bad	13/02/2003 8:55 PM	BAD File	1 KB
Material.bat	24/10/2003 7:29 PM	Windows Batch File	1 KB
Material.ctl	26/09/2002 3:16 PM	Visual Basic User ...	1 KB
Material.log	13/02/2003 8:55 PM	Text Document	3 KB
Material.txt	26/09/2002 3:13 PM	Text Document	3,807 KB

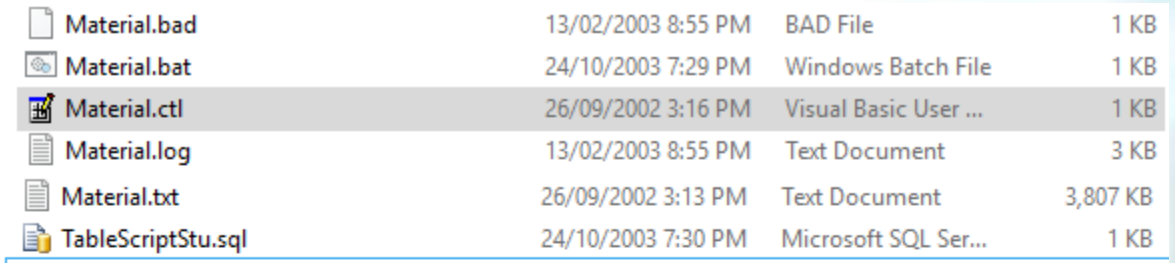
Material.bat - Notepad

File Edit Format View Help

```
SQLLDR user/password CONTROL=c:\test\Material.ctl
LOG=c:\test\Material.log BAD=c:\test
\Material.bad DATA=c:\test\Material.txt
```

Sample Example to Load Data-2

Data Loading Files with Contents



Material.bad	13/02/2003 8:55 PM	BAD File	1 KB
Material.bat	24/10/2003 7:29 PM	Windows Batch File	1 KB
Material.ctl	26/09/2002 3:16 PM	Visual Basic User ...	1 KB
Material.log	13/02/2003 8:55 PM	Text Document	3 KB
Material.txt	26/09/2002 3:13 PM	Text Document	3,807 KB
TableScriptStu.sql	24/10/2003 7:30 PM	Microsoft SQL Ser...	1 KB

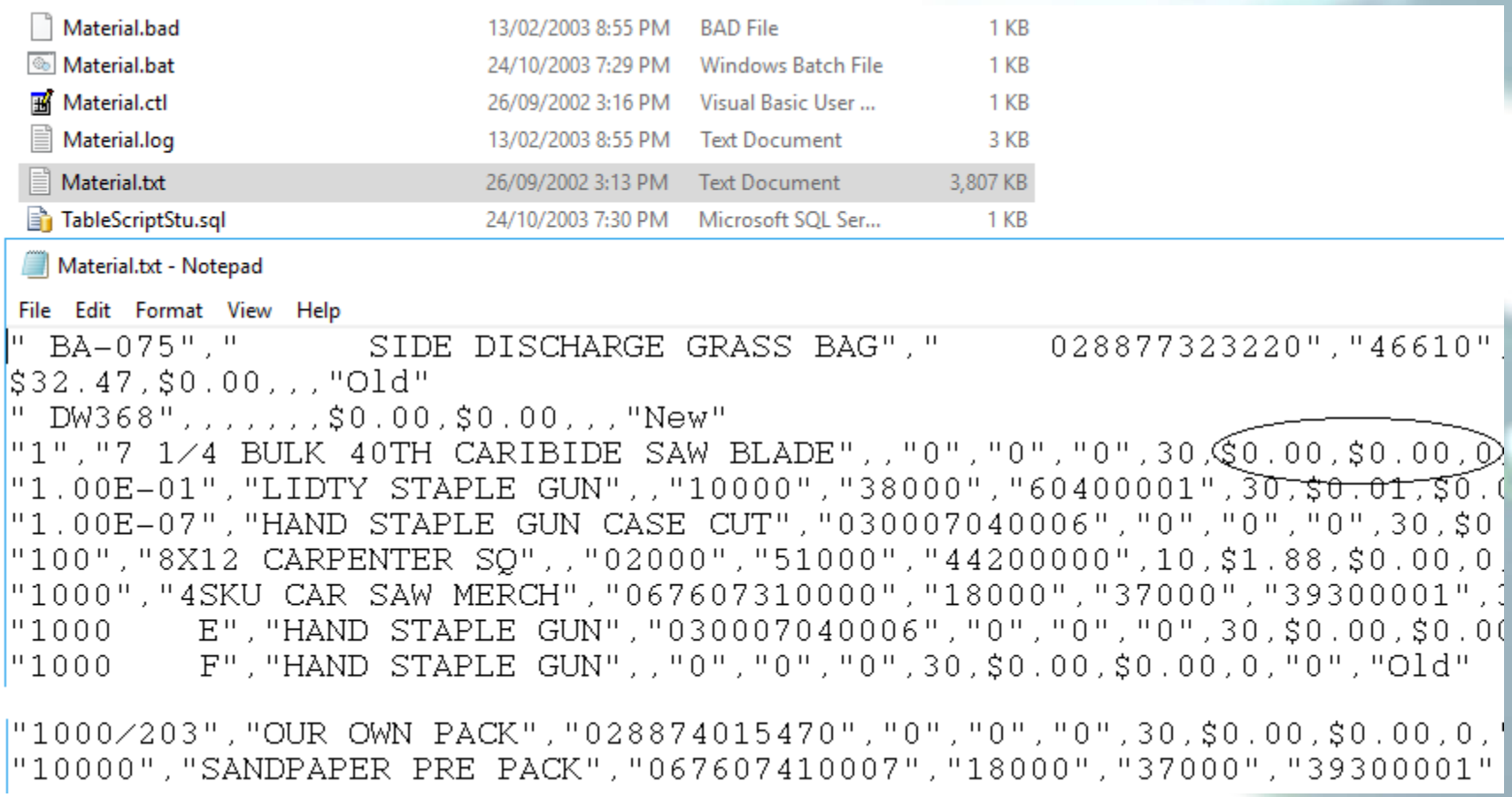
Material.ctl - Notepad

File Edit Format View Help

```
LOAD DATA
REPLACE INTO TABLE TMP_Material
FIELDS TERMINATED BY ','
OPTIONALLY ENCLOSED BY '"'
(BDSKU   ,
BDDescrip       ,
BDUPC           ,
BDMainProdGrp   ,
BDProdGroup     ,
BDSubProdGrp    ,
BDDivision,
BDSysCost       ,
BDBasePrice     ,
BDCurrentInv    ,
BDSalesStatus   ,
Remarks         )
```

Sample Example to Load Data-2

Data Loading Files with Contents



Material.bad	13/02/2003 8:55 PM	BAD File	1 KB
Material.bat	24/10/2003 7:29 PM	Windows Batch File	1 KB
Material.ctl	26/09/2002 3:16 PM	Visual Basic User ...	1 KB
Material.log	13/02/2003 8:55 PM	Text Document	3 KB
Material.txt	26/09/2002 3:13 PM	Text Document	3,807 KB
TableScriptStu.sql	24/10/2003 7:30 PM	Microsoft SQL Ser...	1 KB

Material.txt - Notepad

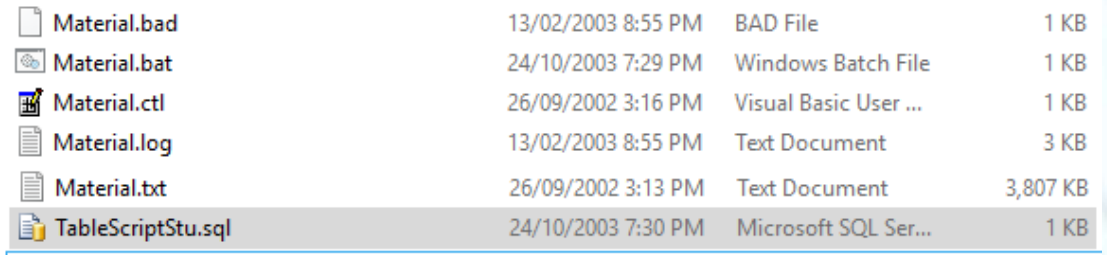
File Edit Format View Help

```
" BA-075","      SIDE DISCHARGE GRASS BAG","     028877323220","46610",
$32.47,$0.00,,,"Old"
" DW368",,,,,,,$0.00,$0.00,,,"New"
"1","7 1/4 BULK 40TH CARIBIDE SAW BLADE",,"0","0","0",30,$0.00,$0.00,0,
"1.00E-01","LIDTY STAPLE GUN",,"10000","38000","60400001",30,$0.01,$0.0
"1.00E-07","HAND STAPLE GUN CASE CUT","030007040006","0","0","0",30,$0
"100","8X12 CARPENTER SQ",,"02000","51000","44200000",10,$1.88,$0.00,0,
"1000","4SKU CAR SAW MERCH","067607310000","18000","37000","39300001",
"1000    E","HAND STAPLE GUN","030007040006","0","0","0",30,$0.00,$0.00
"1000    F","HAND STAPLE GUN",,"0","0","0",30,$0.00,$0.00,0,"0","Old"

"1000/203","OUR OWN PACK","028874015470","0","0","0",30,$0.00,$0.00,0,
"10000","SANDPAPER PRE PACK","067607410007","18000","37000","39300001"
```

Sample Example to Load Data-2

Data Loading Files with Contents



Material.bad	13/02/2003 8:55 PM	BAD File	1 KB
Material.bat	24/10/2003 7:29 PM	Windows Batch File	1 KB
Material.ctl	26/09/2002 3:16 PM	Visual Basic User ...	1 KB
Material.log	13/02/2003 8:55 PM	Text Document	3 KB
Material.txt	26/09/2002 3:13 PM	Text Document	3,807 KB
TableScriptStu.sql	24/10/2003 7:30 PM	Microsoft SQL Ser...	1 KB

TableScriptStu.sql - Notepad

File Edit Format View Help

```
create table TMP_Material
(BDSKU            VARCHAR2(255),
BDDescrip         VARCHAR2(255),
BDUPC             VARCHAR2(255),
BDMainProdGrp     VARCHAR2(255),
BDProdGroup       VARCHAR2(255),
BDSubProdGrp      VARCHAR2(255),
BDDivision        VARCHAR2(255),
BDSysCost         VARCHAR2(255),
BDBasePrice       VARCHAR2(255),
BDCurrentInv      VARCHAR2(255),
BDSalesStatus     VARCHAR2(255),
Remarks           VARCHAR2(255)
);

create table Material
(BDSKU            VARCHAR2(15),
```

Sample Example to Load Data-2

Camtesia





Sample Example to Load Data-3

Sample Example to Load Data-3

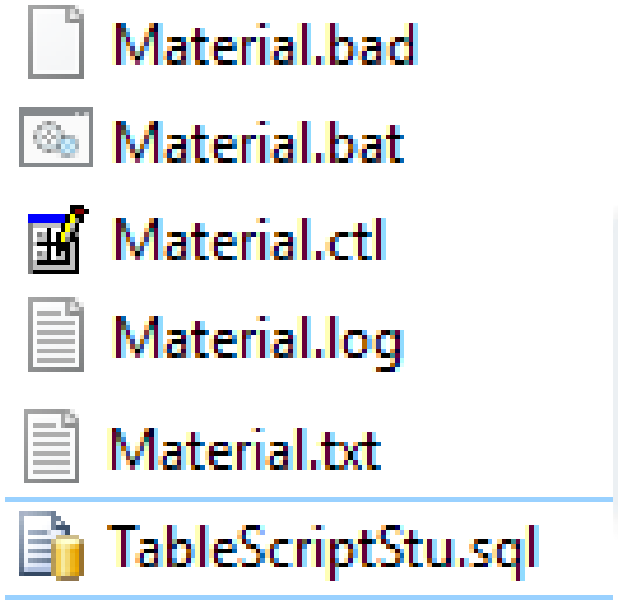
Example's Contents

- Executing Data Loading for Material (for Slide#195)
- Data Migration - Files to Database

Sample Example to Load Data-3

Executing Data Loading for Material

We will execute SQLLDR command to load data of material into scott's schema



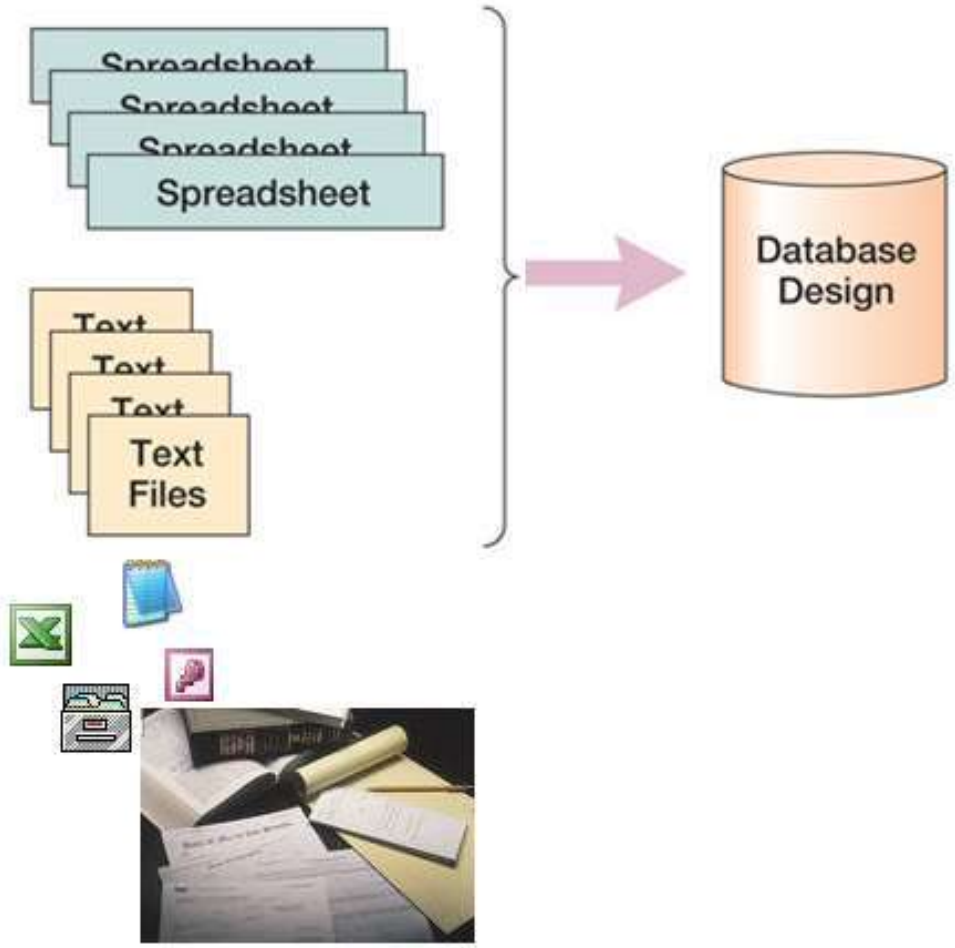
Material.bat - Notepad

File Edit Format View Help

```
SQLLDR user/password CONTROL=c:\test\Material.ctl LOG=c:\test\Material.log BAD=c:\test
\Material.bad DATA=c:\test\Material.txt
```

Sample Example to Load Data-3

Data Migration – Files to Database



Sample Example to Load Data-3

Camtesia





Sample Example to Load Data-4

Sample Example to Load Data-4

Data Loading Tasks in Schema

- Loading data from TEMP to actual Material Table
- Tasks to load data from TEMP to actual Material Table

Sample Example to Load Data-4

Loading data from TEMP to actual Material Table

- You have data in TMP_Material and you want to move or copy data into Material Table

Sample Example to Load Data-4

Tasks to load data from TEMP to actual Material Table

- Use INSERT INTO … SELECT command
- Understand an error
- See where data discrepancies are found
- Run SQL UPDATE statements to fix discrepancies

Sample Example to Load Data-4

Camtesia





Sample Example to Load Data-5

Sample Example to Load Data-5

Data Loading Tasks using MS Access

- Tasks to Connect MS Access & Oracle
- Choosing MS Access by Connecting with Oracle Schema
- Link Tables of MS Access & Oracle Schema

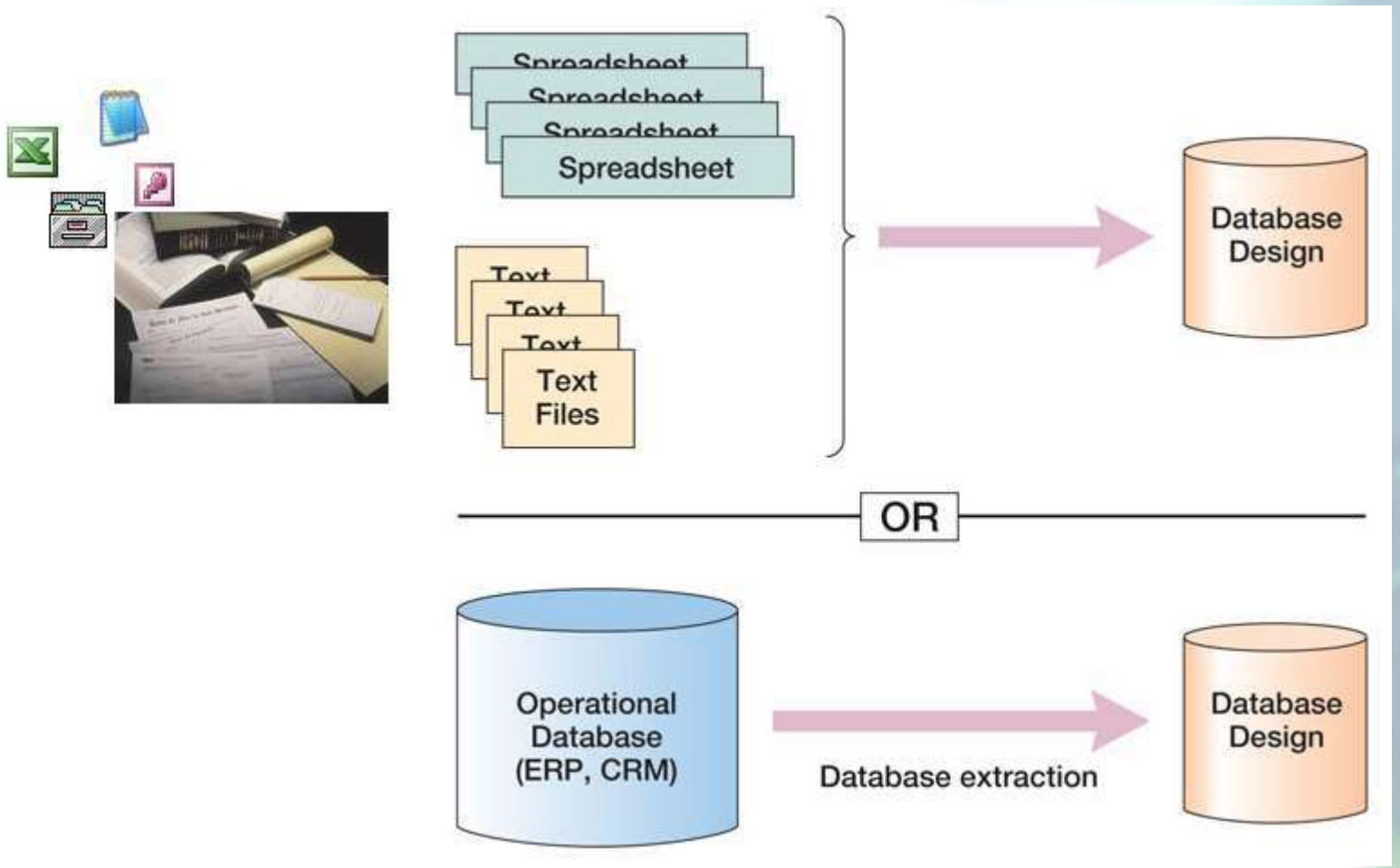
Sample Example to Load Data-5

Tasks to Connect MS Access & Oracle

- Create ODBC data source of Oracle with use scott
- Open sample MS Access File and connect it with ODBC source user scott
- Link or data import is possible in MS Access
- Data can be inserted from MS Access into Oracle schema scott

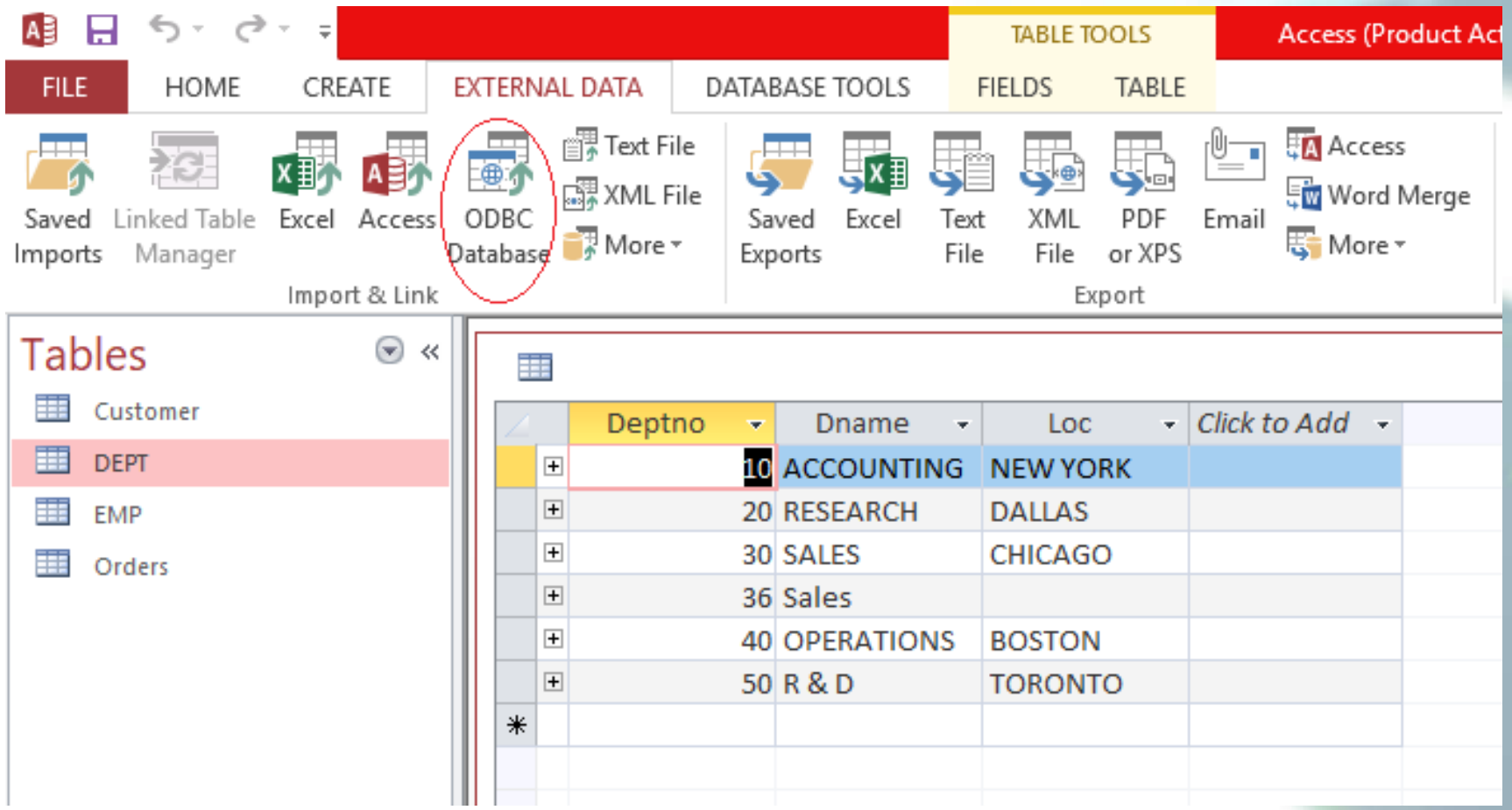
Sample Example to Load Data-5

Choosing MS Access by Connecting with Oracle Schema



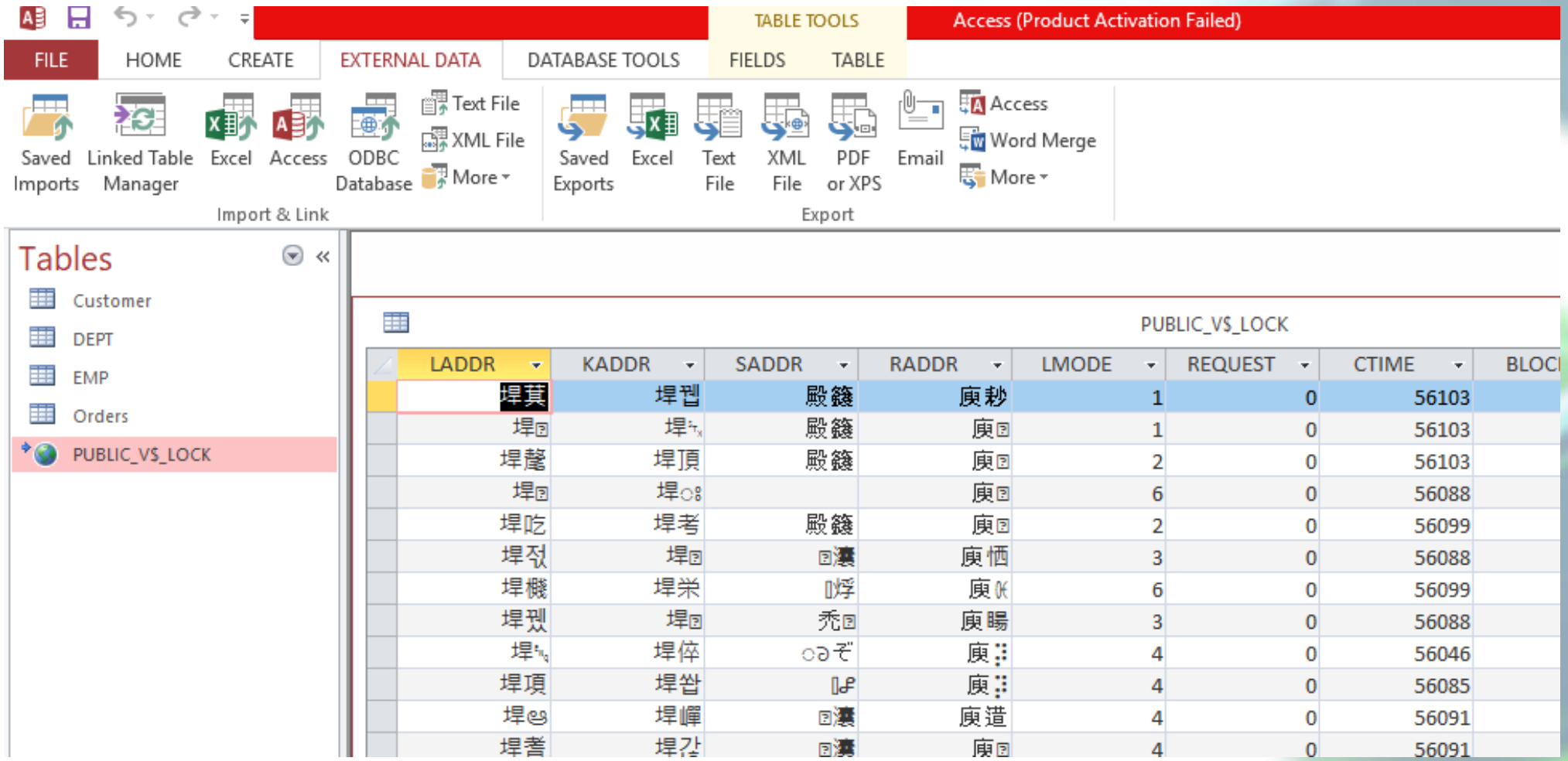
Sample Example to Load Data-5

Link Tables of MS Access & Oracle Schema



Sample Example to Load Data-5

Link Tables of MS Access & Oracle Schema



Sample Example to Load Data-5

Camtesia





Sample Example to Load Data-6

Sample Example to Load Data-6

Data Loading Tasks for Schema to Schema/ Files using Text Files

- Preparing Spool File of Schema's Table using SQL
- Data Load from Oracle Schema into Files

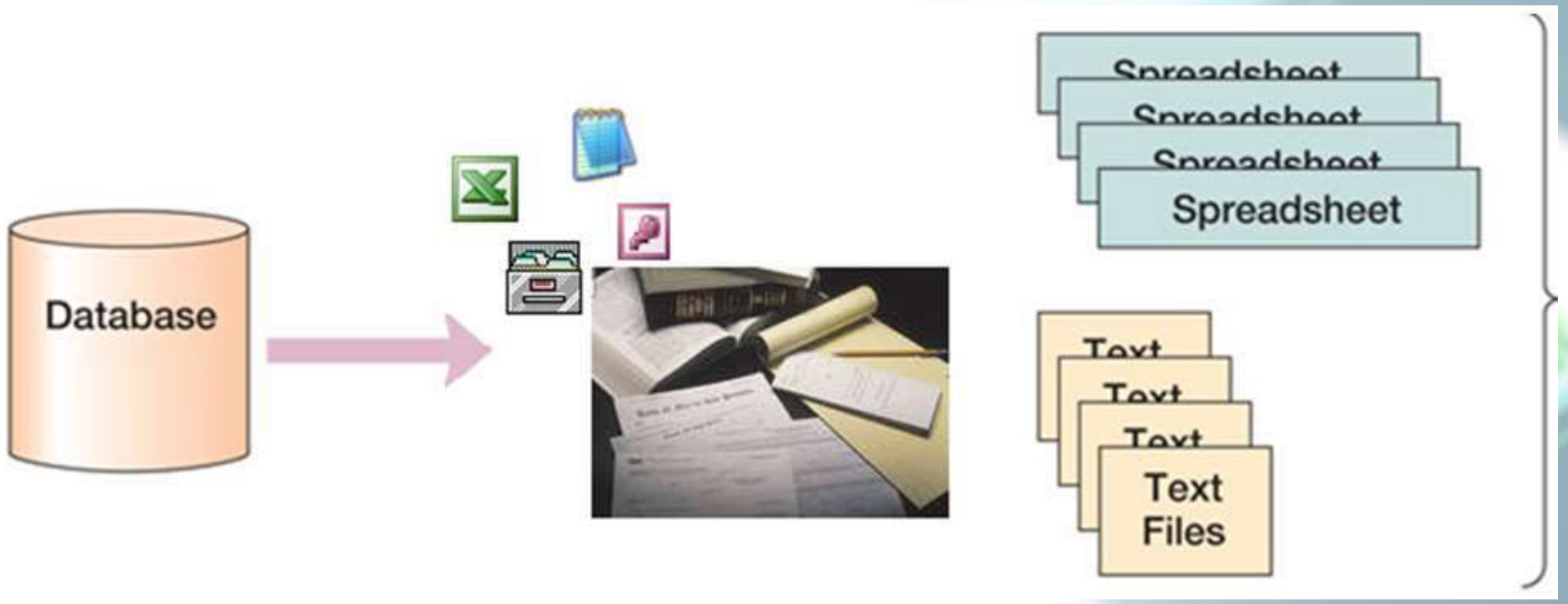
Sample Example to Load Data-6

Preparing Spool File of Schema's Table using SQL

- In Scott schema
  - Spool c:\test\emp.txt
  - Select … from EMP;
  - Spool off
- Spool file contains comma delaminated rows
- File can be used to load data into any DBMS

Sample Example to Load Data-6

Data Load from Oracle Schema into Files



Sample Example to Load Data-6

Camtesia





Sample Example to Load Data-7

Sample Example to Load Data-7

Data Preparation for Tasks using XML data

- Preparing Spool File of Schema's Table using XML SQL
- Sample SQL using XML Elements
- Data Load from Oracle Schema into Files
- Output Results of Sample XML SQLs

Sample Example to Load Data-7

Preparing Spool File of Schema's Table using XML SQL

- In Scott schema
  - Spool c:\test\emp.txt
  - XML: Select … from EMP;
  - Spool off
- Spool file contains XML-taged rows
- File can be used to load data into any DBMS

Sample Example to Load Data-7

Sample SQL using XML Elements

```
select xmlelement(name "emp",
      xmlelement(name "eno",empno),
      xmlelement(name "names",ename))
XML_output
from emp
/
```

Sample Example to Load Data-7

Sample SQL using XML Elements

```
create table XML1 as
select xmlelement("EMP",
      xmlelement("empno",e.empno),
      xmlelement("ename",e.ename),
      xmlelement("job", e.job),
      xmlelement("mgr", e.mgr),
      xmlelement("mgr1", e.mgr)) xml_desc
from emp e
where rownum<=1;
```

Sample Example to Load Data-7

Data Load from Oracle Schema into Files



Sample Example to Load Data-7

Output Results of Sample XML SQLs

```
SQL> select xmlelement(name "emp",
  2         xmlelement(name "eno",empno),
  3         xmlelement(name "names",ename)) XML_output
  4  from emp
  5  /

XML_OUTPUT
--------------------------------------------------------------
<emp><eno>1234</eno><names></names></emp>
<emp><eno>8822</eno><names>ABC</names></emp>
<emp><eno>3513</eno><names>XYZ</names></emp>
<emp><eno>7369</eno><names>SMITH</names></emp>
<emp><eno>7499</eno><names>ALLEN</names></emp>
<emp><eno>7521</eno><names>WARD</names></emp>
<emp><eno>7566</eno><names>JONES</names></emp>
<emp><eno>7654</eno><names>MARTIN</names></emp>
<emp><eno>7698</eno><names>BLAKE</names></emp>
<emp><eno>7782</eno><names>CLARK</names></emp>
<emp><eno>7788</eno><names>SCOTT</names></emp>
```

```
SQL> create table XML1 as
  2  select xmlelement("EMP",
  3         xmlelement("empno",e.empno),
  4         xmlelement("ename",e.ename),
  5         xmlelement("job", e.job),
  6         xmlelement("mgr", e.mgr),
  7         xmlelement("mgr1", e.mgr)) xml_desc
  8  from emp e
  9  where rownum<=1;

Table created.

SQL> select * from XML1;

XML_DESC
--------------------------------------------------------------------------------
<EMP><empno>1234</empno><ename></ename><job></job><mgr></mgr><mgr1></mgr1></EMP>
```

Sample Example to Load Data-7

Camtesia



Introduction to Database Administration



Backup & Recovery Strategies

Backup & Recovery Strategies

- Backup & Recovery Concepts
- Physical & Logical Backup



Backup & Recovery Strategies

Back & Recovery Concepts

- Backup and recovery is the set of concepts, procedures, and strategies involved in protecting the database against data loss caused by media failure or users errors.
- In general, the purpose of a backup and recovery strategy is to protect the database against data loss and reconstruct lost data.

Backup & Recovery Strategies

Back & Recovery Concepts

- A backup is a copy of data. A backup can include crucial parts of the database such as data files, the server parameter file, and control file. A sample backup and recovery scenario is a failed disk drive that causes the loss of a data file.
- If a backup of the lost file exists, then you can restore and recover it. Media recovery refers to the operations involved in restoring data to its state before the loss occurred.

Backup & Recovery Strategies

Physical & Logical Backup

- Backups can be divided into physical backups and logical backups.
- Physical backups are backups of the physical files used in storing and recovering your database, such as datafiles, control files, and archived redo logs. Ultimately, every physical backup is a copy of files storing database information to some other location, whether on disk or some offline storage such as tape.

Backup & Recovery Strategies

Physical & Logical Backup

- Logical backups contain logical data (for example, tables or stored procedures) exported from a database with an Oracle export utility and stored in a binary file, for later re-importing into a database using the corresponding Oracle import utility.

Backup & Recovery Strategies

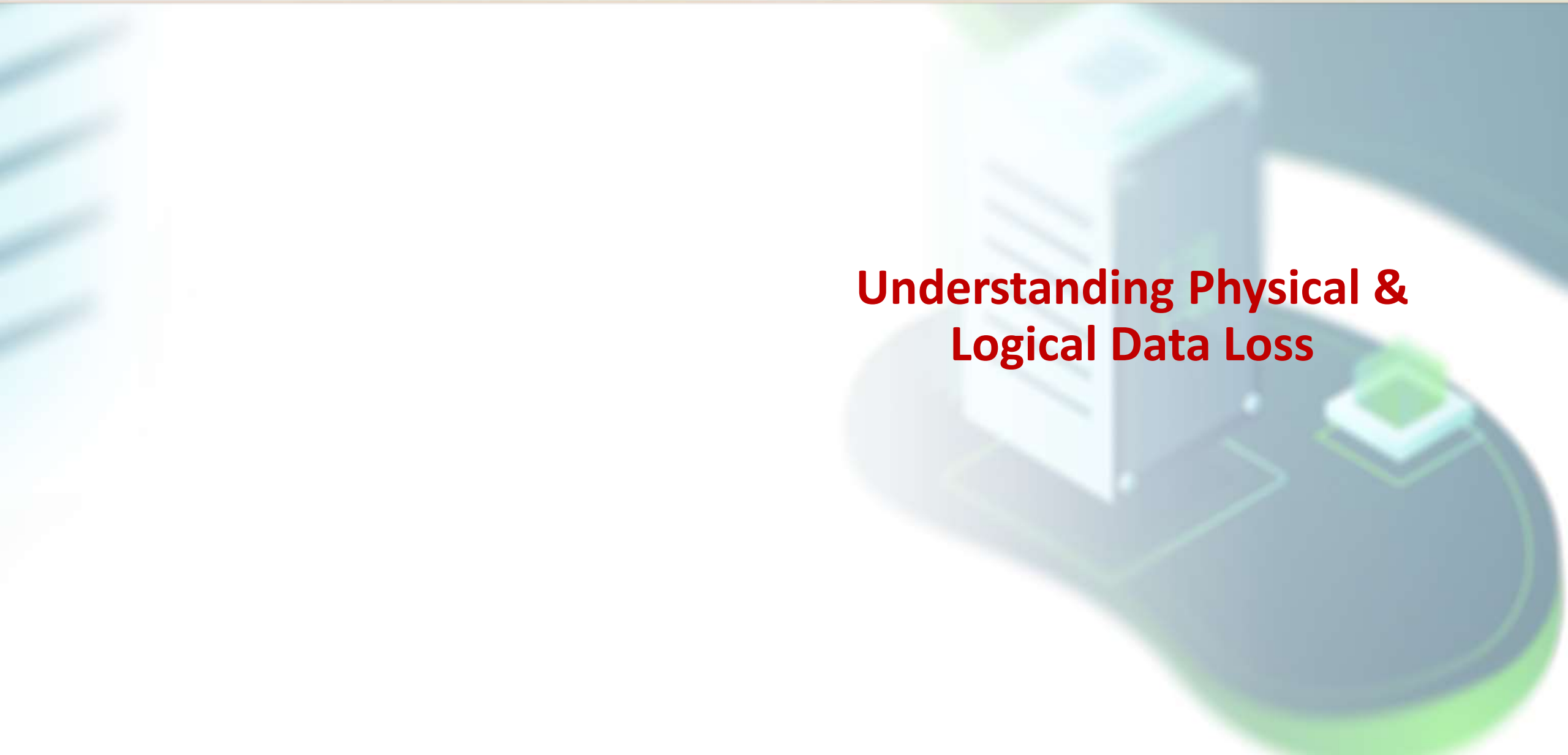
Physical & Logical Backup

- Physical backups are the foundation of any sound backup and recovery strategy.
- Logical backups are a useful supplement to physical backups in many circumstances but are not sufficient protection against data loss without physical backups.

Backup & Recovery Strategies

Camtesia





Understanding Physical & Logical Data Loss

Understanding Physical & Logical Data Loss

Understanding Physical & Logical Data Loss

- Errors & Failures
- Understanding User Error
- Understanding Media Failure

Understanding Physical & Logical Data Loss

Errors & Failures

- While there are several types of problem that can halt the normal operation of an Oracle database or affect database I/O operations, only two typically require DBA intervention and media recovery: media failure, and user errors.

Understanding Physical & Logical Data Loss

Errors & Failures

- Other failures may require DBA intervention to restart the database (after an instance failure) or allocate more disk space (after statement failure due to, for instance, a full datafile) but these situations will not generally cause data loss or require recovery from backup.

Understanding Physical & Logical Data Loss

Understanding User Error

- User errors occur when, either due to an error in application logic or a manual mis-step, data in your database is changed or deleted incorrectly.
- Data loss due to user error includes such missteps as dropping important tables or deleting or changing the contents of a table.

Understanding Physical & Logical Data Loss

Understanding User Error

- While user training and careful management of privileges can prevent most user errors, your backup strategy determines how gracefully you recover the lost data when user error does cause data loss.

Understanding Physical & Logical Data Loss

Understanding Media Failure

- A **media failure** is the failure of a read or write of a disk file required to run the database, due to a physical problem with the disk such as a head crash. Any database file can be vulnerable to a media failure.
- The appropriate recovery technique following a media failure depends on the files affected and the types of backup available.

Understanding Physical & Logical Data Loss

Camtesia



Introduction to Database Administration



Full Logical Backup

Full Logical Backup

Full Logical Backup

- Logical Backup
- Logical Backup Entire Applications
- Outcome of Logical Backup
- Periodic Logical Backup

Full Logical Backup

Logical Backup

- A logical backup copies data, but not physical files, from one location to another. A logical backup is used to move or archive a database, tables, or schemas and to verify database structures.

Full Logical Backup

Logical Backup Entire Applications

A full logical backup enables you to copy these items across environments that use different components, such as operating systems:

Entire applications

Data repositories such as the Oracle Hyperion Shared Services Registry and Oracle Essbase cubes

Individual artifacts such as scripts, data forms, and rule files

Full Logical Backup

Outcome of Logical Backup

A logical export backup generates necessary Structured Query Language (SQL) statements to obtain all table data that is written to a binary file.

A logical export backup does not contain database instance-related information, such as the physical disk location, so you can restore the same data on another database machine.

Full Logical Backup

Periodic Logical Backup

Periodic logical export backups (at least weekly) are recommended in case physical backups fail or the database machine becomes unavailable.

Full Logical Backup

Logical Backup using EXPORT

```
E:\app\Haider\product\11.2.0\dbhome_1\BIN>exp

Export: Release 11.2.0.1.0 - Production on Wed Nov 11 19:29:15 2020

Copyright (c) 1982, 2009, Oracle and/or its affiliates.  All rights reserved.


Username: scott/tiger

Connected to: Oracle Database 11g Enterprise Edition Release 11.2.0.1.0 - Production
With the Partitioning, OLAP, Data Mining and Real Application Testing options
Enter array fetch buffer size: 4096 >

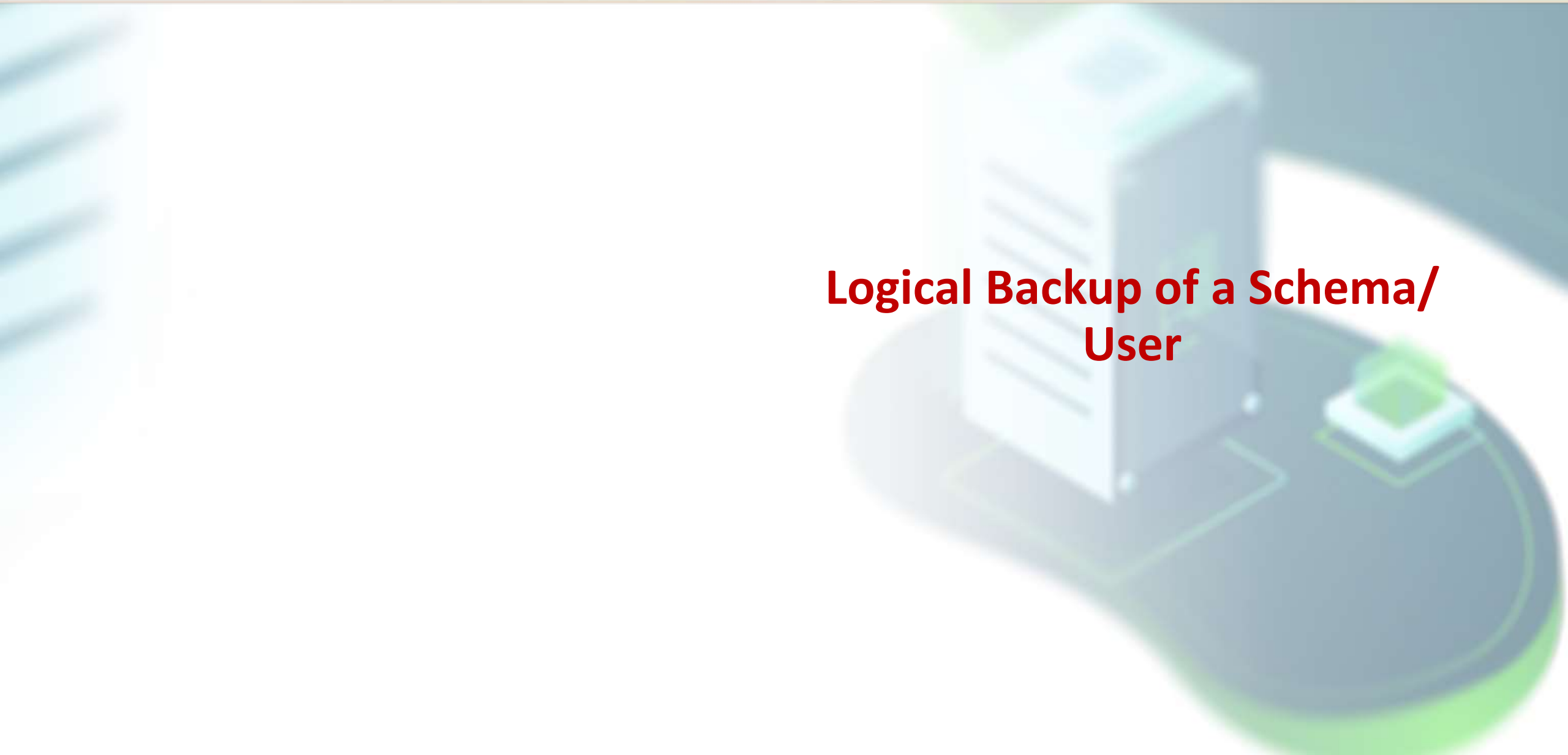
Export file: EXPDAT.DMP > c:\t1.dmp

(1)E(ntire database), (2)U(sers), or (3)T(ables): (2)U >
```

Full Logical Backup

Camtesia





Logical Backup of a Schema/ User

Logical Backup of a Schema/ User

Logical Backup of a Schema/ User

- Logical Backup of Schema
- Export utility for Logical Backup
- Benefits of Export Utility for Logical Backup

Logical Backup of a Schema/ User

Logical Backup of Schema

- A logical backup copies the data in the database and does not records the location of data.
- It can copy the data of schema, database and tables

Logical Backup of a Schema/ User

Export utility for Logical Backup

- The export utility (export and export pump) offered by the Oracle can be used to take logical backup.
- The export utility copies the data and database definitions, and saves in binary OS file in Oracle internal format

Logical Backup of a Schema/ User

Benefits of Export Utility for Logical Backup

- Data block corruption can be detected while exporting data and export procedure will fail.
- If use drops some table and it will recover from import command.
- What data and definition/ structure can be exported?
- Portable backup, can be imported in same or higher version of Oracle database.

Logical Backup of a Schema/ User

Camtesia





Logical Backup of Schema Contents Tables

Logical Backup of Schema Contents Tables

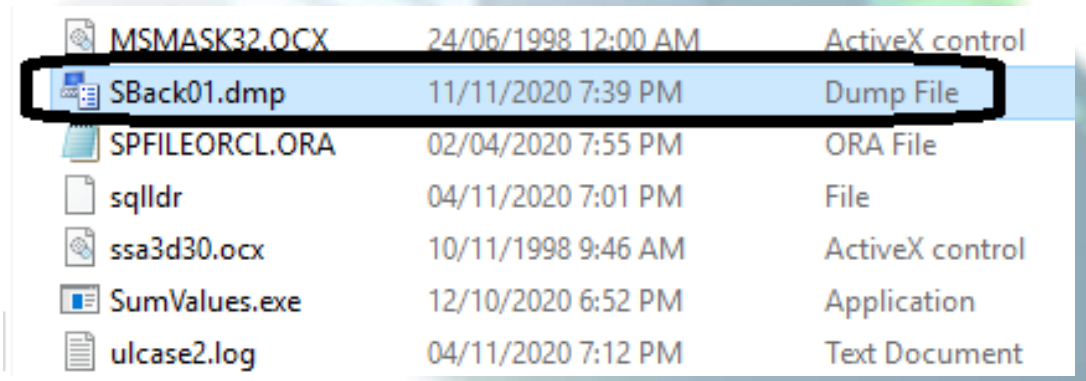
Logical Backup of Schema Contents Tables

- Logical Backup of Schema Contents Tables
- Logical Backup Full/ Tables
- Export using Full Option
- Export with Individual Table

Logical Backup of Schema Contents Tables

Logical Backup Full/ Tables

- Run export command with Full=y option.
- User will be scott/tiger
- Export data of some table



Logical Backup of Schema Contents Tables

Export using Full Option

```
E:\app\Haider\product\11.2.0\dbhome_1\BIN>exp scott/tiger full=y file=c:\test\FBack01.dmp constraints=Y

Export: Release 11.2.0.1.0 - Production on Wed Nov 11 19:49:30 2020

Copyright (c) 1982, 2009, Oracle and/or its affiliates.  All rights reserved.


Connected to: Oracle Database 11g Enterprise Edition Release 11.2.0.1.0 - Production
With the Partitioning, OLAP, Data Mining and Real Application Testing options
Export done in WE8MSWIN1252 character set and AL16UTF16 NCHAR character set

About to export the entire database ...
. exporting tablespace definitions
. exporting profiles
. exporting user definitions
. exporting roles
. exporting resource costs
. exporting rollback segment definitions
. exporting database links
. exporting sequence numbers
. exporting directory aliases
. exporting context namespaces
. exporting foreign function library names
. exporting PUBLIC type synonyms
. exporting private type synonyms
. exporting object type definitions
. exporting system procedural objects and actions
```

Logical Backup of Schema Contents Tables

Export with Individual Table

```
Username: scott/tiger

Connected to: Oracle Database 11g Enterprise Edition Release 11.2.0.1.0 - Production
With the Partitioning, OLAP, Data Mining and Real Application Testing options
Enter array fetch buffer size: 4096 >

Export file: EXPDAT.DMP > c:\test\emp.dmp

(1)E(ntire database), (2)U(sers), or (3)T(ables): (2)U > t

Export table data (yes/no): yes >

Compress extents (yes/no): yes >

Export done in WE8MSWIN1252 character set and AL16UTF16 NCHAR character set

About to export specified tables via Conventional Path ...
Table(T) or Partition(T:P) to be exported: (RETURN to quit) > EMP

. . exporting table                            EMP         17 rows exported
Table(T) or Partition(T:P) to be exported: (RETURN to quit) > DEPT

. . exporting table                           DEPT          4 rows exported
Table(T) or Partition(T:P) to be exported: (RETURN to quit) >

Export terminated successfully without warnings.

F:\app\Haider\product\11.2.0\dbhome_1\BIN>
```

Logical Backup of Schema Contents Tables

Camtesia





Cold Physical Backup

Cold Physical Backup

Cold Physical Backup

- Cold Physical Backup
- Cold Physical Backup Files
- Performing Cold Backup

Cold Physical Backup

Cold Physical Backup

- Offline or cold backups are performed when the database is completely shutdown.
- The disadvantage of an offline backup is that it cannot be done if the database needs to be run 24/7.
- Additionally, you can only recover the database up to the point when the last backup was made unless the database is running in ARCHIVELOG mode.

Cold Physical Backup

Cold Physical Backup Files

An offline backup consists of physically copying the following files:

- Data files
- Control files
- Init.ora and config.ora files

Warning: If you make a cold backup in ARCHIVELOG mode do not backup redo log files.

Cold Physical Backup

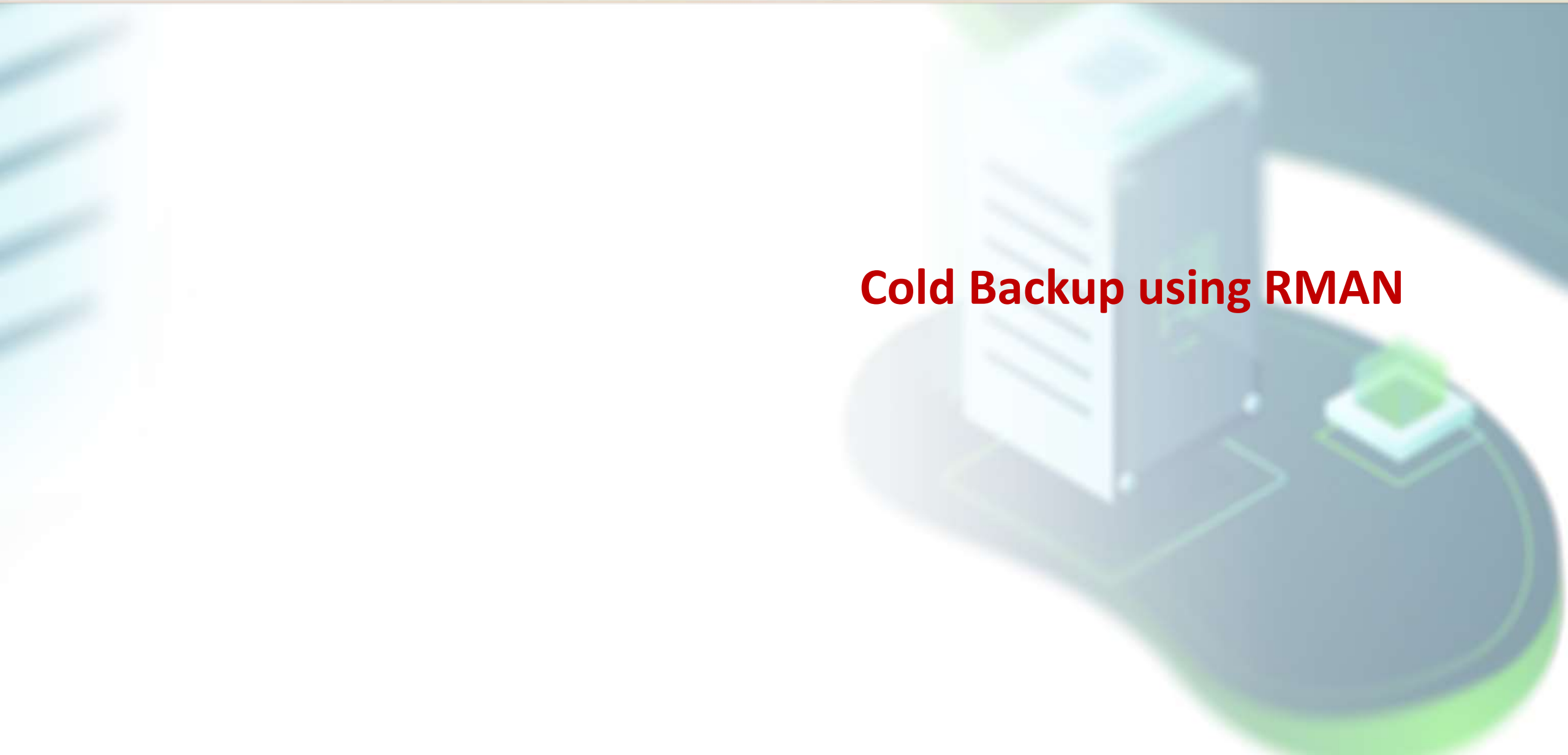
Performing Cold Backup

- Before performing a cold backup, you need to know the location of the files that need to be backed up.
- Because the database structure changes day to day as more files get added or moved between directories, it is always better to query the database to get the physical structure of database before making a cold backup.

Cold Physical Backup

Camtesia





Cold Backup using RMAN

Cold Backup using RMAN

- Cold Backup using RMAN
- Start Backup using RMAN

Cold Backup using RMAN

Cold Backup using RMAN

- Keep Database in Archive Mode
- Go to location where RMAN can run

E:\app\Haider\product\11.2.0\dbhome_1
\BIN>rman target /

RMAN>show all;

Cold Backup using RMAN

Start Backup using RMAN

RMAN> Backup as Backupset database;

```
C:\>dir \rmanbackup\rman
 Volume in drive C has no label.
 Volume Serial Number is 4E83-C14F

 Directory of C:\rmanbackup\rman

11/26/2014  10:16 PM    <DIR>          .
11/26/2014  10:16 PM    <DIR>          ..
11/26/2014  10:16 PM       650,985,472 FULL_0HPOK2RP_17_1
11/26/2014  10:16 PM         7,143,424 FULL_0IPOK2TH_18_1
               2 File(s)    658,128,896 bytes
               2 Dir(s)  447,372,935,168 bytes free
```

Cold Backup using RMAN

Camtesia



Introduction to Database Administration



Hot Backup Concepts

Hot Backup Concepts

- Hot Backup Concepts
- Startup Archive Mode



Hot Backup Concepts

Hot Backup Concepts

- A hot backup is taken when the database needs to run all the time.
- It is an online backup.
- All files of the database are copied and there may be changes to the database during the copy.

Hot Backup Concepts

Hot Backup Concepts

- An online backup or hot backup is also referred to as ARCHIVE LOG backup. An online backup can only be done when the database is running in ARCHIVELOG mode and the database is open.
- When the database is running in ARCHIVELOG mode, the archiver (ARCH) background process will make a copy of the online redo log file to archive backup location.

Hot Backup Concepts

Hot Backup Concepts

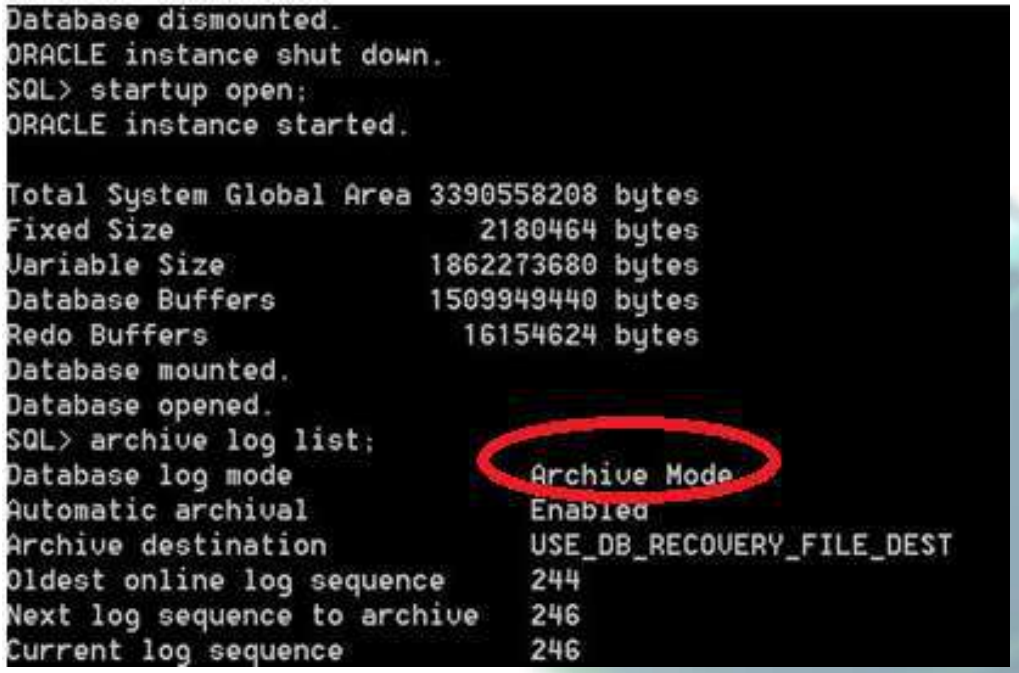
An online backup consists of backing up the following files. But, because the database is open while performing a backup, you have to follow the procedure to backup the following files:

- Data files of each tablespace
- Archived redo log files
- Control file
- Init.ora and config.ora files

Hot Backup Concepts

Startup Archive Mode

Shutdown database and startup NoArchive to Archive database process



Hot Backup Concepts

Camtesia



Introduction to Database Administration



Hot Backup Demo

Hot Backup Demo

Hot Backup Demo

- Hot Backup Demo
- Advise for Hot Backup

Hot Backup Demo

Hot Backup Demo

Following steps are required for Hot Backup:

- Put the tablespace in the Backup mode and copy the data files.
- Back up the control and Init.ora files.
- Stop archiving
- Back up the archive files
- Restart the archive process.

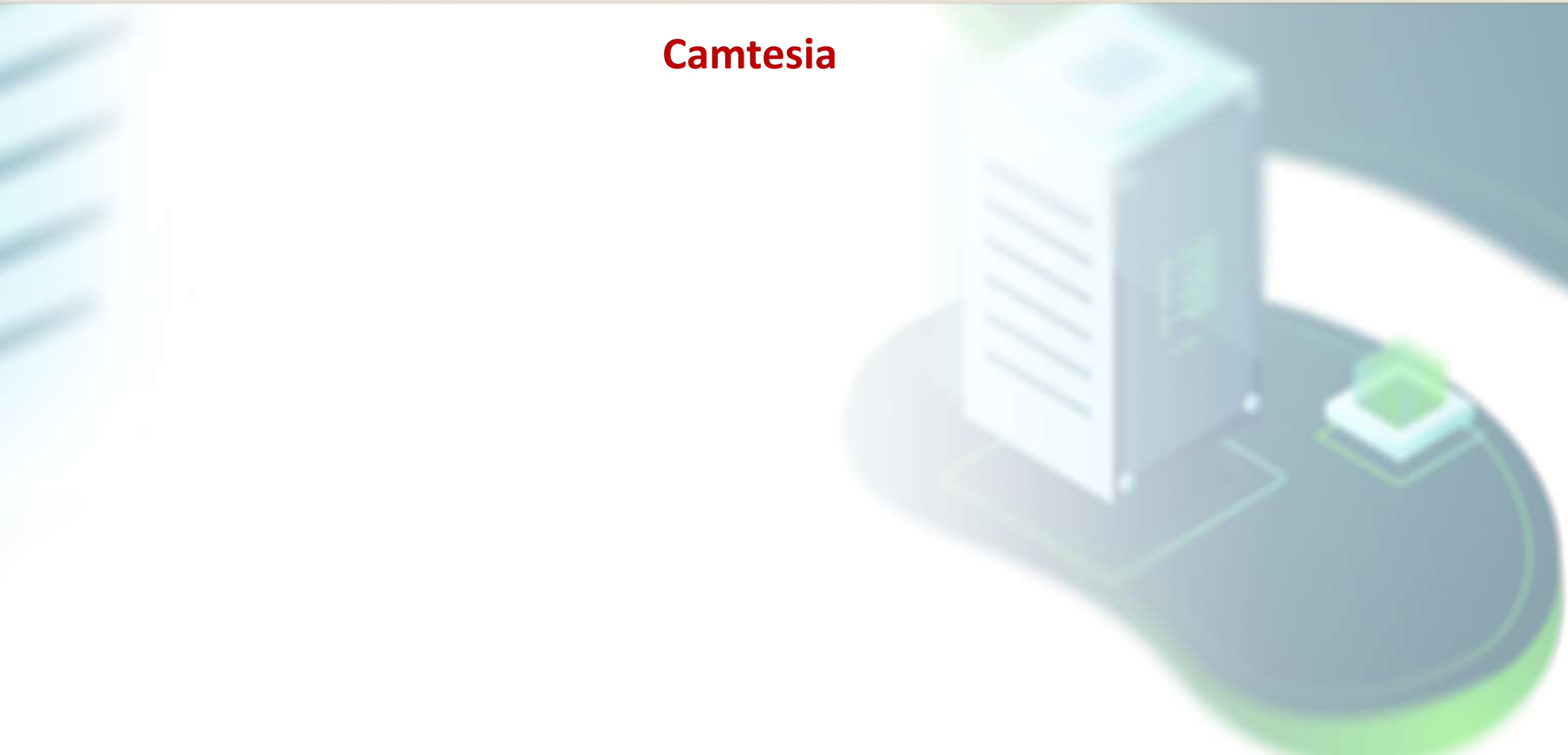
Hot Backup Demo

Advise for Hot Backup

An online backup of a database will keep the database open and functional for 24/7 operations. It is advised to schedule online backups when there is the least user activity on the database, because backing up the database is very I/O intensive and users can see slow response during the backup period. Additionally, if the user activity is very high, the archive destination might fill up very fast.

Hot Backup Demo

Camtesia





Restore from Backup using RMAN

Restore from Backup using RMAN

Restore from Backup using RMAN

- Shutdown database
- Remove files from E:\app\Haider\oradata\orcl
- Startup nomount
- Login with RMAN target /
- Restore controlfile
- Alter database mount;
- Restore database
- Recover database;
- Resetlogs & Recovered Files

Restore from Backup using RMAN

Shutdown Database

Now for recovery first off all shutdown your database using

“shutdown abort”

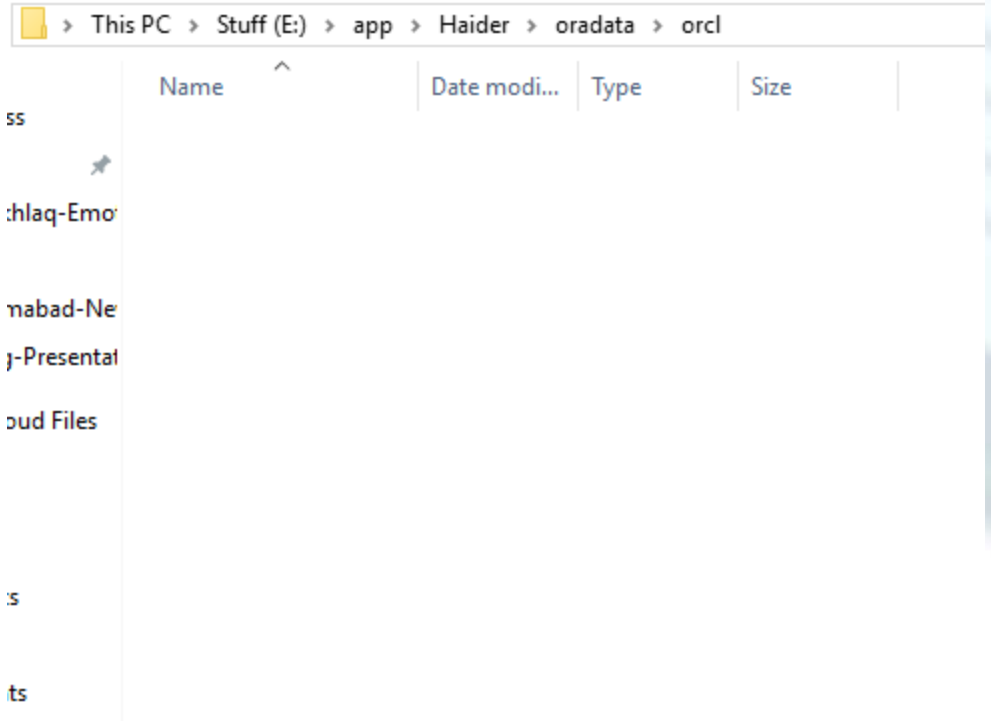
```
RMAN> shutdown abort

using target database control file instead of recovery catalog
Oracle instance shut down
```

Restore from Backup using RMAN

Remove Files from E:\app\Haider\oradata\orcl

Now remove your database or do any change in database, here we physically removed all database files from database



Restore from Backup using RMAN

Startup nomount

```
RMAN> startup nomount;

Oracle instance started

Total System Global Area     293601280 bytes

Fixed Size                     1290208 bytes
Variable Size                146800672 bytes
Database Buffers             138412032 bytes
Redo Buffers                   7098368 bytes

RMAN>
```

Restore from Backup using RMAN

Login with RMAN target /

```
C:\oracle\product\10.2.0\db_1\bin>rman target /

Recovery Manager: Release 10.2.0.3.0 - Production on Thu Nov 27 06:58:55 2014

Copyright (c) 1982, 2005, Oracle.  All rights reserved.

connected to target database (not started)

RMAN>
```

Restore from Backup using RMAN

Restore Control Files

```
RMAN> restore controlfile from 'C:\rmanbackup\rman\FULL_0IPOK2TH_18_1';

Starting restore at 27-NOV-14
using target database control file instead of recovery catalog
allocated channel: ORA_DISK_1
channel ORA_DISK_1: sid=156 devtype=DISK

channel ORA_DISK_1: restoring control file
channel ORA_DISK_1: restore complete, elapsed time: 00:00:10
output filename=C:\ORACLE\PRODUCT\10.2.0\ORADATA\ORCL\CONTROL01.CTL
output filename=C:\ORACLE\PRODUCT\10.2.0\ORADATA\ORCL\CONTROL02.CTL
output filename=C:\ORACLE\PRODUCT\10.2.0\ORADATA\ORCL\CONTROL03.CTL
Finished restore at 27-NOV-14
```

Restore from Backup using RMAN

Alter Database Mount

```
RMAN> alter database mount;

database mounted
released channel: ORA_DISK_1
```

Restore from Backup using RMAN

Restore Database

```
RMAN> restore database;

Starting restore at 27-NOV-14
Starting implicit crosscheck backup at 27-NOV-14
allocated channel: ORA_DISK_1
channel ORA_DISK_1: sid=156 devtype=DISK
```

Restore from Backup using RMAN

Recover Database

```
restoring datafile 00001 to C:\ORACLE\PRODUCT\10.2.0\ORADATA\ORCL\SYSTEM01.DBF
restoring datafile 00002 to C:\ORACLE\PRODUCT\10.2.0\ORADATA\ORCL\UNDOTBS01.DBF
restoring datafile 00003 to C:\ORACLE\PRODUCT\10.2.0\ORADATA\ORCL\SYSAUX01.DBF
restoring datafile 00004 to C:\ORACLE\PRODUCT\10.2.0\ORADATA\ORCL\USERS01.DBF
restoring datafile 00005 to C:\ORACLE\PRODUCT\10.2.0\DB_1\ORADATA\TBS_PERM_04.DA
T
restoring datafile 00006 to C:\ORACLE\PRODUCT\10.2.0\ORADATA\WORK01.DBF
restoring datafile 00007 to C:\ORACLE\PRODUCT\10.2.0\ORADATA\WORK02.DBF
restoring datafile 00008 to C:\ORACLE\PRODUCT\10.2.0\ORADATA\MCARE_PVISIT01.DBF
restoring datafile 00009 to C:\ORACLE\PRODUCT\10.2.0\ORADATA\MCARE_MEDLAB01.DBF
restoring datafile 00010 to C:\ORACLE\PRODUCT\10.2.0\ORADATA\MCARE_INDX01.DBF
restoring datafile 00011 to C:\ORACLE\PRODUCT\10.2.0\ORADATA\TESTING01.DBF
restoring datafile 00012 to C:\ORACLE\PRODUCT\10.2.0\ORADATA\MED_PHARMA01.DBF
restoring datafile 00013 to C:\ORACLE\PRODUCT\10.2.0\ORADATA\MED_INDX01.DBF
channel ORA_DISK_1: reading from backup piece C:\RMANBACKUP\RMAN\FULL_0HPOK2RP_1
7_1
channel ORA_DISK_1: restored backup piece 1
piece handle=C:\RMANBACKUP\RMAN\FULL_0HPOK2RP_17_1 tag=TAG20141126T221521
channel ORA_DISK_1: restore complete, elapsed time: 00:04:25
Finished restore at 27-NOV-14

RMAN>
```

```
RMAN> recover database;

Starting recover at 27-NOV-14
using channel ORA_DISK_1

starting media recovery
```

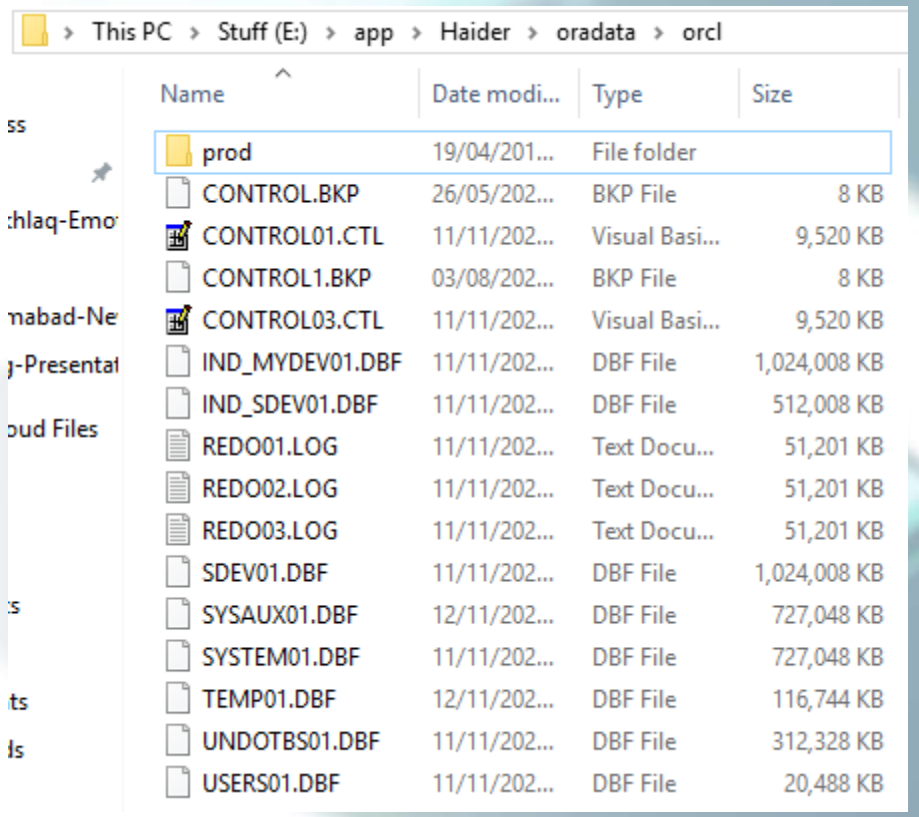
Restore from Backup using RMAN

Resetlogs & Recovered Files

```
RMAN> alter database open resetlogs;

database opened

RMAN> _
```



Restore from Backup using RMAN

Camtesia





Partial Restore Logical Backup

Partial Restore Logical Backup

Partial Restore Logical Backup

- Partial Restore Logical Backup
- Steps Partial Restore Logical Backup
- Partial Restore Tablespace
- Restore Archive Logs

Partial Restore Logical Backup

Partial Restore Logical Backup

- A partial database backup includes a subset of the database: individual tablespaces or data files.
- A tablespace backup is a backup of all the data files in a tablespace or in multiple tablespaces.

Partial Restore Logical Backup

Partial Restore Logical Backup

- Tablespace backups, whether consistent or inconsistent, are valid only if the database is operating in ARCHIVELOG mode because redo is required to make the restored tablespace consistent with the rest of the database.

Partial Restore Logical Backup

Steps of Partial Restore Logical Backup

- Before beginning a backup of a tablespace, identify all of the datafiles in the tablespace with the DBA_DATA_FILES data dictionary view:

```
SELECT TABLESPACE_NAME, FILE_NAME
FROM SYS.DBA_DATA_FILES
WHERE TABLESPACE_NAME = 'USERS';
```

Partial Restore Logical Backup

Steps Partial Restore Logical Backup

- Mark the beginning of the online tablespace backup:
- Back up the online datafiles of the online tablespace with operating system commands
- After backing up the datafiles of the online tablespace, run the SQL statement ALTER TABLESPACE with the END BACKUP option

Partial Restore Logical Backup

Steps Partial Restore Logical Backup

- Archive the unarchived redo logs so that the redo required to recover the tablespace backup is archived

Partial Restore Logical Backup

Partial Restore Tablespace

RMAN> restore tablespace users;

RMAN> recover tablespace users;

Partial Restore Logical Backup

Restore Archive Logs

```
mkdir d:\archive
rman target /
RMAN> run {
2> set archivelog destination to
'd:\archive';
3> restore archivelog all;
4> }
```

Partial Restore Logical Backup

Camtesia



Introduction to Database Administration



Complete & Incomplete Recovery

Complete & Incomplete Recovery

Complete & Incomplete Recovery

- Complete Recovery
- Incomplete Recovery
- Database point-in-time Recovery
- Requirements for DPITR
- Parents, Ancestor & Sibling Incarnations
- Database Incarnation History with Multiple Resetlogs
- DPITR Incarnations
- DPITR Commands

Complete & Incomplete Recovery

Complete Recovery

- Complete recovery applies all redo changes contained in the archived and online logs to a backup. Typically, you perform complete media recovery after a media failure damages data files or the control file. You can perform complete recovery on a database, tablespace, or data file.

Complete & Incomplete Recovery

Incomplete Recovery

- Incomplete recovery, also called database point-in-time recovery, results in a noncurrent version of the database. In this case, you do not apply all of the redo generated after the restored backup. Typically, you perform point-in-time database recovery to undo a user error when Flashback Database is not possible.

Complete & Incomplete Recovery

Incomplete Recovery

- To perform incomplete recovery, you must restore all data files from backups created before the time to which you want to recover and then open the database with the RESETLOGS option when recovery completes. Resetting the logs creates a new stream of log sequence numbers starting with log sequence 1.

Complete & Incomplete Recovery

Database point-in-time Recovery

- **Database point-in-time recovery (DBPITR)** restores the database from backups prior to the target time for recovery, then uses incremental backups and redo to roll the database forward to the target time.
- DBPITR is sometimes called **incomplete recovery** because it does not use all of the available redo or completely recover all changes to your database.

Complete & Incomplete Recovery

Database point-in-time Recovery

- The target SCN can be specified using a date and time, in which case the operation is sometimes called **time-based recovery**.

Complete & Incomplete Recovery

Requirements for DPITR

The requirements for database point-in-time recovery are as follows:

- Your database must be running in ARCHIVELOG mode.
- You must have backups of all data files from before the target SCN for DBPITR and archived redo logs for the period between the SCN of the backups and the target SCN.

Complete & Incomplete Recovery

Parents, Ancestor & Sibling Incarnations

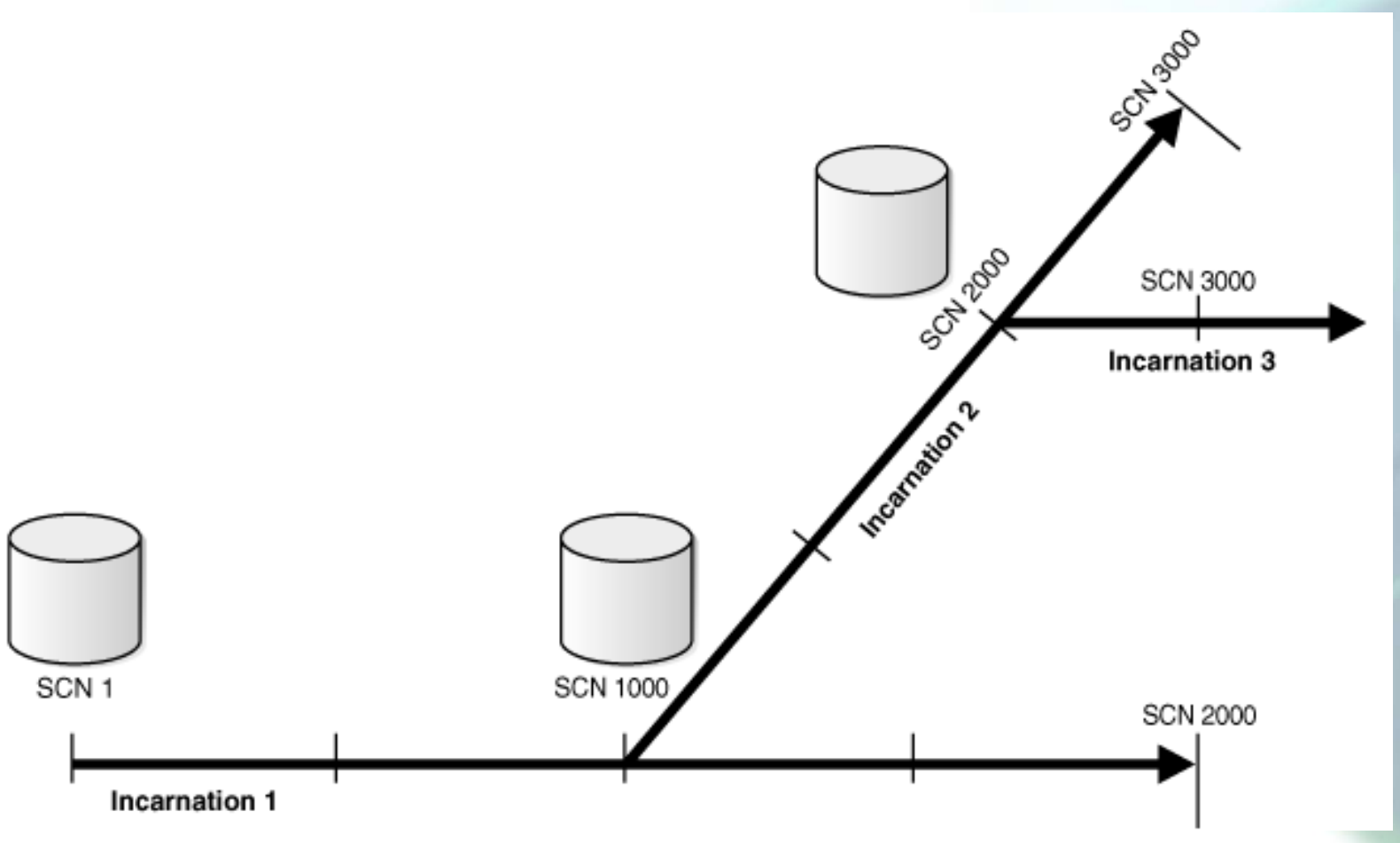
The incarnation from which the current incarnation branched following an OPEN RESETLOGS operation is called the parent incarnation of the current incarnation.

The parent incarnation and all of its parent incarnations are the ancestor incarnations of the current incarnation.

Two incarnations that share a common ancestor are sibling incarnations if neither one is an ancestor of the other.

Complete & Incomplete Recovery

Database Incarnation History with Multiple Resetlogs



Complete & Incomplete Recovery

DPITR Incarnations

The requirements for database point-in-time recovery are as follows:

- Your database must be running in ARCHIVELOG mode.
- You must have backups of all data files from before the target SCN for DBPITR and archived redo logs for the period between the SCN of the backups and the target SCN.

Complete & Incomplete Recovery

DPITR Commands

RMAN> RECOVER DATABASE TO SCN 1500;

RMAN> RESET DATABASE INCARNATION TO 1;

RMAN> RECOVER DATABASE TO SCN 1500;

Complete & Incomplete Recovery

Camtesia



Introduction to Database Administration



Understanding of Import/ Export

Understanding of Import/ Export

Understanding of Import/ Export

- Export Utility
- Export Dump File
- Export Commands
- Import Utility
- Import Ordering
- Export Commands

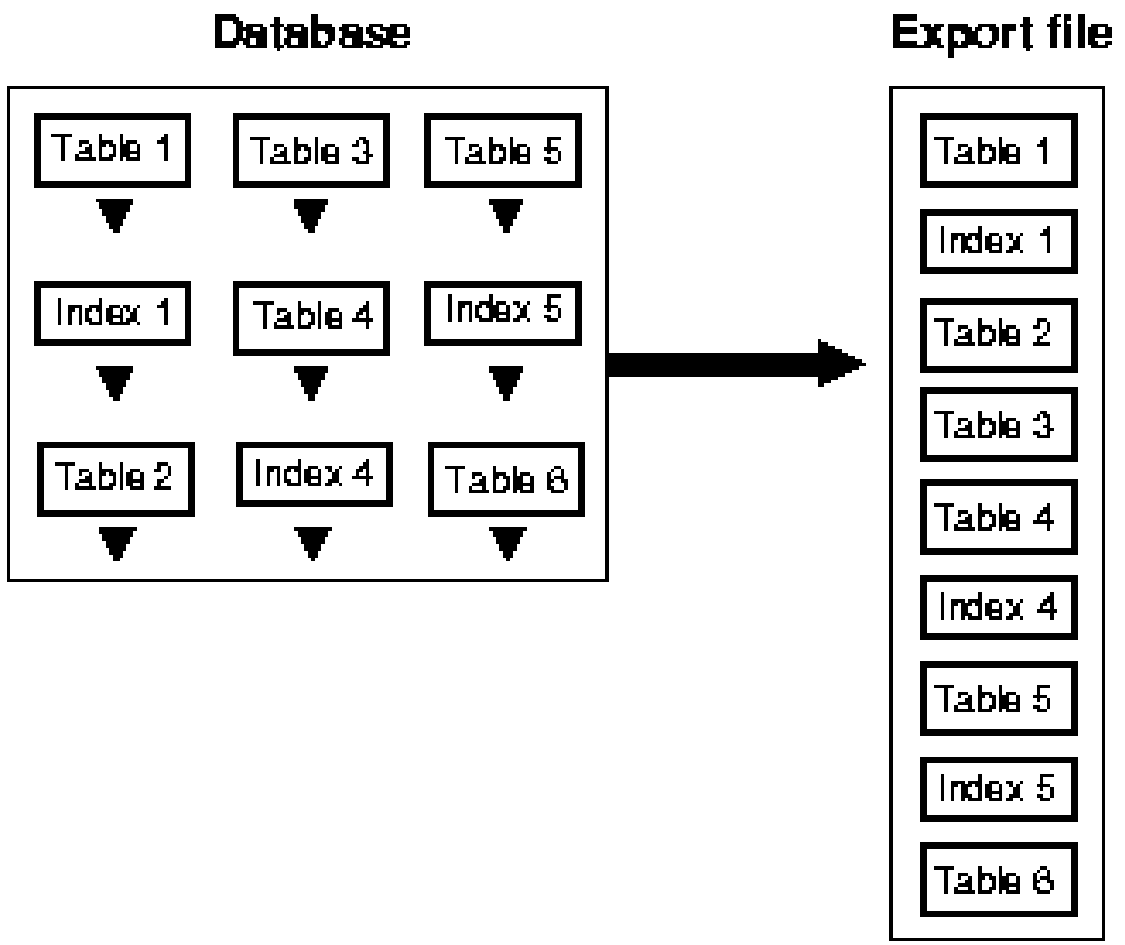
Understanding of Import/ Export

Export Utility

- The Export utility provides a simple way for you to transfer data objects between Oracle databases, even if they reside on platforms with different hardware and software configurations.
- When you run Export against an Oracle database, objects (such as tables) are extracted, followed by their related objects (such as indexes, comments, and grants), if any. The extracted data is written to an Export file.

Understanding of Import/ Export

Export Utility



Understanding of Import/ Export

Export Dump File

- An Export file is an Oracle binary-format dump file that is typically located on disk or tape. The dump files can be transferred using FTP or physically transported (in the case of tape) to a different site. The files can then be used with the Import utility to transfer data between databases that are on systems not connected through a network. The files can also be used as backups in addition to normal backup procedures.

Understanding of Import/ Export

Export Dump File

- Export dump files can only be read by the Oracle Import utility. The version of the Import utility cannot be earlier than the version of the Export utility used to create the dump file.
- You can also display the contents of an export file without actually performing an import. To do this, use the Import SHOW parameter.

Understanding of Import/ Export

Export Commands

exp PARFILE=filename

exp username/password PARFILE=filename

FULL=y FILE=dba.imp GRANTS=y INDEXES=y CONSISTENT=y

exp username/password PARFILE=params.dat INDEXES=n

exp \'username/password AS SYSDBA\'

Understanding of Import/ Export

Import Utility

- The Import utility reads the object definitions and table data from an Export dump file. It inserts the data objects into an Oracle database.

Understanding of Import/ Export

Import Ordering

The export file contains objects in the following order

1. Type definitions
2. Table definitions
3. Table data
4. Table indexes
5. Integrity constraints, views, procedures, and triggers
6. Bitmap, functional, and domain indexes

Understanding of Import/ Export

Export Commands

imp username/password PARAMETER=value

or

imp username/password PARAMETER=(value1,value2,...,valuen)

imp PARFILE=filename

imp username/password PARFILE=filename

FULL=y FILE=dbay INDEXES=y CONSISTENT=y

Introduction to Database Administration



Controlling & Configuring Import/ Export

Controlling & Configuring Import/ Export

Controlling & Configuring Import/ Export

- Error Handling
- Failed Integrity Constraints
- Invalid Data
- Errors Importing Objects
- Object Already Exists
- Sequence
- Partition Level Import

Controlling & Configuring Import/ Export

Error Handling

- If a row is rejected due to an integrity constraint violation or invalid data, Import displays a warning message but continues processing the rest of the table. Some errors, such as "tablespace full," apply to all subsequent rows in the table. These errors cause Import to stop processing the current table and skip to the next table.
- A "tablespace full" error can suspend the import if the RESUMABLE=y parameter is specified.

Controlling & Configuring Import/ Export

Failed Integrity Constraints

A row error is generated if a row violates one of the integrity constraints in force on your system, including:

- NOT NULL constraints
- Uniqueness constraints
- Primary key (not null and unique) constraints
- Referential integrity constraints
- Check constraints

Controlling & Configuring Import/ Export

Invalid Data

- Row errors can also occur when the column definition for a table in a database is different from the column definition in the export file.
- The error is caused by data that is too long to fit into a new table's columns, by invalid datatypes, or by any other INSERT error.

Controlling & Configuring Import/ Export

Errors Importing Objects

- Errors can occur for many reasons when you import database objects.
- When these errors occur, import of the current database object is discontinued. Import then attempts to continue with the next database object in the export file.

Controlling & Configuring Import/ Export

Object Already Exists

- If a database object to be imported already exists in the database, an object creation error occurs. What happens next depends on the setting of the IGNORE parameter.

- If IGNORE=n (the default), the error is reported, and Import continues with the next database object. The current database object is not replaced.

Controlling & Configuring Import/ Export

Object Already Exists

- If IGNORE=y, object creation errors are not reported. The database object is not replaced. If the object is a table, rows are imported into it. Note that only object creation errors are ignored; all other errors (such as operating system, database, and SQL errors) are reported and processing may stop.

Controlling & Configuring Import/ Export

Sequence

- If sequence numbers need to be reset to the value in an export file as part of an import, you should drop sequences.
- If a sequence is not dropped before the import, it is not set to the value captured in the export file, because Import does not drop and re-create a sequence that already exists. If the sequence already exists, the export file's CREATE SEQUENCE statement fails and the sequence is not imported.

Controlling & Configuring Import/ Export

Partition Level Import

- You can import tables, partitions, and sub-partitions in the following ways:
- Table-level Import: imports all data from the specified tables in an Export file.
- Partition-level Import: imports only data from the specified source partitions or sub-partitions.
- You must set the parameter IGNORE=y when loading data into an existing table.

End



Walkthrough Import when Table Data Exist

Walkthrough Import when Table Data Exist

Walkthrough Import when Table Data Exist

- Import When Table Already Exists

Walkthrough Import when Table Data Exist

Import when Table Already Exist

- When tables are manually created before data is imported, the CREATE TABLE statement in the export dump file will fail because the table already exists. To avoid this failure and continue loading data into the table, set the import parameter IGNORE=y. Otherwise, no data will be loaded into the table because of the table creation error or use table_exists_action=append using expdp and impdp

Walkthrough Import when Table Data Exist

Camtesia





Walkthrough Import with Moving Objects

Walkthrough Import with Moving Objects

Walkthrough Import with Moving Objects

- Portal Import
- Moving Data to the Target System

Walkthrough Import with Moving Objects

Portal Import

- Create transport sets and extract the content of the transport sets to transport tables. Transport sets contain the portal objects that you want to export to your target portal environment. This information is displayed in a manifest. The manifest is a list of objects in a transport set, used to provide a granular level of control over the export. For more information on the export process and using the Transport Sets

Walkthrough Import with Moving Objects

Portal Import

- Move the transport sets from one system (source) to another (target) using the Transport Sets - Acquire Service portlet, or using the Oracle Portal Export and Import command-line scripts if there is an intervening firewall. Moving the transport set from the source to the target system using the Transport Sets

Walkthrough Import with Moving Objects

Portal Import

- Import the objects from the transport tables to the target portal repository using the Import Transport Set portlet.

Walkthrough Import with Moving Objects

Moving Data to the Target System

- To move your content using the Acquire Transport Sets portlet:
- In the Transport Set - Acquire Services portlet, click the Browse Source Portals on the Administer page and check that the source portal you're moving data from is registered on the target instance.
- If the source portal has not previously been registered, from the Acquire Transport Sets portlet, click Register A Source Portal and register. From the Transport Set - Acquire Services portlet, click Acquire Transport Sets to display the Acquire a Transport Set page.

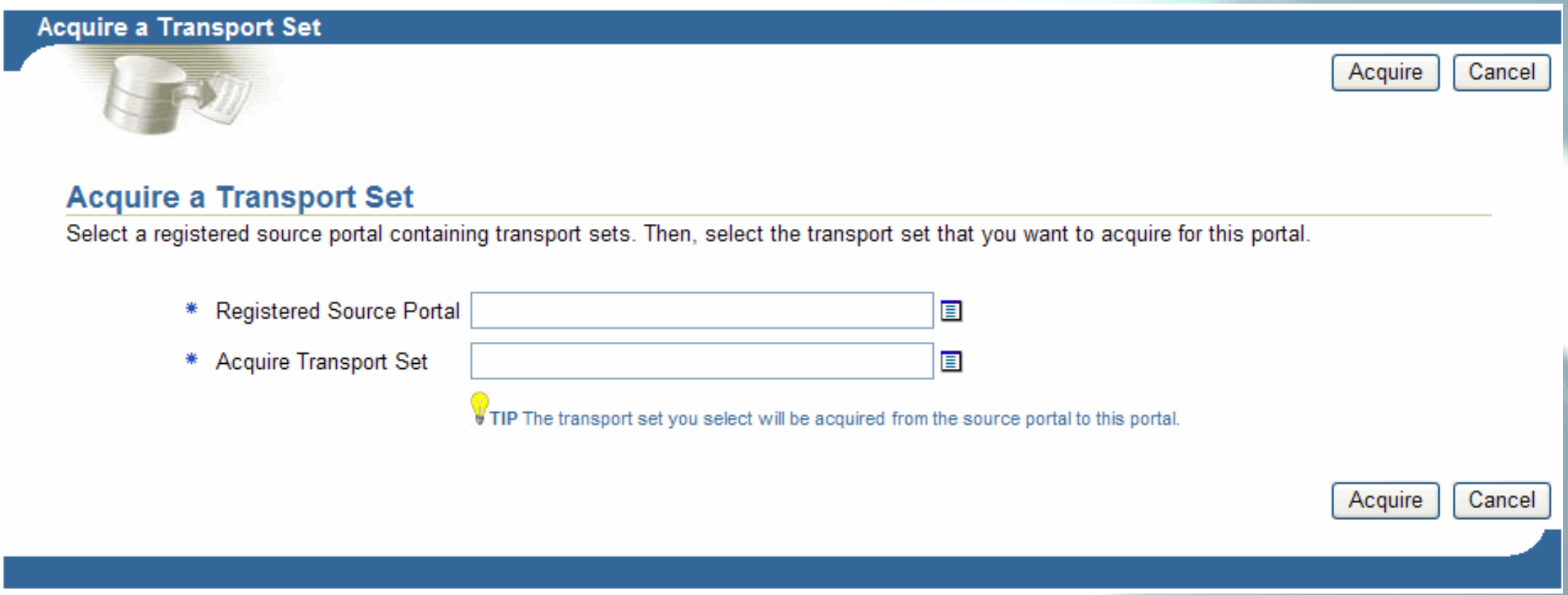
Walkthrough Import with Moving Objects

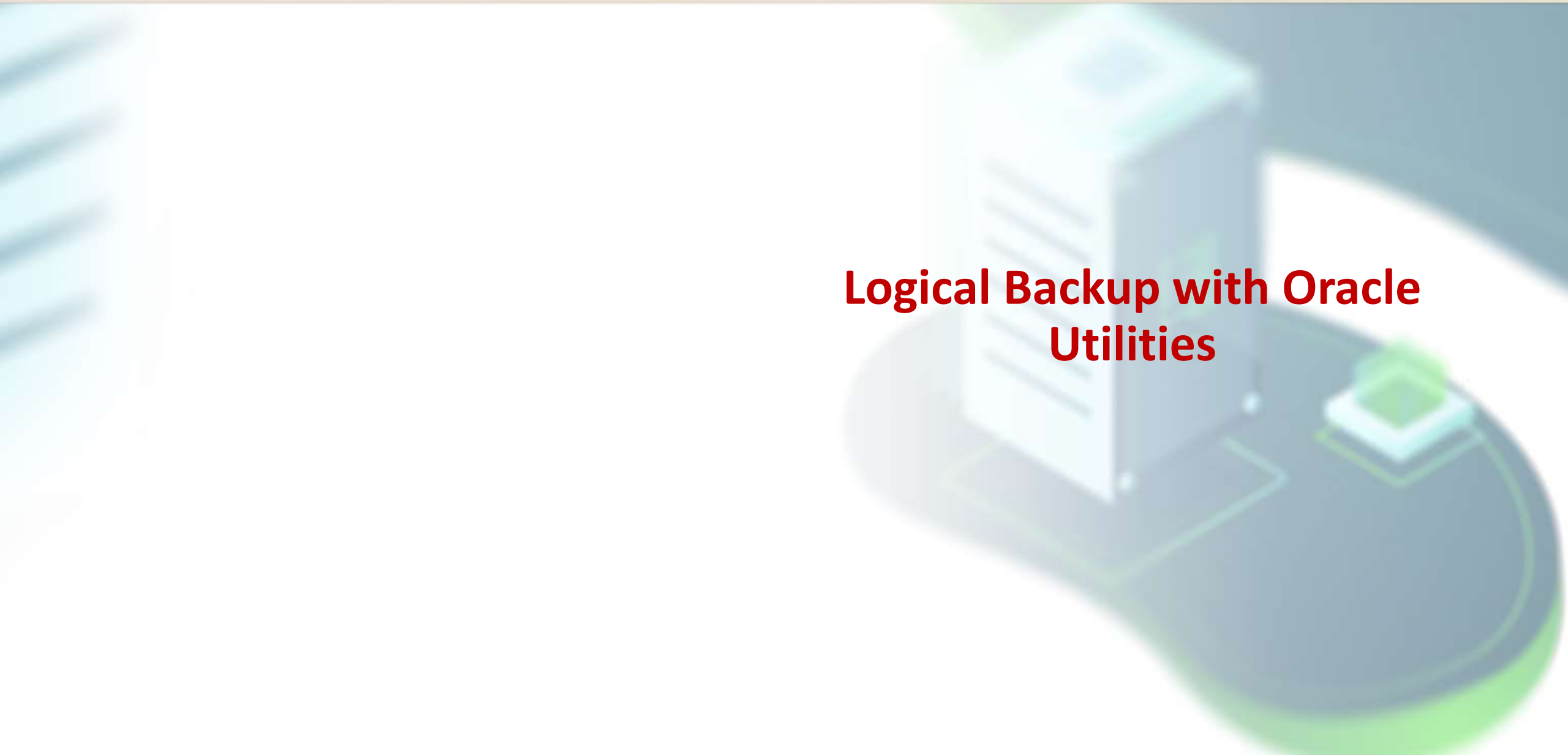
Moving Data to the Target System

- Specify the source portal and the transport set and click **Acquire** to start transferring the data.
- When the transfer is complete, click **Display Manifest** to display the transport set manifest. Click on the explicitly selected object to display the detailed manifest. Click **Tree View** to see the nested dependencies.
- Click **Close** to return to the manifest, and then click **Pre-check Now** to perform a pre-check of the transferred data. The system checks that all object dependencies have been resolved.
- Click **Display Manifest** when the pre-check is complete and check the status column for errors. Resolve any errors before continuing with importing the transport set.

Walkthrough Import with Moving Objects

Moving Data to the Target System





Logical Backup with Oracle Utilities

Logical Backup with Oracle Utilities

Logical Backup with Oracle Utilities

- Logical Backup
- Oracle Data Pump Architecture
- Modes of Data Pump
- Examples of Data Pump

Logical Backup with Oracle Utilities

Logical Backup

- Logical backups are backups in which the export utility (for example Exp) uses SQL to read database data and then export it into a binary file at the operating system level. You can then import the data back into a database using the import utility (Imp).

Logical Backup with Oracle Utilities

Logical Backup

- The Oracle Export Utility can create a file that contains all the objects in a particular schema. The file that is created can then be imported to the same schema to recover from a failure or to a different instance or schema for archiving purposes.

Logical Backup with Oracle Utilities

Oracle Data Pump Architecture

Data Pump consists of the following components:

expdp (Data Pump export utility)

impdp(Data Pump import utility)

DBMS_DATAPUMP PL/SQL package(Data Pump application interface [API])

DBMS_METADATA PL/SQL package (Data Pump Metadata API)

Logical Backup with Oracle Utilities

Modes of Data Pump

- Entire database
- Schema level
- Table level
- Tablespace level
- Transportable tablespace level

Logical Backup with Oracle Utilities

Examples of Data Pump

An example of Export:-

$ expdp mv_maint/foo directory=dp_dir dumpfile=full.dmp logfile=full.log full=y

An example of Import:-

$ impdp mv_maint/foo directory=dp_dir dumpfile=full.dmp logfile=fullimp.log full=y



Walkthrough Export with Moving Objects

Walkthrough Export with Moving Objects

Walkthrough Export with Moving Objects

- Pre-requisite Portal Export & Import
- Command-line Access
- Oracle Export & Import with Character Sets
- Role of Character Sets in Export & Import

Walkthrough Export with Moving Objects

Pre-requisite Portal Export & Import

Oracle Portal instance information:
Before beginning the export and import process, ensure you have the following information:

- System Requirements
- Additional Considerations

Walkthrough Export with Moving Objects

Pre-requisite Portal Export & Import

- Privileges for Exporting and Importing Content
  - Portal schema name.
  - Portal schema password.
  - Portal connect string information.
  - Portal user name.
  - Portal user password.
  - Company name (used only for hosted portal installations). In most cases, this should be left blank.

Walkthrough Export with Moving Objects

Oracle Export & Import with Character Sets

- The Oracle Database exp utility always exports user data, including Unicode data, in the character sets of the export server. The character sets are specified when the database is created.
- The Oracle Database imp utility automatically converts the data to the character sets of the import server.

Walkthrough Export with Moving Objects

Command-line Access

- If you are using the Oracle Portal Export and Import command-line scripts to move the transport sets from one system to another, you must have command-line access to run the shell or command utilities generated by the export import process. The command-line utilities, in turn, access the Oracle Database exp and imp utilities, and the Oracle Portal instance.

Walkthrough Export with Moving Objects

Oracle Export & Import with Character Sets

- Some 8-bit characters can be lost (that is, converted to 7-bit equivalents) when you import an 8-bit character set export file. This occurs if the client system has a native 7-bit character set or if the NLS_LANG operating system environment variable is set to a 7-bit character set. Most often, you notice that accented characters lose their accent marks.

Walkthrough Export with Moving Objects

Oracle Export & Import with Character Sets

- Both the Oracle Database exp and imp utilities alert you of any required character set conversion before exporting or importing the data.

Walkthrough Export with Moving Objects

Role of Character Sets in Export & Import

```
E:\app\Haider\product\11.2.0\dbhome_1\BIN>imp scott/tiger IGNORE=y FILE = c:\test\Scott.d

Import: Release 11.2.0.1.0 - Production on Fri Nov 13 18:54:32 2020

Copyright (c) 1982, 2009, Oracle and/or its affiliates.  All rights reserved.


Connected to: Oracle Database 11g Enterprise Edition Release 11.2.0.1.0 - Production
With the Partitioning, OLAP, Data Mining and Real Application Testing options

Export file created by EXPORT:V11.02.00 via conventional path
import done in WE8MSWIN1252 character set and AL16UTF16 NCHAR character set
Import terminated successfully without warnings.
```

Introduction to Database Administration

General Overview of Database Manager



General Overview of Database Manager

General Overview of Oracle Enterprise Manager (OEM)

- Starting OEM
- Oracle Database Control Login
- OEM Home Page
- OEM - Grant Management Privileges
- OEM - Users Credentials

General Overview of Database Manager

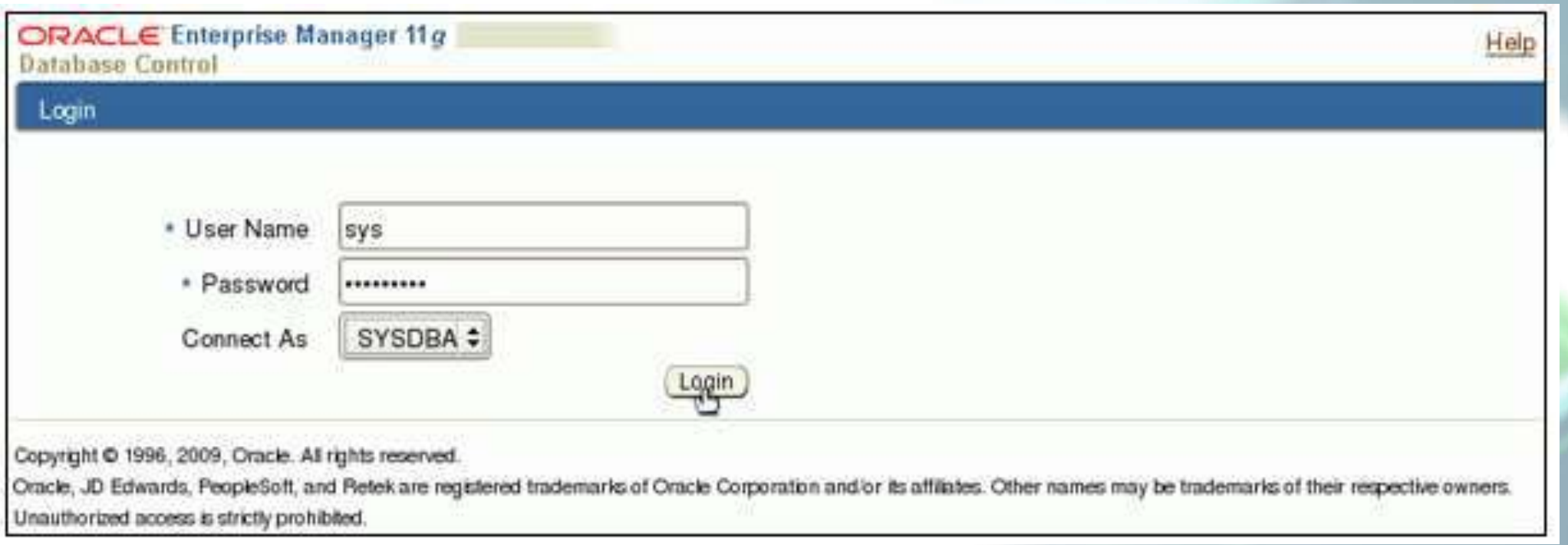
Starting OEM

On cmd (DOS prompt), you may run
set oracle_sid=orcl
emctl stop dbconsole
emctl start dbconsole

In the browser, you can run URL
https://localhost:1158/em

General Overview of Database Manager

Oracle Database Control Login



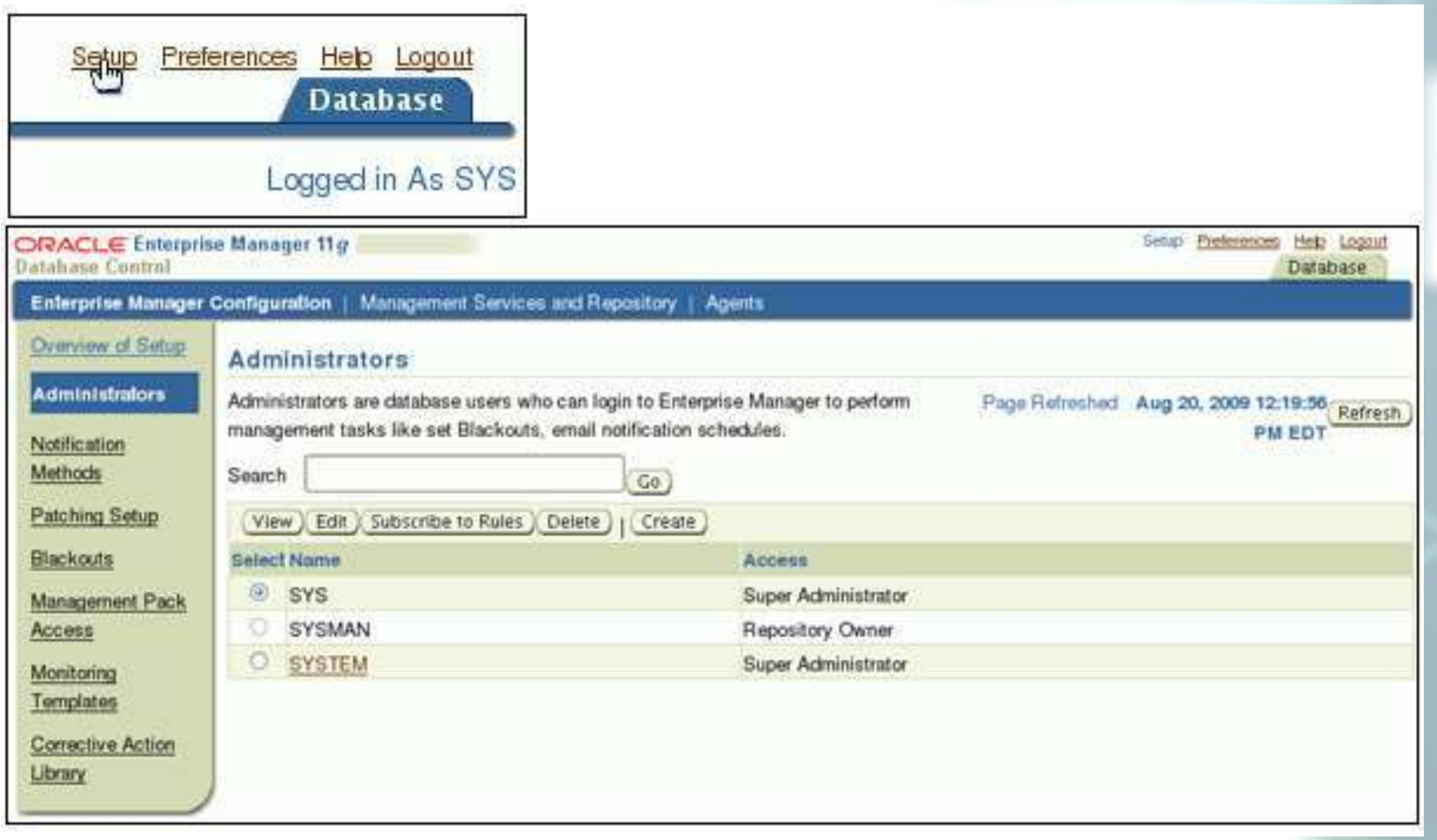
General Overview of Database Manager

OEM Home Page



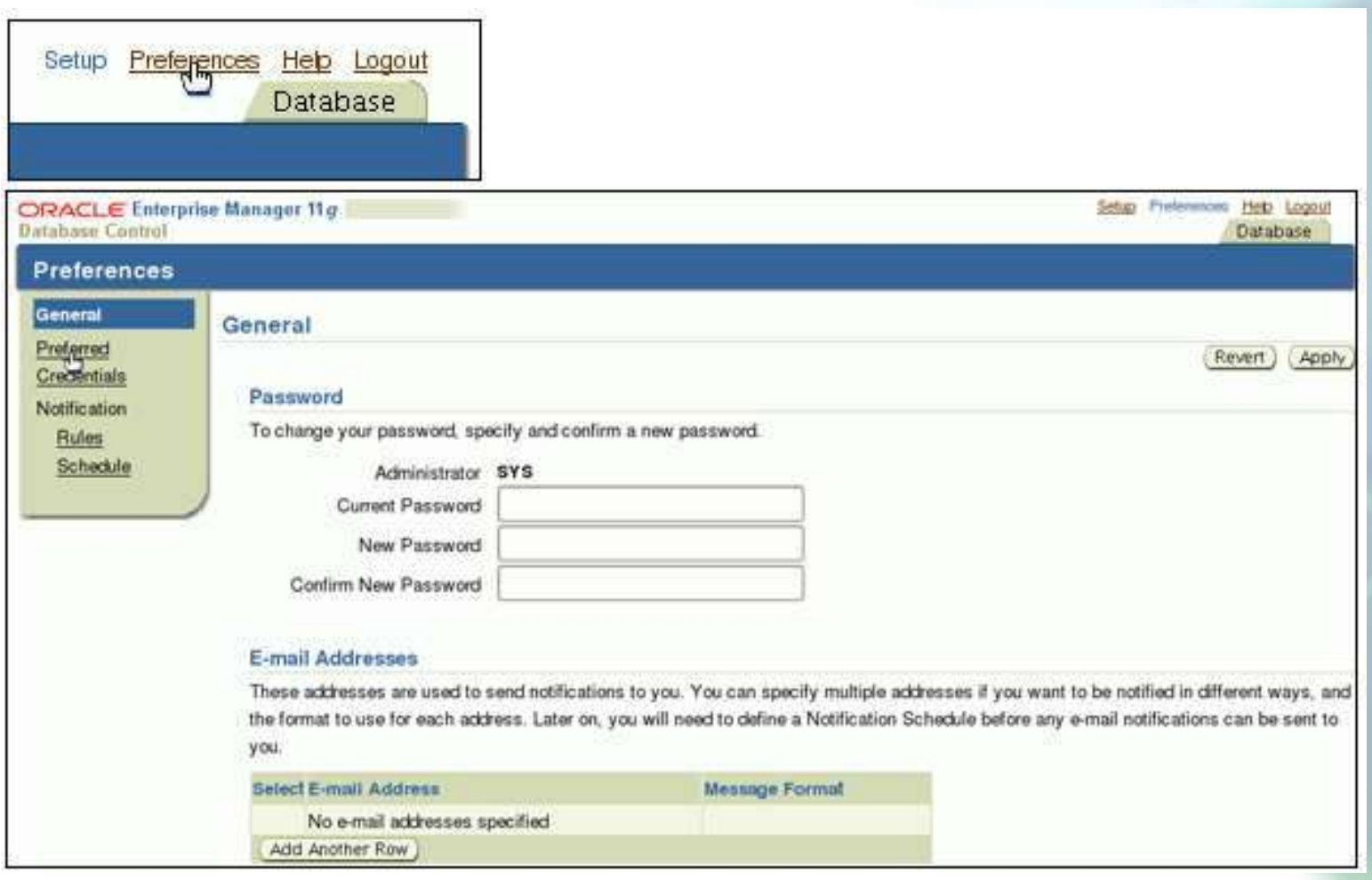
General Overview of Database Manager

OEM – Grant Management Privileges



General Overview of Database Manager

OEM – Users Credentials

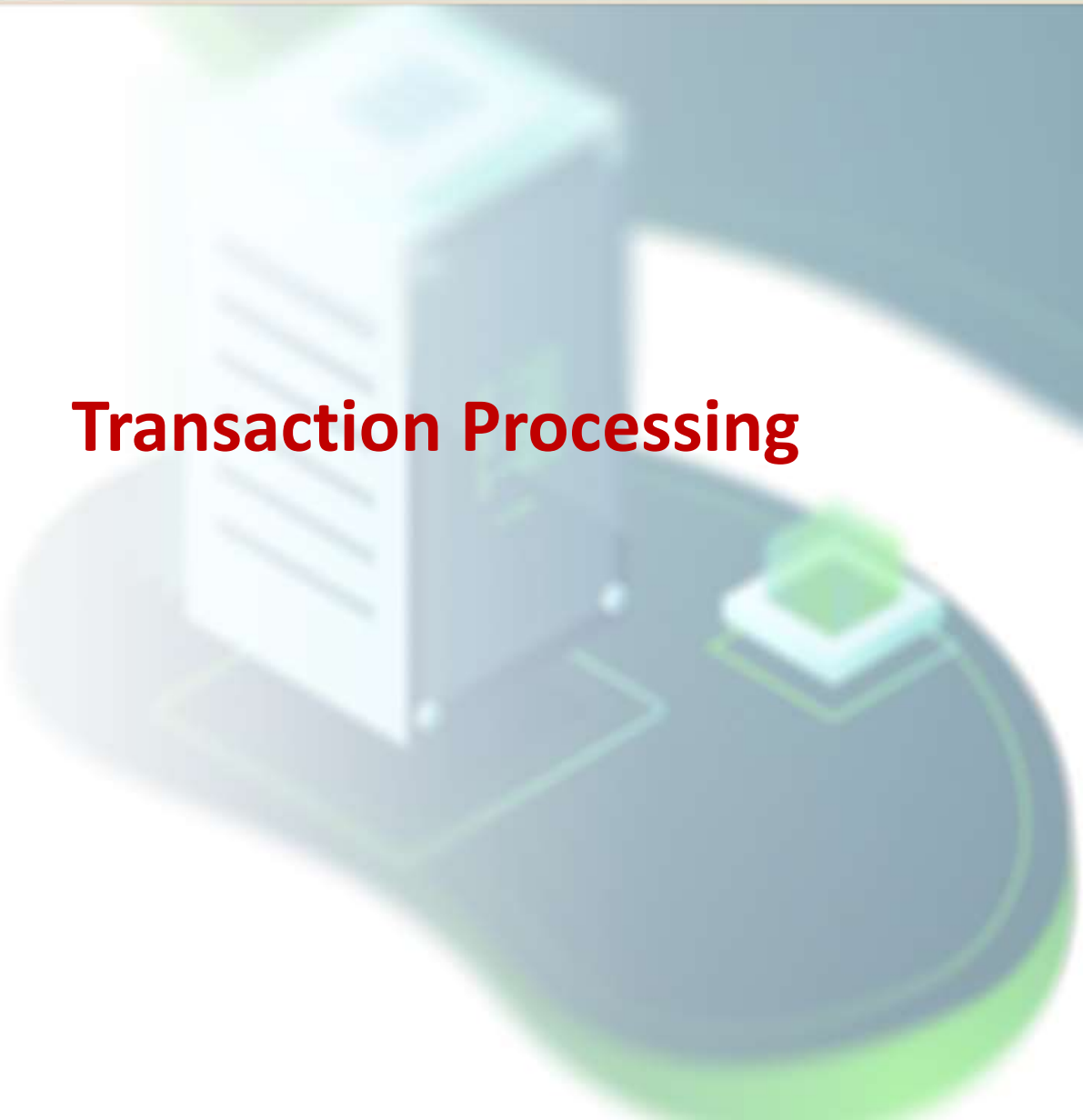


General Overview of Database Manager

OEM – Users Credentials



Introduction to Database Administration



Transaction Processing

Transaction Processing

Transaction Processing

- What is a Transaction?
- Processing of Transactions
- Processing of Transactions in Oracle

Transaction Processing

What is a Transaction?

SQL is initiated at user level (client) with Start-End and it executes on Server (central Database) at a certain distance.

A transaction is a logical, atomic unit of work (block) that contains one or more SQL statements.

Either transaction is done (committed) or not done (rolled back). No partial execution of transaction is allowed.

Transaction Processing

Processing of Transactions

Generally reflects the activities in the real world system

Depositing some amount in an account, will be a transaction

Transferring amount from one account to another; if ok then DB is in consistent state otherwise inconsistent.

One transaction can be calculating GPA in our exam system

Transaction Processing

Processing of Transactions in Oracle

In Oracle Database, each user must see a consistent view of the data, including visible changes made by a user's own transactions and committed transactions of other users.

Transaction Processing

Processing of Transactions in Oracle

- Single-user VS multi-user systems
  - A DBMS is **single-user** if at most one user can use the system at a time
  - A DBMS is **multi-user** if many users can use the system concurrently
- Problem
  - How to make the simultaneous interactions of multiple users with the database safe, consistent, correct, and efficient?

Introduction to Database Administration



Defining Transaction

Defining Transaction

Defining Transaction

- What is a Transaction?
- Transaction with SQL
- Converting SQL into Transaction

Defining Transaction

What is a Transaction?

SQL is initiated at user level (client) with Start-End and it executes on Server (central Database) at a certain distance.

A transaction is a logical, atomic unit of work (block) that contains one or more SQL statements.

Either transaction is done (committed) or not done (rolled back). No partial execution of transaction is allowed.

Defining Transaction

What is a Transaction?

A transaction T is a logical unit of database processing that includes one or more database access operations

- Embedded within an application program
- Specified interactively (e.g., via SQL)

Transaction boundaries:

- Begin/end transaction

Types of transactions

- Read transaction
- write transaction

Defining Transaction

Transaction with SQL

For example, following SQL update will change the salary of some Employee

```
Update EMP
Set sal = 850
Where empno = 7369;
```

Is it statement or a transaction?

Defining Transaction

Converting SQL into Transaction

```
begin

    Update EMP
    Set sal = 850
    Where empno = 7369;

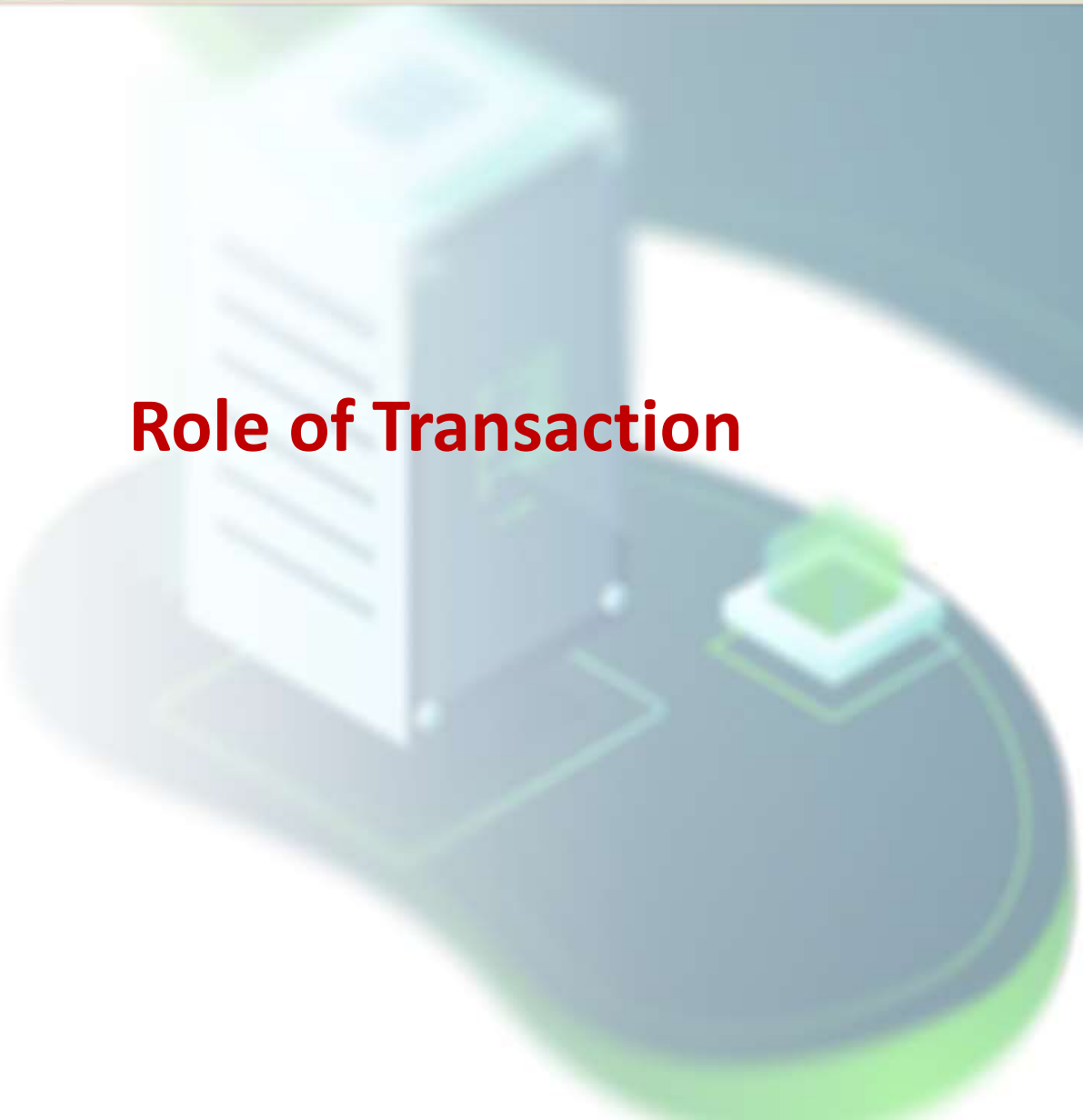
end;
/
```

Defining Transaction

Camtesia



Introduction to Database Administration



Role of Transaction

Role of Transaction

Role or Importance of Transactions

- Transaction & SQL
- Importance of Transaction Processing System
- Transaction with Client-Server Environment

Role of Transaction

Transaction & SQL

Transaction is travelled on network, it may have chances of lost and it requires transaction logs for tracing

Can a simple SQL manages logs?

Can transaction in block manageable?

Issues of Multi-user environment.

Issues of Consistent, Correct and Efficiency are important.

Role of Transaction

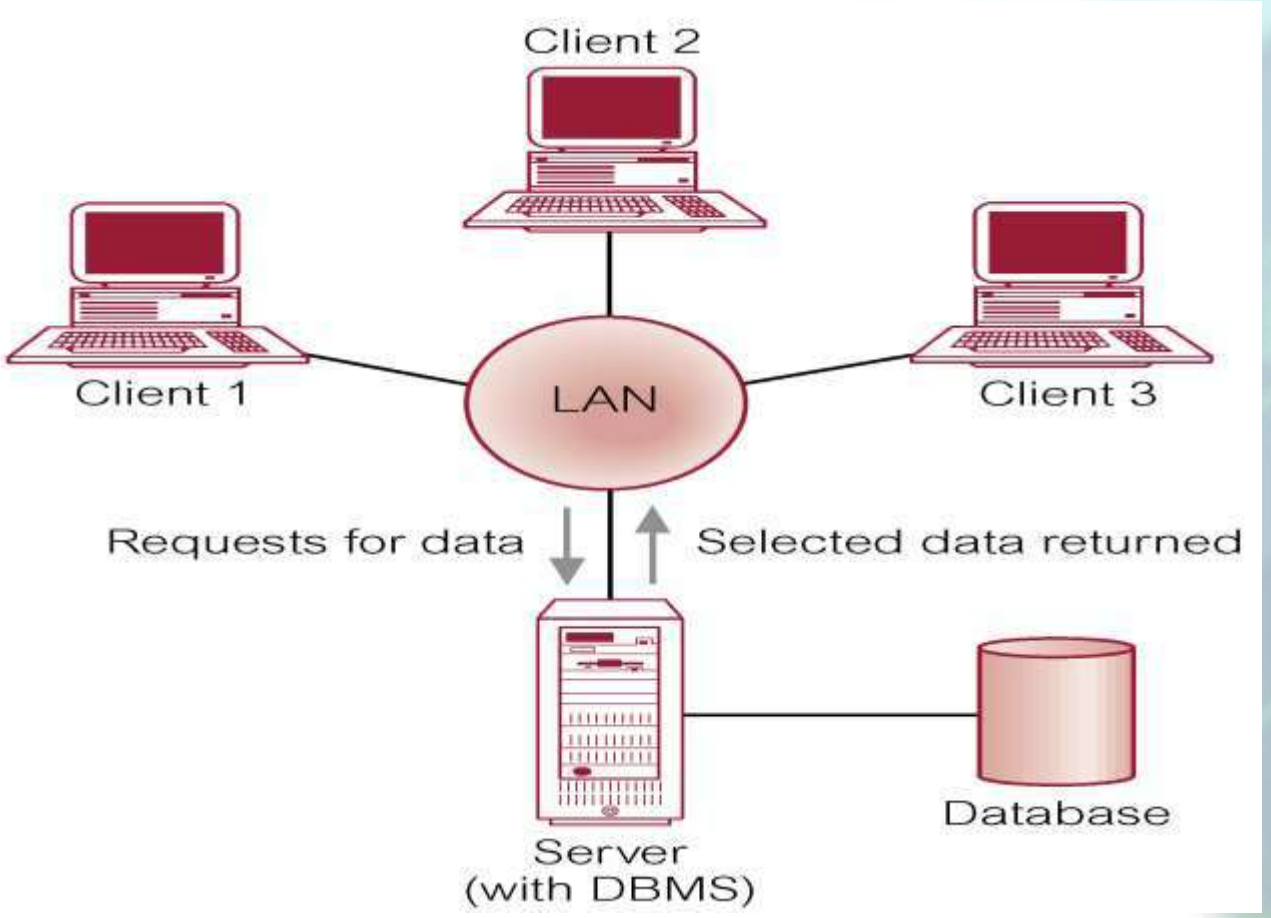
Importance of Transaction Processing System

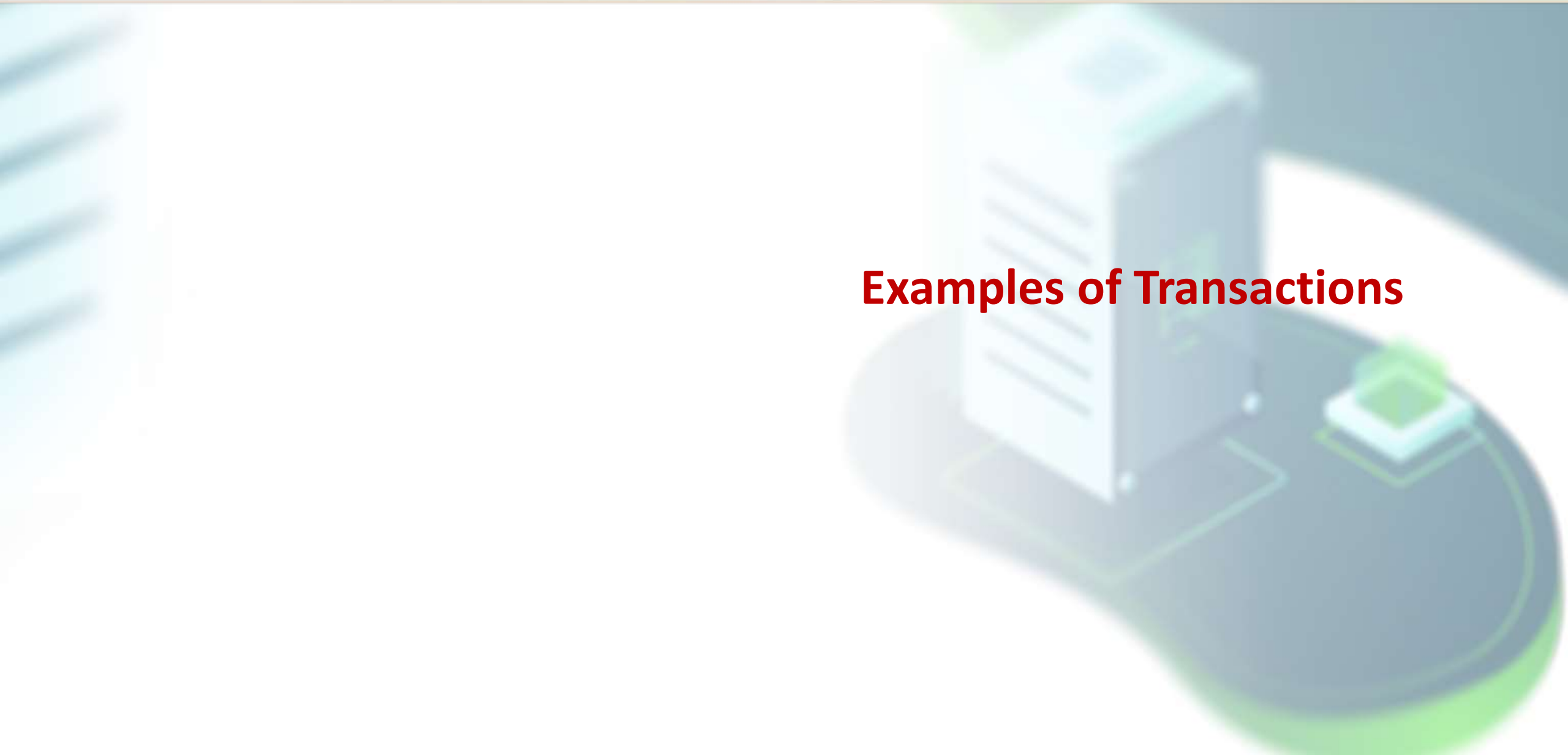
Transaction processing (TP) systems underlie the online applications. a transaction rate of 15 simple Debit Credit transactions per second was “common”, 50 TPS was a “good” system, 100 TPS was “fast”, and a 400 TPS system was characterized as being a very “lean and mean” system.

Teradata capabilities, usage of parallel databases, configuration and tuning etc.

Role of Transaction

Transaction with Client-Server Environment





Examples of Transactions

Examples of Transactions

Example of Transactions

- Usage of ATM - An Example
- Funds Transfer Concept
- Writing a Transaction



Examples of Transactions

Usage of ATM – An Example

Usage of ATM to draw/ transfer money, to collect currency with dispenser



Examples of Transactions

Funds Transfer Concept

Transfer 400,000 from checking account to savings account

For a user it is one activity

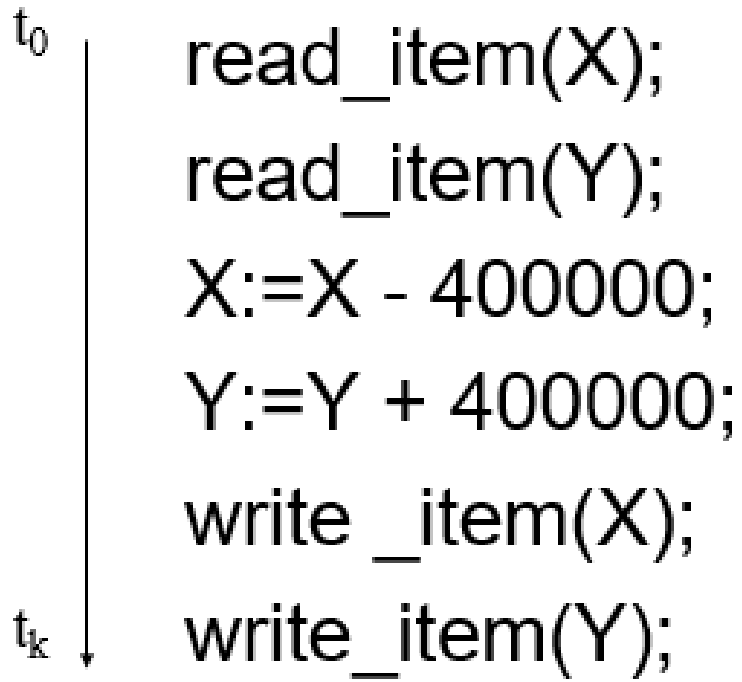
To database:

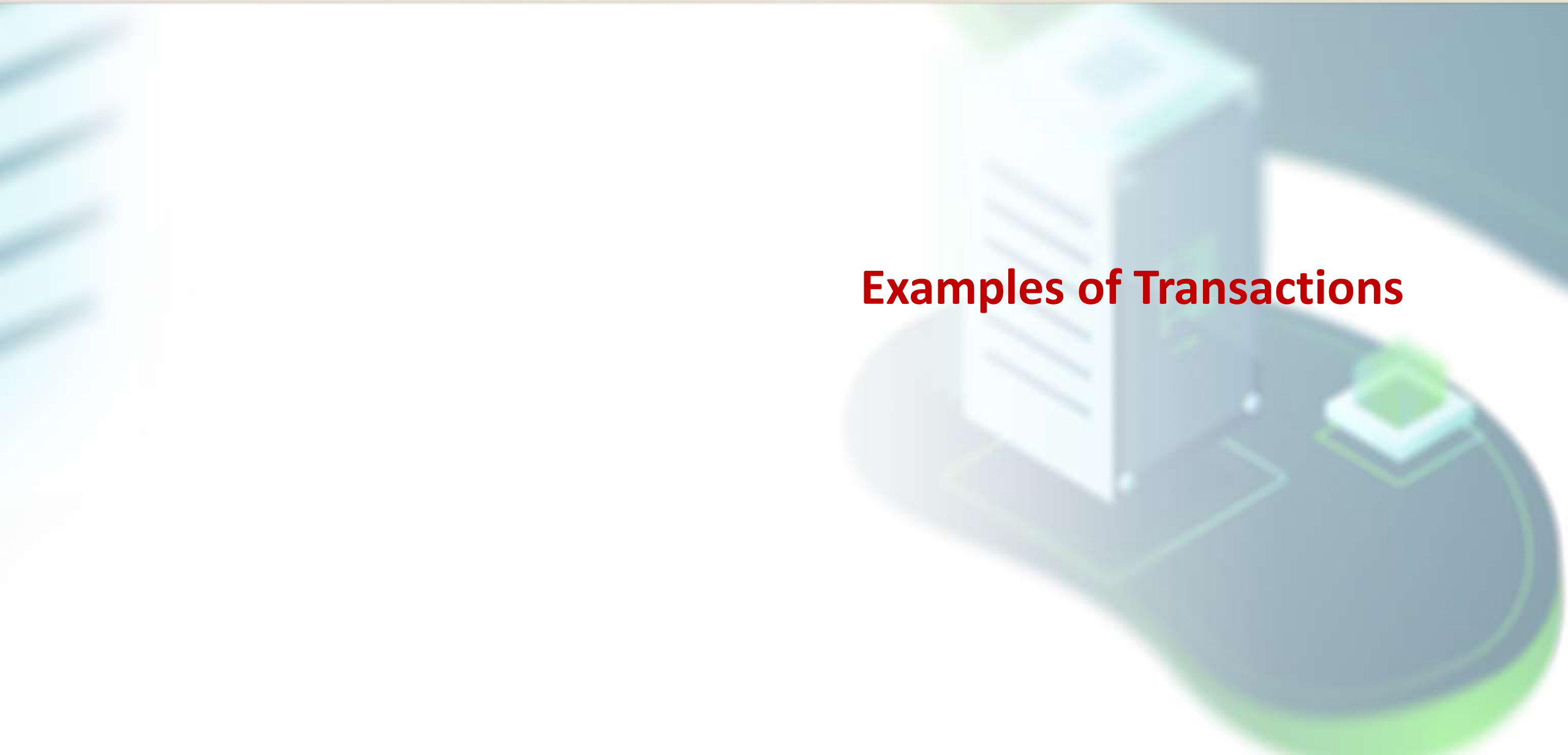
- Read balance of checking account: read(X)
- Read balance of savings account: read (Y)
- Subtract 400,000 from X
- Add 400,000 to Y
- Write new value of X back to disk
- Write new value of Y back to disk

Examples of Transactions

Writing a Transaction

T1





Examples of Transactions

Examples of Transactions

Example of Transactions

- Retail System - An Example
- Transactions & Their Issues

Examples of Transactions

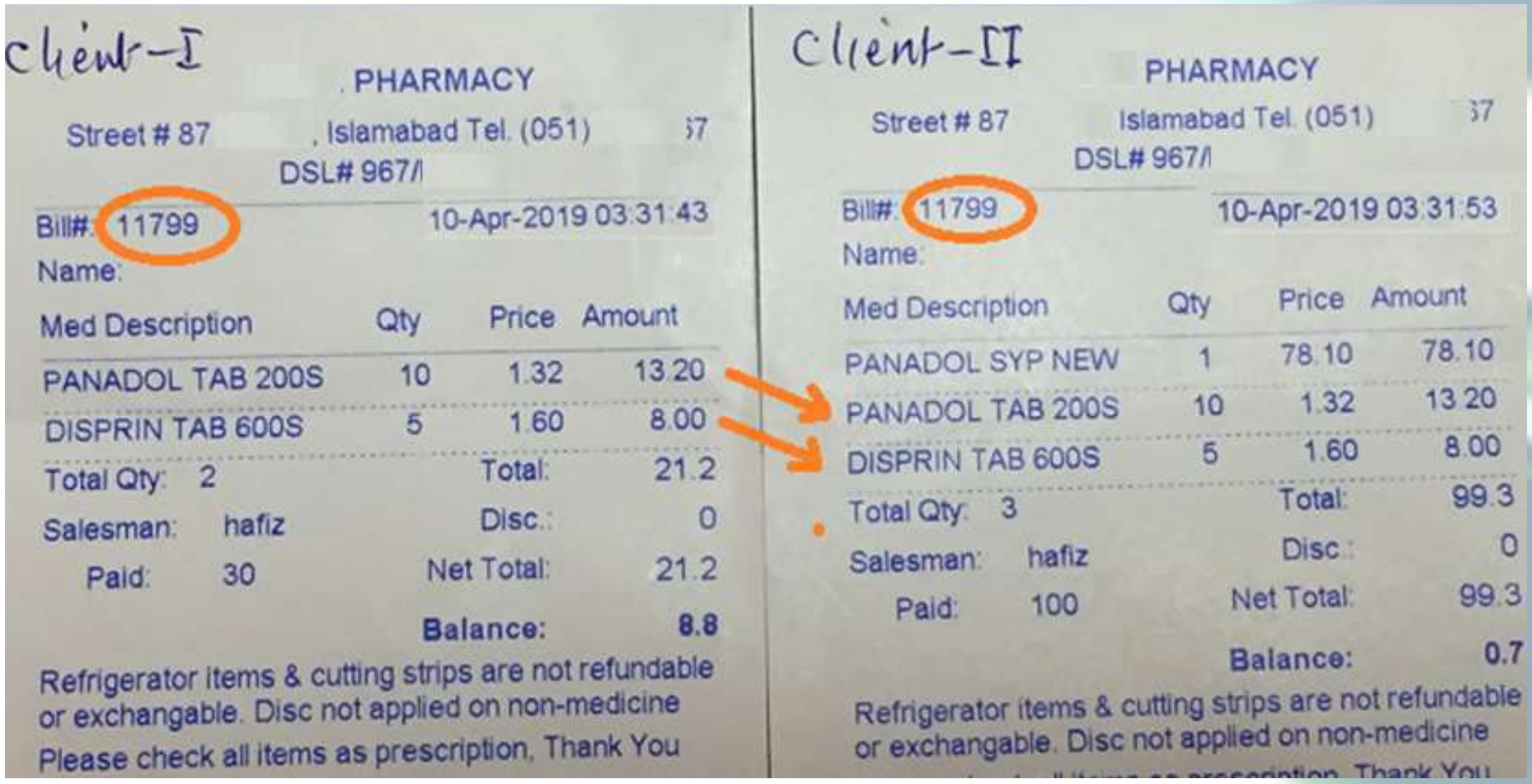
Retail Store – An Example

Invoice is issued a transaction when buying items

What happened multiple invoices are issued in Multi-user environment

Examples of Transactions

Transactions and their Issues



client-I

. PHARMACY

Street # 87 , Islamabad Tel. (051) 57

DSL# 967/

Bill#: 11799 10-Apr-2019 03:31:43

Name:

Med Description	Qty	Price	Amount
PANADOL TAB 200S	10	1.32	13.20
DISPRIN TAB 600S	5	1.60	8.00

Total Qty: 2 Total: 21.2

Salesman: hafiz Disc.: 0

Paid: 30 Net Total: 21.2

Balance: 8.8

Refrigerator items & cutting strips are not refundable or exchangable. Disc not applied on non-medicine

Please check all items as prescription. Thank You

client-II

PHARMACY

Street # 87 Islamabad Tel. (051) 57

DSL# 967/

Bill#: 11799 10-Apr-2019 03:31:53

Name:

Med Description	Qty	Price	Amount
PANADOL SYP NEW	1	78.10	78.10
PANADOL TAB 200S	10	1.32	13.20
DISPRIN TAB 600S	5	1.60	8.00

Total Qty: 3 Total: 99.3

Salesman: hafiz Disc.: 0

Paid: 100 Net Total: 99.3

Balance: 0.7

Refrigerator items & cutting strips are not refundable or exchangable. Disc not applied on non-medicine

Introduction to Database Administration



Transactions Read & Write

Transactions Read & Write

Example of Transactions

- Transaction Read Item
- Transaction Write Item
- Two Transactions

Transactions Read & Write

Transaction Read Item

A database is represented as a collection of named data items

Read-item (X)

- Find the address of the disk block that contains item X
- Copy the disk block into a buffer in main memory
- Copy the item X from the buffer to the program variable named X

Transactions Read & Write

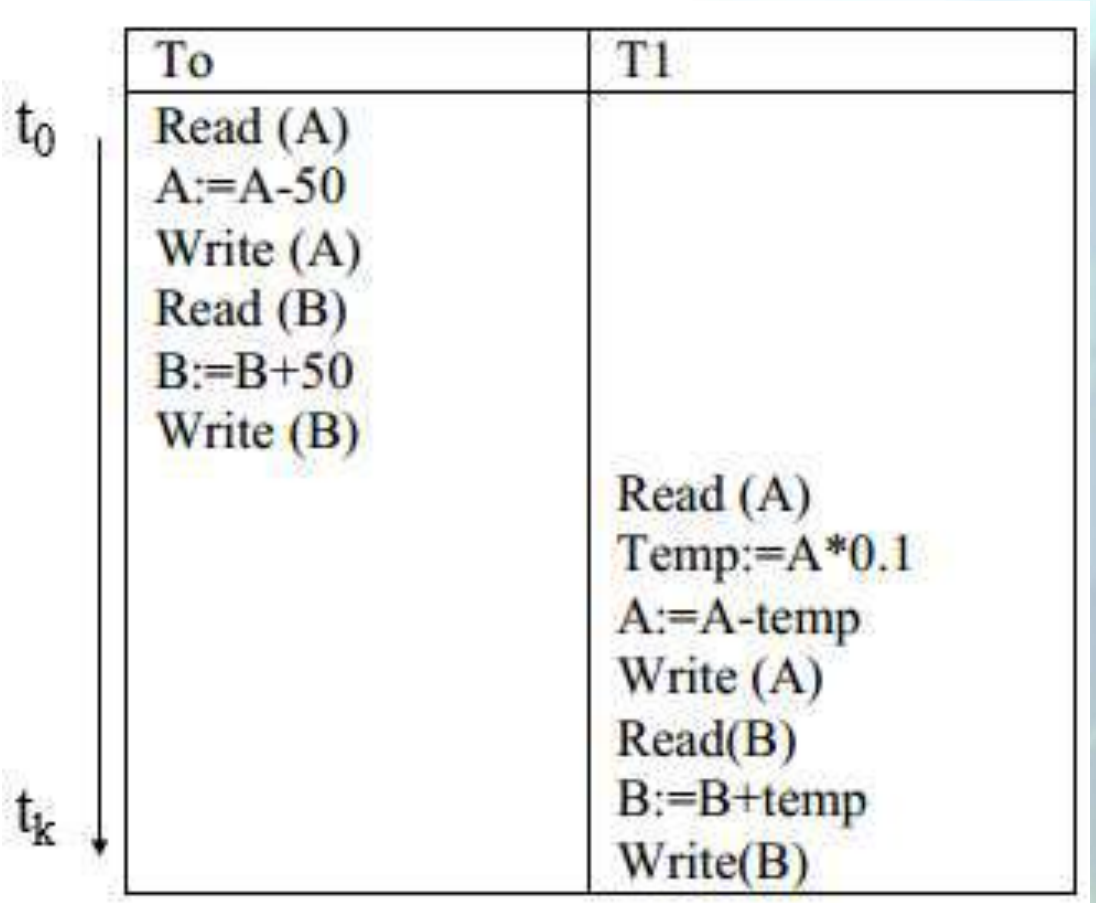
Transactions Write Item

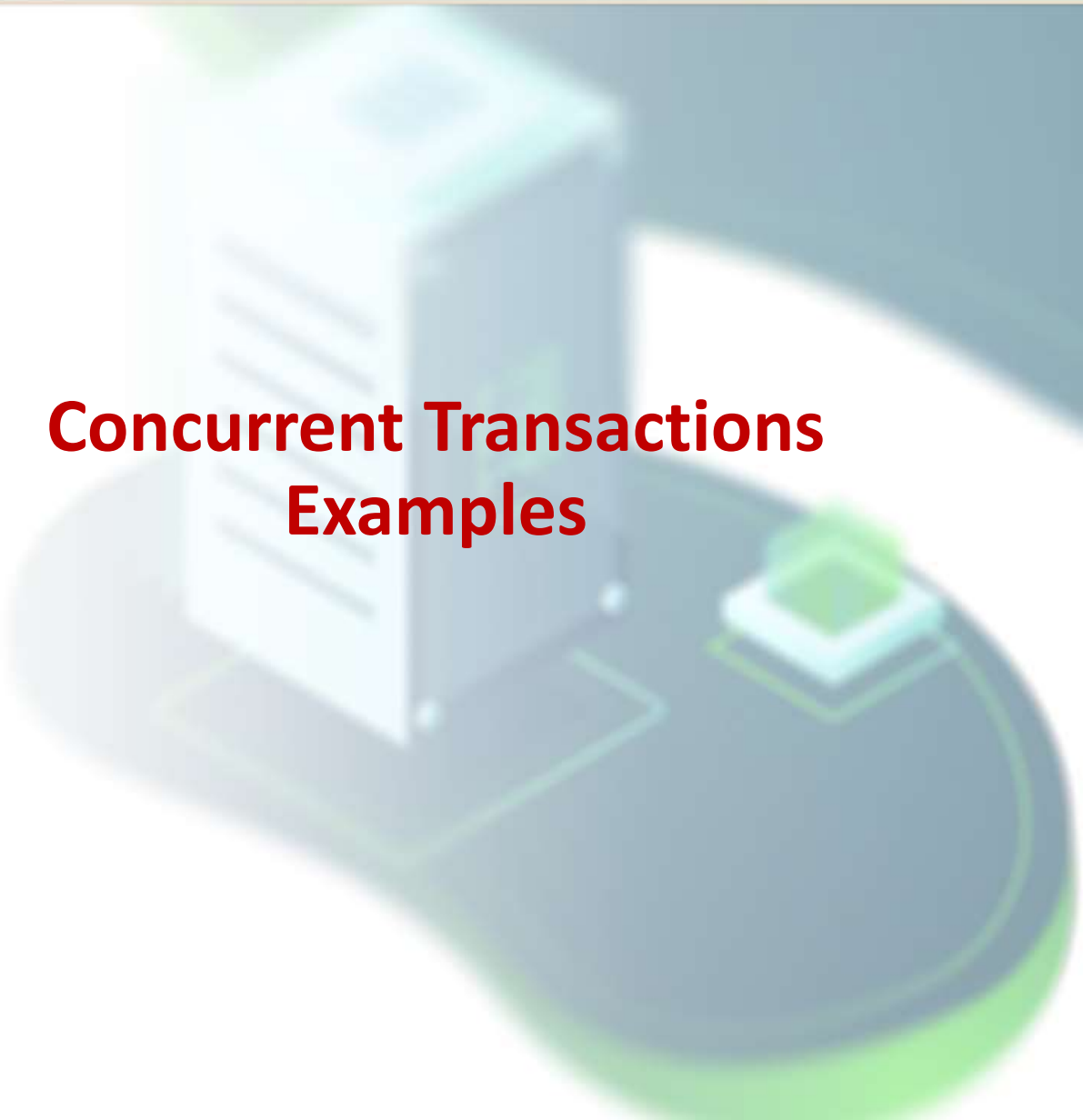
Write-item (X)

- Find the address of the disk block that contains item X.
- Copy that disk block into a buffer in main memory
- Copy item X from the program variable named X into its correct location in the buffer.
- Store the updated block from the buffer back to disk (either immediately or at some later point in time).

Transactions Read & Write

Two Transactions





Concurrent Transactions Examples

Concurrent Transactions Examples

Example of Transactions

- Concurrent Transaction
- Interleaved & Parallel Processing
- Concurrent Processing - An Example

Concurrent Transactions Examples

Concurrent Transaction

In the transaction process, a system usually allows executing more than one transaction simultaneously. This process is called a concurrent execution.

The reasons for wanting to run transactions concurrently are slow input/output devices (usually disks) and the presence of multiple processors on which different transactions can run at the same time.

Concurrent Transactions Examples

Concurrent Transaction

Informally, if there are two transactions in progress concurrently (that is, the Begin of one happens between the Begin and the Commit of the other), we want all the observable effects to be as though all the actions of one transaction happen either before or after all the actions of the other.

Concurrent Transactions Examples

Interleaved & Parallel Processing

Concurrent transactions refer to two or more transactions that appear to users as they are being processed against a database at the same time

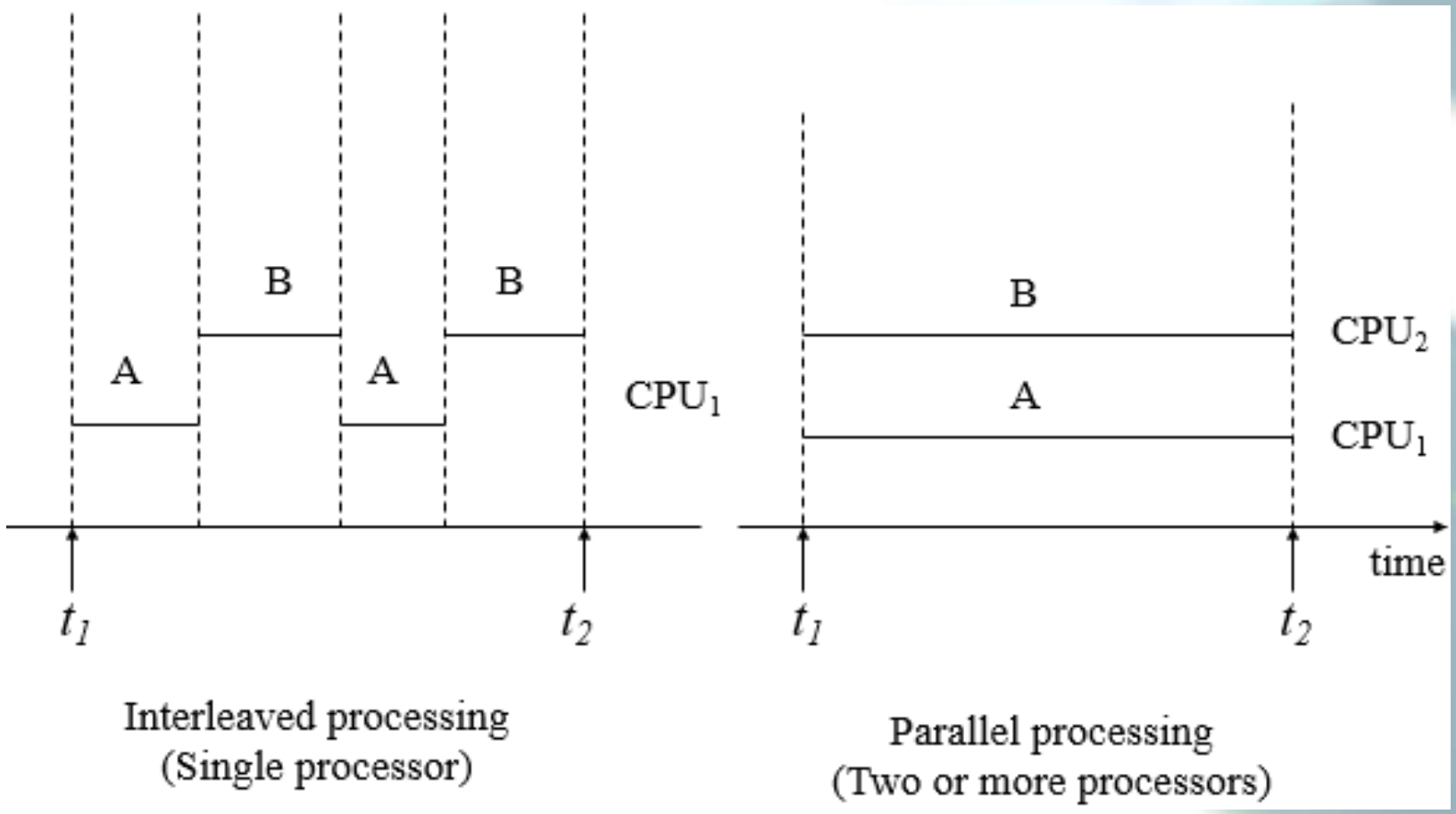
In reality, CPU can execute only one instruction at a time

- **Transactions are interleaved** meaning that the operating system quickly switches CPU services among tasks so that some portion of each of them is carried out in a given interval

Concurrency problems: lost update and inconsistent reads

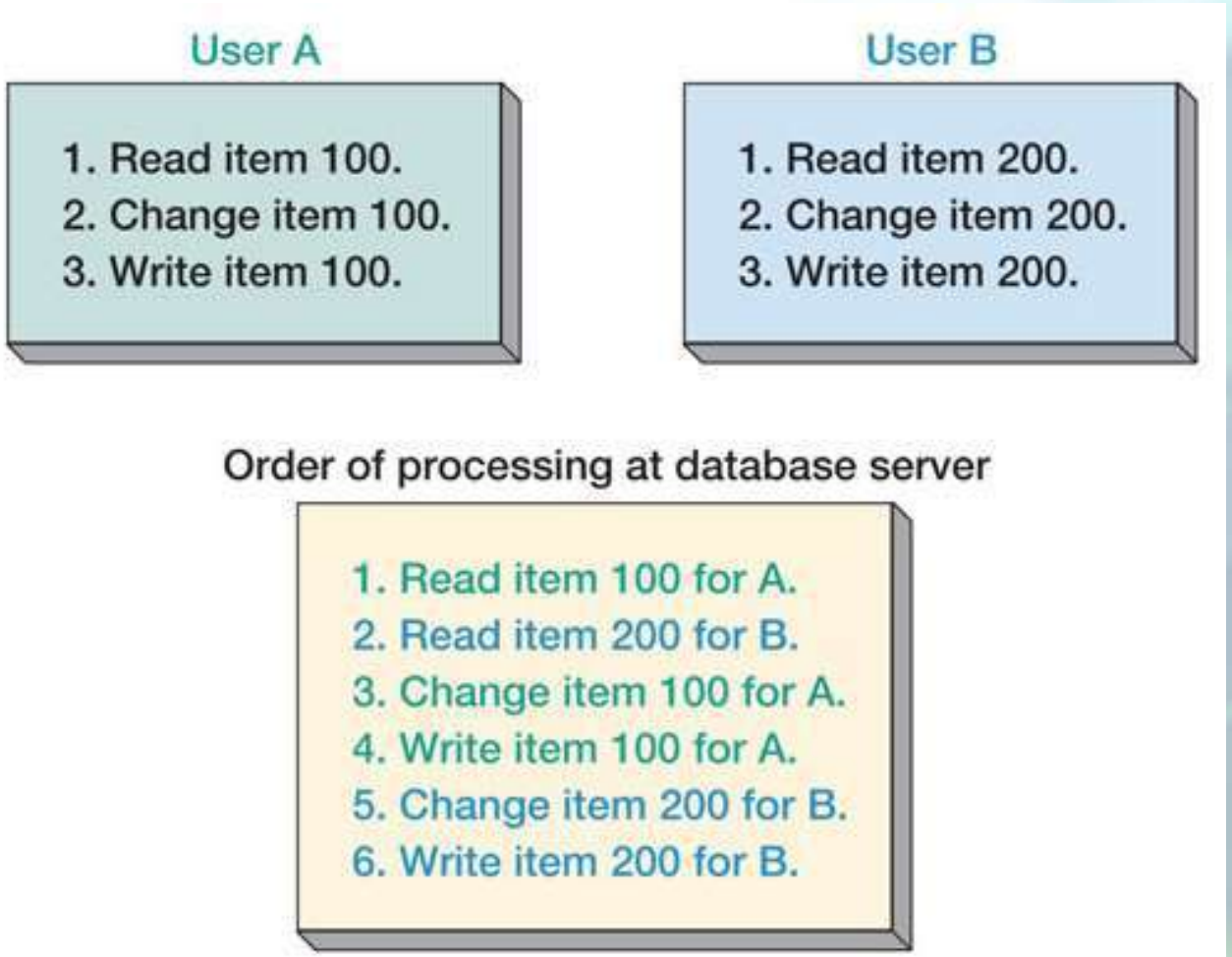
Concurrent Transactions Examples

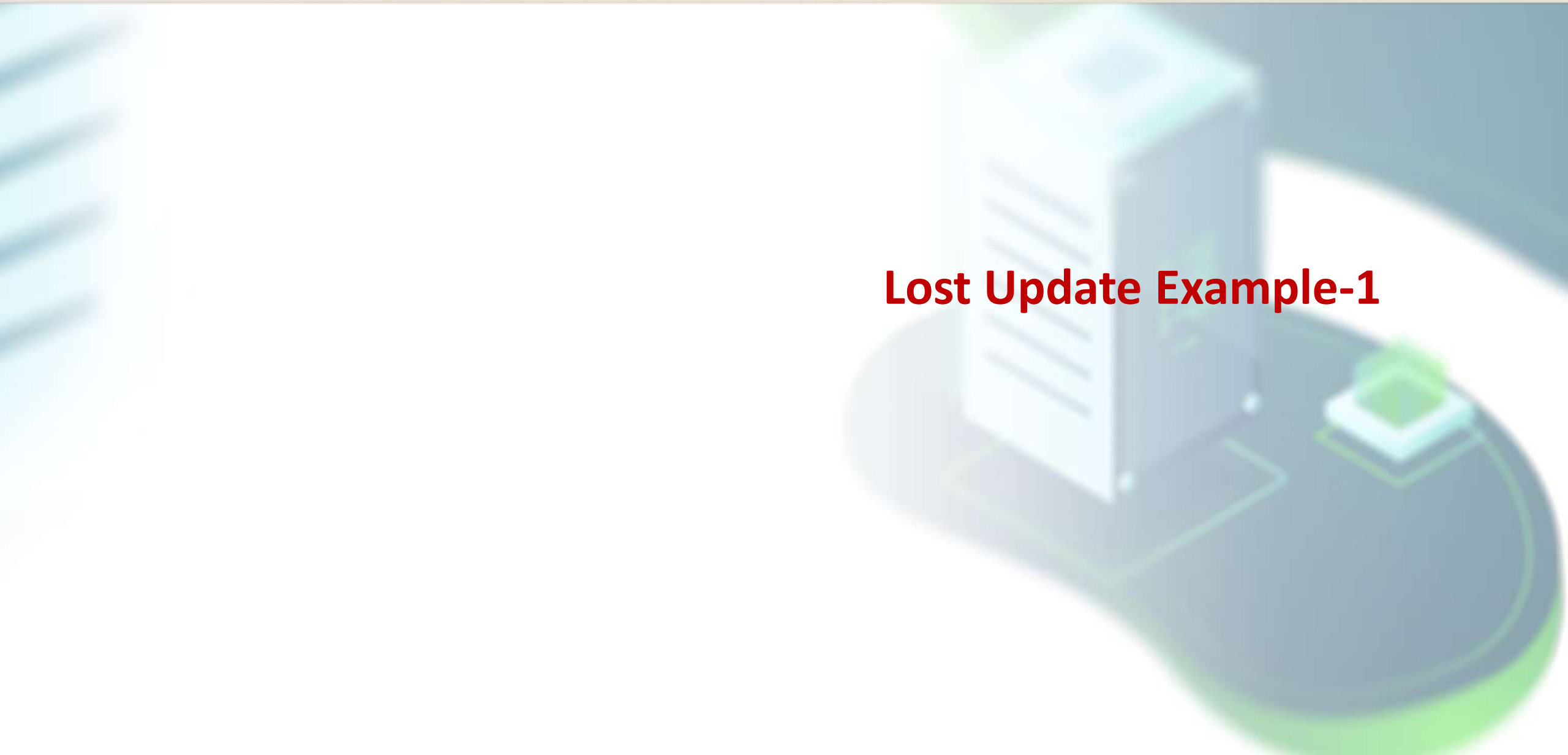
Interleaved & Parallel Processing



Concurrent Transactions Examples

Concurrent Processing – An Example





Lost Update Example-1

Lost Update Example-1

Example of Transactions

- Concurrency Problems
- Lost Update Problems
- Lost Update Example-1

Lost Update Example-1

Concurrency Problems

Several problems can occur when concurrent transactions are run in an uncontrolled manner, such type of problems is known as concurrency problems.

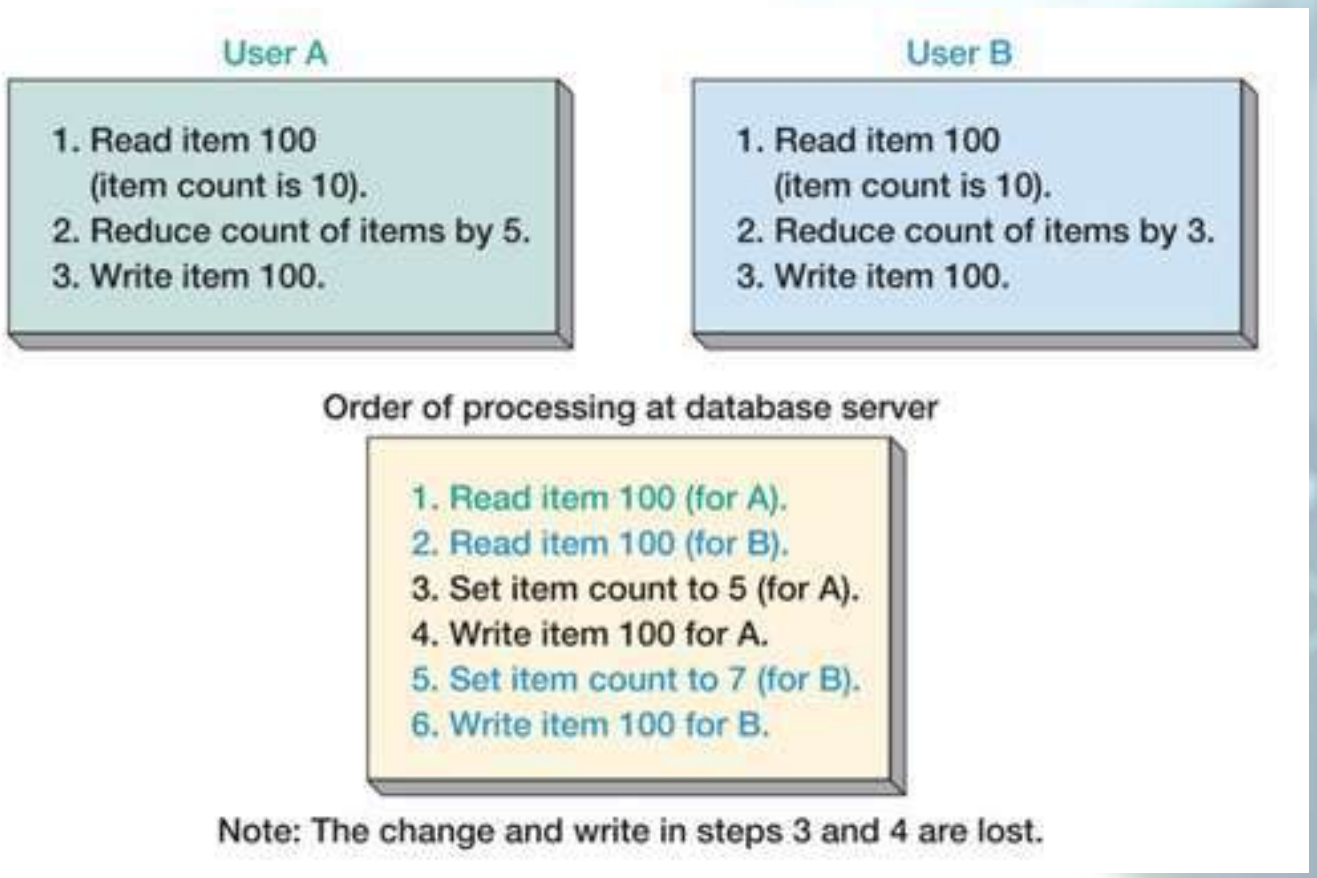
Lost Update Example-1

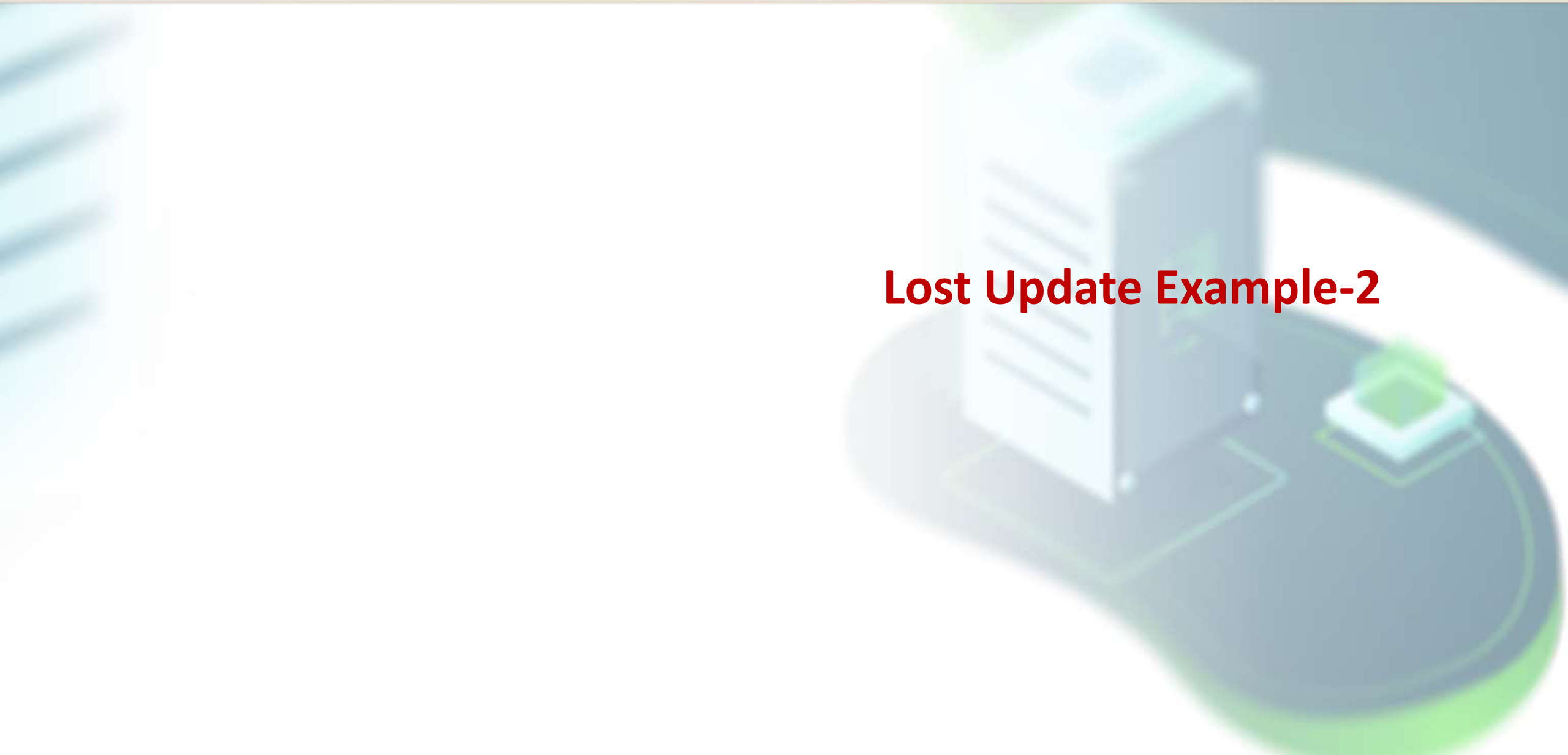
Lost Update Problems

This type of problem occurs when two transactions in database access the same data item and have their operations in an interleaved manner that makes the value of some database item incorrect.

Lost Update Example-1

Lost Update Example-1





Lost Update Example-2

Lost Update Example-2

Example of Transactions

- Concurrency Problems
- Lost Update Problems
- Lost Update Example-2

Lost Update Example-2

Concurrency Problems

Several problems can occur when concurrent transactions are run in an uncontrolled manner, such type of problems is known as concurrency problems.

Lost Update Example-2

Lost Update Problems

This type of problem occurs when two transactions in database access the same data item and have their operations in an interleaved manner that makes the value of some database item incorrect.

Lost Update Example-2

Lost Update Example-2

Time	T1	T2	balx
t1		BEGIN	100
t2	BEGIN	read(balx)	100
t3	read(balx)	balx=balx+100	100
t4	balx=balx-10	write(balx)	200
t5	write(balx)	commit	90
t6	commit	END	90
t7	END		

Lost Update Example-2

Lost Update Example-2

Two users are reading a same Item#45667, reading current quantity 120 simultaneously

Transaction-1



Transaction-2



Dirty Read Example

Dirty Read Example

Dirty Read Transactions

- Dirty Read Problem
- Dirty Read Example

Dirty Read Example

Dirty Read Problem

This type of problem occurs when one transaction T1 updates a data item of the database, and then that transaction fails due to some reason, but its updates are accessed by some other transaction.

Dirty Read Example

Dirty Read Example

Time	T3	T4	balx
t1		BEGIN	100
t2		read(balx)	100
t3		balx=balx+100	100
t4	BEGIN	write(balx)	200
t5	read(balx)	...	200
t6	balx=balx-10	rollback	100
t7	write(balx)	END	190
t8	commit		190
t9	END		

Dirty Read Example

Dirty Read Example

Time	Transaction T1	Transaction T2
t1	Read(A)	
t2	A=A+20	
t3	Write(A)	
t4		Read(A)
t5		A=A+30
t6		Write(A)
t7	Write(B)	



Incorrect Summary Problem

Incorrect Summary Problem

Incorrect Summary Problem

- Incorrect Summary Problem
- Incorrect Summary Problem - An Example

Incorrect Summary Problem

Incorrect Summary Problem

It is also known as an inconsistent retrieval problem. If a transaction T_1 reads a value of data item twice and the data item is changed by another transaction T_2 in between the two read operation. Hence T_1 access two different values for its two read operation of the same data item.

Incorrect Summary Problem

Incorrect Summary Problem – An Example

Time	Transaction T1	Transaction T2
t1	Read(A)	
t2		Read(A)
t3		A=A+30
t4		Write(A)
t5	Read(A)	

Incorrect Summary Problem

Incorrect Summary Problem – An Example

Time	T3	T4	balx	baly	balz	sum
t1		BEGIN	100	50	25	
t2	BEGIN	sum=0				
t3	read(balx)	read(balx)				
t4	balx=balx-10	sum=sum+balx				
t5	write(balx)	read(baly)				
t6	read(balz)	sum=sum+baly				
t7	balz=balz+10					
t8	write(balz)					
t9	commit	read(balz)				
t10	END	sum=sum+balz				
t11		commit				
t12		END				

Introduction to Database Administration

Data Inconsistency in Two Invoices

Data Inconsistency in Two Invoices

Data Inconsistency with Multiple Users

- Single User in a Retail Store
- Retail Store - An Example
- Transactions and their Issues

Data Inconsistency in Two Invoices

Single User in a Retail Store

Single user issue an Invoice one at a time when only one POS terminal is active

When two or more users are issuing invoices, there is a chance of data inconsistency

Data Inconsistency in Two Invoices

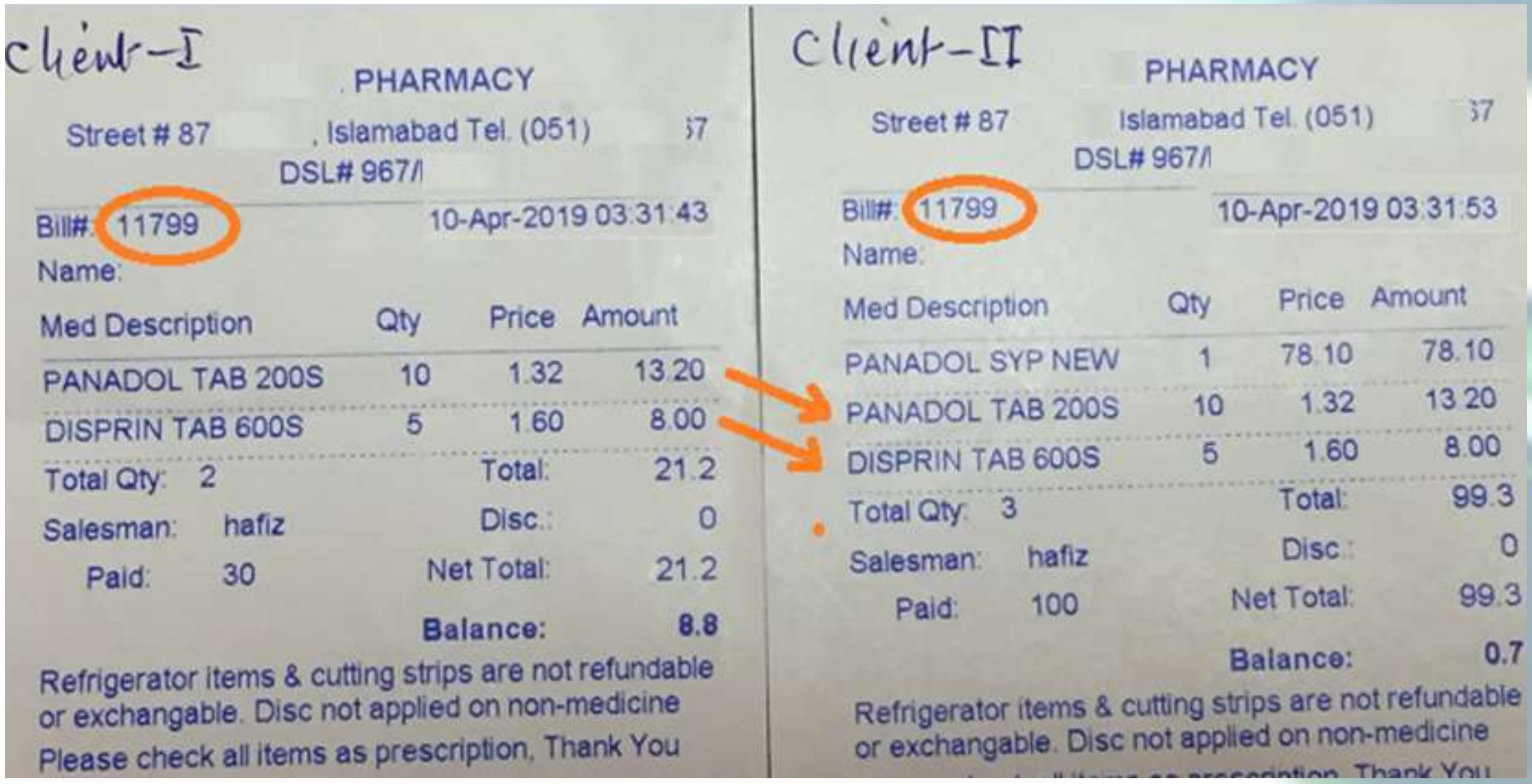
Retail Store – An Example

For example, two invoices are issued at the same time from two POS terminals

What happened multiple invoices are issued in Multi-user environment

Data Inconsistency in Two Invoices

Transactions and their Issues



client-I

PHARMACY

Street # 87 , Islamabad Tel. (051) 57

DSL# 967/

Bill#: 11799 10-Apr-2019 03:31:43

Name:

Med Description	Qty	Price	Amount
PANADOL TAB 200S	10	1.32	13.20
DISPRIN TAB 600S	5	1.60	8.00

Total Qty: 2 Total: 21.2

Salesman: hafiz Disc.: 0

Paid: 30 Net Total: 21.2

Balance: 8.8

Refrigerator items & cutting strips are not refundable or exchangable. Disc not applied on non-medicine

Please check all items as prescription. Thank You

client-II

PHARMACY

Street # 87 Islamabad Tel. (051) 57

DSL# 967/

Bill#: 11799 10-Apr-2019 03:31:53

Name:

Med Description	Qty	Price	Amount
PANADOL SYP NEW	1	78.10	78.10
PANADOL TAB 200S	10	1.32	13.20
DISPRIN TAB 600S	5	1.60	8.00

Total Qty: 3 Total: 99.3

Salesman: hafiz Disc.: 0

Paid: 100 Net Total: 99.3

Balance: 0.7

Refrigerator items & cutting strips are not refundable or exchangable. Disc not applied on non-medicine

Introduction to Database Administration



System Failure Problems

System Failure Problems

System Failure Problems

- What Can Go Wrong?
- System Failure Problems

System Failure Problems

What Can Go Wrong?

- System may crash before data is written back to disk
  - Problem of *atomicity*
- Some transaction is modifying shared data while another transaction is ongoing (or vice versa)
  - Problem of *serialization* and *isolation*

System Failure Problems

What Can Go Wrong?

- System may not be able to obtain one or more of the data items
- System may not be able to write one or more of the data items (Problems of *atomicity)*
- DBMS has a *Concurrency Control* subsytem to assure database remains in consistent state despite concurrent execution of transactions

System Failure Problems

System Failure Problems

System failures may occur

- Types of failures:
  - System crash
  - Transaction or system error
  - Local errors
  - Concurrency control enforcement
  - Disk failure
  - Physical failures

DBMS has a *Recovery* Subsystem to protect database against system failures

End

Introduction to Database Administration



System Log

System Log

System Log

- Introduction
- System Log
- How the Log File is used?
- Transaction Log Example

System Log

Introduction

- **Log or Journal** : The log keeps track of all transaction operations that affect the values of database items. This information may be needed to permit recovery from transaction failures. The log is kept on disk, so it is not affected by any type of failure except for disk or catastrophic failure. In addition, the log is periodically backed up to archival storage (tape) to guard against such catastrophic failures.

System Log

Introduction

- T refers to a unique **transaction-id** that is generated automatically by the system and is used to identify each transaction

System Log

System Log

- Transaction –id
- System log
  - Multiple record-type file
  - Log is kept on disk
  - Periodically backed up

System Log

System Log

- Log records
  - [start_transaction, T]
  - [write_item, T,X,old_value,new_value]:
  - [read_item, T,X]
  - [commit,T]
  - [abort,T]
  - [checkpoint]
- Commit point of a transaction

System Log

How the Log File is used?

- All permanent changes to data are recorded
  - Possible to undo changes to data
- After crash, search log backwards until find last checkpoint
  - Know that beyond this point, effects of transaction are permanently recorded

System Log

How the Log File is used?

- Need to either *redo* or *undo* everything that happened since last checkpoint
  - Undo: When transaction only partially completed (before crash)
  - Redo: Transaction completed but we are unsure whether data was written to disk

System Log

Transaction Log Example

TransID	Table	RowID	column	Before	After
100	**start**				
100	EMPLOYEE	1005472	Salary	8000	8250
100	PROJECT	1423PRJ	PrjRate	253	256
100	**end committed**				

Introduction to Database Administration



Schedules & Recoverability

Schedules & Recoverability

- Transaction and Schedule
- Schedule
- Example of Schedule

Schedules & Recoverability

Transaction and Schedule

We have discussed that

- Multiple transactions can be executed concurrently by interleaving their operations

Schedule

- Ordering of execution of operations from various transactions T_1, T_2, … , T_n is called a schedule *S*

Schedules & Recoverability

Schedule

Schedule S of n transactions T_1, T_2, … , T_n is an ordering of the operations of the transactions subject to the constraint that, for each transaction T_i that participates in S, the operations of T_i in S must appear in the same order in which they occur in T_i.

Schedules & Recoverability

Example of Schedule

Transaction T_1: $r_1(X)$; $w_1(X)$; $r_1(Y)$; $w_1(Y)$; c_1

Transaction T_2: $r_2(X)$; $w_2(X)$; c_2

A schedule, S:

$r_1(X)$; $r_2(X)$; $w_1(X)$; $r_1(Y)$; $w_2(X)$; $w_1(Y)$; c_1; c_2

Introduction to Database Administration



Conflicts

Conflicts

Schedule with Conflicts

- Conflicts
- Conditions for Conflicts
- Example of Conflicts

Conflicts

Conflicts

Conflict operations can creates inconsistency

Two operations *conflict* if they satisfy ALL three conditions:

1. they belong to different transactions **AND**
2. they access the same item **AND**
3. at least one is a **write_item()**operation

Conflicts

Conditions for Conflicts

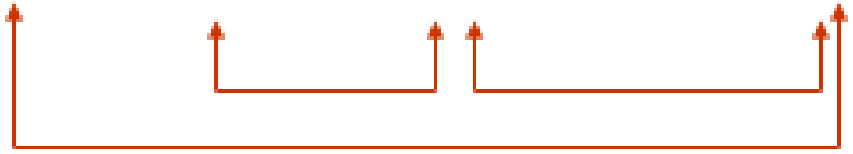
Two operations *conflict* if they satisfy ALL three conditions:

1. they belong to different transactions **AND**
2. they access the same item **AND**
3. at least one is a **write_item()**operation

Conflicts

Example of Conflicts

S: $r_1(X)$; $r_2(X)$; $w_1(X)$; $r_1(Y)$; $w_2(X)$; $w_1(Y)$;



conflicts

Introduction to Database Administration



Serializability

Serializability

Schedule with Serializability

- Serializability
- Serial Schedule
- Characteristics of Serial Schedule

Serializability

Serializability

- Serial Schedule
- Non-serial schedule
- Serializable schedule
  - Conflict-serializable schedule
  - View-serializable schedule

Serializability

Serializability

- **Serial and Non-serial schedule**
  A schedule *S* is **serial** if, for every transaction *T* participating in the schedule, all the operations of *T* are executed consecutively in the schedule; otherwise, the schedule is called **non-serial**
- **Serializable schedule**
  A schedule *S* of *n* transactions is **serializable** if it is *equivalent to some serial schedule* of the same *n* transactions

Serializability

Serial Schedule

- Serial schedule is guaranteed correct, should we adopt it?
- No, that will be inefficient, under the utilization of resources
- Dislike by the users

Serializability

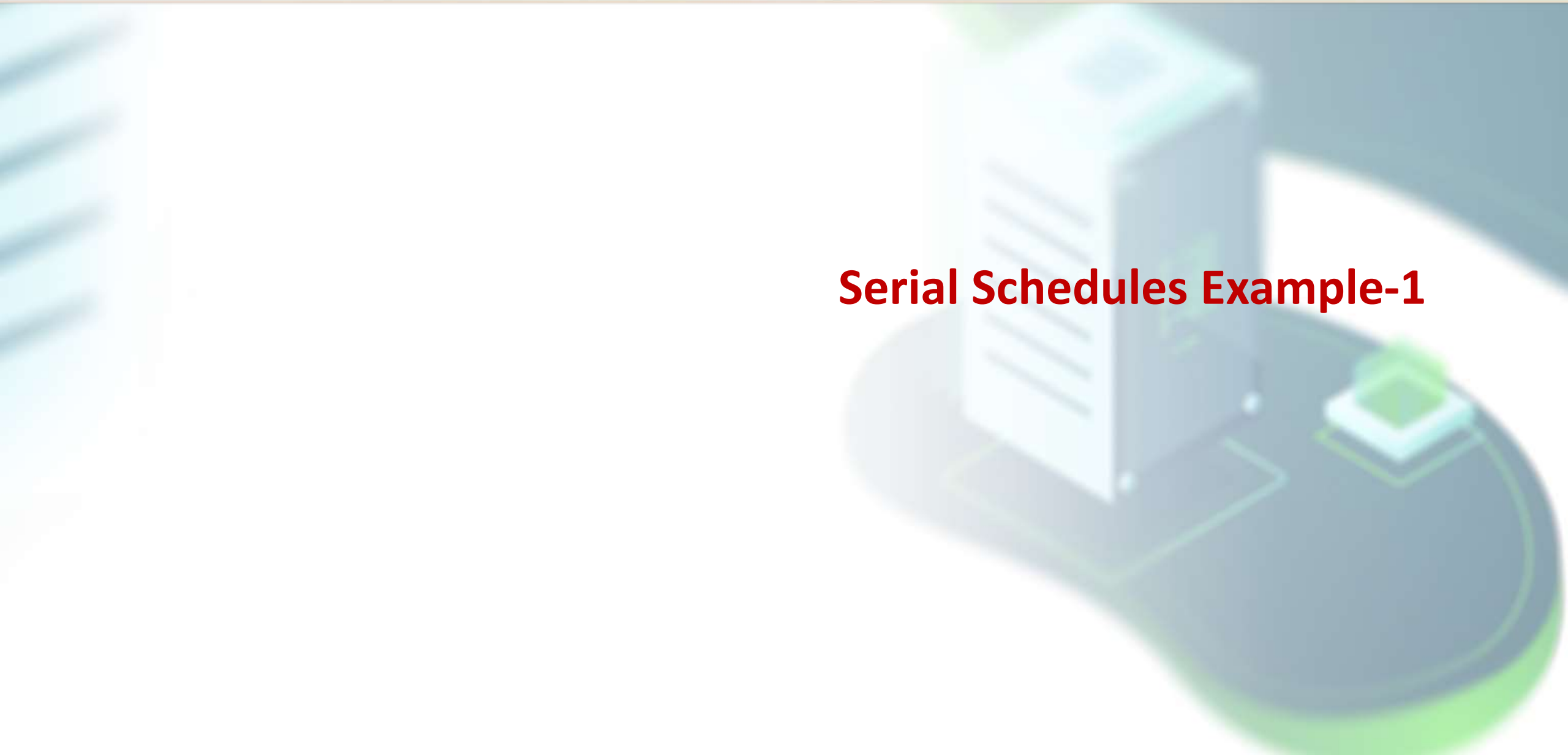
Characteristics of Serial Schedule

Result equivalent: Two schedules are called result equivalent if they produce the same final state of the database.

Conflict equivalent: Two schedules are said to be conflict equivalent if the order of any two conflicting operations is the same in both schedules.

Conflict serializable: A schedule S is said to be conflict serializable if it is conflict equivalent to some serial schedule S'.

End



Serial Schedules Example-1

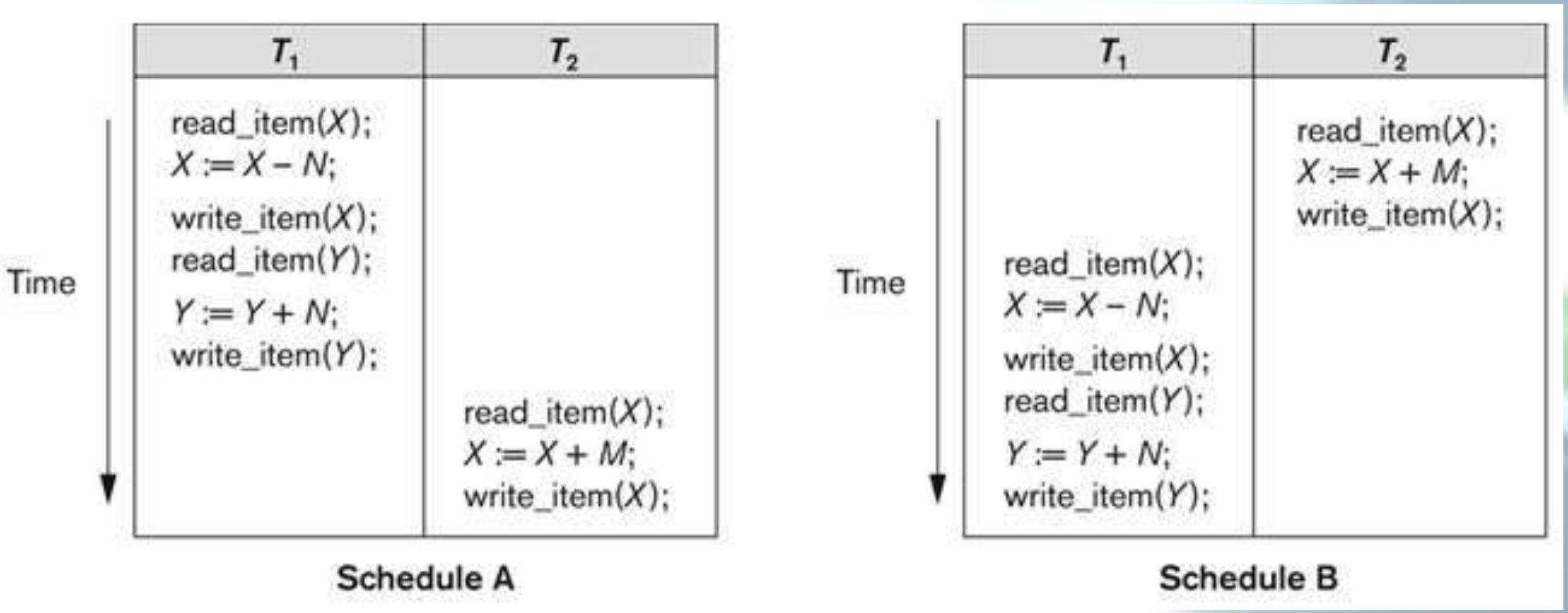
Serial Schedules Example-1

Schedule with Serial Schedules Example-1

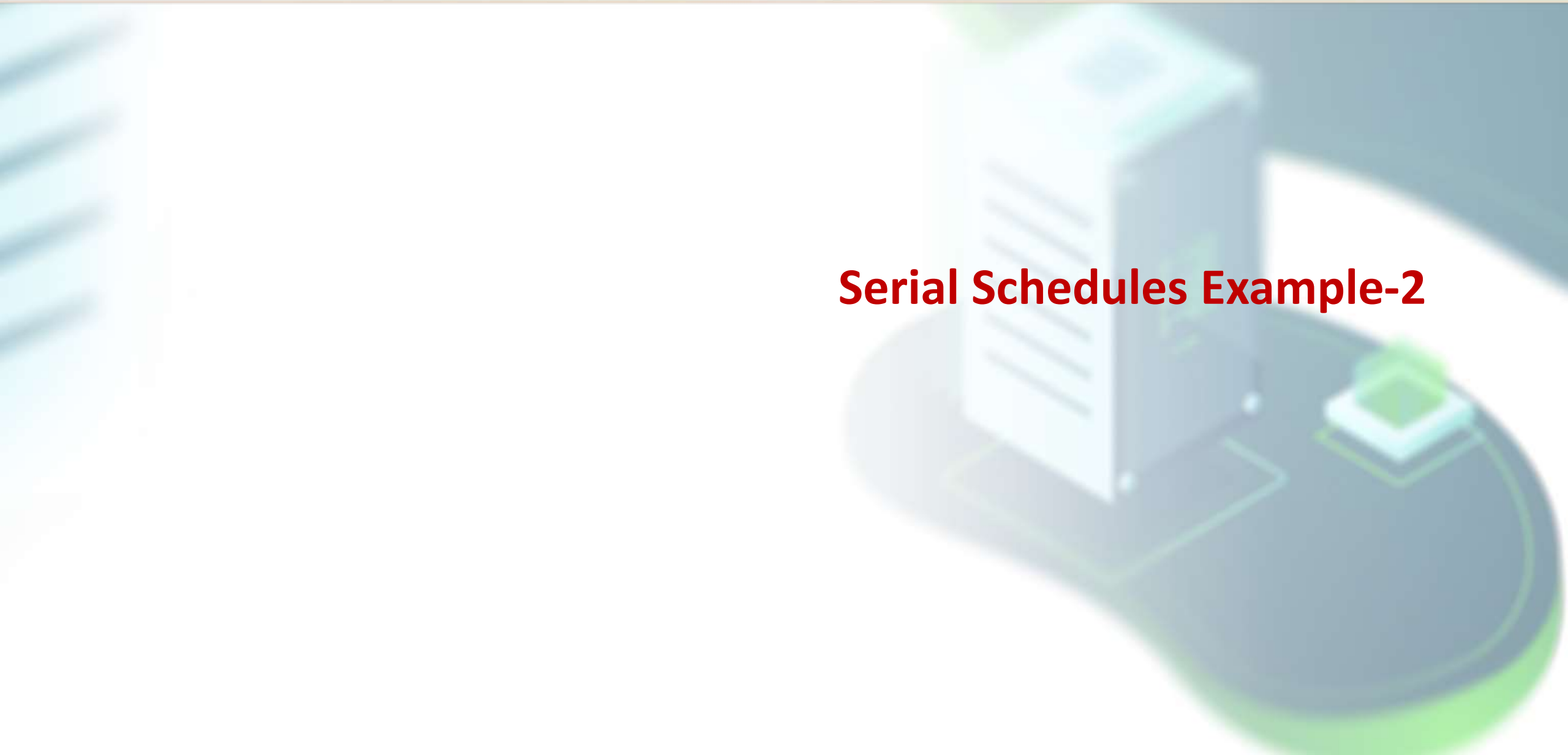
- Example-1

Serial Schedules Example-1

Example-1



Assume X=90, Y=90, N=3, M=2



Serial Schedules Example-2

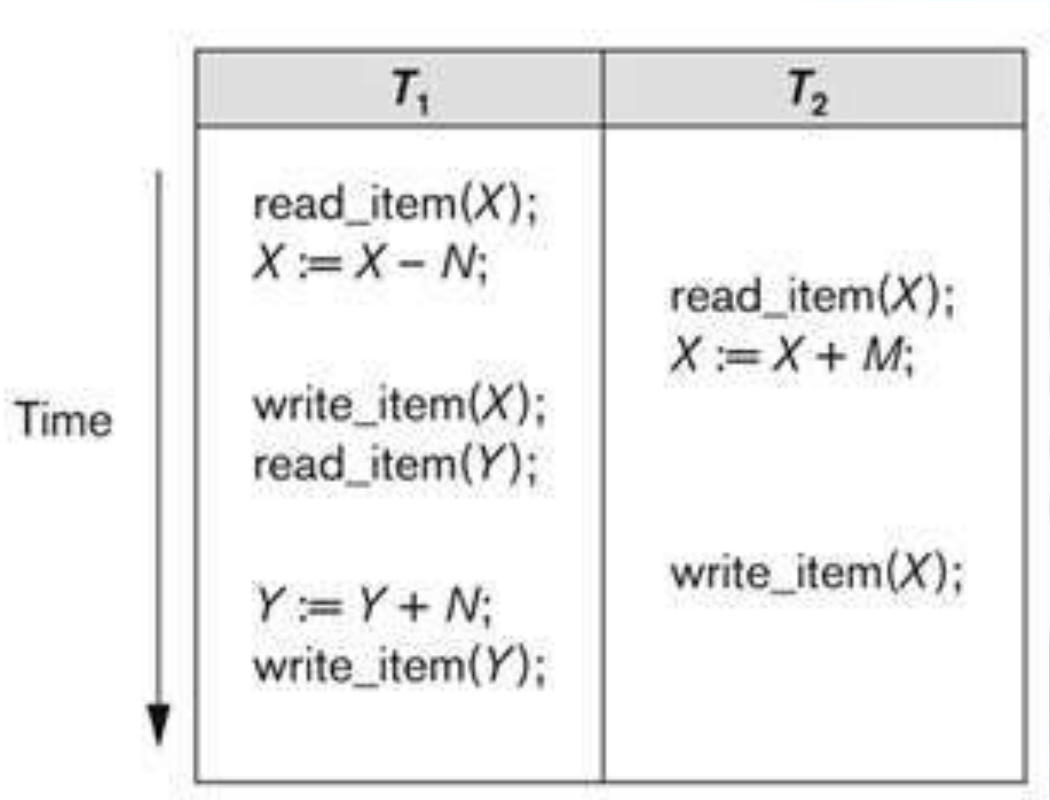
Serial Schedules Example-2

Schedule with Serial Schedules Example-2

- Example-2

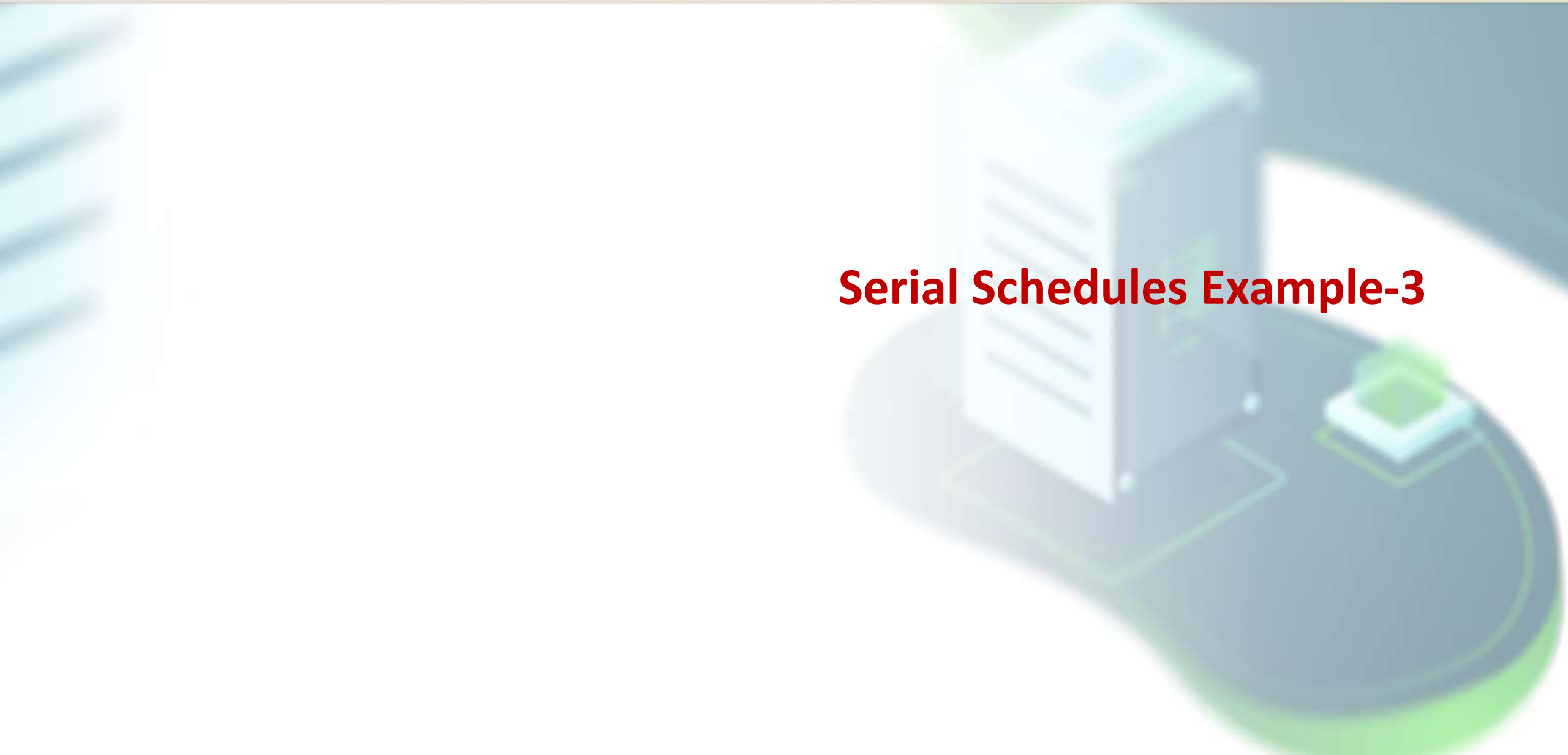
Serial Schedules Example-2

Example-2 (Schedule S')



Assume X=90, Y=90, N=3, M=2

Not Correct Values



Serial Schedules Example-3

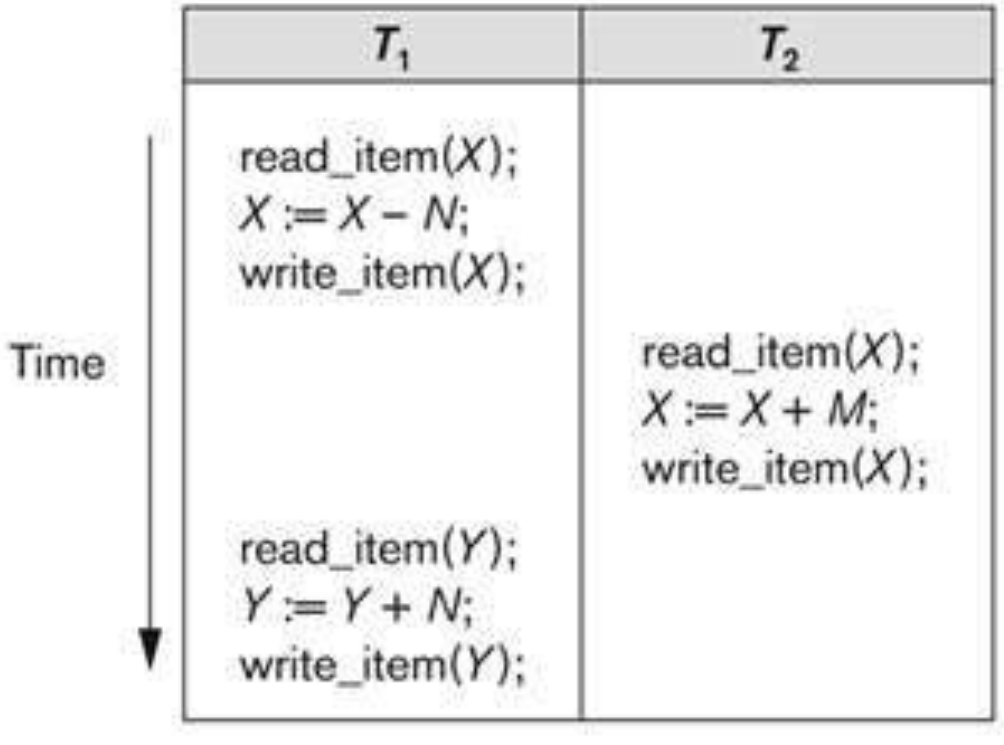
Serial Schedules Example-3

Schedule with Serial Schedules Example-3

- Example-3

Serial Schedules Example-3

Example-3 (Schedule S'')



Assume X=90, Y=90, N=3, M=2

Produces same result as serial schedule



Conflict Serializability Example-1

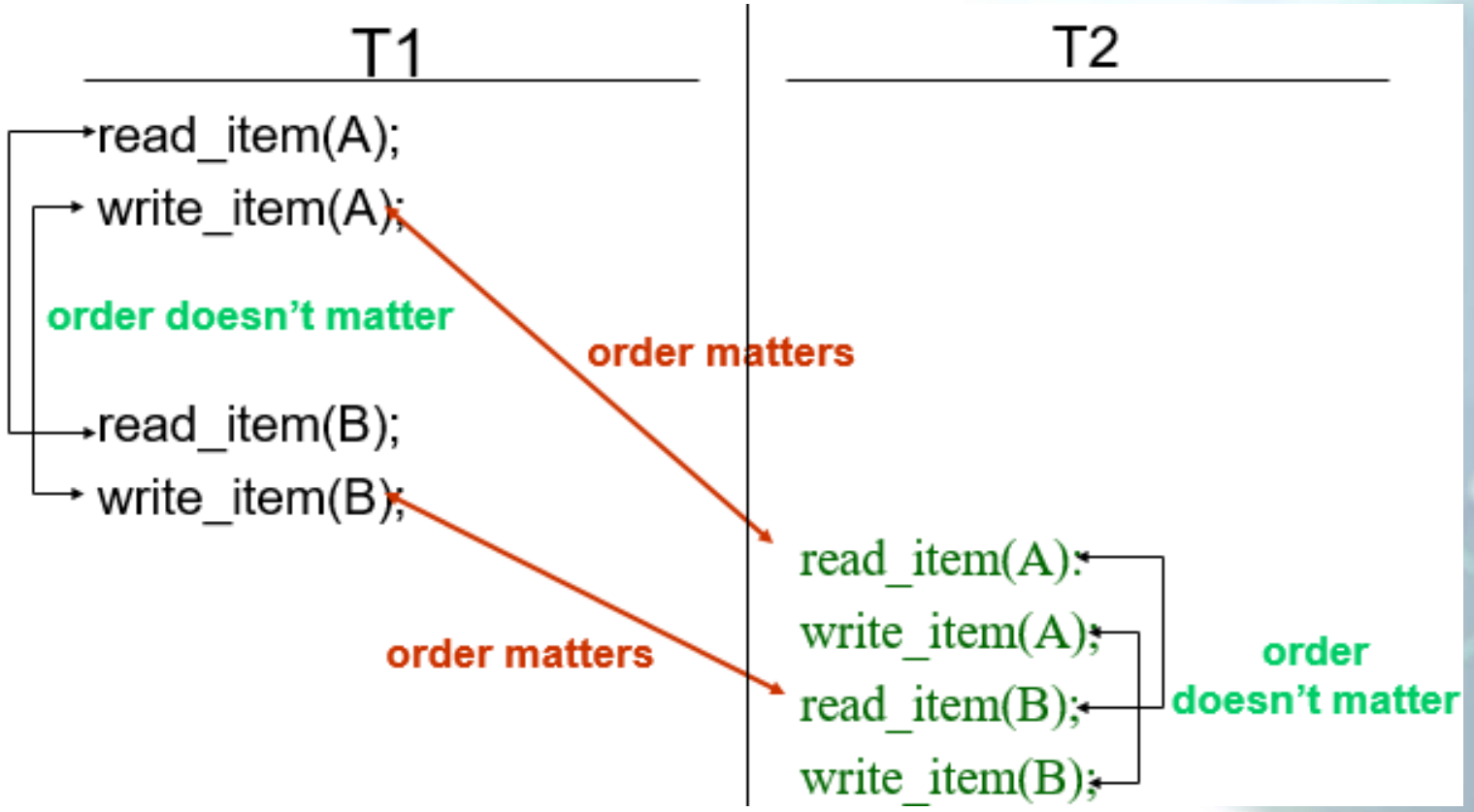
Conflict Serializability Example-1

Schedule with Conflict Serializability Example-1

- Example-1 (S1)
- Example-1 (S1’)
- Example-1 (Schedules A, B)

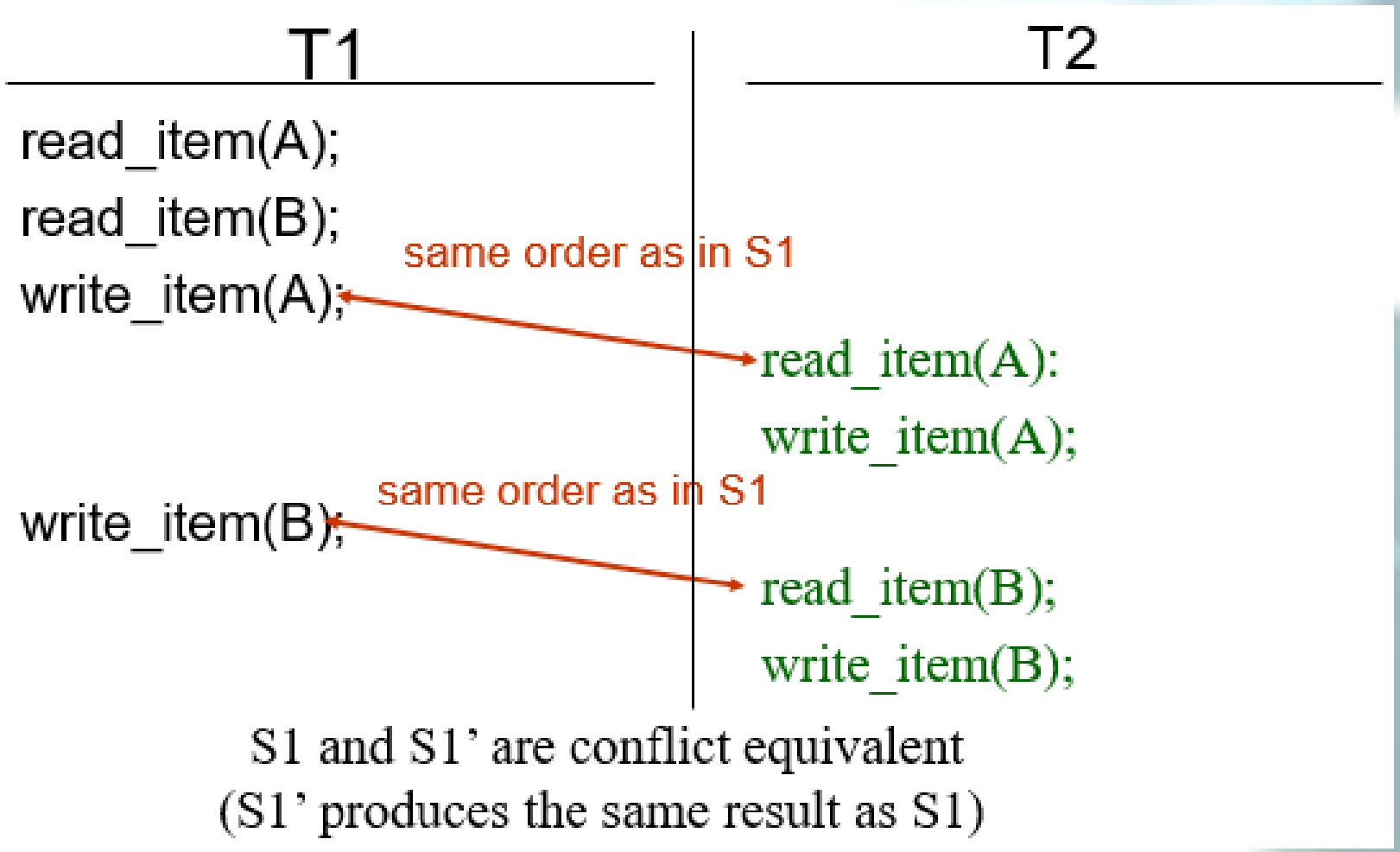
Conflict Serializability Example-1

Example-1 (Schedule S)



Conflict Serializability Example-1

Example-1 (Schedule S1')



S1 and S1' are conflict equivalent
(S1' produces the same result as S1)

Conflict Serializability Example-1

Example-1 (Schedules A, B)

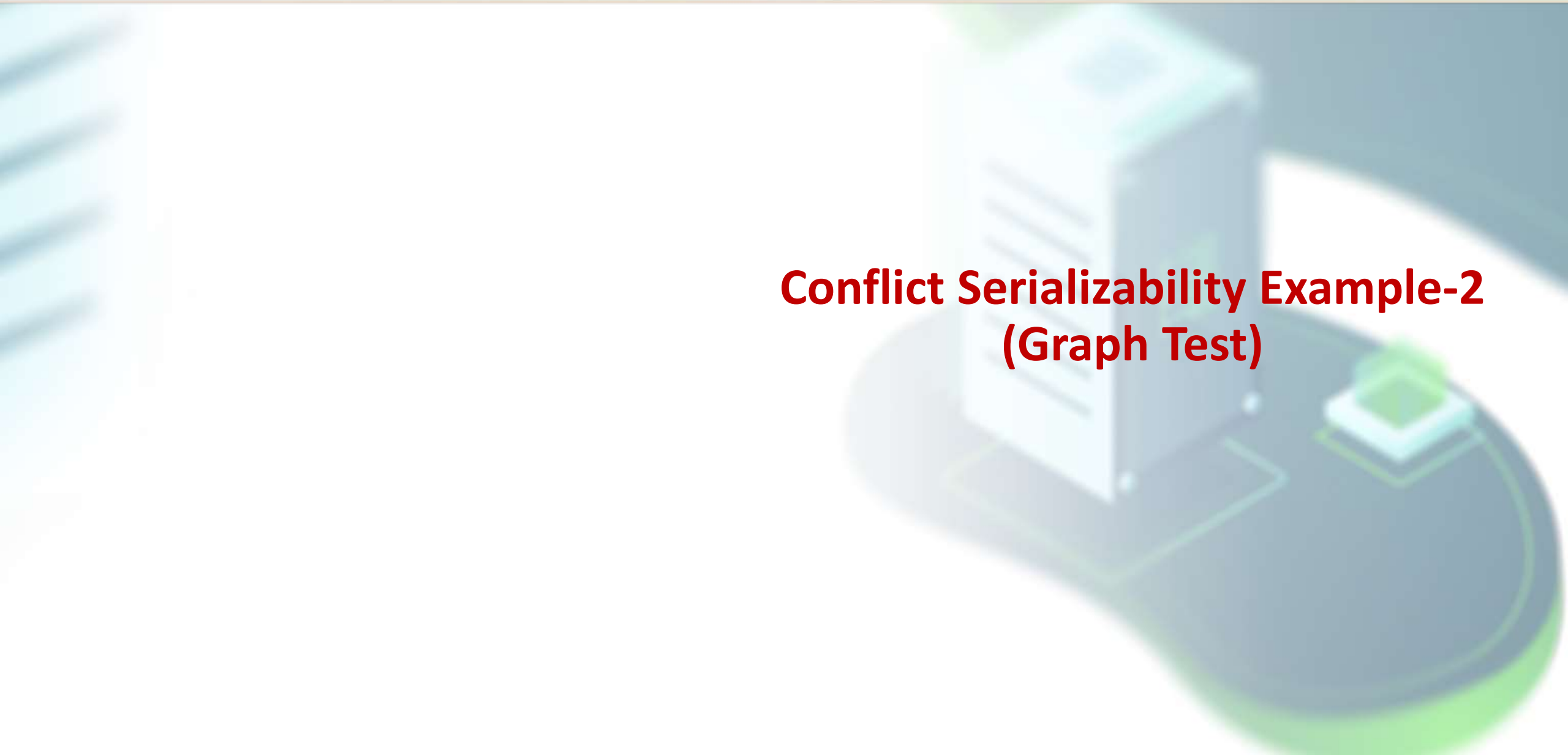
Schedule A

T1	T2
read_item(X);	
X:=X-N;	
write_item(X);	
read_item(Y);	
Y:=Y+N;	
write_item(Y);	
	read_item(X);
	X:=X+M;
	write_item(X);

Schedule B

T1	T2
read_item(X);	
X:=X-N;	
write_item(X);	
	read_item(X);
	X:=X+M;
	write_item(X);
read_item(Y);	
Y:=Y+N;	
write_item(Y);	

B is conflict equivalent to A ⇒ B is serializable



Conflict Serializability Example-2 (Graph Test)

Conflict Serializability Example-2 (Graph Test)

Conflict Serializability Example-2 (Graph Test)

- Test for Serializability
- Example-2 (Graph Test)

Conflict Serializability Example-2 (Graph Test)

Test for Serializability

Construct a directed graph, *precedence graph,* G = (V, E)

V: set of all transactions participating in schedule

E: set of edges $T_i \rightarrow T_j$ for which one of the following holds:

T_i executes a write_item(X) before T_j executes read_item(X)

T_i executes a read_item(X) before T_j executes write_item(X)

T_i executes a write_item(X) before T_j executes write_item(X)

Conflict Serializability Example-2 (Graph Test)

Test for Serializability

An edge $T_i \rightarrow T_j$ means that in any serial schedule equivalent to S, T_i must come before T_j

If G has a cycle, then S is not conflict serializable

If not, use topological sort to obtain serializable schedule (linear order consistent with precedence order of graph)

To check consistency if graph is acyclic otherwise schedule has deadlock means inconsistent

Conflict Serializability Example-2 (Graph Test)

Example-2 (Graph Test)

T1	T2	T3
		read_item(Y);
		read_item(Z);
read_item(X);		
write_item(X);		
		write_item(Y);
		write_item(Z);
	read_item(Z);	
read_item(Y);		
write_item(Y);		
	read_item(Y);	
	write_item(Y);	
	read_item(X);	
	write_item(X);	

Introduction to Database Administration



Apply Locking

Apply Locking

Apply Locking

- Database Locks
- Locks in Oracle SQL
- Locks Update

Apply Locking

Database Locks

- When two sessions or users of database try to update or delete the same data in a table, then there will be a concurrent update problem. In order to avoid this problem, database locks the data for the first user and allows him to update/delete the data.
- Once he is done with his update/delete, he COMMITs or ROLLBACK the transaction, which will release the lock on the data. When lock on the data is released, another user can lock it for his changes.

Apply Locking

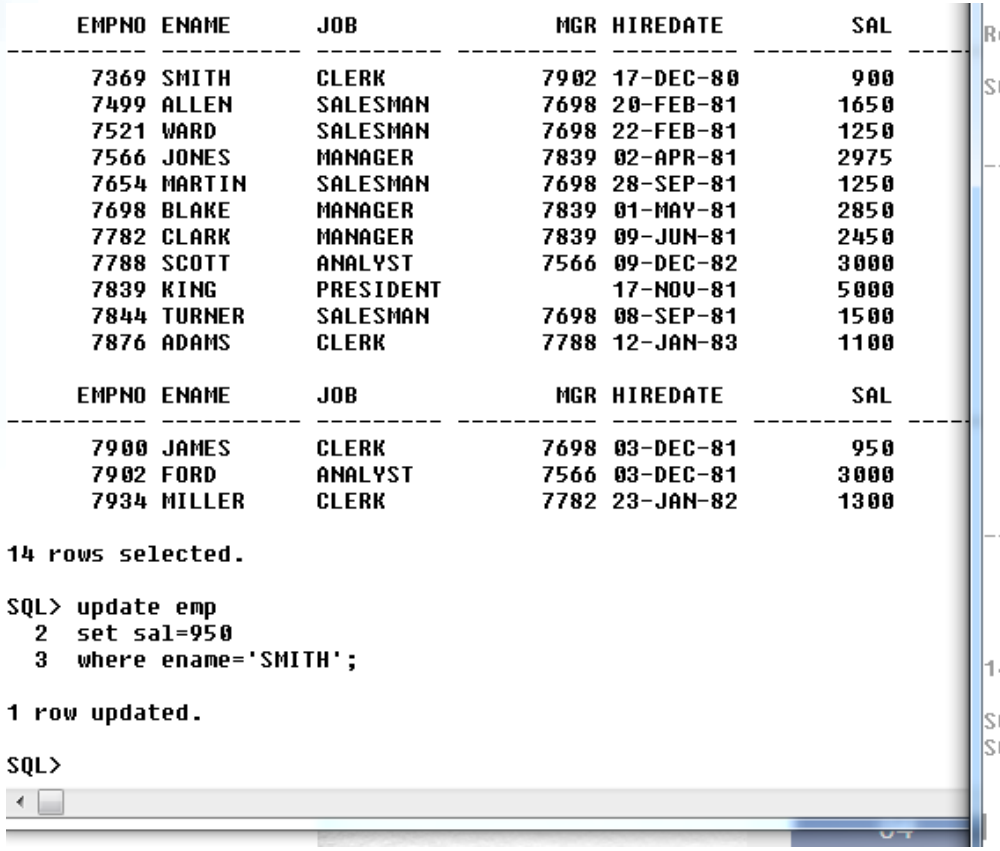
Database Locks

- An Object is locked before it performs any operation
- Two types of operations, two types of locks

  Shared & Exclusive Locks

Apply Locking

Locks in Oracle SQL



```
     EMPNO ENAME      JOB              MGR HIREDATE         SAL
---------- ---------- --------- ---------- --------- ----------
      7369 SMITH      CLERK           7902 17-DEC-80        900
      7499 ALLEN      SALESMAN        7698 20-FEB-81       1650
      7521 WARD       SALESMAN        7698 22-FEB-81       1250
      7566 JONES      MANAGER         7839 02-APR-81       2975
      7654 MARTIN     SALESMAN        7698 28-SEP-81       1250
      7698 BLAKE      MANAGER         7839 01-MAY-81       2850
      7782 CLARK      MANAGER         7839 09-JUN-81       2450
      7788 SCOTT      ANALYST         7566 09-DEC-82       3000
      7839 KING       PRESIDENT            17-NOV-81       5000
      7844 TURNER     SALESMAN        7698 08-SEP-81       1500
      7876 ADAMS      CLERK           7788 12-JAN-83       1100

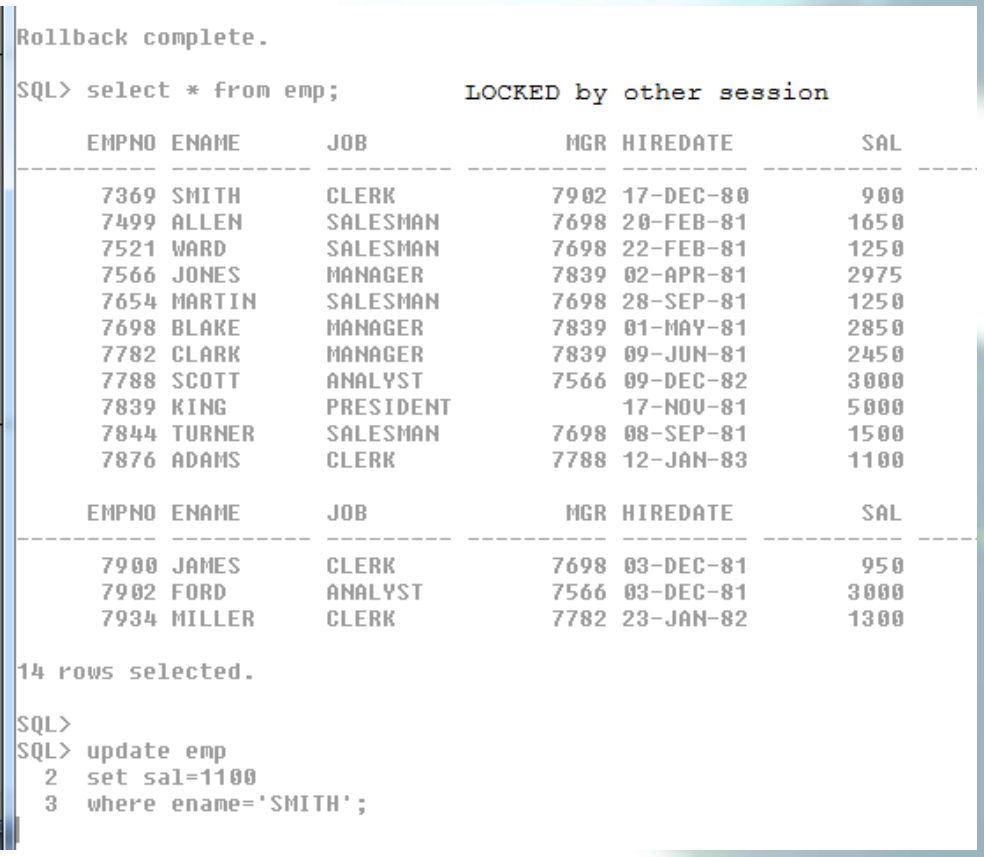
     EMPNO ENAME      JOB              MGR HIREDATE         SAL
---------- ---------- --------- ---------- --------- ----------
      7900 JAMES      CLERK           7698 03-DEC-81        950
      7902 FORD       ANALYST         7566 03-DEC-81       3000
      7934 MILLER     CLERK           7782 23-JAN-82       1300

14 rows selected.

SQL> update emp
  2  set sal=950
  3  where ename='SMITH';

1 row updated.

SQL>
```



```
Rollback complete.

SQL> select * from emp;          LOCKED by other session

     EMPNO ENAME      JOB              MGR HIREDATE         SAL
---------- ---------- --------- ---------- --------- ----------
      7369 SMITH      CLERK           7902 17-DEC-80        900
      7499 ALLEN      SALESMAN        7698 20-FEB-81       1650
      7521 WARD       SALESMAN        7698 22-FEB-81       1250
      7566 JONES      MANAGER         7839 02-APR-81       2975
      7654 MARTIN     SALESMAN        7698 28-SEP-81       1250
      7698 BLAKE      MANAGER         7839 01-MAY-81       2850
      7782 CLARK      MANAGER         7839 09-JUN-81       2450
      7788 SCOTT      ANALYST         7566 09-DEC-82       3000
      7839 KING       PRESIDENT            17-NOV-81       5000
      7844 TURNER     SALESMAN        7698 08-SEP-81       1500
      7876 ADAMS      CLERK           7788 12-JAN-83       1100

     EMPNO ENAME      JOB              MGR HIREDATE         SAL
---------- ---------- --------- ---------- --------- ----------
      7900 JAMES      CLERK           7698 03-DEC-81        950
      7902 FORD       ANALYST         7566 03-DEC-81       3000
      7934 MILLER     CLERK           7782 23-JAN-82       1300

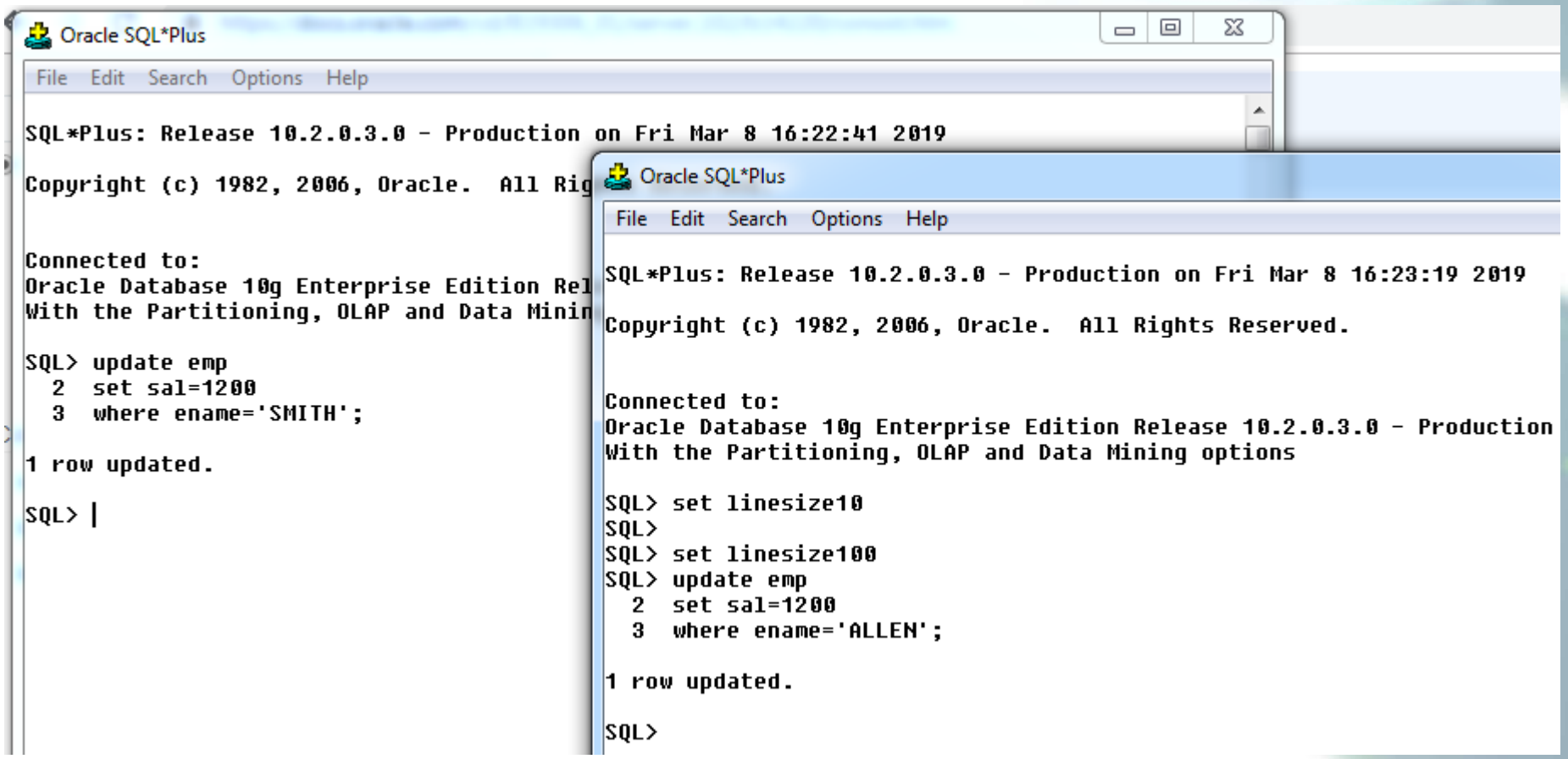
14 rows selected.

SQL>
SQL> update emp
  2  set sal=1100
  3  where ename='SMITH';
```

Record level lock/ Row Lock – same record

Apply Locking

Locks Update



Introduction to Database Administration



Deadlocks

Deadlocks

Deadlocks

- Database Deadlocks
- Deadlocks - Two Transactions
- Applying Locks at Table Level
- Deadlock - Example

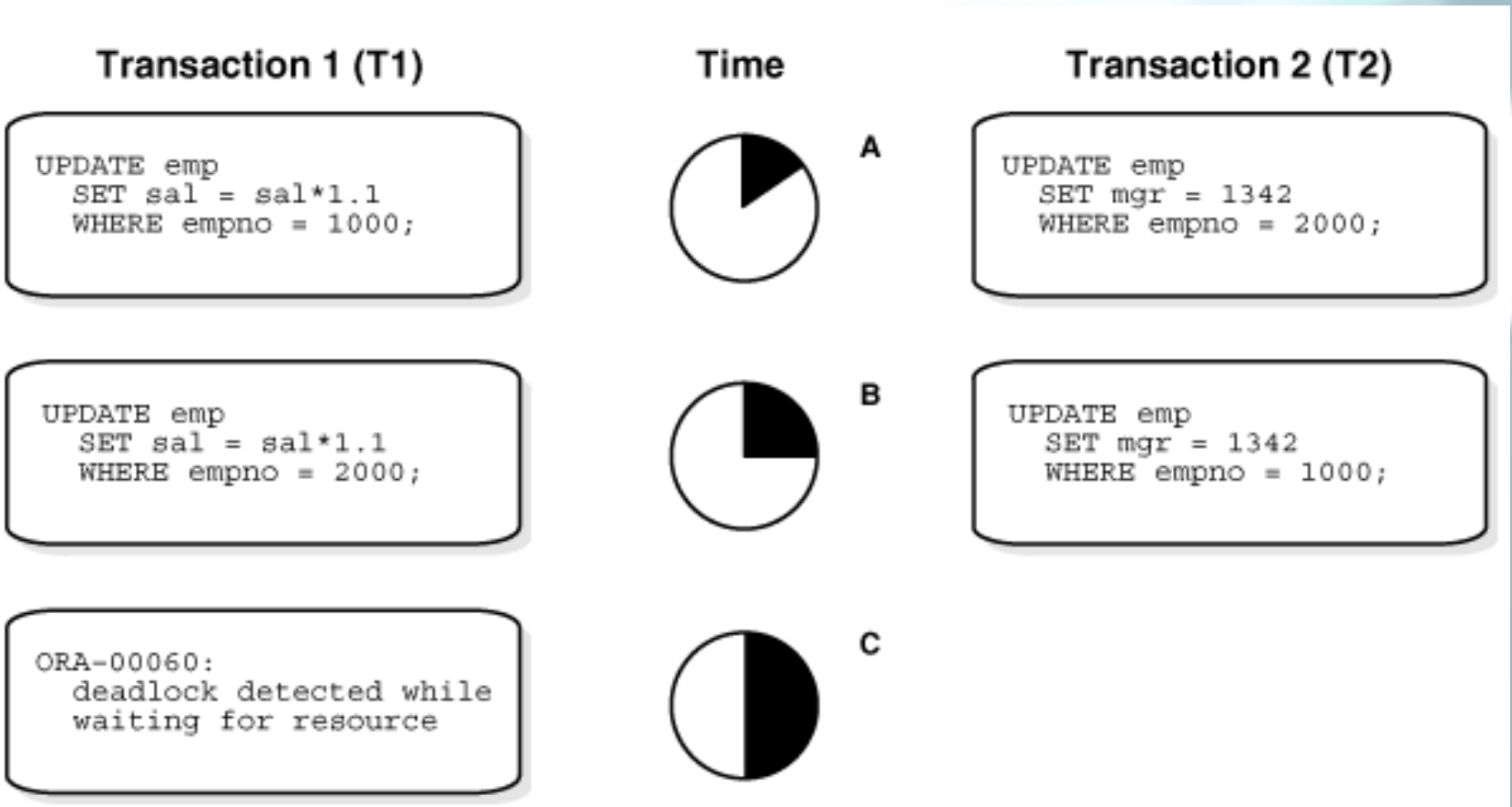
Deadlocks

Database Deadlocks

In a **database**, a **deadlock** is a situation in which two or more transactions are waiting for one another to give up locks. For example, Transaction A might hold a lock on some rows in the Accounts table and needs to update some rows in the Orders table to finish.

Deadlocks

Deadlocks – Two Transactions



Deadlocks

Applying Locks at Table Level

```
--user scott
Lock table dept IN EXCLUSIVE MODE;

--user can use rollback to unlock this table

--user sys
 select object_id, session_id, oracle_username, locked_mode
 from V$Locked_Object;

 OBJECT_ID SESSION_ID ORACLE_USERNAME                 LOCKED_MODE
---------- ---------- ------------------------------ -----------
     53768        140 SCOTT                                    6  (Locked applied)
     53768        148 SCOTT                                    0  (Locked in other session)

--in session 148
SQL> update dept
  2  set loc='BOSTON1'
  3  where deptno=40;

----LOCKED-------

How to put in share mode?
Lock table dept in share mode;
```

Deadlocks

Deadlock - Example

Time	T1	T2
t1	XLock(x)	…
t2	…	XLock(y)
t3	Request y	…
t4	wait	Request x
t5	wait	wait
t6	wait	wait



DDL Schema Script for Project

DDL Schema Script for Project

DDL Schema Script for Project

- Plan the Script for Schema
- Sample Script



DDL Schema Script for Project

Plan the Script for Schema

ANSI standard script of SQL consists of

Create and Alter Statements

Standard & Additional Constraints

Tables with Tablespace Storage

Database Security

Shared Logical Objects such as Views, Synonyms

Logical & Physical Storage

DML - SQL

DDL Schema Script for Project

Sample Script

```
Drop table employee;
Drop table orders;
Drop table customer;
Drop table Item;

create table Customer
(custID          CHAR(10),
CustName         VARCHAR2(25) NOT NULL,
Address          VARCHAR2(50),
City             VARCHAR2(50),
TelNo            VARCHAR2(15),
Email            VARCHAR2(50) UNIQUE)
tablespace TS_Sales;

Alter table Customer
add constraint PK_Custid primary key (CustID);

Create table Orders
(InvoiceNo       CHAR(12),
InvType          CHAR(1),
InvDate          Date NOT NULL,
TotalPrice       Number(6,2),
SalesEmpID       CHAR(5),
CustID           CHAR(8))
tablespace TS_Sales;

Alter table Orders
add constraint PK_InvNo primary key (InvoiceNO)
USING INDEX TABLESPACE Sale_indx;

Alter table Orders
add constraint FK_Cust_CustID foreign key (custID)
references Customer(CustID);
```

DDL Schema Script for Project

Alter Table with Move on Storage

```
ALTER TABLE hr.admin_emp MOVE
      STORAGE ( INITIAL 20K
                NEXT 40K
                MINEXTENTS 2
                MAXEXTENTS 20
                PCTINCREASE 0 )
  TABLESPACE hr_tbs;

ALTER TABLE hr.admin_emp
      ADD (bonus NUMBER (7,2));
```

Introduction to Database Administration

DDL Schema Script including Tables, Columns and Data Types

DDL Schema Script including Tables, Columns and Data Types

DDL Schema Script including Tables, Columns and Data Types

- DDL Script for Schema
- Sample Create Table - SQL
- Sample Create/ Alter Table with Constraints
- Sample Create Table with Additional Constraints

DDL Schema Script including Tables, Columns and Data Types

DDL Script for Schema

Scripts are used to create objects (structures) such as tables, sequences, views and PL/SQL programs etc in a schema. Many script contains DDL SQL.

Script can contain DML for populating data for development and testing.

Similarly, structure of schema objects can be populated using exported DMP file.

DDL Schema Script including Tables, Columns and Data Types

Sample Create Table - SQL

```
CREATE TABLE hr.admin_emp (
        empno       NUMBER(5) PRIMARY KEY,
        ename       VARCHAR2(15) NOT NULL,
        ssn         NUMBER(9) ENCRYPT USING 'AES256',
        job         VARCHAR2(10),
        mgr         NUMBER(5),
        hiredate    DATE DEFAULT (sysdate),
        photo       BLOB,
        sal         NUMBER(7,2),
        hrly_rate   NUMBER(7,2) GENERATED ALWAYS AS (sal/2080),
        comm        NUMBER(7,2),
        deptno      NUMBER(3) NOT NULL
                     CONSTRAINT admin_dept_fkey REFERENCES hr.departments
                     (department_id))
   TABLESPACE admin_tbs
   STORAGE ( INITIAL 50K);

COMMENT ON TABLE hr.admin_emp IS 'Enhanced employee table';
```

DDL Schema Script including Tables, Columns and Data Types

Sample Create/ Alter Table with Constraints

```
Drop table employee;
Drop table orders;
Drop table customer;
Drop table Item;

create table Customer
(custID          CHAR(10),
CustName         VARCHAR2(25) NOT NULL,
Address          VARCHAR2(50),
City             VARCHAR2(50),
TelNo            VARCHAR2(15),
Email            VARCHAR2(50) UNIQUE)
tablespace TS_Sales;

Alter table Customer
add constraint PK_Custid primary key (CustID);

Create table Orders
(InvoiceNo       CHAR(12),
InvType          CHAR(1),
InvDate          Date NOT NULL,
TotalPrice       Number(6,2),
SalesEmpID       CHAR(5),
CustID           CHAR(8))
tablespace TS_Sales;

Alter table Orders
add constraint PK_InvNo primary key (InvoiceNO)
USING INDEX TABLESPACE Sale_indx;

Alter table Orders
add constraint FK_Cust_CustID foreign key (custID)
references Customer(CustID);
```

DDL Schema Script including Tables, Columns and Data Types

Sample Create Table with Additional Constraints

```
create table Employee
(EmpID              CHAR(3),
EmpName             VARCHAR2(25) NOT NULL,
Address             VARCHAR2(50),
City                VARCHAR2(50),
TelNo               VARCHAR2(15),
Email               VARCHAR2(50) UNIQUE,
Status              CHAR(1))
tablespace TS_Sales;

Alter table Employee
add constraint ck_status
check (Status in ('Y', 'N'));

Alter table Employee
add constraint PK_EmpID primary key (EmpID)
USING INDEX TABLESPACE Sale_indx;
```

DDL Schema Script including Allocation of Tablespaces with Sizes

DDL Schema Script including Allocation of Tablespaces with Sizes

DDL Schema Script for Tablespaces

- Logical & Physical Storage
- Data Tablespaces for Tables
- Tablespace for Indexes
- Temporary Tablespace for Project

DDL Schema Script including Allocation of Tablespaces with Sizes

Logical & Physical Storage

- Creating tablespaces for data, indexes
- Tablespaces for table or tables
- Tablespace with OS level storage

DDL Schema Script including Allocation of Tablespaces with Sizes

Data Tablespaces for Tables

```
Create tablespace MedCare Datafile
    'D:\oracle9i\ora92\oradata\MCare01.dbf' Size 150M online
    MINIMUM EXTENT 64K
    DEFAULT STORAGE (INITIAL 64K NEXT 64K);


Create tablespace MCare_IVF Datafile
     'D:\oracle9i\ora92\oradata\MCIVF01.dbf' Size 50M online
     MINIMUM EXTENT 64K
     DEFAULT STORAGE (INITIAL 64K NEXT 64K);

CREATE TABLESPACE MCare_IVFCHART DATAFILE
    'D:\oracle9i\ora92\oradata\MCare_IVFCHART01.dbf' SIZE 50M
    MINIMUM EXTENT 1024K
    DEFAULT STORAGE (INITIAL 1024K NEXT 1024K);
```

DDL Schema Script including Allocation of Tablespaces with Sizes

Tablespace for Indexes

```
CREATE TABLESPACE MCare_indx
DATAFILE 'D:\oracle9i\ora92\oradata\MCare_indx01.dbf' SIZE 100M
MINIMUM EXTENT 64K
DEFAULT STORAGE (INITIAL 64K NEXT 64K);
```

DDL Schema Script including Allocation of Tablespaces with Sizes

Temporary Tablespace for Project

```
CREATE TEMPORARY TABLESPACE "TEMP_DEFAULT3" TEMPFILE
    'D:\ORACLE9I\ORA92\ORADATA\eahmed\TEMP_DEFAUL3.DBF' SIZE
    1000M EXTENT MANAGEMENT LOCAL UNIFORM SIZE 1M;

ALTER DATABASE DEFAULT TEMPORARY TABLESPACE "TEMP_DEFAULT3";
```



Creation of DML Insert Statements for Data

Creation of DML Insert Statements for Data

DML Schema Script

- DDL & DML Script
- Sample DML Insert Statements
- Sample DML Insert & Update Statements

Creation of DML Insert Statements for Data

DDL & DML Script

- One script file can be created to manage DDL & DML commands
- Separate files for DDL & DML
- DML Insert, Delete and Update commands
- All commands are in an order
- Well-defined datasets are selected

Creation of DML Insert Statements for Data

Sample DML Insert Statements

```
insert into customer(
CUSTID,
CUSTNAME,
ADDRESS,
CITY,
TELNO,
EMAIL)
values(
'88700110',
'3M Co',
'41 Main st.',
'Lahore',
'0514568877',
'ejaz@nu.edu.pk'
);

insert into customer(
CUSTID,
CUSTNAME,
ADDRESS,
```

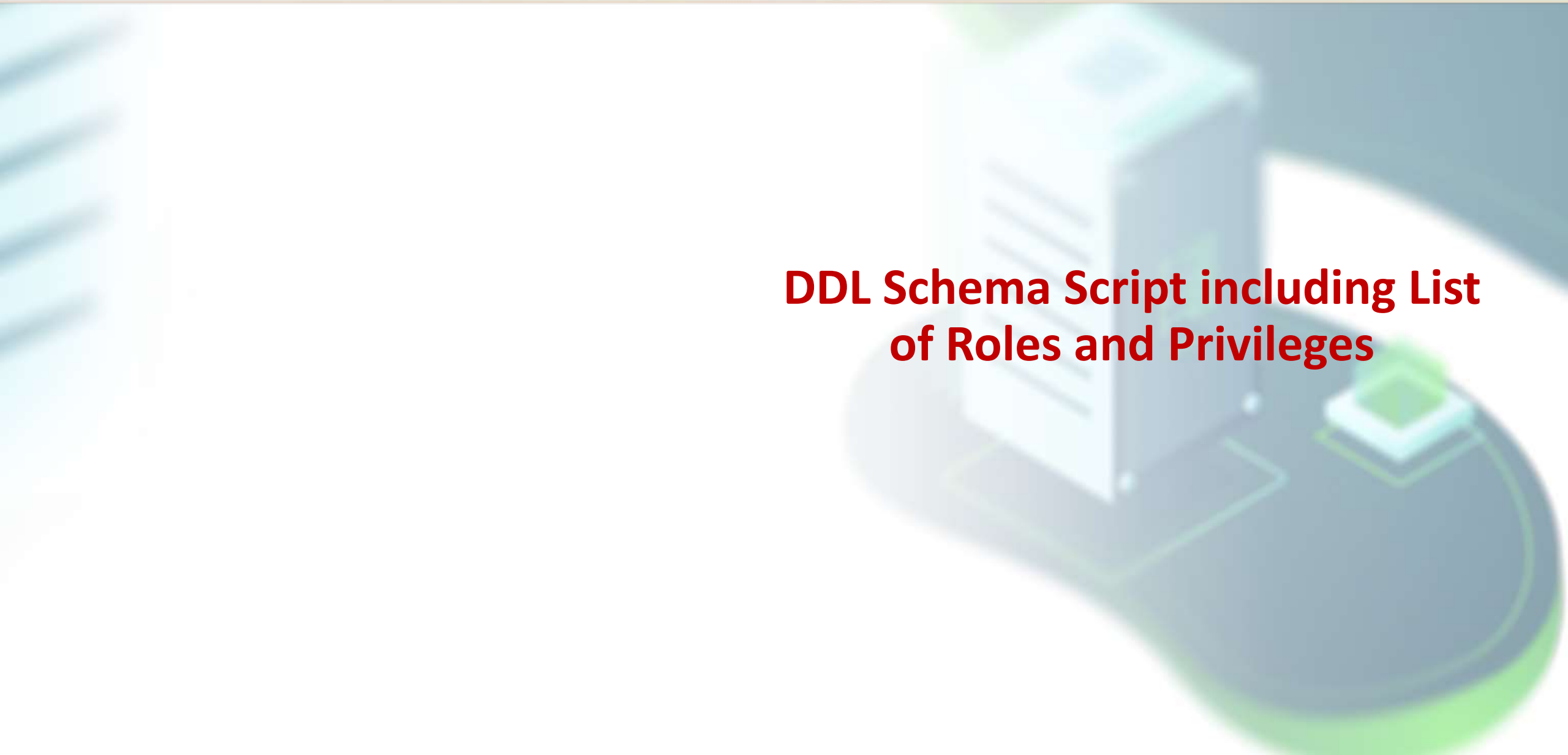
Creation of DML Insert Statements for Data

Sample DML Insert & Update Statements

```
Insert into Customer

...

Update Sold
set qty = (select qty
           from Item
           where ItemID='254011');
```



DDL Schema Script including List of Roles and Privileges

DDL Schema Script including List of Roles and Privileges

Script for Security

- Roles & Privileges
- Creating User with Grants

DDL Schema Script including List of Roles and Privileges

Roles & Privileges

- Create roles
- Assign privileges to roles
- Assign privileges to users

DDL Schema Script including List of Roles and Privileges

Creating User with Grants

```
create tablespace work
datafile 'E:\oracle\product\10.2.0\db_2\oradata\work01.dbf'size 500M online;

create user uwork
identified by xyz
default tablespace work
temporary tablespace temp;

grant connect, resource to uwork;
grant create table to uwork;


create tablespace work_indx
datafile 'E:\oracle\product\10.2.0\db_2\oradata\WIndx01.dbf'size 200M online;

create tablespace work_depend
datafile 'E:\oracle\product\10.2.0\db_2\oradata\WDepend01.dbf'size 300M online;
```

DDL Schema Script including List of Roles and Privileges

Creating User with Grants

```
grant select on MED_STOCK  to public;
grant select on TRANSACTION   to public;
grant select on TRANS_DETAIL  to public;


grant select, insert, update on EMP to public;
grant select, insert, update on DEPT to ejaz;
```

Introduction to Database Administration

DDL Schema Script including Synonyms of Tables & Drop of all Objects

DDL Schema Script including Synonyms of Tables & Drop of all Objects

Synonyms & Drop Objects

- Generate SQL Script using SELECT Clause
- Creating Synonyms
- Drop Synonyms
- Drop Tables

DDL Schema Script including Synonyms of Tables & Drop of all Objects

Generate SQL Script using Select Clause

```
--Table EMP owner (scott) gives grant:
grant select on emp to u1;
--generate the following SQLs from scott schema and run in u1 schema
select 'create synonym '||tname||' for scott.'||tname||';'
from tab;
```

DDL Schema Script including Synonyms of Tables & Drop of all Objects

Creating Synonyms

```
grant create synonym to ul;
grant create public synonym to ul;
grant create synonym to scott;
grant create public synonym to scott;
```

DDL Schema Script including Synonyms of Tables & Drop of all Objects

Drop Synonyms

```
--inside same schema of scott:
create public synonym syn_emp for scott.emp;

drop synonym dept;
drop synonym emp;
```

DDL Schema Script including Synonyms of Tables & Drop of all Objects

Drop Tables

```
drop table EMP;
drop table ProjectAssign;
drop table DEPT;
drop project;
```

Introduction to Database Administration



Revision of Course Contents

Revision of Course Contents

Revision of DBA Course

- Introduction to Database & Architecture
- Database Design (Development)
- DBA
- PL/SQL
- Project’s Preparations & Scripts
- Logical & Physical Storage

Revision of Course Contents

Introduction to Database & Architecture

- What is Database?
- What is DBMS?
- ANSI SPARK 3 Level Architecture
- 2 & 3 Tiers Client Server Architecture
- Keys & Constraints

Revision of Course Contents

Database Design (Development)

- What is ER Model?
- ER Model Artifacts
- ER Mapping & Implementation

Revision of Course Contents

DBA

- DBA tasks & responsibilities
- Data Dictionary
- Oracle Installation & Configuration
- Logical & Physical Storage
- Backup & Recovery
- Database Tuning & Performance
- Support & Services
- Transaction Processing

Revision of Course Contents

PLSQL

- Procedure programming using Blocks
- Programming exceptions
- Procedures, functions & packages
- Cursor and loops
- Advance topics using Arrays, Dynamic SQL.

Revision of Course Contents

Project's Preparation and Scripts

- Database Mapping or Implementation
- Scripts for DDL
- Scripts for logical tablespaces
- Scripts for Security
- Data Migration

Revision of Course Contents

Logical & Physical Storage

